

Data Science Core Skills

Build your background before you arrive

August 2026

Table of contents

1	Data Science Core Skills	3
1.1	What is here	3
1.2	Where to start	3
1.3	How much time it takes	4
1.4	Downloading it	4
1.5	About the data	4
I	Before you start	5
2	How to use this	6
2.1	Every unit has the same ten parts	6
2.2	How to work through a unit	6
2.3	What you can skip	7
2.4	If you get stuck	7
2.5	A note on honesty	7
3	Set up Python	8
3.1	1. Install Python	8
3.2	2. Make a folder and an environment	8
3.3	3. Install the packages	9
3.4	4. Check it worked	9
3.5	5. Get the data	9
3.6	When something goes wrong	9
4	The diagnostic	10
4.1	Python	10
4.2	Mathematics	11
4.3	Statistics	13
4.4	What to do with your answers	14
II	Python	15
5	Getting Python running	16
5.1	Before you start	16
5.2	The idea	16
5.3	The mechanics	17
5.4	Worked example	19
5.5	Try it yourself	20
5.6	Check it in Python	20
5.7	Common wrong turns	21
5.8	Check yourself	21

6	Values, names and types	23
6.1	Before you start	23
6.2	The idea	24
6.3	The mechanics	24
6.4	Worked example	27
6.5	Try it yourself	28
6.6	Check it in Python	29
6.7	Common wrong turns	29
6.8	Check yourself	30
7	Collections	31
7.1	Before you start	31
7.2	The idea	31
7.3	The mechanics	32
7.4	Worked example	35
7.5	Try it yourself	36
7.6	Check it in Python	38
7.7	Common wrong turns	38
7.8	Check yourself	39
8	Making decisions, repeating work	40
8.1	Before you start	40
8.2	The idea	40
8.3	The mechanics	41
8.4	Worked example	44
8.5	Try it yourself	45
8.6	Check it in Python	47
8.7	Common wrong turns	47
8.8	Check yourself	48
9	Functions and modules	49
9.1	Before you start	49
9.2	The idea	49
9.3	The mechanics	51
9.4	Worked example	54
9.5	Try it yourself	55
9.6	Check it in Python	57
9.7	Common wrong turns	58
9.8	Check yourself	58
10	NumPy: arrays and indexing	60
10.1	Before you start	60
10.2	The idea	60
10.3	The mechanics	61
10.4	Worked example	65
10.5	Try it yourself	66
10.6	Check it in Python	67
10.7	Common wrong turns	68
10.8	Check yourself	68
11	NumPy: computing across arrays	70
11.1	Before you start	70

11.2	The idea	70
11.3	The mechanics	71
11.4	Worked example	75
11.5	Try it yourself	76
11.6	Check it in Python	77
11.7	Common wrong turns	78
11.8	Check yourself	79
12	pandas: the DataFrame	81
12.1	Before you start	81
12.2	The idea	81
12.3	The mechanics	82
12.4	Worked example	85
12.5	Try it yourself	87
12.6	Check it in Python	88
12.7	Common wrong turns	89
12.8	Check yourself	89
13	pandas: cleaning, grouping, joining	91
13.1	Before you start	91
13.2	The idea	91
13.3	The mechanics	92
13.4	Worked example	97
13.5	Try it yourself	99
13.6	Check it in Python	100
13.7	Common wrong turns	101
13.8	Check yourself	101
14	Plotting with Matplotlib	103
14.1	Before you start	103
14.2	The idea	103
14.3	The mechanics	104
14.4	Worked example	109
14.5	Try it yourself	110
14.6	Check it in Python	110
14.7	Common wrong turns	112
14.8	Check yourself	112
15	Work someone else can rerun	114
15.1	Before you start	114
15.2	The idea	114
15.3	The mechanics	115
15.4	Worked example	118
15.5	Try it yourself	120
15.6	Check it in Python	120
15.7	Getting help, and using an assistant	122
15.8	Common wrong turns	122
15.9	Check yourself	122

III Mathematics	124
16 Reading mathematics	125
16.1 Before you start	125
16.2 The idea	125
16.3 The mechanics	126
16.4 Worked example	128
16.5 Try it yourself	129
16.6 Check it in Python	130
16.7 Common wrong turns	131
16.8 Check yourself	131
17 Functions, exponentials and logarithms	133
17.1 Before you start	133
17.2 The idea	133
17.3 The mechanics	134
17.4 Worked example	136
17.5 Try it yourself	137
17.6 Check it in Python	138
17.7 Common wrong turns	139
17.8 Check yourself	140
18 Rates of change	141
18.1 Before you start	141
18.2 The idea	141
18.3 The mechanics	144
18.4 Worked example	146
18.5 Try it yourself	147
18.6 Check it in Python	148
18.7 Common wrong turns	149
18.8 Check yourself	150
19 The differentiation toolkit	152
19.1 Before you start	152
19.2 The idea	152
19.3 The mechanics	153
19.4 Worked example	154
19.5 Try it yourself	155
19.6 Check it in Python	156
19.7 Common wrong turns	157
19.8 Check yourself	158
20 Finding the best value	159
20.1 Before you start	159
20.2 The idea	159
20.3 The mechanics	160
20.4 Worked example	162
20.5 Try it yourself	163
20.6 Check it in Python	164
20.7 Common wrong turns	165
20.8 Check yourself	166

21 Several variables and the gradient	167
21.1 Before you start	167
21.2 The idea	168
21.3 The mechanics	169
21.4 Worked example	171
21.5 Try it yourself	173
21.6 Check it in Python	174
21.7 Common wrong turns	176
21.8 Check yourself	177
22 Accumulation and area	179
22.1 Before you start	179
22.2 The idea	180
22.3 The mechanics	181
22.4 Worked example	183
22.5 Try it yourself	184
22.6 Check it in Python	185
22.7 Common wrong turns	187
22.8 Check yourself	187
23 Data as vectors	189
23.1 Before you start	189
23.2 The idea	189
23.3 The mechanics	190
23.4 Worked example	192
23.5 Try it yourself	193
23.6 Check it in Python	194
23.7 Common wrong turns	196
23.8 Check yourself	196
24 Matrices as operations on data	198
24.1 Before you start	198
24.2 The idea	199
24.3 The mechanics	200
24.4 Worked example	201
24.5 Try it yourself	203
24.6 Check it in Python	205
24.7 Common wrong turns	206
24.8 Check yourself	207
25 Solving $Ax = b$	208
25.1 Before you start	208
25.2 The idea	208
25.3 The mechanics	210
25.4 Worked example	211
25.5 Try it yourself	213
25.6 Check it in Python	214
25.7 Common wrong turns	215
25.8 Check yourself	216
26 When there is no solution: least squares	218
26.1 Before you start	218

26.2	The idea	219
26.3	The mechanics	219
26.4	Worked example	221
26.5	Try it yourself	222
26.6	Check it in Python	224
26.7	Common wrong turns	226
26.8	Check yourself	227
27	Directions in data: eigenvectors and the SVD	229
27.1	Before you start	229
27.2	The idea	230
27.3	The mechanics	230
27.4	Worked example	232
27.5	Try it yourself	234
27.6	Check it in Python	235
27.7	Common wrong turns	237
27.8	Check yourself	237
IV	Statistics	239
28	What is the question?	240
28.1	Before you start	240
28.2	The idea	241
28.3	The mechanics	242
28.4	Worked example	244
28.5	Try it yourself	247
28.6	Check it in Python	248
28.7	Common wrong turns	250
28.8	Check yourself	250
29	Describing one variable	252
29.1	Before you start	252
29.2	The idea	252
29.3	The mechanics	253
29.4	Worked example	255
29.5	Try it yourself	257
29.6	Check it in Python	259
29.7	Common wrong turns	260
29.8	Check yourself	261
30	Describing two variables	263
30.1	Before you start	263
30.2	The idea	264
30.3	The mechanics	264
30.4	Worked example	266
30.5	Try it yourself	267
30.6	Check it in Python	269
30.7	Common wrong turns	270
30.8	Check yourself	271

31 Probability, conditioning and Bayes	273
31.1 Before you start	273
31.2 The idea	274
31.3 The mechanics	274
31.4 Worked example	277
31.5 Try it yourself	278
31.6 Check it in Python	280
31.7 Common wrong turns	281
31.8 Check yourself	282
32 Random variables	284
32.1 Before you start	284
32.2 The idea	285
32.3 The mechanics	286
32.4 Worked example	288
32.5 Try it yourself	289
32.6 Check it in Python	291
32.7 Common wrong turns	292
32.8 Check yourself	293
33 Distributions chosen by mechanism	295
33.1 Before you start	295
33.2 The idea	296
33.3 The mechanics	296
33.4 Worked example	299
33.5 Try it yourself	300
33.6 Check it in Python	302
33.7 Common wrong turns	304
33.8 Check yourself	304
34 Sampling distributions and the CLT	306
34.1 Before you start	306
34.2 The idea	307
34.3 The mechanics	307
34.4 Worked example	310
34.5 Try it yourself	311
34.6 Check it in Python	312
34.7 Common wrong turns	314
34.8 Check yourself	315
35 Confidence intervals and the bootstrap	317
35.1 Before you start	317
35.2 The idea	318
35.3 The mechanics	318
35.4 Worked example	320
35.5 Try it yourself	321
35.6 Check it in Python	323
35.7 Common wrong turns	324
35.8 Check yourself	325
36 Hypothesis tests	327
36.1 Before you start	327

36.2	The idea	328
36.3	The mechanics	328
36.4	Worked example	330
36.5	Try it yourself	332
36.6	Check it in Python	334
36.7	Common wrong turns	335
36.8	Check yourself	336
37	Comparing groups, measuring association	338
37.1	Before you start	338
37.2	The idea	338
37.3	The mechanics	340
37.4	Worked example	341
37.5	Try it yourself	344
37.6	Check it in Python	345
37.7	Common wrong turns	347
37.8	Check yourself	347
38	Simple linear regression	349
38.1	Before you start	349
38.2	The idea	350
38.3	The mechanics	350
38.4	Worked example	352
38.5	Try it yourself	354
38.6	Check it in Python	356
38.7	Common wrong turns	358
38.8	Check yourself	359
39	How analyses go wrong	361
39.1	Before you start	361
39.2	The idea	362
39.3	The mechanics	363
39.4	Worked example	366
39.5	Try it yourself	367
39.6	Check it in Python	369
39.7	Common wrong turns	370
39.8	Check yourself	371
V	Lookup	373
40	Formula card	374
40.1	Reading notation	374
40.2	Exponentials and logarithms	374
40.3	Differentiation	374
40.4	Integration	375
40.5	Vectors and matrices	375
40.6	Describing data	375
40.7	Probability	376
40.8	Random variables	376
40.9	Distributions	376
40.10	Inference	377

40.11	Regression	377
41	Python card	378
41.1	Core Python	378
41.2	NumPy	378
41.3	pandas	379
41.4	Matplotlib	379
41.5	scipy.stats	380
41.6	Work someone else can rerun	380
42	Glossary	381

1 Data Science Core Skills

Build your background before you arrive

Welcome. This is a self-study course in the mathematics, statistics and Python you will need for an MSc in Data Science. It is open, free, and designed to be worked through **before** your course begins.

You do not need to have done any of it to start your degree. It exists because students arrive from very different backgrounds — some have not seen calculus in a decade, some have never written a line of code, some have both and want to refresh — and because arriving with these ideas already familiar makes the first term substantially easier.

1.1 What is here

Thirty-five short units across three tracks.

Track	Units	Reading	With practice
Python	11	49 pages	20 hours
Mathematics	12	57 pages	24 hours
Statistics	12	57 pages	23 hours

Python comes first, and that is deliberate. Every mathematics and statistics unit ends by checking its own claim in a few lines of code — a derivative against a numerical estimate, a confidence interval against a simulation. Those checks are the most useful part of each unit, and they are only useful if you can read them.

Every unit is built to the same shape and sized for one sitting: a few pages to read, a worked example, exercises with full solutions, a short piece of code that checks the idea, and five questions to test yourself. Nothing is hidden in an appendix, because there are no appendices.

1.2 Where to start

i Three things, in this order

1. **How to use this** — five minutes. How the units work, how much to do at a time, and how to tell what you can skip.
2. **Set up Python** — thirty minutes, once. You can read the mathematics without it, but you will get much more from every unit if the code runs on your own machine.
3. **The diagnostic** — twenty minutes. Fifteen questions whose only purpose is to tell you which units you can skip.

1.3 How much time it takes

Working through everything takes roughly **65 to 70 hours**. How you spread that is up to you, and the units do not expire:

- **Four weeks, most of a working week** — the intensive route, if you have the summer free.
- **Ten weeks, an evening or two a week** — the steady route, and the one most people take.
- **Only what the diagnostic flags** — perfectly legitimate. If you last used matrices two years ago, refresh matrices and leave the rest.

1.4 Downloading it

Everything on this site is also available as a **PDF**, linked in the sidebar, if you prefer to read away from a screen or work on paper. The site and the PDF contain exactly the same material.

1.5 About the data

Every worked example uses one small table describing a fictional previous cohort: their prior background, hours of practice, and final scores. **The data are invented.** They describe no real student and no real cohort, and the script that generates them is included in the repository so you can see exactly how they were made.

Using one dataset throughout means you never have to learn a new context to follow an example — by the third unit you will know what the columns mean, and you can spend your attention on the idea instead.

Part I

Before you start

2 How to use this

Five minutes now will save you a lot of time later.

2.1 Every unit has the same ten parts

They are always in the same order, so after the first one you will know where you are without looking.

Part	What it is for
Why this unit is here	What you will be able to do that you cannot do now. Read it — the motivation is the part that keeps you going.
Before you start	What the unit assumes, each item linked to where it is covered, plus a quick check.
The idea	The concept, in prose, with as little notation as it can survive.
The mechanics	The rules, the notation, and the cases that catch people out.
Worked example	One problem done fully, with the reasoning shown rather than compressed.
Try it yourself	Two or three exercises. Full solutions, hidden until you ask.
Check it in Python	Code that verifies the unit's main claim.
Common wrong turns	The specific mistakes people make here, stated and corrected.
Check yourself	Five questions, answers at the foot of the page.
Where this shows up next	What this unit is needed for, so you can see it going somewhere.

2.2 How to work through a unit

Read it once without stopping, then go back and do the exercises. The reading takes twenty minutes or so; the exercises take longer, and they are where the learning actually happens.

Three habits are worth building deliberately:

Predict before you run. When you reach a piece of code, decide what it will print before you run it. When you get it wrong you have learned something specific; when you get it right you have earned some confidence.

Reveal solutions late. The solutions are one click away, which makes them very easy to look at too early. Write something down first — even something you suspect is wrong. Comparing a wrong attempt with a correct solution teaches far more than reading a correct solution cold.

Say the answer in words. If you can compute a derivative but cannot say what it means, you are not finished. Every worked example in these notes ends with an interpretation for exactly this reason.

2.3 What you can skip

Quite a lot, possibly. The [diagnostic](#) exists to tell you what.

The three tracks are largely independent, and within each one the units say what they assume. If a unit's *Before you start* list holds no surprises and you can do its quick check, you can move on. Nobody is marking this, and there is no credit for reading pages you did not need.

The tracks are presented in the order **Python, Mathematics, Statistics**. That is a suggestion, not a rule, but it is the order the material was built for: the mathematics and statistics units check themselves in code, so a little Python first makes everything after it more useful. If you already program, skim the Python track for the NumPy and pandas units and move on.

Three dependencies cross between tracks, and the units flag them:

- Every **Mathematics** and **Statistics** unit has a *Check it in Python* block, which assumes roughly **Python 1–5**.
- **Statistics 5** (random variables) uses integration from **Mathematics 7**.
- **Statistics 11** (regression) uses least squares from **Mathematics 11**.

2.4 If you get stuck

Getting stuck is not a signal that you are behind. It is the ordinary texture of learning mathematics, and everybody doing this course is experiencing it.

Try, in this order: reread the *Before you start* list and follow the link back; work the numbers on paper rather than in your head; try the exercise with smaller or simpler numbers first; leave it and come back tomorrow, which works far more often than it has any right to.

If it still will not come, note it down and move on. Several ideas here only land in retrospect, once you have seen what they are used for.

2.5 A note on honesty

There is no assessment attached to any of this, which means there is nothing to be gained by getting through it quickly. You can have an AI assistant produce every answer on this site in a few minutes and learn nothing at all.

If you use one — and it is a reasonable tool — use it the way you would use a tutor rather than an answer key: ask it to explain a step you did not follow, ask it why your attempt was wrong, ask it for another example of the same type. Then close it and do the next exercise yourself.

3 Set up Python

You can read the mathematics without running anything. But every unit ends with a short piece of code that checks the idea, and typing those in yourself — and breaking them, and fixing them — is worth a great deal.

This takes about half an hour, once.

i If you would rather start reading now and set this up later, that is fine. Come back before P1.

3.1 1. Install Python

Download Python from python.org/downloads and install it. This course is written against **Python 3.11**; anything from 3.10 onwards will work.

On Windows, tick “**Add python.exe to PATH**” on the first installer screen. It is easy to miss and awkward to fix afterwards.

3.2 2. Make a folder and an environment

Open a terminal — Terminal on macOS or Linux, PowerShell on Windows — and run:

```
mkdir bootcamp
cd bootcamp
python -m venv .venv
```

That creates an isolated environment, so that anything you install here cannot disturb anything else on your machine. Activate it:

```
# macOS or Linux
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\activate
```

Your prompt should now start with `(.venv)`. You need to do this each time you open a new terminal.

3.3 3. Install the packages

```
python -m pip install numpy==1.26.4 pandas==2.3.3 matplotlib==3.10.8
↳ scipy==1.17.0 jupyterlab==4.5.3
```

The versions are pinned so that the numbers you get match the numbers printed in these notes exactly.

3.4 4. Check it worked

```
python -c "import sys, numpy, pandas; print(sys.version);
↳ print(numpy.__version__, pandas.__version__)"
```

You should see a 3.11 version string and then 1.26.4 2.3.3.

3.5 5. Get the data

Every example uses one small table. Download `cohort.csv` and `background_meta.csv` and put them in a `data` folder inside `bootcamp`.

Check that it loads:

```
python -c "import pandas as pd; print(pd.read_csv('data/cohort.csv').shape)"
```

You should see `(240, 7)`.

3.6 When something goes wrong

It very often does, and the error message is usually more helpful than it looks.

Read the last line first. A Python error prints a long traceback, but the final line names the problem. `ModuleNotFoundError: No module named 'pandas'` means step 3 did not run in the environment that step 4 used — check that your prompt still shows `(.venv)`.

python: command not found usually means the `PATH` box was missed during installation. Reinstalling and ticking it is faster than fixing it by hand.

Keep the whole error. If you need to ask someone, the exact text of the last line is the single most useful thing you can give them.

None of this is a test, and struggling with an install says nothing whatsoever about whether you can program. It is the least interesting part of the course and the only part where the computer, not the idea, is the obstacle.

4 The diagnostic

Fifteen questions, about twenty minutes. Its only purpose is to tell you which units you can skip.

It is not a test, nothing is recorded, and there is no pass mark. Answer honestly — the cost of pretending you can do something is that you skip a unit you needed.

How to take it. Work on paper, without a calculator except where one is obviously needed, and without running any code. Write your answer down before opening the solution: an answer you nearly had is not the same as one you had, and only you can tell the difference. Keep a note of which questions you missed; the [routing table](#) at the end turns that list into a reading plan.

4.1 Python

1. What does this print?

```
print(3 / 2, 3 // 2, 7 % 3)
```

💡 Show the answer

1.5 1 1

/ always gives a float, // divides and throws away the remainder, and % gives the remainder. If the first one surprised you, that is Python 3 behaving differently from most people's expectations and from Python 2.

Covers [P2](#).

2. What does this print?

```
marks = [41, 58, 67, 72, 90]
print(marks[1:3], marks[-1])
```

💡 Show the answer

[58, 67] 90

A slice `a:b` starts at `a` and stops *before* `b`, so it has $b - a$ elements. Negative indices count back from the end.

Covers [P3](#).

3. What array does this produce, and what shape is it?

```
import numpy as np
grid = np.arange(6).reshape(2, 3)
print(grid + np.array([10, 20, 30]))
```

💡 Show the answer

```
[[10 21 32]
 [13 24 35]]
```

Shape (2, 3). The one-dimensional array of length 3 is *broadcast* down both rows. If you expected an error, or expected the addition to go the other way, read the NumPy units. Covers [P6](#) and [P7](#).

4. A pandas DataFrame `df` has a column `score` containing some missing values. What does `df[df["score"] > 70]["score"].mean()` compute, and what happens to the missing values?

💡 Show the answer

It keeps the rows whose score exceeds 70 and averages that column. Rows with a missing score are **silently dropped by the comparison** — `NaN > 70` is `False` — and `.mean()` would skip any that survived anyway.

The important half of the answer is “silently”. Nothing warns you, and the count you would have reported is smaller than you think.

Covers [P8](#) and [P9](#).

4.2 Mathematics

5. Evaluate $\sum_{i=1}^4 (2i - 1)$.

💡 Show the answer

$1 + 3 + 5 + 7 = 16$.

The notation says “put $i = 1, 2, 3, 4$ into $2i - 1$ in turn, and add the results”. If reading it aloud was the hard part rather than the arithmetic, [M1](#) is written for you.

Covers [M1](#).

6. Solve $2^x = 32$. Then write $\ln(e^3)$ and $\log(ab)$ more simply.

💡 Show the answer

$x = 5$, since $2^5 = 32$. $\ln(e^3) = 3$, because a logarithm undoes the exponential. $\log(ab) = \log a + \log b$ — logs turn multiplication into addition, which is the whole reason they exist.

Covers [M2](#).

7. A straight line passes through (2, 5) and (6, 17). What is its gradient, and what does a gradient of 3 mean in words?

💡 Show the answer

$$(17 - 5)/(6 - 2) = 12/4 = 3.$$

In words: **for every 1 the input increases, the output increases by 3**. If the arithmetic was fine but the sentence was not, do not skip M3 — the sentence is the part everything later depends on.

Covers M3.

8. Differentiate $f(x) = x^3 - 4x + 1$.

💡 Show the answer

$$f'(x) = 3x^2 - 4.$$

Bring the power down and reduce it by one; the derivative of a constant is zero.

Covers M4.

9. Where does $g(x) = x^2 - 6x + 5$ take its smallest value, and what is that value?

💡 Show the answer

$$g'(x) = 2x - 6, \text{ which is zero at } x = 3. \text{ There } g(3) = 9 - 18 + 5 = -4.$$

It is a minimum rather than a maximum because the parabola opens upwards — a check worth making rather than assuming.

Covers M5.

10. Evaluate $\int_0^2 3x^2 dx$.

💡 Show the answer

$$[x^3]_0^2 = 8 - 0 = 8.$$

If you also said “that is the area under the curve $y = 3x^2$ between 0 and 2”, you have the reading that S5 needs.

Covers M7.

11. A is 3×4 and B is 4×2 . What is the shape of AB ? What about BA ?

💡 Show the answer

AB is 3×2 : the inner dimensions (4 and 4) must match and they cancel, leaving the outer ones. BA **does not exist** — B has 2 columns and A has 3 rows, and $2 \neq 3$.

Shape arithmetic is most of what goes wrong in practice, and it is checkable before you run anything.

Covers M8 and M9.

12. Solve $2x + y = 7$ and $x - y = 2$.

💡 Show the answer

Add the two equations: $3x = 9$, so $x = 3$, and then $y = 1$.

Check both: $2(3) + 1 = 7$ and $3 - 1 = 2$. Substituting your answer back is a five-second habit that catches most arithmetic slips.

Covers [M10](#).

4.3 Statistics

13. For the values 4, 4, 5, 9, 13: give the mean, the median, and say which of the two is larger and why.

💡 Show the answer

Mean = $35/5 = 7$. Median = 5 (the middle value once sorted).

The mean is larger because the two big values pull it up, while the median notices only that they are above the middle. (For completeness, the sample standard deviation is 3.94, dividing by $n - 1 = 4$.)

Covers [S2](#).

14. Of 200 students, 80 use the library weekly and 60 of those 80 passed. Of the 120 who do not, 60 passed. What is the probability that a student passed *given* that they use the library? And that they use the library *given* that they passed?

💡 Show the answer

$P(\text{pass} \mid \text{library}) = 60/80 = 0.75$.

For the other one, count all the passers: $60 + 60 = 120$. So $P(\text{library} \mid \text{pass}) = 60/120 = 0.50$.

Two different numbers from the same 60 students, because the denominator changed. If those came out equal, [S4](#) is the most valuable unit on this site for you.

Covers [S4](#).

15. A sample of 64 measurements has a standard deviation of 16.

- (a) What is the standard error of the mean?
- (b) The resulting 95% confidence interval is (10.2, 13.8). Is it correct to say “there is a 95% probability the population mean lies between 10.2 and 13.8”?

💡 Show the answer

(a) $16/\sqrt{64} = 16/8 = 2$.

(b) **No.** The 95% describes the *procedure*: across many repeated studies, 95% of the intervals built this way contain the true mean. This particular interval either contains it or does not.

If (a) was fine and (b) was not, read [S8](#) — the distinction it draws is the one most often got wrong in published work.

Covers [S7](#) and [S8](#).

4.4 What to do with your answers

Find each question you missed and read the units in its row. If you answered a row's questions confidently, skip or skim those units — but do glance at each unit's *Before you start* box before moving past it, since that is where a unit declares what it assumes.

Table 4.1: What each question routes to

Missed	Read	If you got it, you can
1	P1, P2	start the Python track at P3
2	P3, P4	skip to P5
3	P6, P7	skim P6–P7 for the broadcasting rules
4	P8, P9	skim P8–P9, then read P11
5	M1	start the Mathematics track at M2
6	M2	skip to M3
7	M3	skip to M4
8	M4	skip to M5
9	M5, M6	skim M5, then read M6 for the many-variable case
10	M7	skip M7, but read it anyway before S5
11	M8, M9	skip to M10
12	M10, M11	skim M10, then read M11 and M12
13	S1, S2, S3	skip to S4
14	S4, S5	skip to S6
15	S7, S8, S9	start the Statistics track at S10

Three things the table cannot tell you.

A confident wrong answer is worse than a blank. If you were sure and wrong, read the unit rather than skimming it: what needs replacing is a belief, not a gap.

S1 and S12 are not on the table, because no short question tests them. They are about choosing what to ask and recognising how an answer goes wrong, and they are the two units most likely to change how you work. Read both regardless of how you scored.

Nobody skips everything. If you answered all fifteen comfortably, you do not need the mechanics — but the *Common wrong turns* block in each unit is worth twenty minutes on its own, and it is where most of the material that is new to experienced readers lives.

Part II

Python

5 Getting Python running

Python · Unit 1

P1 · Python Reading: 4 pages With practice: 2 hours Assumes: nothing — start here

Why this unit is here

You are going to spend more time reading error messages than writing code. That is not a sign of doing it badly; it is what programming is.

So this unit is only half about installing Python. The other half is learning to read what the machine tells you when something goes wrong — because for the next few weeks you are working alone, and the traceback is the only thing that can tell you what happened.

5.1 Before you start

Nothing. This is the first unit, and it assumes you have never written a line of code.

You will need a computer you can install software on, and about an hour.

Quick check. Can you find your computer's terminal? On Windows it is **PowerShell**; on macOS, **Terminal**; on Linux you already know.

 Show the answer

Windows: press the Start button and type `powershell`. macOS: press `⌘` + Space and type `terminal`.

A window opens with a line of text and a blinking cursor. That is a prompt: it is waiting for you to type a command. Nothing you type there will break your computer.

5.2 The idea

Python is a language for giving a computer instructions. You write instructions in a plain text file; a program called the **interpreter** reads that file and carries them out, one line at a time, top to bottom.

That is genuinely all that happens. There is no compilation step to think about and no magic. When something goes wrong, it went wrong on one specific line, and Python will tell you which.

There are two ways to run instructions, and you will use both:

- A **script** is a file ending in `.py`. You run the whole thing at once. Use scripts when a task should run start-to-finish the same way every time.
- A **notebook** is a file ending in `.ipynb`, made of cells you run one at a time, with output appearing underneath. Use notebooks to explore, and to mix explanation with code.

These notes are written as notebooks-in-spirit: every code block you see was actually run, and its output is printed underneath.

5.3 The mechanics

5.3.1 1. Install Python

Download it from python.org/downloads. These notes use **Python 3.11**; anything from 3.10 will work.

! On Windows, tick “**Add python.exe to PATH**” on the first installer screen. It is easy to miss and annoying to fix afterwards. If you missed it, the quickest repair is to run the installer again and choose Modify.

5.3.2 2. Make a folder and an isolated environment

In your terminal:

```
mkdir bootcamp
cd bootcamp
python -m venv .venv
```

`venv` stands for *virtual environment*. It creates a private copy of Python inside this folder, so anything you install here cannot disturb anything else on your machine. Activate it:

```
# macOS or Linux
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\activate
```

Your prompt now starts with `(.venv)`. **You must do this each time you open a new terminal.** Forgetting is the single commonest cause of “but I installed that already”.

5.3.3 3. Install the packages

```
python -m pip install numpy==1.26.4 pandas==2.3.3 matplotlib==3.10.8
↪  scipy==1.17.0 jupyterlab==4.5.3
```

Versions are pinned so the numbers you get match the numbers printed here exactly.

5.3.4 4. Your first program

Create a file called `hello.py` containing one line:

```
print("Hello from my own machine")
```

Run it:

```
python hello.py
```

5.3.5 5. Read the traceback

Now make it fail on purpose. Change the file to:

```
temperature = "22C"  
print(temperature + 5)
```

Python stops and prints a **traceback**. It looks alarming and is not. It is a report with a fixed shape, and you read it *from the bottom*.

```
temperature = "22C"  
print(temperature + 5)
```

```
TypeError: can only concatenate str (not "int") to str
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[1], line 2  
      1 temperature = "22C"  
----> 2 print(temperature + 5)  
TypeError: can only concatenate str (not "int") to str
```

```
Traceback (most recent call last):  
  File "hello.py", line 2, in <module>  
    print(temperature + 5)
```

```
~~~~~  
TypeError: can only concatenate str (not "int") to str  
what went wrong — read this first
```

Figure 5.1: Every traceback has this shape. The last line names what went wrong; the lines above say where. Read the bottom line first, then look at the line of your own code it points to.

The bottom line is the answer: you tried to join a piece of text to a number, and Python will not guess what you meant. The lines above tell you it happened in `hello.py`, on line 2.

5.4 Worked example

A traceback, diagnosed and fixed.

Suppose you are asked to add up three measurements and one of them came from a text file, so it is still text:

```
measurements = [12.5, 13.1, "12.9"]
total = sum(measurements)
```

```
TypeError: unsupported operand type(s) for +: 'float' and 'str'
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[3], line 2
      1 measurements = [12.5, 13.1, "12.9"]
----> 2 total = sum(measurements)
TypeError: unsupported operand type(s) for +: 'float' and 'str'
```

Work through it in the fixed order.

Bottom line first. `TypeError: unsupported operand type(s) for +: 'float' and 'str'`. Something tried to add a number to a piece of text.

Which line? The traceback points at the `sum(...)` call.

What actually reached it? Look at the data:

```
measurements = [12.5, 13.1, "12.9"]
for value in measurements:
    print(repr(value), type(value).__name__)
```

```
12.5 float
13.1 float
'12.9' str
```

There it is. The third value is `'12.9'` with quotation marks — text that looks like a number. `repr()` shows the quotation marks that `print()` hides, which is why it is the right tool for this question.

The fix, stated deliberately rather than patched:

```
measurements = [12.5, 13.1, "12.9"]
numbers = [float(value) for value in measurements]
print(sum(numbers))
```

```
38.5
```

Notice what we did *not* do: we did not delete the awkward value, and we did not wrap the whole thing in something that swallows the error. We found out what was actually in the data and converted it on purpose. That habit is worth more than any syntax in this unit.

5.5 Try it yourself

Exercise 1 Predict the final line of the traceback, then run it.

```
print(x)
```

💡 Show the solution

NameError: name 'x' is not defined.

Python has no idea what `x` is, because nothing has ever assigned a value to that name. This is different from a **TypeError**, which means the name *does* exist but holds something the operation cannot use.

Exercise 2 This one fails before it runs at all. Why is that different?

```
print("Hello
```

💡 Show the solution

SyntaxError: unterminated string literal.

The closing quotation mark is missing, so Python cannot even parse the file into instructions. A **SyntaxError** is raised *before* anything executes, which is why you can get one from a line far below code that would otherwise have run.

The practical tell: if nothing at all printed, suspect a syntax error. If some output appeared and then it stopped, the program was running and hit a problem partway.

Exercise 3 You run a script and get **ModuleNotFoundError: No module named 'pandas'**, but you installed pandas ten minutes ago. What is the most likely cause?

💡 Show the solution

You are not in the environment you installed into. Check whether your prompt still shows `(.venv)`; if not, activate it again.

The general shape of this error: Python is looking, and not finding, so either it was never installed *or you are asking a different Python*. Confirm which interpreter is running with the check in the next section.

5.6 Check it in Python

Every other unit ends by checking a claim. This one checks your setup, which is the claim that matters here.

```
import sys
import numpy, pandas, matplotlib, scipy

print("python      ", sys.version.split()[0])
print("interpreter", sys.executable)
```

```
for package in (numpy, pandas, matplotlib, scipy):
    print(f"{package.__name__:<11}", package.__version__)
```

```
python      3.11.14
interpreter /home/hyperspace/anaconda3/bin/python
numpy       1.26.4
pandas      2.3.3
matplotlib  3.10.8
scipy       1.17.0
```

Two things to look at. The version should start with **3.11** (3.10 or later is fine). And the interpreter path should point **inside your .venv folder** — if it points somewhere else, you are running the wrong Python, and that explains almost every “but I installed it” problem you will hit.

Keep this snippet. When you ask anyone for help, the answer is more useful if you paste its output with your question.

5.7 Common wrong turns

Where this usually goes wrong

Reading the traceback from the top The top is the least useful part. The bottom line names the problem; work upwards only if you need to know how execution got there.

Assuming an error means you cannot program Everyone who writes code sees errors constantly, including people who have done it for thirty years. An error is the program telling you something specific and true. It is information, not a verdict.

Forgetting to activate the environment If your prompt does not begin `(.venv)`, you are using a different Python with different packages. This causes more confusion in week one than any other single thing.

Typing the prompt character Examples sometimes show `$` or `>` at the start of a line. That is the prompt, not part of the command. Type only what follows it.

Changing several things at once when stuck Then you cannot tell which change helped. Make one change, run it, look at the result.

Pasting a fix you do not understand It may work today and leave you unable to fix it tomorrow. Before you keep any snippet — from a forum, or from an assistant — make sure you can say what each line does.

5.8 Check yourself

Answers are at the foot of the page.

1. Which line of a traceback should you read first?
 - a. the first
 - b. the last
 - c. the one naming your file
 - d. all of them, in order
2. What is the practical difference between a `SyntaxError` and a `NameError`?
3. Why does a virtual environment matter, in one sentence?

4. You see `ModuleNotFoundError` for a package you know you installed. What is the first thing to check?
5. A script prints two lines and then stops with an error. Can the problem be a syntax error? Why or why not?

 Show the answers

1. **(b) the last.** It names the exception type and the message — what actually went wrong. The lines above tell you where, which only matters once you know what.
2. A `SyntaxError` means Python could not read your file as instructions, so **nothing ran at all**. A `NameError` means the file ran fine until it reached a name that had never been given a value.
3. It keeps this project's packages separate from everything else on your machine, so installing something here cannot break something there.
4. Whether the environment is active — does your prompt show `(.venv)`? Then confirm with `sys.executable` that the running interpreter is the one inside it.
5. **No.** A syntax error is raised before any code executes, so no output would have appeared. Output followed by a stop means the program was running and hit a problem partway through — which is a useful diagnostic in itself.

Where this shows up next

P2 looks at what values actually are, and why `"22C"` and `22` behave so differently. **P11** returns to environments and versions, and to asking for help in a way that gets a useful answer. Every Mathematics and Statistics unit ends with a short **Check it in Python** block — the setup you have just built is what makes those runnable.

6 Values, names and types

Python · Unit 2

P2 · Python Reading: 4 pages With practice: 1.5 hours Assumes: P1

Why this unit is here

P1 ended with "22C" + 5 refusing to work. That refusal is the subject of this unit.

Every value in Python has a **type**, and the type decides what you are allowed to do with it. Most confusing errors in a beginner's first month come down to a value not being the type they assumed. Learning to ask *what type is this, actually?* — and to answer it in one line — removes most of them.

6.1 Before you start

You should be able to:

- run a script or a notebook cell — P1;
- read the last line of a traceback — P1.

Quick check. Without running it, what does this print?

```
print(3 + 4)
print("3" + "4")
```

💡 Show the answer

7, then 34.

The + symbol does two different jobs depending on what sits either side of it: arithmetic on numbers, joining on text. That is not Python being inconsistent — it is the type deciding the meaning, which is exactly this unit's subject.

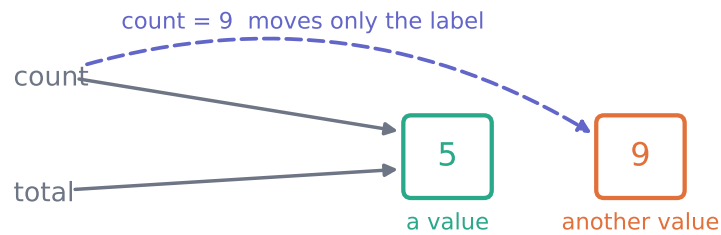


Figure 6.1: Assignment attaches a name to a value. Two names can label the same value; reassigning one moves that label without touching the value or the other name.

6.2 The idea

Writing `x = 5` does not put 5 into a box called `x`. It creates the value 5 and attaches the **name** `x` to it, like a luggage label. The name is not the thing.

That distinction sounds academic until you meet its consequences, so it is worth a picture:

In code:

```
count = 5
total = count      # a second label on the same value
count = 9          # move the first label to a different value

print("count:", count)
print("total:", total)
```

```
count: 9
total: 5
```

`total` still refers to 5. Reassigning `count` moved one label; it did not reach inside `total` and change anything. For the simple values in this unit that is easy to accept, and in [P3](#) it becomes something you must think about.

6.3 The mechanics

6.3.1 The five types to recognise

```
values = [42, 3.14, True, "hello", None]
for value in values:
    print(f"{repr(value):<8} {type(value).__name__}")
```

```
42      int
3.14    float
True    bool
'hello' str
None    NoneType
```


- **int** — a whole number, of any size. Python will happily hold a 200-digit integer.
- **float** — a number with a decimal part, stored to finite precision. More on that below, because it will bite you.
- **bool** — **True** or **False**. Use these for logical state, not as 1 and 0.
- **str** — text, in quotation marks. "5" is text; 5 is a number.
- **None** — the deliberate absence of a value. Not zero, not an empty string; a positive statement that there is nothing here.

6.3.2 Converting on purpose

```
reading = "22.5"
print(float(reading) + 5)
```

27.5

Conversion works when the value has a sensible reading in the target type, and raises when it does not:

```
float("22C")
```

```
ValueError: could not convert string to float: '22C'
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[5], line 1
----> 1 float("22C")
ValueError: could not convert string to float: '22C'
```

That failure is a feature. Python could have guessed 22 and thrown away the C, and then your temperature data would silently lose its units.

6.3.3 Floats are approximate

This is the one that surprises everyone:

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

```
0.30000000000000004
False
```

Nothing is broken. Computers store numbers in binary, and 0.1 has no exact binary representation — just as 1/3 has no exact decimal one. The tiny error is real and unavoidable.

The consequence for you is a rule: **never test computed floats with ==**. Test whether they are close enough:

```
import numpy as np
print(np.isclose(0.1 + 0.2, 0.3))
```

True

Integers are exact, so `==` is fine for them. It is floats that need care, and this is why [M4](#) checks a derivative with `np.isclose` rather than `==`.

6.3.4 f-strings

To build text that contains values, put an `f` before the quotation mark and the values in braces:

```
name = "Life sciences"
mean = 60.5137
count = 57

print(f"{name}: mean {mean:.2f} over {count} students")
```

Life sciences: mean 60.51 over 57 students

`:.2f` means “as a float, to two decimal places”. This is how every number in these notes is printed, and it saves you from reporting a mean to twelve digits you cannot justify.

6.3.5 Naming

A name should say what the value is, and what unit it is in:

```
temperature_c = 21.5
elapsed_seconds = 90
proportion_passed = 0.83

print(f"{temperature_c}°C, {elapsed_seconds}s, {proportion_passed:.0%}
↳ passed")
```

21.5°C, 90s, 83% passed

`t = 21.5` is shorter and tells a reader — including you in a fortnight — nothing. Units in the name have prevented more errors than any other habit in this course.

6.4 Worked example

Reading a value whose type is not what you assumed.

The cohort file is text on disk. Every column arrives as characters, and something has to decide what they mean. Watch what happens when that decision is skipped:

```
score_from_file = "68.4"
bonus = 5

print("what we have:", repr(score_from_file),
      ↪ type(score_from_file).__name__)
```

what we have: '68.4' str

```
score_from_file + bonus
```

TypeError: can only concatenate str (not "int") to str

```
-----
TypeError                                Traceback (most recent call last)
Cell In[11], line 1
----> 1 score_from_file + bonus
TypeError: can only concatenate str (not "int") to str
```

The error is exactly the one from [P1](#). Now convert on purpose, and check:

```
score = float(score_from_file)
print("converted:", repr(score), type(score).__name__)
print("with bonus:", score + bonus)
```

converted: 68.4 float
with bonus: 73.4

One thing worth noticing before you move on. "68.4" + "5" would *not* have raised — it would have produced "68.45", silently, and everything downstream would have been wrong without a single error message.

```
print(repr(score_from_file + "5"))
```

'68.45'

An error that stops you is a good outcome. The dangerous case is the operation that succeeds and means something else.

6.5 Try it yourself

Exercise 1 Predict each result, then run them.

```
print(7 / 2)
print(7 // 2)
print(type(7 / 2).__name__)
```

💡 Show the solution

3.5, 3, float.

/ always produces a float, even when the division is exact — 4 / 2 gives 2.0, not 2.

// is floor division: it divides and rounds down to a whole number.

The third line is the one people get wrong. 7 / 2 is a **float** because / always is.

Exercise 2 Why does the first of these print **False** and the second **True**?

```
print(0.1 + 0.2 == 0.3)
print(1 + 2 == 3)
```

💡 Show the solution

The second uses integers, which are stored exactly, so the comparison is exact.

The first uses floats. 0.1 and 0.2 cannot be represented exactly in binary, so their sum lands a hair away from 0.3 — about 5.6×10^{-17} above it. `==` asks whether they are *identical*, and they are not.

Use `np.isclose` for computed floats. Reserve `==` for integers, strings, and booleans.

Exercise 3 A colleague stores “no reading was taken” as the number **-999**. Give two distinct reasons **None** would be better, and say when **-999** might still be forced on you.

💡 Show the solution

Two reasons:

- **-999** participates in arithmetic. Take a mean and it silently drags the answer down; nothing warns you. **None** cannot be added to anything, so the mistake becomes an error you can see.
- **-999** is indistinguishable from a genuine measurement of **-999**. If the quantity could ever legitimately take that value, the two meanings are now merged and cannot be separated.

When it is forced on you: a legacy file format or an instrument that emits a sentinel. Then convert it to a proper missing value *as you load the data*, record how many you converted, and never let the sentinel reach a calculation. **P9** does exactly this.

6.6 Check it in Python

Two habits, both one line each.

Ask what you actually have rather than assuming:

```
mystery = "12.9"
print(repr(mystery), "→", type(mystery).__name__)
```

'12.9' → str

`repr()` rather than `print()`, because `print("12.9")` and `print(12.9)` look identical on screen while `repr` shows the quotation marks that distinguish them.

Compare computed floats with a tolerance, and see the difference the choice makes:

```
total = 0.1 + 0.2
print("exact equality :", total == 0.3)
print("np.isclose      :", np.isclose(total, 0.3))
print("the gap         :", total - 0.3)
```

```
exact equality : False
np.isclose     : True
the gap        : 5.551115123125783e-17
```

The gap is about 5.6×10^{-17} — far too small to matter for any measurement, and far too large for `==` to ignore.

6.7 Common wrong turns

Where this usually goes wrong

Testing computed floats with `==` Arithmetic on floats accumulates tiny errors. Use `np.isclose`, and choose a tolerance that means something for your data.

Assuming a number-looking thing is a number "5" is text. It arrives that way from files, forms and web pages. Convert deliberately and check the type when a result surprises you.

Treating None as zero or as an empty string None means *nothing was recorded*. Zero is a measurement. Conflating them changes your answers and hides the fact that data is missing.

Using a sentinel like -999 for missing data It survives into every calculation it touches. If a format forces it on you, convert it the moment you load the file.

Naming variables `x`, `df`, `data`, `temp` You will not remember what they held. `temperature_c` costs eleven characters and carries the unit with it.

Assuming print shows you the type `print` shows a value's *appearance*. "12.9" and 12.9 look the same. Use `repr()` or `type()` when the distinction matters.

6.8 Check yourself

Answers are at the foot of the page.

1. What is the type of `7 / 7`?
a. `int` b. `float` c. `bool` d. depends on the values
2. `a = [1, 2]`, then `b = a`. Has `b` got its own copy of the list, or another label on the same one?
3. Give a case where `"5" + "5"` is a bug even though nothing raises.
4. Why is `float("22C")` raising an error better than it guessing 22?
5. Someone reports a mean of `-142.7` from scores that should lie between 0 and 100. What would you check first?

Show the answers

1. **(b) `float`.** `/` always produces a float, so `7 / 7` is `1.0`. Only `//` gives an integer.
2. **Another label on the same list.** Nothing was copied. This matters far more for lists than for numbers, because a list can be changed in place — which is **P3**'s subject.
3. Any time the two values are numbers that arrived as text. `"5" + "5"` gives `"55"`, which is not 10 and not an error. Totals that are wildly too large are a classic symptom of adding numbers that were still strings.
4. Because guessing would silently discard information — the `C` might mean Celsius, or a grade, or a data-entry slip. An error forces you to decide what the value means, once, on purpose.
5. Whether a missing-value sentinel like `-999` is in the data and got included in the mean. Count how many values are impossible before doing anything else.

Where this shows up next

P3 puts many values in one container, where the name-versus-value distinction from this unit starts to matter a great deal. **P6** adds a fixed type to a whole array at once, which is what makes NumPy fast. **P9** converts a whole column deliberately, and records what would not convert.

7 Collections

Python · Unit 3

P3 · Python Reading: 4 pages With practice: 1.5 hours Assumes: P2

Why this unit is here

Data arrives in quantity. A single score is not interesting; two hundred and forty of them are.

Python gives you four containers for holding many values, and they are not interchangeable. Choosing the wrong one usually still works — which is the problem, because it works slowly, or it lets a duplicate through, or it lets something change underneath you. Choosing well takes one question, asked before you type.

7.1 Before you start

You should be able to:

- say what type a value has, and convert it deliberately — P2;
- explain what `a = b` does to names — P2.

Quick check. After `a = 5` and `b = a` and `a = 9`, what is `b`?

 Show the answer

5. Reassigning `a` moved one label; `b` still points at the original value. This unit is where that idea stops being a curiosity.

7.2 The idea

The four containers differ along three questions, and that is the whole of it:

- A **list** is an ordered, changeable sequence. It is the default, and most of the time it is right.
- A **tuple** is an ordered sequence you cannot change. Use it when a group of values belongs together and should stay put.
- A **dictionary** stores values under keys, so you look things up by name rather than by position.
- A **set** holds unique values with no order. Use it to ask *have I seen this before?*

	order	can change?	duplicates
list	ordered	changeable	duplicates ok
tuple	ordered	fixed	duplicates ok
dict	by key	changeable	keys unique
set	unordered	changeable	no duplicates

Figure 7.1: The four containers, distinguished by three questions. Answer them for your data and the choice is made.

7.3 The mechanics

7.3.1 Lists

```
scores = [68.4, 55.1, 72.0, 61.3]

print(scores[0], scores[-1])      # first and last
print(scores[1:3])                # a slice: from 1, up to but not
    ↪ including 3
print(len(scores))
```

```
68.4 61.3
[55.1, 72.0]
4
```

Python counts from **zero**, so the first item is at index 0 and the last is at `len(scores) - 1`. Negative indices count back from the end, which saves you writing `scores[len(scores) - 1]`.

Slicing takes `start:stop`, and `stop` is **not included**. `scores[1:3]` gives two items, not three. That convention looks odd for a day and then becomes convenient, because `scores[:2]` and `scores[2:]` split the list with no overlap and no gap.

Lists change in place:

```
scores.append(59.8)
scores[0] = 70.0
print(scores)
```

```
[70.0, 55.1, 72.0, 61.3, 59.8]
```


7.3.2 The part that catches people

Because a list can change in place, two names on the same list see each other's changes:

```
original = [1, 2, 3]
alias = original          # another label on the SAME list
alias.append(4)

print("original:", original)
print("alias:    ", alias)
```

```
original: [1, 2, 3, 4]
alias:    [1, 2, 3, 4]
```

That is the P2 picture with real consequences. If you want an independent list, ask for one:

```
original = [1, 2, 3]
independent = original.copy()
independent.append(4)

print("original:  ", original)
print("independent:", independent)
```

```
original:  [1, 2, 3]
independent: [1, 2, 3, 4]
```

7.3.3 Tuples

Parentheses instead of brackets, and no changing afterwards:

```
point = (52.24, -0.90)    # latitude, longitude
print(point[0])
point[0] = 0
```

```
52.24
```

```
TypeError: 'tuple' object does not support item assignment
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[6], line 3
      1 point = (52.24, -0.90)    # latitude, longitude
      2 print(point[0])
----> 3 point[0] = 0
TypeError: 'tuple' object does not support item assignment
```

The error is the point of a tuple. A coordinate pair, a date, an RGB colour — things whose parts belong together and should not be edited one at a time.

7.3.4 Dictionaries

```
student = {"id": "S0042", "background": "Life sciences", "score": 68.4}

print(student["background"])
student["completed"] = True          # add a new key
print(list(student.keys()))
```

```
Life sciences
['id', 'background', 'score', 'completed']
```

Look-up is by key, and `student["score"]` says what it means in a way that `row[2]` never will. Asking for a key that does not exist raises:

```
student["age"]
```

```
KeyError: 'age'
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 student["age"]
KeyError: 'age'
```

Use `.get()` when absence is expected rather than exceptional:

```
print(student.get("age", "not recorded"))
```

```
not recorded
```

7.3.5 Sets

```
backgrounds = ["Computing", "Life sciences", "Computing", "Social sciences"]
unique = set(backgrounds)

print(unique)
print(len(backgrounds), "values,", len(unique), "distinct")
```

```
{'Computing', 'Life sciences', 'Social sciences'}
4 values, 3 distinct
```

Sets are unordered. The printed order is not sorted and must never be relied on — if you want order, ask for it:

```
print(sorted(unique))
```

```
['Computing', 'Life sciences', 'Social sciences']
```

Membership testing is what sets are really for, and on a large collection it is enormously faster than scanning a list:

```
print("Computing" in unique)
```

```
True
```

7.4 Worked example

Summarising the cohort with the right container for each job.

```
import csv
from pathlib import Path

rows = list(csv.DictReader(Path("../data/cohort.csv").open()))
print(len(rows), "rows")
print(rows[0])
```

```
240 rows
```

```
{'student_id': 'S0001', 'background': 'Mathematics or Physics', 'diagnostic_score': '89.1'
09-23'}
```

`csv.DictReader` gives one **dictionary per row**, keyed by column name — so `row["final_score"]` rather than `row[4]`. Note that every value is a string: the file is text, and nothing has yet decided what the columns mean.

Now three questions, each answered by a different container.

Which backgrounds appear? Uniqueness matters, order does not — a set:

```
backgrounds = {row["background"] for row in rows if row["background"]}
for name in sorted(backgrounds):
    print(" ", name)
```

```
Computing
Engineering
Life sciences
Mathematics or Physics
Social sciences
```

What is the mean score in each? Look-up by name — a dictionary of lists:

```

by_background = {}
skipped = 0

for row in rows:
    group, raw = row["background"], row["final_score"]
    if not group or not raw:
        skipped += 1
        continue
    score = float(raw)
    if not 0 <= score <= 100:
        skipped += 1
        continue
    by_background.setdefault(group, []).append(score)

for name in sorted(by_background):
    scores = by_background[name]
    print(f"  {name:<24} n={len(scores):>3}
          ↪ mean={sum(scores)/len(scores):5.2f}")
print(f"\n  {skipped} rows skipped")

```

```

Computing          n= 69  mean=65.85
Engineering        n= 36  mean=67.15
Life sciences      n= 57  mean=60.51
Mathematics or Physics n= 36  mean=75.28
Social sciences    n= 39  mean=60.21

```

```
3 rows skipped
```

`setdefault` creates the empty list the first time a group is seen, so you do not have to check whether the key exists on every row.

Notice the three skipped rows are **counted, not silently dropped**. That number is part of the result; P9 makes it an audit file.

The five highest scores? Order matters — a sorted list:

```

all_scores = [s for scores in by_background.values() for s in scores]
print(sorted(all_scores, reverse=True)[:5])

```

```
[91.6, 88.8, 88.5, 86.6, 85.3]
```

7.5 Try it yourself

Exercise 1 Predict the output, then run it.

```

values = [10, 20, 30, 40, 50]
print(values[1:4])
print(values[:2])
print(values[-2:])

```

💡 Show the solution

[20, 30, 40], [10, 20], [40, 50].

The first stops *before* index 4. The second takes everything up to index 2. The third starts two from the end. Together they show why the convention is convenient: `values[:2] + values[2:]` reconstructs the whole list exactly.

Exercise 2 This is meant to leave `baseline` untouched. It does not. Fix it, and say what went wrong.

```
baseline = [1, 2, 3]
adjusted = baseline
adjusted.append(4)
```

💡 Show the solution

`adjusted = baseline` makes a second label on the same list, so `.append` is visible through both names. `baseline` is now [1, 2, 3, 4].

The fix is to ask for an independent list:

```
adjusted = baseline.copy()
```

This is the commonest source of “but I never changed that variable” in a beginner’s code, and it only happens with containers that can change in place — which is why tuples do not have the problem.

Exercise 3 For each, name the container you would use and why in one clause.

- Every distinct postcode in a delivery file, to check whether a new one has been seen before.
- A student’s id, name and score, passed around as one thing.
- Look up a country’s population by its name.
- Daily temperature readings, in the order taken.

💡 Show the solution

- Set** — you want uniqueness, and `in` on a set stays fast however many postcodes there are.
- Tuple** — the three belong together and should not be edited individually. (A dictionary is also defensible if you want the fields named.)
- Dictionary** — look-up by key is exactly what it is for.
- List** — order carries meaning, and you will want to append.

The question to ask each time: *do I look things up by position, by name, or just ask whether something is present?*

7.6 Check it in Python

Two checks worth building into the way you work.

Confirm you have an independent copy when you meant to:

```
baseline = [1, 2, 3]
adjusted = baseline.copy()
adjusted.append(4)

assert baseline == [1, 2, 3], "baseline was modified"
print("baseline intact:", baseline)
print("same object?     ", adjusted is baseline)
```

```
baseline intact: [1, 2, 3]
same object?     False
```

`is` asks whether two names point at the *same* object; `==` asks whether two objects have the same contents. Two separate lists holding `[1, 2, 3]` are `==` but not `is`.

Count what you dropped, every time:

```
raw = ["68.4", "", "72.1", "not recorded", "55.0"]
usable, dropped = [], []
for value in raw:
    try:
        usable.append(float(value))
    except ValueError:
        dropped.append(value)

print(f"kept {len(usable)} of {len(raw)}; dropped {dropped}")
assert len(usable) + len(dropped) == len(raw)
```

```
kept 3 of 5; dropped ['', 'not recorded']
```

The assertion says nothing went missing unaccounted for. It is one line and it turns a silent loss into a loud one.

7.7 Common wrong turns

Where this usually goes wrong

Assuming assignment copies a container `b = a` gives a second name for the same list. Use `.copy()` when you want an independent one, and remember that numbers and strings never have this problem because they cannot be changed in place.

Relying on the order a set prints in Sets are unordered. The printed order can differ between runs and versions. Call `sorted()` if you need an order.

Using a list for membership tests on large data `x in some_list` scans from the start. `x in some_set` does not. On a hundred thousand items the difference is dramatic.

Indexing rows by number `row[4]` breaks the moment a column is inserted, and it tells a reader nothing. `row["final_score"]` survives, and explains itself.

Being caught by the exclusive slice end `values[1:3]` gives two items. It is a convention worth learning rather than fighting: adjacent slices join up with no overlap.

Dropping rows without counting them Every filtered-out row is a decision. Count them and say how many, or you have no idea whether you removed three or three hundred.

7.8 Check yourself

Answers are at the foot of the page.

1. What does `[10, 20, 30, 40][1:3]` give?
 - a. `[10, 20]`
 - b. `[20, 30]`
 - c. `[20, 30, 40]`
 - d. `[10, 20, 30]`
2. `a = [1, 2]; b = a; b.append(3)`. What is `a`, and why?
3. Why can a tuple be a dictionary key when a list cannot?
4. You need to know whether each of a million ids has been seen before. List or set, and why?
5. `print(some_set)` gives a different order on a colleague's machine. Is that a bug?

💡 Show the answers

1. **(b) `[20, 30]`**. Start at index 1, stop before index 3.
2. `[1, 2, 3]`. `b` is another label on the same list, and `.append` changes the list itself rather than making a new one.
3. Because a dictionary key must not change after it is stored — the dictionary files it by its contents. A tuple cannot change, so it is safe; a list can, which would leave the entry unfindable. (Python enforces this: using a list as a key raises `TypeError: unhashable type.`)
4. **A set.** `in` on a set does not depend on how many items it holds; on a list it scans, so a million ids means up to a million comparisons each time.
5. **No.** Sets have no order to be wrong about. If order matters to you, that is a signal you should be sorting, or should not be using a set.

Where this shows up next

P4 iterates over these containers and makes decisions about what it finds. **P8** replaces the dictionary-of-lists in the worked example with a labelled table that does the grouping for you. The counted-and-reported skipping in that example becomes the exclusions audit in **P9**.

8 Making decisions, repeating work

Python · Unit 4

P4 · Python Reading: 4 pages With practice: 2 hours Assumes: P3

Why this unit is here

So far your programs have run straight down the page. Two constructions change that, and between them they cover almost everything a program does: **choosing** between alternatives, and **repeating** work over many values.

Everything from cleaning a column to fitting a model is built from these. The skill worth acquiring here is not the syntax, which is small, but the habit of tracing what a loop does *before* you run it.

8.1 Before you start

You should be able to:

- index and slice a list, and iterate the idea of “many values” — P3;
- compare values and know what type each is — P2.

Quick check. What does `len([10, 20, 30])` give, and what is the index of its last item?

💡 Show the answer

3, and the last index is 2. Length counts items; indices start at zero, so the last index is always `len(...) - 1`. Off-by-one errors live in that gap.

8.2 The idea

A **condition** is an expression that evaluates to `True` or `False`, and it decides which block of code runs:

```
score = 68.4

if score >= 70:
    print("distinction")
elif score >= 50:
    print("pass")
```



```
else:
    print("below the pass mark")
```

pass

Two things to notice, because both are Python-specific.

Indentation is syntax. The four spaces are not decoration — they are how Python knows which lines belong to which branch. Get them wrong and the meaning changes, or the file will not parse at all.

elif chains are checked in order, and stop at the first match. A score of 75 satisfies both `>= 70` and `>= 50`, but only the first branch runs. That is why the tests must be written from most to least demanding; reverse them and everything above 50 becomes a pass.

8.2.1 Comparison and logic

```
n = 57
print(n > 50, n == 57, n != 57, 50 <= n <= 100)
print(n > 50 and n < 60)
print(n < 50 or n > 100)
print(not n > 50)
```

```
True True False True
True
False
False
```

`==` compares, `=` assigns. Confusing them is a syntax error in a condition, which is fortunate — in some languages it silently assigns instead.

`50 <= n <= 100` chains, and reads exactly as it looks. That is unusual among programming languages and worth using.

8.3 The mechanics

8.3.1 for — repeat once per item

```
scores = [68.4, 55.1, 72.0]

for score in scores:
    print(score, "→", "pass" if score >= 50 else "fail")
```

68.4 → pass
55.1 → pass
72.0 → pass

The name `score` is created fresh on each pass, taking each value in turn. You do not manage an index and you cannot run off the end.

Three helpers you will use constantly:

```
groups = ["Computing", "Life sciences", "Engineering"]
counts = [69, 57, 36]

for position, name in enumerate(groups, start=1):
    print(position, name)

print()
for name, count in zip(groups, counts):
    print(f"{name:<16} {count}")
```

```
1 Computing
2 Life sciences
3 Engineering
```

```
Computing      69
Life sciences  57
Engineering    36
```

`enumerate` gives you the position alongside the item. `zip` walks two sequences together — and stops at the shorter one, silently, which is worth remembering.

8.3.2 while — repeat until a condition changes

```
remaining = 100.0
halvings = 0

while remaining > 1.0:
    remaining = remaining / 2
    halvings += 1

print(f"{halvings} halvings to get below 1.0, ending at {remaining:.4f}")
```

```
7 halvings to get below 1.0, ending at 0.7812
```

Use `for` when you know what you are iterating over, and `while` when you are waiting for a condition. The danger of `while` is that **something in the loop must change the condition** — if `remaining` never shrank, this would run forever. Interrupt a runaway loop with Control-C.

8.3.3 Tracing a loop before running it

This is the habit that matters. Take a loop and write down what each name holds after each pass:

pass	value	total before	total after
1	3	0	3
2	1	3	4
3	4	4	8

Figure 8.1: Tracing for `value` in `[3, 1, 4]` accumulating a running total. Writing the state after each pass turns a loop from something you hope works into something you have checked.

```
total = 0
for value in [3, 1, 4]:
    total = total + value
print(total)
```

8

Three lines of paper, and you knew the answer before the machine did. Do this whenever a loop surprises you.

8.3.4 Comprehensions

A very common loop shape — build a new list by transforming or filtering an old one — has a shorthand:

```
scores = [68.4, 42.0, 72.0, 55.1, 38.5]

passed = [s for s in scores if s >= 50]
rounded = [round(s) for s in scores]

print(passed)
print(rounded)
```

```
[68.4, 72.0, 55.1]
[68, 42, 72, 55, 38]
```

Read `[s for s in scores if s >= 50]` as “s, for each s in scores, where s is at least 50”. It does exactly what the equivalent loop does:

```

passed_long = []
for s in scores:
    if s >= 50:
        passed_long.append(s)

print(passed_long == passed)

```

True

Use a comprehension when it fits on one line and reads as a sentence. When it needs two conditions and a nested loop, write the loop — a comprehension that has to be decoded is worse than the four lines it replaced.

8.3.5 break and continue

```

for value in ["68.4", "", "72.1", "stop", "55.0"]:
    if value == "":
        continue          # skip this one, carry on
    if value == "stop":
        break             # leave the loop entirely
    print(float(value))

```

68.4

72.1

8.4 Worked example

Classifying the cohort, and counting what you could not classify.

```

import csv
from pathlib import Path

rows = list(csv.DictReader(Path("../data/cohort.csv").open()))

bands = {"distinction": 0, "merit": 0, "pass": 0, "below": 0}
unusable = []

for row in rows:
    raw = row["final_score"]
    if raw == "":
        unusable.append((row["student_id"], "no score recorded"))
        continue

    score = float(raw)
    if not 0 <= score <= 100:

```

```

        unusable.append((row["student_id"], f"impossible score {score}"))
        continue

    if score >= 70:
        bands["distinction"] += 1
    elif score >= 60:
        bands["merit"] += 1
    elif score >= 50:
        bands["pass"] += 1
    else:
        bands["below"] += 1

for name, count in bands.items():
    print(f"  {name:<12} {count:>3}")
print(f"\n  classified {sum(bands.values())} of {len(rows)}")
for student, reason in unusable:
    print(f"  excluded {student}: {reason}")

```

```

distinction    69
merit          103
pass           60
below           6

```

```

classified 238 of 240
excluded S0018: no score recorded
excluded S0189: impossible score 187.0

```

Three things in that loop are deliberate and worth copying.

The `continue` cases come **first**, before any arithmetic. A value you cannot use should leave the loop body before it can contaminate anything.

Every exclusion is **recorded with a reason**, not just counted. “Two rows dropped” invites the question “which, and why?”; this answers it in advance.

And the bands are checked from **highest to lowest**, because `elif` stops at the first match. Written the other way round, every score above 50 would land in `pass` and the top two bands would always be zero.

8.5 Try it yourself

Exercise 1 Trace this on paper before running it. What does it print?

```

total = 0
for i in range(1, 5):
    total += i
    print(i, total)

```

💡 Show the solution

```
1 1
2 3
3 6
4 10
```

`range(1, 5)` gives 1, 2, 3, 4 — it starts at 1 and stops **before** 5, the same exclusive-end convention as slicing. `total += i` is shorthand for `total = total + i`.

Exercise 2 This is meant to report every score band. Why does it always print **pass** for anything above 50?

```
if score >= 50:
    print("pass")
elif score >= 60:
    print("merit")
elif score >= 70:
    print("distinction")
```

💡 Show the solution

An `elif` chain stops at the **first** true condition. A score of 85 satisfies `>= 50`, so it prints **pass** and the later branches are never reached.

The fix is to order the tests from most to least demanding — `>= 70`, then `>= 60`, then `>= 50` — so the first one that matches is the right one.

A useful check: if any branch of an `elif` chain can never run, the ordering is wrong.

Exercise 3 Rewrite as a comprehension, then say whether you would keep it.

```
long_names = []
for name in groups:
    if len(name) > 10:
        long_names.append(name.upper())
```

💡 Show the solution

```
long_names = [name.upper() for name in groups if len(name) > 10]
```

Whether to keep it is a judgement, and the honest answer here is yes: it fits on one line and reads as a sentence — “name uppercased, for each name in groups, where the name is longer than ten characters”.

If it needed a second condition and a nested loop, the four-line version would be clearer. Brevity is not the goal; being read correctly on the first pass is.

8.6 Check it in Python

Two checks that catch the two commonest loop bugs.

Did every item get accounted for? A loop that filters should conserve the total:

```
values = [68.4, 42.0, 72.0, 55.1, 38.5]
kept = [v for v in values if v >= 50]
dropped = [v for v in values if v < 50]

assert len(kept) + len(dropped) == len(values)
print(f"kept {len(kept)}, dropped {len(dropped)}, of {len(values)}")
```

kept 3, dropped 2, of 5

If a value could satisfy both conditions or neither, this assertion fires — and that is exactly the `elif`-ordering bug from Exercise 2, caught automatically.

Does the while condition actually change? Add a guard while developing:

```
remaining, steps = 100.0, 0

while remaining > 1.0:
    remaining /= 2
    steps += 1
    assert steps < 1000, "loop is not converging - check the update"

print(f"finished in {steps} steps at {remaining:.4f}")
```

finished in 7 steps at 0.7812

The assertion costs nothing and turns an infinite loop into an immediate, readable failure. Remove it once the logic is settled, or leave it in — a bound you can justify is documentation.

8.7 Common wrong turns

Where this usually goes wrong

Ordering an `elif` chain from least to most demanding The first true branch wins, so `>= 50` placed first captures everything. If a branch can never run, the order is wrong.

Forgetting that `range(a, b)` excludes `b` `range(1, 5)` gives four numbers, not five. It is the same convention as slicing, and it makes adjacent ranges join up cleanly.

Writing a while whose condition never changes Something in the body must move the loop towards ending. If nothing does, it runs forever. Control-C interrupts it.

Modifying a list while looping over it Removing items shifts everything after them, so the loop skips values. Build a new list instead — which is what a comprehension does.

Assuming `zip` complains about unequal lengths It stops at the shorter sequence without a word. If the two should match, assert that they do.

Reaching for a comprehension because it is shorter A comprehension you have to decode is worse than the loop it replaced. Use one when it reads as a sentence.

8.8 Check yourself

Answers are at the foot of the page.

1. How many numbers does `range(2, 8)` produce?
a. 5 b. 6 c. 7 d. 8
2. What is the difference between `continue` and `break`?
3. `zip([1, 2, 3], ["a", "b"])` — how many pairs, and what happened to the 3?
4. Why must something inside a `while` body change the condition?
5. A classification loop reports zero in your top band every time, though you can see qualifying values in the data. What would you check first?

Show the answers

1. **(b) 6** — 2, 3, 4, 5, 6, 7. It starts at 2 and stops before 8.
2. `continue` abandons the current pass and moves to the next item; `break` leaves the loop entirely and carries on with the code after it.
3. **Two pairs.** `zip` stops at the shorter sequence, so the 3 is silently dropped. If that would be a bug, assert the lengths match first.
4. Because the condition is re-tested before every pass. If nothing in the body moves it towards `False`, the test never changes and the loop never ends.
5. The order of the `elif` chain. If a less demanding test sits above a more demanding one, the top band is unreachable — every qualifying value is caught by the earlier branch.

Where this shows up next

P5 wraps a loop in a name, so the same logic can be used in several places and tested once. **P7** replaces many of these loops with a single array expression — clearer *and* faster, once the shapes are right. **P9** does this unit's worked example in three lines, with the exclusions written to a file.

9 Functions and modules

Python · Unit 5

P5 · Python Reading: 4 pages With practice: 2 hours Assumes: P4

Why this unit is here

A function gives a piece of work a **name**, so you can use it in several places and fix it in one.

That is the obvious benefit. The one that matters more for data science is that a named piece of work can be **checked**. A calculation buried in the middle of a script can only be verified by reading it; the same calculation in a function can be handed known inputs and asked whether it returns the right answer. Every unit after this one leans on that.

9.1 Before you start

You should be able to:

- write a loop and a conditional, and indent them correctly — P4;
- work with lists and dictionaries — P3.

Quick check. What is `len("hello")`, and what kind of thing is `len`?

💡 Show the answer

5, and `len` is a **function** — you have been using them since P1. `print`, `float`, `sorted` and `range` are all functions. This unit is about writing your own.

9.2 The idea

```
def standardise(value, mean, sd):  
    """Return how many standard deviations `value` sits above the mean."""  
    return (value - mean) / sd  
  
print(standardise(72.0, 65.3, 8.4))
```

0.7976190476190479

Four parts:

- **def** starts a definition, followed by the name.
- The names in parentheses are **parameters** — placeholders for values supplied when the function is called.
- The indented text in triple quotes is a **docstring**: what the function does, for a human.
- **return** hands a value back. Without it, a function returns **None**.

The values you pass in at the call site are **arguments**. Parameters live in the definition; arguments arrive at the call. Keeping the two words apart makes error messages much easier to read.

9.2.1 What is visible where

Names created inside a function are private to it:

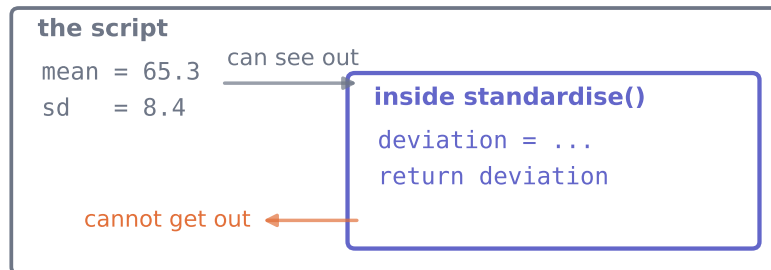


Figure 9.1: A function can see the names around it, but names it creates are its own. `deviation` does not exist once the function returns.

```
def standardise(value, mean, sd):
    deviation = value - mean
    return deviation / sd

print(standardise(72.0, 65.3, 8.4))
print(deviation)
```

0.7976190476190479

NameError: name 'deviation' is not defined

```
-----
NameError                                Traceback (most recent call last)
Cell In[3], line 7
      3     return deviation / sd
      6 print(standardise(72.0, 65.3, 8.4))
----> 7 print(deviation)
NameError: name 'deviation' is not defined
```

`deviation` was created inside and does not exist outside. That is a feature: a function cannot quietly leave things lying around, so you can use one without knowing how it works inside.

9.3 The mechanics

9.3.1 Defaults and keyword arguments

```
def summarise(values, decimals=2, label="sample"):
    mean = sum(values) / len(values)
    return f"{label}: n={len(values)}, mean={mean:.{decimals}f}"

scores = [68.4, 55.1, 72.0, 61.3]
print(summarise(scores))
print(summarise(scores, decimals=4))
print(summarise(scores, label="Life sciences"))
```

```
sample: n=4, mean=64.20
sample: n=4, mean=64.2000
Life sciences: n=4, mean=64.20
```

Parameters with defaults come after those without. At the call site, naming the argument — `decimals=4` rather than a bare `4` — makes the line readable and means you can supply them in any order.

! Never use a list or dictionary as a default value. It is created once, when the function is defined, and then shared by every call:

```
def add_reading(value, readings=[]):      # do not do this
    readings.append(value)
    return readings

print(add_reading(1))
print(add_reading(2))      # the list from the first call is still
    ↪ there
```

```
[1]
[1, 2]
```

Use None as the default and create the list inside:

```
def add_reading(value, readings=None):
    if readings is None:
        readings = []
    readings.append(value)
    return readings

print(add_reading(1))
print(add_reading(2))
```

```
[1]
[2]
```

9.3.2 Doing one thing

A function should be describable in one sentence without using “and”. Compare:

```
def mean(values):
    """Return the arithmetic mean of a non-empty sequence."""
    return sum(values) / len(values)

def usable(values, low=0, high=100):
    """Return only the values inside [low, high]."""
    return [v for v in values if low <= v <= high]

raw = [68.4, 187.0, 55.1, 72.0]
print(mean(usable(raw)))
```

65.16666666666667

Two small functions, each checkable on its own, beat one that filters *and* averages and can only be tested as a lump.

9.3.3 assert — a check you leave in the code

```
def mean(values):
    assert len(values) > 0, "mean of an empty sequence is undefined"
    return sum(values) / len(values)

print(mean([1, 2, 3]))
```

2.0

```
mean([])
```

AssertionError: mean of an empty sequence is undefined

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 mean([])
Cell In[8], line 2, in mean(values)
      1 def mean(values):
----> 2     assert len(values) > 0, "mean of an empty sequence is undefined"
      3     return sum(values) / len(values)
AssertionError: mean of an empty sequence is undefined
```

Without the assertion this raises `ZeroDivisionError` from inside `sum`, which tells you what broke but not what you did wrong. With it, the message names the actual mistake.

9.3.4 Modules

A module is a file of Python you can use from another file. Import at the top:

```
import math
import statistics as stats

print(math.sqrt(2))
print(stats.median([68.4, 55.1, 72.0]))
```

1.4142135623730951
68.4

`import math` keeps the origin visible — `math.sqrt` says where `sqrt` came from. `import statistics as stats` shortens a long name; `import numpy as np` is the same move, and the abbreviations for `numpy`, `pandas` and `matplotlib` are universal conventions worth following exactly.

i Avoid `from module import *`. It pulls every name in, so you cannot tell what came from where, and a later import can silently replace an earlier name. The cost of typing `math.` is small; the cost of a name collision you cannot see is not.

A short standard-library list worth knowing exists: `math`, `statistics`, `pathlib` for file paths, `csv`, `json`, `random`, `datetime`.

9.4 Worked example

A checked function, used on the cohort.

```
def summarise_group(scores, low=0, high=100):
    """Summarise a group's scores, reporting what was excluded.

    Returns a dictionary with the count kept, the count excluded, the mean
    ↪ and
    the sample standard deviation. Raises if nothing usable remains.
    """
    usable = [s for s in scores if low <= s <= high]
    excluded = len(scores) - len(usable)

    if len(usable) < 2:
        raise ValueError(f"need at least 2 usable scores, got
        ↪ {len(usable)}")

    mean = sum(usable) / len(usable)
    variance = sum((s - mean)**2 for s in usable) / (len(usable) - 1)

    return {"n": len(usable), "excluded": excluded,
            "mean": mean, "sd": variance**0.5}
```

Check it against values you can verify by hand *before* pointing it at real data:

```
result = summarise_group([2, 4, 4, 4, 5, 5, 7, 9])
print(result)

assert result["n"] == 8
assert abs(result["mean"] - 5.0) < 1e-12
assert abs(result["sd"] - 2.13809) < 1e-4          # sample sd, n-1 divisor
assert summarise_group([1, 2, 3, 187])["excluded"] == 1
print("all checks passed")
```

```
{'n': 8, 'excluded': 0, 'mean': 5.0, 'sd': 2.138089935299395}
all checks passed
```

The mean of those eight numbers is 5 by inspection, and the divisor is $n - 1$ as [S8](#) requires. Now use it:

```

import csv
from pathlib import Path

rows = list(csv.DictReader(Path("../data/cohort.csv").open()))

by_group = {}
for row in rows:
    if row["background"] and row["final_score"]:
        by_group.setdefault(row["background"],
↪ []).append(float(row["final_score"]))

for name in sorted(by_group):
    r = summarise_group(by_group[name])
    note = f" ({r['excluded']} excluded)" if r["excluded"] else ""
    print(f" {name:<24} n={r['n']:>3} mean={r['mean']:5.2f}
↪ sd={r['sd']:4.2f}{note}")

```

Computing	n= 69	mean=65.85	sd=6.60	
Engineering	n= 36	mean=67.15	sd=5.33	
Life sciences	n= 57	mean=60.51	sd=7.29	(1 excluded)
Mathematics or Physics	n= 36	mean=75.28	sd=7.63	
Social sciences	n= 39	mean=60.21	sd=6.58	

The one excluded value is the impossible score of 187 planted in the file. The function found it, reported it, and did not average it in — because the exclusion rule lives in one place and is tested.

9.5 Try it yourself

Exercise 1 Write `larger(a, b)` returning the larger of two numbers, then write three assertions that would catch a wrong implementation — including the equal case.

 Show the solution

```

def larger(a, b):
    """Return the larger of two numbers; either if they are equal."""
    return a if a >= b else b

assert larger(3, 7) == 7      # second larger
assert larger(7, 3) == 7      # first larger
assert larger(4, 4) == 4      # equal

```

The equal case is the one people omit, and it is where an implementation using `>` versus `>=` differs. Testing only `larger(3, 7)` would pass a function that always returns its second argument.

Exercise 2 Why does this print `[1, 2]` on the second call rather than `[2]`?

```
def collect(value, into=[]):
    into.append(value)
    return into

print(collect(1))
print(collect(2))
```

💡 Show the solution

The default `[]` is created **once**, when the `def` line runs — not on each call. Every call that does not supply `into` shares that same list, so values accumulate across calls. The fix:

```
def collect(value, into=None):
    if into is None:
        into = []
    into.append(value)
    return into
```

The rule: a default value must be immutable. Numbers, strings, `True`, `None` and tuples are safe; lists, dictionaries and sets are not.

Exercise 3 This function does three things. Split it, and say what you gain.

```
def process(rows):
    scores = [float(r["final_score"]) for r in rows if r["final_score"]]
    valid = [s for s in scores if 0 <= s <= 100]
    return sum(valid) / len(valid)
```

💡 Show the solution

```
def extract_scores(rows):
    """Pull final_score out of rows that have one, as floats."""
    return [float(r["final_score"]) for r in rows if r["final_score"]]

def usable(scores, low=0, high=100):
    """Keep only scores inside the plausible range."""
    return [s for s in scores if low <= s <= high]

def mean(values):
    assert values, "mean of an empty sequence is undefined"
    return sum(values) / len(values)
```

What you gain: each piece can be tested with two lines and no CSV file; the plausible range becomes a visible, changeable parameter rather than a hard-coded 0 and 100 buried three lines deep; and when a mean comes out wrong you can find out *which* stage was responsible.

What you also gain, less obviously: `usable` now has a name, so the fact that filtering

happened is visible at the call site instead of hidden.

9.6 Check it in Python

Write the checks **before** the function is used in anger, on inputs whose answer you already know.

```
def sample_sd(values):
    """Sample standard deviation, with the n-1 divisor."""
    if len(values) < 2:
        raise ValueError("need at least two values")
    mean = sum(values) / len(values)
    return (sum((v - mean)**2 for v in values) / (len(values) - 1))**0.5

import numpy as np
for case in ([2, 4, 4, 4, 5, 5, 7, 9], [1.5, 2.5], [10, 10, 10]):
    mine, theirs = sample_sd(case), np.std(case, ddof=1)
    print(f"{str(case):<28} {mine:8.5f} vs numpy {theirs:8.5f} "
          f"agree: {np.isclose(mine, theirs)}")
```

[2, 4, 4, 4, 5, 5, 7, 9]	2.13809	vs numpy	2.13809	agree: True
[1.5, 2.5]	0.70711	vs numpy	0.70711	agree: True
[10, 10, 10]	0.00000	vs numpy	0.00000	agree: True

Comparing against an independent implementation is the strongest cheap check there is. Note the three cases: an ordinary one, the smallest legal input, and one with no variation at all — ordinary, edge, and degenerate.

And check that it refuses what it should:

```
try:
    sample_sd([5])
except ValueError as error:
    print("correctly refused:", error)
```

correctly refused: need at least two values

A function that fails loudly on bad input is more useful than one that returns a number you cannot trust.

9.7 Common wrong turns

Where this usually goes wrong

A mutable default argument `def f(x, acc=[])` creates that list once and shares it across all calls. Use `None` and create it inside.

Printing instead of returning A function that prints gives you nothing to test or reuse. Return the value and let the caller decide whether to print it.

One function doing several jobs If describing it needs the word “and”, it is two functions. Small pieces are testable pieces.

Testing only the easy case `larger(3, 7)` passes for a function that always returns its second argument. Test ordinary, edge and degenerate inputs.

`from module import *` Hides where names came from and lets a later import silently replace an earlier one. Import the module, or import specific names.

Assuming a function can change the caller’s variables Names created inside are local. A function communicates back through its return value — which is why a function with no `return` gives you `None`.

9.8 Check yourself

Answers are at the foot of the page.

1. What does a function return if it has no `return` statement?
 - a. 0 b. `None` c. the last value computed d. it is an error
2. What is the difference between a parameter and an argument?
3. Why is `def f(x, acc=[])` dangerous?
4. Give one reason to prefer returning a value over printing it.
5. You test `larger(a, b)` only with `larger(2, 9)` and it passes. Name an incorrect implementation that would also pass.

 Show the answers

1. **(b) `None`.** Which is why a function that prints but does not return gives you `None` if you try to use its result.
2. A **parameter** is the name in the definition; an **argument** is the value supplied at the call. `def f(x)` declares a parameter; `f(3)` passes an argument.
3. The default list is created once at definition time and shared by every call that does not supply one, so values accumulate between unrelated calls.
4. Any of: the value can be tested; it can be reused in a larger calculation; the caller chooses the formatting; and the function stays usable in a script that should print nothing at all.
5. `def larger(a, b): return b` — it returns 9, which is correct for that one case. So would `return max(a, b) if a < b else b`. Testing the reversed order and the equal case rules both out.

Where this shows up next

P6 starts using functions from NumPy in earnest, and the import conventions from this unit become universal. **P11** moves functions into their own .py files and runs the checks with **pytest** instead of bare assertions. Every **Check it in Python** block in the Mathematics and Statistics tracks is this unit's habit applied to a mathematical claim.

10 NumPy: arrays and indexing

Python · Unit 6

P6 · Python Reading: 5 pages With practice: 2 hours Assumes: P5

Why this unit is here

A Python list can hold anything, in any mix, and grow at either end. That flexibility costs speed and, more importantly here, costs *meaning* — a list gives you no way to say “these 240 numbers are one variable”.

A NumPy **array** trades the flexibility away. Every element has the same type, the shape is fixed, and the whole thing sits in one block of memory. In exchange you get arithmetic on entire columns at once, and a vocabulary — shape, axis, dtype — that the rest of data science is written in.

10.1 Before you start

You should be able to:

- index and slice a list — [P3](#);
- import a module under an alias — [P5](#);
- say what type a value has — [P2](#).

Quick check. What does `[10, 20, 30][1:3]` give?

💡 Show the answer

`[20, 30]`. Start at index 1, stop *before* index 3. Array slicing uses exactly the same convention, so this transfers directly.

10.2 The idea

```
import numpy as np

scores = np.array([68.4, 55.1, 72.0, 61.3])
print(scores)
print("shape", scores.shape, " ndim", scores.ndim, " dtype", scores.dtype)
```

```
[68.4 55.1 72.  61.3]
shape (4,)  ndim 1  dtype float64
```

Three attributes describe every array, and you should be able to state all three before doing arithmetic:

- **shape** — how many elements along each dimension, as a tuple.
- **ndim** — how many dimensions. `len(shape)`.
- **dtype** — the single type every element has.

`shape` is `(4,)` with a trailing comma because it is a one-element tuple: four values along one dimension. `(4,)` and `(4, 1)` are different things, and [P7](#) is where that difference bites.

The immediate payoff is that arithmetic applies to everything at once:

```
print(scores + 5)
print(scores > 60)
```

```
[73.4 60.1 77.  66.3]
[ True False  True  True]
```

A list will not do either. `[1, 2] + 5` raises, and `[1, 2] > 1` is meaningless.

10.3 The mechanics

10.3.1 Making arrays

```
print(np.zeros(4))
print(np.ones((2, 3)))
print(np.arange(0, 10, 2))
print(np.linspace(0, 1, 5))
```

```
[0.  0.  0.  0.]
[[1.  1.  1.]
 [1.  1.  1.]]
[0  2  4  6  8]
[0.   0.25 0.5  0.75 1.   ]
```

`arange(start, stop, step)` follows the usual exclusive-stop convention, so it never reaches `stop`. `linspace(start, stop, n)` gives exactly `n` points and **does** include both ends — a deliberate difference, and the reason `linspace` is the right choice for plotting a curve.

10.3.2 dtype is part of the calculation

```
whole = np.array([1, 2, 3])
print(whole.dtype)

whole[0] = 2.7          # assigning a float into an integer array
print(whole)
```

```
int64
[2 2 3]
```

The 2.7 was truncated to 2 without a word of warning. `dtype` is not a display detail — it changes results. If a column will hold fractions, make it float from the start:

```
decimal = np.array([1, 2, 3], dtype=float)
decimal[0] = 2.7
print(decimal, decimal.dtype)
```

```
[2.7 2.  3. ] float64
```

Mixing types in the input forces a common dtype, which is rarely what you want:

```
print(np.array([1, 2.5, 3]).dtype)
print(np.array([1, 2, "3"]).dtype)
```

```
float64
<U21
```

That second one turned your numbers into text. Inspect the dtype whenever an array comes from somewhere you did not control.

10.3.3 Two dimensions

```
table = np.arange(12).reshape(3, 4)
print(table)
print("shape", table.shape)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
shape (3, 4)
```

Index with `[row, column]`:

```
print(table[0, 0])      # first row, first column
print(table[2, -1])    # last row, last column
print(table[1])         # a whole row
print(table[:, 1])     # a whole column
```

```
0
11
[4 5 6 7]
[1 5 9]
```

`table[:, 1]` reads as “every row, column 1”. The colon means *all of this dimension*, and it is how you take a column out of a table.

10.3.4 Boolean indexing

The single most useful thing in this unit:

```
scores = np.array([68.4, 42.0, 72.0, 55.1, 187.0])

mask = (scores >= 50) & (scores <= 100)
print(mask)
print(scores[mask])
```

```
[ True False  True  True False]
[68.4 72.  55.1]
```

The comparison produces an array of `True/False`, and indexing with it keeps only the `True` positions. Two rules:

- Use `&` and `|`, not `and` and `or`. The word forms ask for one answer about the whole array; the symbols work element by element.
- **Parenthesise each comparison.** `scores >= 50 & scores <= 100` binds in the wrong order and fails confusingly.

Counting is then free, because `True` sums as 1:

```
print(f"{mask.sum()} of {mask.size} usable")
```

```
3 of 5 usable
```

10.3.5 Views and copies

This is the concept that catches people, and it is worth slowing down for.

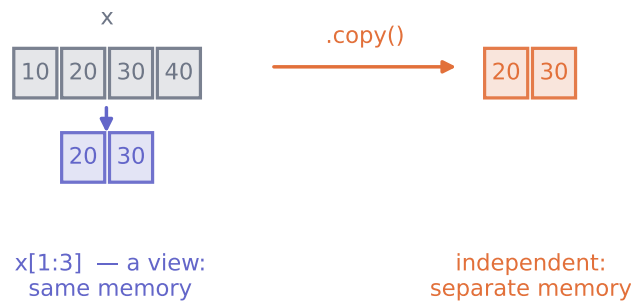


Figure 10.1: A basic slice is a *view* — a second way of looking at the same memory, so writing through it changes the original. Advanced indexing, and `.copy()`, produce independent data.

```
x = np.array([10, 20, 30, 40])
window = x[1:3]          # a view
window[0] = 99

print("x      ", x)
print("window", window)
```

```
x      [10 99 30 40]
window [99 30]
```

Changing the view changed **x**. Nothing was copied — **window** is a second way of looking at the same block of memory. That is deliberate: slicing a large array would be expensive if it duplicated the data.

When you want independence, say so:

```
x = np.array([10, 20, 30, 40])
window = x[1:3].copy()
window[0] = 99

print("x      ", x)
print("window", window)
```

```
x      [10 20 30 40]
window [99 30]
```

You can always ask:

```
x = np.array([10, 20, 30, 40])
print("slice shares memory: ", np.shares_memory(x, x[1:3]))
print("boolean shares memory:", np.shares_memory(x, x[x > 15]))
```



```
slice shares memory: True
boolean shares memory: False
```

Basic slicing gives a view; boolean and fancy indexing give a copy. When in doubt, `np.shares_memory` answers it in one line.

10.4 Worked example

Loading the cohort's numeric columns and finding the usable rows.

```
import numpy as np

raw = np.genfromtxt("../data/cohort.csv", delimiter=",", skip_header=1,
                    usecols=(2, 3, 4))          # diagnostic, hours, final
print("shape", raw.shape, " dtype", raw.dtype)
print(raw[:3])
```

```
shape (240, 3) dtype float64
[[89.1 12.9 83.6]
 [42.4 22.8 56.6]
 [68.4  1.7 58.5]]
```

`genfromtxt` turned anything unparseable into `nan` — “not a number”, the float value meaning *this is missing*. It is there in the data:

```
missing = np.isnan(raw).any(axis=1)
print(f"rows with a missing value: {missing.sum()}")
print("row indices:", np.flatnonzero(missing))
```

```
rows with a missing value: 1
row indices: [17]
```

Now build the usable mask, one condition at a time so each is visible:

```
complete = ~np.isnan(raw).any(axis=1)
in_range = (raw[:, 2] >= 0) & (raw[:, 2] <= 100)
usable = complete & in_range

print(f"complete rows : {complete.sum()}")
print(f"score in range: {in_range.sum()}")
print(f"both           : {usable.sum()} of {raw.shape[0]}")

data = raw[usable]
print("kept shape", data.shape)
```

```
complete rows : 239
score in range: 238
both           : 238 of 240
kept shape (238, 3)
```

~ negates a boolean array — read it as “not”. And note `in_range` is computed on the raw array, where a `nan` compares `False` against everything, so it already excludes the missing row; combining both conditions explicitly is clearer than relying on that.

Finally, a sanity check on the result rather than a hope:

```
final = data[:, 2]
print(f"n      {final.size}")
print(f"min    {final.min():.1f}")
print(f"max    {final.max():.1f}")
print(f"mean   {final.mean():.2f}")
assert not np.isnan(data).any(), "a missing value survived the filter"
assert (final >= 0).all() and (final <= 100).all(), "an impossible score
↳ survived"
print("filters did what they claimed")
```

```
n      238
min    44.5
max    91.6
mean   65.27
filters did what they claimed
```

Two assertions, two lines, and the class of error where a filter silently fails to filter is now impossible to miss.

10.5 Try it yourself

Exercise 1 Predict `shape`, `ndim` and `dtype` for each, then check.

```
a = np.array([1, 2, 3])
b = np.array([[1.0, 2.0], [3.0, 4.0]])
c = np.zeros((2, 3, 4))
```

Show the solution

- `a` — shape (3,), `ndim` 1, `dtype` `int64` (may be `int32` on Windows).
- `b` — shape (2, 2), `ndim` 2, `dtype` `float64`, because the literals are written with decimal points.
- `c` — shape (2, 3, 4), `ndim` 3, `dtype` `float64`. `zeros` gives floats unless told otherwise.

The `dtype` of `a` is the one worth noting: writing `1` rather than `1.0` gives you an integer array, and assigning `2.7` into it later would silently truncate.

Exercise 2 Why does the first change `x` and the second not?

```
x = np.array([1, 2, 3, 4])
x[:2][0] = 99          # first

y = np.array([1, 2, 3, 4])
y[y < 3][0] = 99      # second
```

💡 Show the solution

`x[:2]` is basic slicing, so it returns a **view** into `x`'s memory. Writing through it writes into `x`.

`y[y < 3]` is boolean indexing, which must gather scattered elements, so it returns a **copy**. The 99 is written into that temporary copy, which is then discarded — `y` is untouched and no error is raised.

The second is the more dangerous of the two, because it silently does nothing. To assign through a boolean mask, put the mask on the left of the assignment:

```
y[y < 3] = 99
```

Exercise 3 Write one expression giving the diagnostic scores of rows whose final score is at least 70, from the `raw` array above. Then say how many there are.

💡 Show the solution

```
high = raw[(raw[:, 2] >= 70) & ~np.isnan(raw[:, 2]), 0]
print(high.size)
```

Reading it outward: `raw[:, 2]` is the final-score column; the comparison makes a boolean mask; `& ~np.isnan(...)` drops the missing row explicitly; and the trailing `, 0` selects the diagnostic column from the rows that survived.

Strictly the `~np.isnan` is redundant, because `nan >= 70` is already **False** — but relying on that is the kind of cleverness that fails when someone changes the comparison to `<`. Say what you mean.

10.6 Check it in Python

Ask whether you are holding a **view** before writing into anything:

```
x = np.array([1.0, 2.0, 3.0, 4.0])
piece = x[1:3]
print("shares memory with x:", np.shares_memory(x, piece))
```

shares memory with x: True

Assert the shape you expect, so a mistake stops the program rather than propagating:

```
table = np.arange(12).reshape(3, 4)
column = table[:, 1]

assert column.shape == (3,), f"expected (3,), got {column.shape}"
print("column", column, "shape", column.shape)
```

```
column [1 5 9] shape (3,)
```

Check that a filter filtered. The failure mode is not an error; it is a filter that quietly kept something:

```
values = np.array([68.4, np.nan, 187.0, 55.1])
kept = values[~np.isnan(values) & (values <= 100)]

print("kept:", kept)
assert not np.isnan(kept).any()
assert (kept <= 100).all()
print(f"{kept.size} of {values.size} kept, and both conditions verified")
```

```
kept: [68.4 55.1]
2 of 4 kept, and both conditions verified
```

10.7 Common wrong turns

Where this usually goes wrong

Writing through a boolean index and expecting it to stick `y[y < 3][0] = 99` modifies a temporary copy and is silently discarded. Put the mask on the left: `y[y < 3] = 99`.

Forgetting a slice is a view `piece = x[1:3]` then `piece[0] = 99` changes `x`. Use `.copy()` when you want independence, and `np.shares_memory` when you are unsure.

Using and / or on arrays They ask for a single truth value about the whole array and raise. Use `&` and `|`, with each comparison in its own parentheses.

Ignoring dtype An integer array truncates anything fractional assigned into it, without warning. Create it as float if it will hold fractions.

Assuming a mixed list gives you numbers `np.array([1, 2, "3"])` produces an array of *strings*. Check the dtype when the source is not under your control.

Comparing against nan with == `nan == nan` is False, and so is every other comparison. Use `np.isnan`.

10.8 Check yourself

Answers are at the foot of the page.

1. What is the shape of `np.zeros((3, 4))[:, 2]`?

- a. (4,) b. (3,) c. (3, 1) d. (1, 3)
2. `x = np.array([1,2,3]); y = x[:2]; y[0] = 9`. What is `x`, and why?
3. Why must each comparison be parenthesised in `(a > 1) & (a < 5)`?
4. `np.array([1, 2, 3])[0] = 2.9` — what does the array hold afterwards?
5. A colleague filters missing values with `values[values != np.nan]` and gets everything back. What went wrong?

💡 Show the answers

1. **(b) (3,)**. Taking one column from a (3, 4) array leaves one value per row, and the column dimension disappears.
2. `[9, 2, 3]`. `x[:2]` is a view into the same memory, so writing through `y` writes into `x`.
3. Because `&` binds more tightly than `>` and `<`. Without parentheses, `a > 1 & a < 5` is read as `a > (1 & a) < 5`, which is not what you meant and usually raises.
4. `[2, 2, 3]`. The array is integer dtype, so 2.9 is truncated to 2 — no error, no warning.
5. `nan` is not equal to anything, including itself, so `values != np.nan` is `True` everywhere and nothing is filtered. Use `~np.isnan(values)`.

Where this shows up next

P7 does arithmetic *across* arrays — broadcasting, reductions by axis, and the shape traps that come with them. **P8** puts names on the columns so you stop writing `raw[:, 2]` and start writing `df["final_score"]`. **M8** treats a row of this table as a vector, and gives the arithmetic a geometric meaning.

11 NumPy: computing across arrays

Python · Unit 7

P7 · Python Reading: 5 pages With practice: 2 hours Assumes: P6

Why this unit is here

Data science is arithmetic done to whole tables at once. Three ideas make that possible: operations that apply to every element, *broadcasting* that lets differently-shaped arrays combine, and *reductions* that collapse an array along one direction.

They are also the three places where beginners most often get a wrong answer rather than an error message. NumPy will happily subtract the wrong thing from the wrong thing and hand you a perfectly shaped result. This unit is as much about noticing that as about the syntax.

11.1 Before you start

You should be able to:

- create an array and inspect its **shape**, **ndim** and **dtype** — P6;
- index and slice, and say whether you are holding a view or a copy — P6;
- write a **for** loop over a list — P4.

Quick check. What are the **shape** and **ndim** of `np.zeros((3, 4))`?

💡 Show the answer

shape is (3, 4) and **ndim** is 2 — three rows, four columns, two dimensions. If that was not immediate, reread P6 first; every idea below is expressed in terms of shape.

11.2 The idea

In plain Python, doing something to every number means writing a loop:

```
scores = [58.0, 71.5, 64.0, 80.5]

adjusted = []
for score in scores:
    adjusted.append(score * 1.1)
print(adjusted)
```

```
[63.800000000000004, 78.65, 70.4, 88.55000000000001]
```

In NumPy you write the operation once and it applies to everything:

```
import numpy as np

scores = np.array([58.0, 71.5, 64.0, 80.5])
print(scores * 1.1)
```

```
[63.8  78.65 70.4  88.55]
```

This is not just shorter. The loop still happens — it happens inside compiled code rather than in the Python interpreter, which for large arrays is dramatically faster. But speed is the lesser reason. The real gain is that `scores * 1.1` says *what* you want, while the loop says *how* to get it, and the first is much easier to read and much harder to get subtly wrong.

The same applies to comparisons, which produce arrays of `True/False`:

```
print(scores > 65)
print(scores[scores > 65])
```

```
[False  True False  True]
[71.5 80.5]
```

11.3 The mechanics

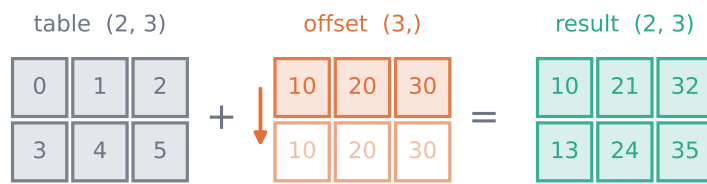
11.3.1 Broadcasting

Two arrays of the same shape combine element by element. Arrays of *different* shapes may still combine, if their shapes are compatible. NumPy compares them **from the right**, and two dimensions are compatible when they are equal or one of them is 1.

```
table = np.arange(6).reshape(2, 3)      # shape (2, 3)
offset = np.array([10, 20, 30])         # shape (3,)

print(table)
print(table + offset)
```

```
[[0 1 2]
 [3 4 5]]
[[10 21 32]
 [13 24 35]]
```



the second row is stretched, not stored

Figure 11.1: Broadcasting a (3,) array against a (2, 3) one. NumPy compares the shapes from the right: 3 matches 3, and the absent first dimension is stretched to 2. The hollow row is not stored — it is the same three numbers, reused.

The (3,) array was stretched The (3,) array was stretched across both rows. Nothing was copied in memory — NumPy simply reuses the same values — but the *result* is as if it had been.

This is concise and it is also where silent errors live. A one-dimensional array of length 3 has shape (3,). The same numbers as a column have shape (3, 1). They do not behave the same way:

```
v = np.array([10.0, 20.0, 30.0])

print("v.shape", v.shape)
print("v[:, None].shape", v[:, None].shape)
print("(v - v).shape", (v - v).shape)
print("(v[:, None] - v).shape", (v[:, None] - v).shape, " <-- 3x3, not 3")
```

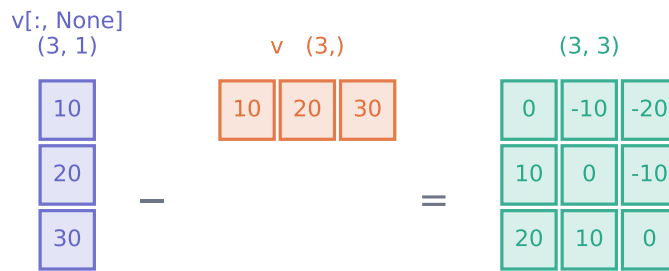
```
v.shape (3,)
v[:, None].shape (3, 1)
(v - v).shape (3,)
(v[:, None] - v).shape (3, 3) <-- 3x3, not 3
```

The last line is the trap. The last line is the trap. You asked for differences between 3 numbers and got a 3×3 table of *all pairwise* differences, with no error raised. If a calculation downstream then averages it, you get a number — the wrong number. **Print the shape whenever you are unsure.**

11.3.2 Reductions and axis

A reduction collapses an array. With no **axis** it collapses everything to a single value:

```
table = np.arange(6).reshape(2, 3)
print(table)
print("sum of everything:", table.sum())
```

three numbers in, nine out — every pairwise difference, and no error

Figure 11.2: Why (3,) and (3, 1) are not interchangeable. Comparing from the right, a column of three against a row of three makes *both* dimensions stretch, and three numbers become nine. Nothing is raised.

```
[[0 1 2]
 [3 4 5]]
sum of everything: 15
```

With an `axis` it collapses only that direction. This is the single most misremembered thing in NumPy, so it is worth stating carefully:

`axis=0` means **collapse the row index**. You end up with one result per column.

```
print("table.sum(axis=0) ->", table.sum(axis=0), " shape",
      ↪ table.sum(axis=0).shape)
print("table.sum(axis=1) ->", table.sum(axis=1), " shape",
      ↪ table.sum(axis=1).shape)
```

```
table.sum(axis=0) -> [3 5 7]  shape (3,)
table.sum(axis=1) -> [ 3 12]  shape (2,)
```

The reliable way to remember it The reliable way to remember it is by shape rather than by words. The table is (2, 3). Reducing over `axis=0` removes the first dimension and leaves (3,); reducing over `axis=1` removes the second and leaves (2,). *The axis you name is the one that disappears.*

11.3.3 ddof, and a default that will catch you

`np.std` and `np.var` divide by n by default. The sample standard deviation that statistics uses divides by $n - 1$, which you request with `ddof=1`:

```
values = np.array([2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0])
print("population (ddof=0):", values.std())
print("sample      (ddof=1):", values.std(ddof=1))
```

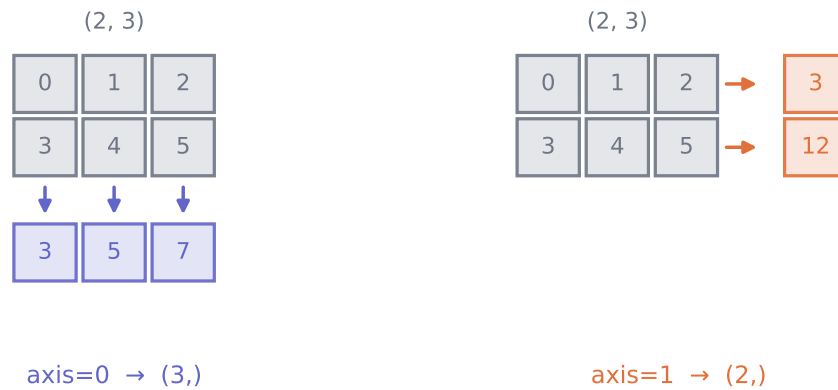


Figure 11.3: The axis you name is the one that disappears. Reducing over `axis=0` collapses the row index and leaves one number per column; over `axis=1` it collapses the column index and leaves one per row.

```
population (ddof=0): 2.0
sample      (ddof=1): 2.138089935299395
```

Neither is wrong; they answer different questions. But NumPy’s default is not the one statistics usually wants, and the gap is largest for exactly the small samples where it matters most. Decide deliberately and say which you used.

11.3.4 Random numbers

Create one generator, near the top of the script, and draw from it repeatedly:

```
rng = np.random.default_rng(2026)
print(rng.normal(loc=0, scale=1, size=4))
print(rng.integers(low=1, high=7, size=4))
```

```
[-0.79312248  0.24057128 -1.89632635  1.39577171]
[4 3 5 5]
```

The seed makes the sequence repeatable, which is what lets these notes print the same numbers on your machine as on mine. Do **not** call `np.random.default_rng(2026)` again inside a loop: each call restarts the sequence from the same place, so every iteration draws the identical “random” value. (Calling `default_rng()` with no seed inside a loop does give different values each time — but then nothing is reproducible, which is its own problem.) Create one generator outside the loop and draw from it repeatedly.

11.3.5 `*` is not `@`

For arrays, `*` multiplies element by element and `@` performs matrix multiplication. They are different questions with different answers:

```
A = np.array([[1.0, 2.0], [3.0, 4.0]])
print("A * A =\n", A * A)
print("A @ A =\n", A @ A)
```

```
A * A =
[[ 1.  4.]
 [ 9. 16.]]
A @ A =
[[ 7. 10.]
 [15. 22.]]
```

If M9 is still ahead of you, take this on trust for now: `*` is arithmetic on matching positions, `@` is the row-times-column operation from linear algebra.

11.4 Worked example

Standardise every numeric column of the cohort table.

Standardising means subtracting each column's mean and dividing by its standard deviation, so that every column ends up centred at 0 with a spread of 1. It uses reductions and broadcasting together, and it is the single most common preprocessing step in data science.

```
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
columns = ["diagnostic_score", "hours_practice", "final_score"]
raw = cohort[columns].to_numpy()

# Keep only rows that are complete and in range. The cohort file contains a
# deliberate missing value and one impossible score; P9 deals with them
# properly, but arithmetic on a NaN silently poisons everything it touches.
valid = np.isfinite(raw).all(axis=1) & (raw[:, 2] >= 0) & (raw[:, 2] <= 100)
data = raw[valid]

print("kept", valid.sum(), "of", len(raw), "rows; shape", data.shape)
```

kept 238 of 240 rows; shape (238, 3)

Now the two lines that matter:

```
means = data.mean(axis=0)          # one per column -> shape (3,)
sds = data.std(axis=0, ddof=1)     # one per column -> shape (3,)

standardised = (data - means) / sds

print("column means:", np.round(means, 2))
print("column sds  :", np.round(sds, 2))
print("shapes:", data.shape, "-", means.shape, "->", standardised.shape)
```

```
column means: [62.43 16.42 65.27]
column sds   : [17.49  8.14  8.38]
shapes: (238, 3) - (3,) -> (238, 3)
```

Read what happened. `data` is (238, 3) and `means` is (3,). Comparing from the right, 3 matches 3, and the missing first dimension is stretched to 238. One mean was subtracted from each column, exactly as intended.

Now check it, rather than assuming:

```
print("largest remaining mean:", np.abs(standardised.mean(axis=0)).max())
print("standard deviations   :", standardised.std(axis=0, ddof=1))
```

```
largest remaining mean: 2.1593371348696847e-15
standard deviations   : [1.  1.  1.]
```

The means are not exactly zero but around 10^{-15} , which is floating-point noise rather than a mistake — see [P2](#).

Two mistakes were available here, and only one of them would have announced itself. Using `axis=1` would have computed one mean per *student*, shape (238,), and subtracting that from a (238, 3) array raises a `ValueError` — 238 against 3 cannot broadcast. That one fails loudly, which is the good case. Forgetting `ddof=1` is the quiet one. It divides by a slightly *smaller* number, so the standardised columns come out with standard deviations of 1.002 rather than 1 — the factor is $\sqrt{n/(n-1)}$. Nothing raises, the shapes are all correct, and the result looks entirely reasonable. Two lines of checking catch it.

11.5 Try it yourself

Exercise 1 `a` has shape (4, 3) and `b` has shape (3,). What is the shape of `a * b`? What about `a * b[:, None]` — and why?

💡 Show the solution

`a * b` has shape (4, 3). Comparing from the right: 3 against 3 matches, and `b`'s missing first dimension stretches to 4. Each row of `a` is multiplied by `b`.

`a * b[:, None]` **raises an error**. `b[:, None]` has shape (3, 1). Comparing from the right: 1 against 3 is fine (the 1 stretches), but then 3 against 4 is not — neither is 1 and they are unequal.

This one at least fails loudly. The dangerous version is when both dimensions happen to be compatible and you get a silently larger array.

Exercise 2 You have `marks` with shape (30, 5) — thirty students, five assignments. Write the expression for each student's average, and the expression for each assignment's average. State the shape of each result.

💡 Show the solution

Per student, average across the five assignments — collapse the *column* index:

```
marks.mean(axis=1)      # shape (30,)
```

Per assignment, average across the thirty students — collapse the *row* index:

```
marks.mean(axis=0)      # shape (5,)
```

Check them by shape rather than by intuition. If you wanted one number per student you should get 30 of them, and only `axis=1` does that.

Exercise 3 This is meant to centre each column, but it does not. Find the bug without running it, then say what shape the result has.

```
data = np.arange(12).reshape(4, 3)
centred = data - data.mean(axis=1)
```

💡 Show the solution

`data.mean(axis=1)` collapses the column index, giving one mean per **row** — shape `(4,)`. To centre columns you need one mean per column, `axis=0`, shape `(3,)`.

Here you are lucky: it raises. Comparing `(4, 3)` with `(4,)` from the right, 4 against 3 is neither equal nor 1, so NumPy refuses and you get a `ValueError`. There is no result shape.

Now change the array to `np.arange(16).reshape(4, 4)` and the luck runs out. `(4, 4)` against `(4,)` broadcasts happily along the last axis, so the value at position `[i, j]` has the mean of **row j** subtracted from it — the row means get lined up against *columns*. That is not row-centring and it is not column-centring; it is nothing at all. You get a plausible `(4, 4)` array of meaningless numbers.

The fix for centring columns is `data - data.mean(axis=0)`. If you genuinely wanted to centre rows, you need `data - data.mean(axis=1, keepdims=True)`, where `keepdims=True` gives shape `(4, 1)` so the means line up against rows.

The lesson: square arrays hide axis errors, so be most careful exactly when the shape looks tidiest.

11.6 Check it in Python

Two habits are worth building deliberately. The first is asserting the shape you expect, so a broadcast that goes wrong stops the program instead of quietly continuing:

```
values = np.array([10.0, 20.0, 30.0])

differences = values - values.mean()
```

```
assert differences.shape == values.shape, f"expected {values.shape}, got
↳ {differences.shape}"
print("shape as expected:", differences.shape)
```

```
shape as expected: (3,)
```

The second is checking that a transformation did what you claimed. Here is the axis bug from Exercise 3, in the square case where it does not raise:

```
data = np.arange(16, dtype=float).reshape(4, 4)

wrong = data - data.mean(axis=1)      # one mean per row - wrong direction
right = data - data.mean(axis=0)      # one mean per column - what we wanted

print("both shapes:", wrong.shape, right.shape)
print("column means after 'wrong':", np.round(wrong.mean(axis=0), 3))
print("column means after 'right':", np.round(right.mean(axis=0), 3))
```

```
both shapes: (4, 4) (4, 4)
column means after 'wrong': [ 4.5  1.5 -1.5 -4.5]
column means after 'right': [0. 0. 0. 0.]
```

Both produced a (4, 4) array and neither complained. In `wrong`, the row means were broadcast against columns — the mean of row 1 subtracted from column 1, and so on — which is not a meaningful operation at all. Only the check distinguishes them: after centring columns correctly every column mean must be zero, and in the wrong version they plainly are not.

If you take one thing from this unit, make it that. **Predict the shape, then verify the property.** It costs one line and it catches the entire class of error that NumPy will not catch for you.

11.7 Common wrong turns

Where this usually goes wrong

Reading axis=0 as “along the rows” The phrasing traps everyone. `axis=0` collapses the row index and gives one result per column. Think in shapes: the axis you name is the one that disappears.

Confusing (n,) with (n, 1) A flat array of length n and a column of n values behave completely differently under broadcasting. Combining the two shapes gives an $n \times n$ result, usually without an error.

Trusting a result because it has a plausible shape NumPy checks that shapes are *compatible*, never that they are *meaningful*. A square array will absorb an axis mistake in total silence.

Leaving ddof at its default `np.std` divides by n . The sample standard deviation divides by $n - 1$. Choose on purpose and record which you used.

Reseeding inside a loop `np.random.default_rng(42)` created afresh on each iteration restarts the same sequence, so every “random” draw is identical. Create the generator once, outside the loop, and pass it in.

Using `*` when you meant `@` For arrays, `*` multiplies matching positions and `@` does matrix multiplication. Both may succeed on the same inputs and return different answers.

Letting a NaN into the arithmetic A single missing value propagates through every sum, mean and product it reaches, turning the whole result into `nan`. Filter first, and be able to say how many rows you dropped and why.

11.8 Check yourself

Answers are at the foot of the page.

1. `x` has shape (5, 8). What is the shape of `x.sum(axis=1)`?
a. (8,) b. (5,) c. (5, 1) d. a single number
2. `a` has shape (3, 4) and `b` has shape (4,). Does `a + b` work? What about `a + b[:, None]`?
3. Why can an axis mistake on a square array be more dangerous than the same mistake on a rectangular one?
4. A colleague reports a standard deviation of 8.37 where you calculated 8.38 from the same numbers. What is the most likely explanation?
5. A loop draws one random number per iteration and every value comes out the same. What is the most likely cause?

 Show the answers

1. **(b) (5,).** `axis=1` removes the second dimension, leaving one result per row. Option (a) is what `axis=0` gives.
2. `a + b` works: comparing from the right, 4 matches 4, and the missing dimension stretches to 3. `a + b[:, None]` has shape (4, 1) against (3, 4); from the right, 1 stretches to 4, but then 4 against 3 fails, so it raises a `ValueError`.
3. Because the shapes stay compatible either way. On a (4, 3) array, using the wrong axis usually raises an error; on a (4, 4) array both directions broadcast happily and you get a correctly shaped, wrong answer with no warning.
4. One of you used `ddof=1` and the other did not — 8.37 is the n divisor and 8.38 the $n - 1$ divisor. Neither is a mistake, but the two answer different questions and the report should say which.
5. The generator is being created inside the loop **with a fixed seed**, so the sequence restarts from the same place on every iteration. Move `rng = np.random.default_rng(2026)` outside the loop. (An unseeded `default_rng()` inside the loop would not repeat — but it would also not be reproducible, so the fix is the same.)

Where this shows up next

P8 moves to labelled tables, where columns have names and types instead of positions — but the arithmetic underneath is everything in this unit. **P10** plots the arrays you build here. **M9** explains what **@** is actually computing, and why the shape rules are what they are.

12 pandas: the DataFrame

Python · Unit 8

P8 · Python Reading: 5 pages With practice: 2 hours Assumes: P7

Why this unit is here

Real data is a **table**: columns with names, columns with different types, values that are missing. A NumPy array cannot express that — it insists every element share one type, and it identifies columns by number, so [P6](#) had you writing `raw[:, 2]` and remembering what 2 meant.

pandas keeps NumPy underneath and adds the two things a table needs: **names** and **mixed types**. It is the tool you will touch every single day, which is why it gets two units.

12.1 Before you start

You should be able to:

- work with arrays, shapes and boolean masks — [P6](#), [P7](#);
- say what a dtype is and why it matters — [P6](#);
- write a function and an assertion — [P5](#).

Quick check. In `raw[:, 2]`, what does the 2 refer to, and what breaks if a column is inserted before it?

 Show the answer

The third column, by position. Insert a column anywhere before it and 2 now means something else — silently. That fragility is most of the reason this unit exists.

12.2 The idea

```
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
print(cohort.shape)
cohort.head()
```

(240, 7)

	student_id	background	diagnostic_score	hours_practice	final_score	completed	st
0	S0001	Mathematics or Physics	89.1	12.9	83.6	True	20
1	S0002	Life sciences	42.4	22.8	56.6	True	20
2	S0003	Engineering	68.4	1.7	58.5	True	20
3	S0004	Computing	64.3	7.5	67.5	True	20
4	S0005	Computing	69.0	13.5	73.8	True	20

A **DataFrame** is a table. Each column is a **Series** — an array with a name and a dtype of its own — and all the columns share an **index** identifying the rows.

	student_id	background	final_score
	index object	object	float64
0	S0001	Maths...	83.6
1	S0002	Life sci...	56.6
2	S0003	Eng...	58.5

one dtype per column — not one for the whole table

Figure 12.1: The three parts of a DataFrame: an index naming the rows, column labels naming the columns, and a dtype per column. The dtype is per column, which is what a NumPy array cannot do.

12.3 The mechanics

12.3.1 The four things you always do after loading

```
print(cohort.shape)
print()
print(cohort.dtypes)
print()
print(cohort.isna().sum())
```

(240, 7)

```
student_id      object
background      object
diagnostic_score float64
hours_practice  float64
final_score     float64
completed       bool
started_on      object
```

```
dtype: object
```

```
student_id      0
background      1
diagnostic_score 0
hours_practice  0
final_score     1
completed       0
started_on      0
dtype: int64
```

Shape tells you whether you loaded what you expected. `dtypes` tells you what pandas *decided* each column was. `isna().sum()` counts missing values per column. Doing these three before anything else takes ten seconds and catches most loading disasters.

Notice `final_score` came in as `float64` and `background` as `object` — pandas' name for "text, or mixed". Notice also that one `final_score` and one `background` are missing, which matches the file's planted defects.

12.3.2 Never trust the inferred type

pandas guesses well and is not always right. Convert deliberately, and let it fail loudly if the data disagrees:

```
cohort["final_score"] = pd.to_numeric(cohort["final_score"], errors="raise")
cohort["started_on"] = pd.to_datetime(cohort["started_on"], errors="raise")
print(cohort[["final_score", "started_on"]].dtypes)
```

```
final_score      float64
started_on      datetime64[ns]
dtype: object
```

`errors="raise"` is the important part. The alternative, `errors="coerce"`, silently turns anything unparseable into `NaT/NaN` — convenient, and a good way to lose data without noticing. Use `raise` first; reach for `coerce` only when you have decided that unparseable values *should* become missing, and then count how many did.

12.3.3 Selecting columns and rows

```
print(type(cohort["final_score"]).__name__)      # one column -> Series
print(type(cohort[["final_score", "background"]]).__name__)  # a list ->
↳ DataFrame
```

```
Series
DataFrame
```

One name gives a Series; a *list* of names gives a DataFrame. The doubled brackets are not a typo — they are a list inside the selection.

For rows, use `.loc` (by label) or `.iloc` (by position):

```
print(cohort.loc[0, "student_id"])      # label 0, that column
print(cohort.iloc[0, 0])                # first row, first column
cohort.loc[0:2, ["student_id", "final_score"]]
```

```
S0001
S0001
```

	student_id	final_score
0	S0001	83.6
1	S0002	56.6
2	S0003	58.5

! `.loc` slices are **inclusive** of the end label — `loc[0:2]` gives three rows. Everywhere else in Python the end is excluded. It is an inconsistency worth knowing rather than being caught by; `.iloc[0:2]` gives two rows as you would expect.

12.3.4 Selecting rows by condition

```
high = cohort[cohort["final_score"] >= 75]
print(f"{len(high)} students scored 75 or more")

quantitative = cohort[cohort["background"].isin(["Computing",
↪ "Engineering"])]
print(f"{len(quantitative)} from computing or engineering")
```

```
25 students scored 75 or more
106 from computing or engineering
```

Combine conditions with `&` and `|`, each in its own parentheses — the same rule as P6:

```
selected = cohort[(cohort["final_score"] >= 60) & (cohort["hours_practice"]
↪ < 15)]
print(f"{len(selected)} scored 60+ while practising under 15 hours")
```

```
98 scored 60+ while practising under 15 hours
```

12.3.5 Adding a column

```
cohort["score_per_hour"] = cohort["final_score"] / cohort["hours_practice"]
print(cohort[["final_score", "hours_practice", "score_per_hour"]].head(3))
```

	final_score	hours_practice	score_per_hour
0	83.6	12.9	6.480620
1	56.6	22.8	2.482456
2	58.5	1.7	34.411765

The arithmetic is elementwise across the whole column, exactly as in [P7](#) — pandas is NumPy underneath.

12.3.6 The trap that catches everyone

```
subset = cohort[cohort["final_score"] >= 75]
subset["flag"] = True # pandas may or may not be modifying a copy
print(f"{len(subset)} rows flagged")
```

25 rows flagged

Selecting rows may give you a view or a copy, and pandas does not promise which. Writing to the result is therefore unreliable — sometimes it silently fails to stick, sometimes it warns. The fix is to say what you want:

```
subset = cohort[cohort["final_score"] >= 75].copy()
subset["flag"] = True
print("independent copy:", len(subset), "rows", subset["flag"].all())
```

independent copy: 25 rows, True

Rule: if you select rows and then write to the result, add `.copy()`. It is one word and it removes an entire category of confusion.

12.4 Worked example

Loading the cohort into a validated analysis table.

```
from pathlib import Path

def load_cohort(path):
    """Load the cohort file, convert types deliberately, and validate the
    ↪ schema."""
    frame = pd.read_csv(path)
```

```

expected = {"student_id", "background", "diagnostic_score",
            "hours_practice", "final_score", "completed", "started_on"}
if set(frame.columns) != expected:
    raise ValueError(f"unexpected columns: {sorted(set(frame.columns) ^
    ↪ expected)}")

for column in ("diagnostic_score", "hours_practice", "final_score"):
    frame[column] = pd.to_numeric(frame[column], errors="raise")
frame["started_on"] = pd.to_datetime(frame["started_on"],
    ↪ errors="raise")

return frame

cohort = load_cohort(Path("../data/cohort.csv"))
print(cohort.shape)
print(cohort.dtypes)

```

```

(240, 7)
student_id          object
background          object
diagnostic_score    float64
hours_practice      float64
final_score         float64
completed           bool
started_on          datetime64[ns]
dtype: object

```

Now split it into what you will analyse and what you will not — keeping both:

```

valid = (
    cohort["final_score"].between(0, 100)
    & cohort["final_score"].notna()
    & cohort["background"].notna()
)

analysis = cohort.loc[valid].copy()
excluded = cohort.loc[~valid].copy()

print(f"analysis: {len(analysis)} rows")
print(f"excluded: {len(excluded)} rows")
excluded[["student_id", "background", "final_score"]]

```

```

analysis: 237 rows
excluded: 3 rows

```

	student_id	background	final_score
17	S0018	Computing	NaN
102	S0103	NaN	66.2
188	S0189	Life sciences	187.0

Three rows excluded, and you can see exactly which and why: one missing score, one missing background, one impossible score of 187. That table is a result, not a byproduct — P9 writes it to a file.

A last check that the split lost nothing:

```
assert len(analysis) + len(excluded) == len(cohort)
assert analysis["final_score"].between(0, 100).all()
assert analysis["background"].notna().all()
print(f"{len(analysis)} + {len(excluded)} = {len(cohort)}; all conditions
↪ verified")
```

237 + 3 = 240; all conditions verified

12.5 Try it yourself

Exercise 1 What is the difference between `cohort["final_score"]` and `cohort[["final_score"]]`?

💡 Show the solution

The first gives a **Series** — one column, one dimension. The second gives a **DataFrame** with one column, because the inner brackets are a *list* of column names. It matters when the next step expects one or the other. `.mean()` on a Series gives a number; on a DataFrame it gives a Series with one entry.

Exercise 2 This is meant to flag high scorers in the original table. It does not. Why, and what are the two ways to fix it?

```
high = cohort[cohort["final_score"] >= 80]
high["distinction"] = True
```

💡 Show the solution

`high` is a new object selected out of `cohort`, so writing to it does not touch `cohort` — and because pandas does not promise whether it is a view or a copy, even the write to `high` is unreliable.

If you wanted an independent table of high scorers:

```
high = cohort[cohort["final_score"] >= 80].copy()
high["distinction"] = True
```

If you wanted to flag rows **in the original**, assign through `.loc` in one statement:

```
cohort["distinction"] = False
cohort.loc[cohort["final_score"] >= 80, "distinction"] = True
```

The second is usually what people actually meant.

Exercise 3 Write one expression giving the mean `diagnostic_score` of students from "Life sciences" whose `final_score` is recorded. Then say why the second condition is needed.

💡 Show the solution

```
mask = (cohort["background"] == "Life sciences") &
↪ cohort["final_score"].notna()
cohort.loc[mask, "diagnostic_score"].mean()
```

Strictly the `notna()` is not needed to compute a mean of *diagnostic scores* — pandas skips missing values automatically, and the missing value is in a different column anyway. It is needed because of what the number will be compared with. If you later report this alongside a mean final score computed on a different set of rows, the two summarise different students. Fixing the analysis set once, up front, is what the worked example does and why.

12.6 Check it in Python

Check the schema, not just the shape. A file with the right number of columns in the wrong order will load without complaint:

```
expected = ["student_id", "background", "diagnostic_score",
            "hours_practice", "final_score", "completed", "started_on"]
assert list(cohort.columns) == expected, f"got {list(cohort.columns)}"
print("schema as expected:", len(expected), "columns in order")
```

schema as expected: 7 columns in order

Check that a filter conserves rows, so nothing vanishes unaccounted for:

```
assert len(analysis) + len(excluded) == len(cohort)
print(f"{len(analysis)} kept + {len(excluded)} excluded = {len(cohort)}"
↪ total")
```

237 kept + 3 excluded = 240 total

Check the ranges you assumed, rather than hoping:


```
summary = analysis[["diagnostic_score", "hours_practice",
    ↪ "final_score"]].agg(
    ["min", "max", "mean"]
).round(2)
print(summary)

assert analysis["final_score"].between(0, 100).all()
assert (analysis["hours_practice"] > 0).all()
print("\nall ranges plausible")
```

	diagnostic_score	hours_practice	final_score
min	27.20	1.00	44.50
max	100.00	36.20	91.60
mean	62.51	16.38	65.27

all ranges plausible

If `hours_practice` ever came back negative, that assertion would fire before the number reached a plot or a model.

12.7 Common wrong turns

Where this usually goes wrong

Writing to the result of a row selection `frame[mask]["col"] = x` may modify a temporary copy and vanish. Use `.copy()` for an independent table, or `frame.loc[mask, "col"] = x` to write into the original.

Trusting the inferred dtype A column of numbers with one stray "n/a" loads as `object`, and every arithmetic operation on it then behaves strangely. Look at `.dtypes` after every load.

Using `errors="coerce"` as a default It converts anything unparseable to missing, silently. Use `raise` first, and only `coerce` once you have decided that is the right treatment — then count how many were affected.

Forgetting `.loc` slices include the end label `loc[0:2]` gives three rows; `iloc[0:2]` gives two. The inconsistency is real; knowing it is easier than being surprised by it.

Selecting on one column and summarising another If the row set differs between two numbers you intend to compare, they are about different students. Fix the analysis set once, up front.

Dropping rows without keeping them An excluded row is a decision that should be inspectable. Keep the excluded table; do not just filter it away.

12.8 Check yourself

Answers are at the foot of the page.

1. `cohort[["final_score"]]` returns what?

- a. a Series b. a DataFrame c. a NumPy array d. a list
- 2. How many rows does `frame.loc[0:2]` return, and how many does `frame.iloc[0:2]`?
- 3. Why does `pd.to_numeric(col, errors="coerce")` deserve caution?
- 4. A numeric column loads with dtype `object`. What is the likely cause?
- 5. You filter to high scorers and add a flag, but the original table is unchanged and no error appeared. What happened?

💡 Show the answers

- 1. **(b) a DataFrame** with one column. The inner brackets make it a list of names, and a list always gives a DataFrame.
- 2. `.loc[0:2]` gives **three** (labels 0, 1 and 2 — the end is included); `.iloc[0:2]` gives **two** (positions 0 and 1, the usual exclusive end).
- 3. It converts every unparseable value to NaN without saying so, so a column that is 30% unparseable looks like a column that is 30% missing. Use `raise` until you have decided otherwise, then count the coerced values.
- 4. At least one value is not a number — a stray "n/a", an empty string, a thousands separator, or a stray unit like "22C". Find it before converting.
- 5. The selection produced a separate object, so the flag was written there and not into the original. Use `frame.loc[mask, "flag"] = True` to write into the original, or `.copy()` if you wanted a separate table.

Where this shows up next

P9 takes the excluded table seriously, groups the analysis table, and joins it to a second file. **P10** plots these columns. **S2** and **S3** use exactly this table to describe one and two variables.

13 pandas: cleaning, grouping, joining

Python · Unit 9

P9 · Python Reading: 5 pages With practice: 2 hours Assumes: P8

Why this unit is here

Three operations account for most of the work in most analyses: deciding what to do about missing values, summarising by group, and combining two tables.

None of them is technically hard. All three are easy to get **quietly** wrong — a `dropna()` that removes a fifth of your data, a join that duplicates rows, a group mean computed over a different set of rows than you thought. This unit is about making each of those decisions visible.

13.1 Before you start

You should be able to:

- load a table, check its dtypes, and select rows by condition — P8;
- explain why `.copy()` matters after a row selection — P8;
- write an assertion — P5.

Quick check. After `subset = frame[frame["x"] > 0]`, why might writing to `subset` not affect `frame`?

💡 Show the answer

Because the selection produced a separate object. pandas does not promise whether it is a view or a copy, which is why you add `.copy()` when you intend an independent table and use `frame.loc[mask, col] = ...` when you intend to write into the original.

13.2 The idea

```
import pandas as pd
from pathlib import Path

cohort = pd.read_csv("../data/cohort.csv")
for column in ("diagnostic_score", "hours_practice", "final_score"):
    cohort[column] = pd.to_numeric(cohort[column], errors="raise")
```

```
print(cohort.shape)
print(cohort.isna().sum())
```

```
(240, 7)
student_id      0
background      1
diagnostic_score 0
hours_practice  0
final_score     1
completed       0
started_on      0
dtype: int64
```

Two missing values, in different columns and different rows. Before touching them, ask the question this unit turns on: **why is it missing?**

- The score was never recorded because the student withdrew — the missingness is related to the outcome, and dropping the row biases the result.
- The score was lost in a transcription error — unrelated to the outcome, and dropping is comparatively harmless.
- The field does not apply to that student at all — structurally missing, and the row may belong to a different population.

pandas cannot tell you which. `dropna()` will happily proceed under all three, which is why it is a **decision**, not a cleaning step.

13.3 The mechanics

13.3.1 Look at missingness before removing it

```
missing_by_group = (
    cohort.assign(no_score=cohort["final_score"].isna())
    .groupby("background", dropna=False)["no_score"]
    .agg(["sum", "size", "mean"])
    .rename(columns={"sum": "missing", "size": "n", "mean": "proportion"})
)
print(missing_by_group)
```

	missing	n	proportion
background			
Computing	1	70	0.014286
Engineering	0	36	0.000000
Life sciences	0	58	0.000000
Mathematics or Physics	0	36	0.000000
Social sciences	0	39	0.000000
NaN	0	1	0.000000

One missing value, in one group. If instead it had been forty, all in the lowest scoring group, complete-case analysis would systematically overstate the cohort mean — and the number would look perfectly reasonable.

Note `dropna=False` in the `groupby`: without it, the row whose *background* is missing would vanish from this very summary, which is the opposite of what a missingness report is for.

13.3.2 Split, and keep both halves

```
valid = (
    cohort["final_score"].between(0, 100)
    & cohort["final_score"].notna()
    & cohort["background"].notna()
)

analysis = cohort.loc[valid].copy()
excluded = cohort.loc[~valid].copy()
excluded["reason"] = [
    "final_score missing" if pd.isna(r.final_score)
    else "background missing" if pd.isna(r.background)
    else f"final_score out of range ({r.final_score})"
    for r in excluded.itertuples()
]

print(f"{len(analysis)} for analysis, {len(excluded)} excluded")
excluded[["student_id", "background", "final_score", "reason"]]
```

237 for analysis, 3 excluded

	student_id	background	final_score	reason
17	S0018	Computing	NaN	final_score missing
102	S0103	NaN	66.2	background missing
188	S0189	Life sciences	187.0	final_score out of range (187.0)

The exclusions table with reasons is the deliverable. “We dropped three rows” is not reproducible; this is.

13.3.3 Grouping: split, apply, combine

```
summary = (
    analysis.groupby("background", as_index=False)
    .agg(n=("final_score", "size"),
         mean_score=("final_score", "mean"),
         sd_score=("final_score", "std"),
         mean_hours=("hours_practice", "mean"))
```

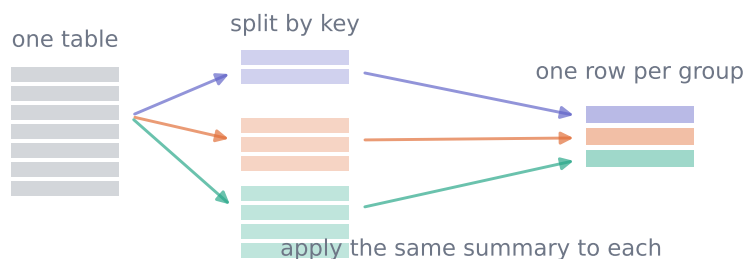


Figure 13.1: Split–apply–combine. The rows are split by a key, the same summary is applied to each part, and the results are stacked back into one table with one row per group.

```

        .round(2)
    )
    summary

```

	background	n	mean_score	sd_score	mean_hours
0	Computing	69	65.85	6.60	12.47
1	Engineering	36	67.15	5.33	10.45
2	Life sciences	57	60.51	7.29	22.61
3	Mathematics or Physics	36	75.28	7.63	8.03
4	Social sciences	39	60.21	6.58	27.37

`as_index=False` keeps `background` as an ordinary column rather than moving it into the index, which makes what follows simpler. The named arguments to `.agg` give each output column an explicit name, so you never have to decode a multi-level heading.

pandas' `.std()` uses the $n - 1$ divisor by default — the sample standard deviation, the opposite of NumPy's default. That inconsistency is real and worth remembering; P7 covers the NumPy side.

13.3.4 Joining, with the relationship checked

```

meta = pd.read_csv("../data/background_meta.csv")
print(meta)

```

	background	quantitative	typical_prior_maths
0	Mathematics or Physics level	True	A-level Further Maths or degree-
1	Computing	True	A-level Maths
2	Engineering	True	A-level Maths
3	Life sciences	False	GCSE, some statistics
4	Social sciences	False	GCSE

```
combined = summary.merge(
    meta,
    on="background",
    how="left",
    validate="one_to_one",
    indicator=True,
)
print(combined["_merge"].value_counts().to_string())
combined = combined.drop(columns="_merge")
combined
```

```
_merge
both      5
left_only 0
right_only 0
```

	background	n	mean_score	sd_score	mean_hours	quantitative	typical_prior_ma
0	Computing	69	65.85	6.60	12.47	True	A-level Maths
1	Engineering	36	67.15	5.33	10.45	True	A-level Maths
2	Life sciences	57	60.51	7.29	22.61	False	GCSE, some stati
3	Mathematics or Physics	36	75.28	7.63	8.03	True	A-level Further M
4	Social sciences	39	60.21	6.58	27.37	False	GCSE

Two arguments do the work here, and they should be habits.

`validate="one_to_one"` states the relationship you believe holds — one summary row per background, one metadata row per background — and pandas **raises** if it does not. A join that unexpectedly matches many-to-many silently multiplies your rows and changes every downstream number.

`indicator=True` adds a column saying, for each row, whether it matched on both sides. Checking it catches the other failure: a key that quietly matched nothing and left NaN where you expected data.

```
broken = pd.concat([meta, meta.head(1)])          # a duplicated key
summary.merge(broken, on="background", how="left", validate="one_to_one")
```

```
MergeError: Merge keys are not unique in right dataset; not a one-to-one
↳ merge
```

```
MergeError                                     Traceback (most recent call last)
```

```
Cell In[8], line 2
```

```
1 broken = pd.concat([meta, meta.head(1)])          # a duplicated key
----> 2 summary.merge(broken, on="background", how="left",
↳ validate="one_to_one")
```

```
File ~/anaconda3/lib/python3.11/site-packages/pandas/core/frame.py:10859, in
↳ DataFrame.merge(self, right, how, on, left_on, right_on, left_index,
↳ right_index, sort, suffixes, copy, indicator, validate)
```

```

10840 @Substitution("")
10841 @Appender(_merge_doc, indents=2)
10842 def merge(
    (...) 10855     validate: MergeValidate | None = None,
10856 ) -> DataFrame:
10857     from pandas.core.reshape.merge import merge
> 10859     return merge(
10860         self,
10861         right,
10862         how=how,
10863         on=on,
10864         left_on=left_on,
10865         right_on=right_on,
10866         left_index=left_index,
10867         right_index=right_index,
10868         sort=sort,
10869         suffixes=suffixes,
10870         copy=copy,
10871         indicator=indicator,
10872         validate=validate,
10873     )
File ~/anaconda3/lib/python3.11/site-
↳ packages/pandas/core/reshape/merge.py:170, in merge(left, right, how,
↳ on, left_on, right_on, left_index, right_index, sort, suffixes, copy,
↳ indicator, validate)
    155     return _cross_merge(
    156         left_df,
    157         right_df,
    (...) 167         copy=copy,
    168     )
    169 else:
--> 170     op = _MergeOperation(
    171         left_df,
    172         right_df,
    173         how=how,
    174         on=on,
    175         left_on=left_on,
    176         right_on=right_on,
    177         left_index=left_index,
    178         right_index=right_index,
    179         sort=sort,
    180         suffixes=suffixes,
    181         indicator=indicator,
    182         validate=validate,
    183     )
    184     return op.get_result(copy=copy)
File ~/anaconda3/lib/python3.11/site-
↳ packages/pandas/core/reshape/merge.py:813, in
↳ _MergeOperation.__init__(self, left, right, how, on, left_on, right_on,
↳ left_index, right_index, sort, suffixes, indicator, validate)
    809 # If argument passed to validate,

```



```

    810 # check if columns specified as unique
    811 # are in fact unique.
    812 if validate is not None:
--> 813     self._validate_validate_kwd(validate)
File ~/anaconda3/lib/python3.11/site-
↳ packages/pandas/core/reshape/merge.py:1658, in
↳ _MergeOperation._validate_validate_kwd(self, validate)
    1654         raise MergeError(
    1655             "Merge keys are not unique in left dataset; not a
↳ one-to-one merge"
    1656         )
    1657     if not right_unique:
-> 1658         raise MergeError(
    1659             "Merge keys are not unique in right dataset; not a
↳ one-to-one merge"
    1660         )
    1662 elif validate in ["one_to_many", "1:m"]:
    1663     if not left_unique:
MergeError: Merge keys are not unique in right dataset; not a one-to-one
↳ merge

```

That is the check earning its place.

13.3.5 Long and wide

```

wide = summary[["background", "mean_score", "mean_hours"]]
long = wide.melt(id_vars="background", var_name="measure",
↳ value_name="value")
print(long.head(6))

```

	background	measure	value
0	Computing	mean_score	65.85
1	Engineering	mean_score	67.15
2	Life sciences	mean_score	60.51
3	Mathematics or Physics	mean_score	75.28
4	Social sciences	mean_score	60.21
5	Computing	mean_hours	12.47

Wide is easier to read; long is easier to plot and group. `melt` goes one way, `pivot` the other. Most plotting libraries, and most statistical models, want long.

13.4 Worked example

The whole pipeline, with every decision recorded.

```

OUT = Path("_outputs")
OUT.mkdir(exist_ok=True)

# 1. load and convert deliberately
raw = pd.read_csv("../data/cohort.csv")
for column in ("diagnostic_score", "hours_practice", "final_score"):
    raw[column] = pd.to_numeric(raw[column], errors="raise")

# 2. split into analysis and exclusions, keeping both
valid = (raw["final_score"].between(0, 100)
        & raw["final_score"].notna()
        & raw["background"].notna())
analysis, excluded = raw.loc[valid].copy(), raw.loc[~valid].copy()

# 3. summarise by group
summary = (analysis.groupby("background", as_index=False)
            .agg(n=("final_score", "size"),
                 mean_score=("final_score", "mean"),
                 sd_score=("final_score", "std"))
            .round(2))

# 4. join metadata, with the relationship asserted
meta = pd.read_csv("../data/background_meta.csv")
report = summary.merge(meta, on="background", how="left",
                       validate="one_to_one", indicator=True)
assert (report["_merge"] == "both").all(), "a group had no metadata"
report = report.drop(columns="_merge")

# 5. write both results
report.to_csv(OUT / "group_summary.csv", index=False)
excluded.to_csv(OUT / "excluded_rows.csv", index=False)

print(f"{len(analysis)} analysed, {len(excluded)} excluded")
print(f"wrote {OUT/'group_summary.csv'} and {OUT/'excluded_rows.csv'}")
report

```

237 analysed, 3 excluded

wrote _outputs/group_summary.csv and _outputs/excluded_rows.csv

	background	n	mean_score	sd_score	quantitative	typical_prior_maths
0	Computing	69	65.85	6.60	True	A-level Maths
1	Engineering	36	67.15	5.33	True	A-level Maths
2	Life sciences	57	60.51	7.29	False	GCSE, some statistics
3	Mathematics or Physics	36	75.28	7.63	True	A-level Further Maths or degree-
4	Social sciences	39	60.21	6.58	False	GCSE

Five steps, and the two that people skip are steps 2 and 4. Keeping the excluded rows means someone can ask *which ones?* and get an answer. Validating the join means the row count cannot silently change under you.

```
assert len(analysis) + len(excluded) == len(raw)
assert report["n"].sum() == len(analysis)
print(f"{report['n'].sum()} students across {len(report)} groups "
      f"={len(analysis)} analysed. Nothing lost, nothing duplicated.")
```

237 students across 5 groups = 237 analysed. Nothing lost, nothing duplicated.

That last assertion is the one that catches a bad join: if the merge had duplicated rows, the group counts would no longer add up to the number of students.

13.5 Try it yourself

Exercise 1 `frame.dropna()` removes 40% of your rows. Name two things you would check before accepting that, and one before rejecting it.

 Show the solution

Before accepting:

- **Which columns** the missingness is in. `dropna()` with no arguments drops a row if *any* column is missing, including ones you never use. `subset=` limits it to the columns that matter.
- **Whether missingness relates to the outcome.** Group the missing indicator by anything you know about the rows, as in the `missing_by_group` table above. If missing rows differ systematically, complete-case analysis is biased.

Before rejecting: whether the rows are missing *structurally* — a field that does not apply to that kind of record. Those rows may simply not belong to the population you are describing, and removing them is correct.

Exercise 2 After a `merge`, your table has 340 rows where both inputs had 240. What happened, and which one argument would have caught it?

 Show the solution

The key is not unique on the right-hand side, so some left rows matched several right rows and were duplicated.

`validate="many_to_one"` — or `"one_to_one"` if you believe both sides are unique — makes pandas raise instead of silently multiplying.

Why it matters beyond tidiness: every duplicated row now contributes more than once to every mean, count and model that follows, and nothing about the output looks wrong.

Exercise 3 Produce a table of the mean `diagnostic_score` and mean `final_score` per background, sorted by mean final score descending. Then say what the ordering shows.

💡 Show the solution

```
(analysis.groupby("background", as_index=False)
 .agg(diagnostic=("diagnostic_score", "mean"),
      final=("final_score", "mean"))
 .round(2)
 .sort_values("final", ascending=False))
```

The ordering by final score is almost exactly the ordering by diagnostic score — groups that arrived ahead finished ahead. That is the confounding structure planted in this dataset, and it is why the overall relationship between practice and score points the wrong way. **S12** takes it apart properly.

13.6 Check it in Python

Assert that a split conserves rows:

```
assert len(analysis) + len(excluded) == len(raw)
print(f"{len(analysis)} + {len(excluded)} = {len(raw)}")
```

237 + 3 = 240

Assert that a join did not change the row count:

```
before = len(summary)
after = len(report)
assert before == after, f"merge changed row count: {before} -> {after}"
print(f"merge preserved {after} rows")
```

merge preserved 5 rows

Assert that group sizes add up to the analysis set — the check that actually catches a silent duplication:

```
assert report["n"].sum() == len(analysis)
print(f"group sizes sum to {report['n'].sum()}, matching {len(analysis)}  
↳ analysed rows")
```

group sizes sum to 237, matching 237 analysed rows

And record what you excluded, in the output, not just in your head:

```
print(pd.read_csv(OUT / "excluded_rows.csv")[["student_id",
↵ "final_score"]].to_string(index=False))
```

```
student_id  final_score
      S0018           NaN
      S0103          66.2
      S0189         187.0
```

13.7 Common wrong turns

Where this usually goes wrong

dropna() as a cleaning step It is a statistical decision about the population you are describing. Look at where the missingness is and whether it relates to the outcome first, and use `subset=` so unrelated columns cannot remove rows.

Merging without validate An unexpected many-to-many join multiplies rows and changes every number that follows, with no error. State the relationship and let pandas enforce it.

Not checking indicator A key that matched nothing leaves `NaN` rather than an error. The indicator column turns that into something you can assert on.

Assuming `.std()` matches NumPy pandas uses the $n - 1$ divisor by default, NumPy uses n . Two “standard deviations” of the same column can differ for this reason alone.

Grouping with `dropna=True` in a missingness report The rows whose key is missing disappear from the very summary meant to find them.

Discarding the excluded rows Then nobody, including you, can answer “which ones, and why?”. Write them out.

13.8 Check yourself

Answers are at the foot of the page.

1. `frame.dropna()` with no arguments removes a row when:
 - a. all columns are missing
 - b. any column is missing
 - c. the first column is missing
 - d. the index is missing
2. What does `validate="one_to_one"` protect against?
3. Why does `groupby(..., dropna=False)` matter when reporting missingness?
4. Group counts sum to 260 but your analysis table has 238 rows. What is the most likely explanation?
5. Give one situation where dropping missing rows is the right call, and one where it is not.

💡 Show the answers

1. **(b) any column.** Which is why a column you never use can silently remove rows. Use `subset=` to name the columns that matter.
2. A join whose key is not unique where you assumed it was, which duplicates rows and changes every downstream number without raising.
3. Because rows whose grouping key is itself missing would be dropped from the report — hiding exactly the cases the report exists to find.
4. The merge duplicated rows: a key matched more than once on the right-hand side. `validate=` would have raised instead.
5. **Right:** the values are missing structurally — the field does not apply — so those rows are not part of the population being described. **Not right:** the values are missing *because of* the outcome, e.g. students who did badly withdrew before the final score was recorded. Then complete-case analysis systematically overstates the result.

Where this shows up next

P10 plots the grouped summary this unit produced. **P11** turns this pipeline into a script another person can run end to end. **S1** asks what population these exclusions leave you describing, and **S12** returns to the confounding visible in the group summary.

14 Plotting with Matplotlib

Python · Unit 10

P10 · Python Reading: 4 pages With practice: 1.5 hours Assumes: P7

Why this unit is here

A plot is an argument. It says *look at this, and conclude that* — and it can make a real pattern obvious or an absent one look convincing.

So this unit is about two things at once: the mechanics of getting a figure out of Matplotlib, and the small number of choices that decide whether the figure tells the truth. Shared scales, honest axes, and not relying on colour alone are not polish. They are the difference between a chart and a claim you can defend.

14.1 Before you start

You should be able to:

- build arrays and reduce them along an axis — [P7](#);
- load and group a table — [P8](#), [P9](#).

Quick check. What does `np.linspace(0, 1, 5)` give, and why is it the right tool for plotting a curve?

💡 Show the answer

`[0, 0.25, 0.5, 0.75, 1]` — exactly five points, **including both ends**. That last part is why: `arange` stops before its endpoint, so a curve drawn with it would fall short of the right edge.

14.2 The idea

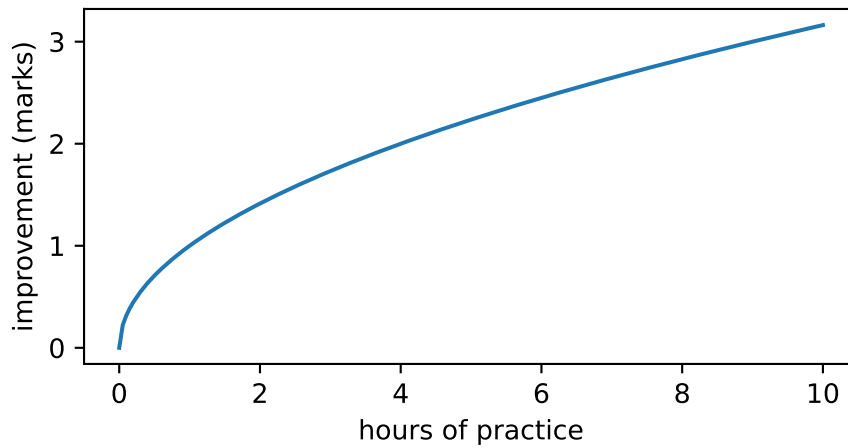
Matplotlib has two interfaces. Use the one where every command names its target:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 200)

fig, ax = plt.subplots(figsize=(5.2, 2.4))
```

```
ax.plot(x, np.sqrt(x))
ax.set(xlabel="hours of practice", ylabel="improvement (marks)")
plt.show()
```



`plt.subplots()` hands back two objects, and knowing which is which removes most confusion:

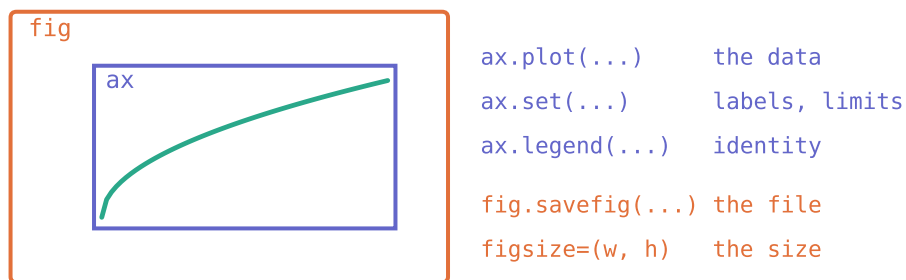


Figure 14.1: The figure is the page; the axes are a set of coordinates drawn on it. Data goes to the axes; saving and sizing belong to the figure.

Data goes to `ax`. Sizing and saving go to `fig`. Once that split is clear the rest of the library follows.

14.3 The mechanics

14.3.1 The four displays that carry most of the work

```
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
cohort["final_score"] = pd.to_numeric(cohort["final_score"], errors="raise")
data = cohort[cohort["final_score"].between(0, 100) &
    ↪ cohort["background"].notna()]
```



```

fig, axes = plt.subplots(2, 2, figsize=(6.4, 4.2))

axes[0, 0].plot(np.linspace(0, 30, 100), 45 + 6*np.linspace(0, 30, 100)
                - 0.25*np.linspace(0, 30, 100)**2)
axes[0, 0].set(title="line - a relationship", xlabel="hours",
               ↪ ylabel="score")

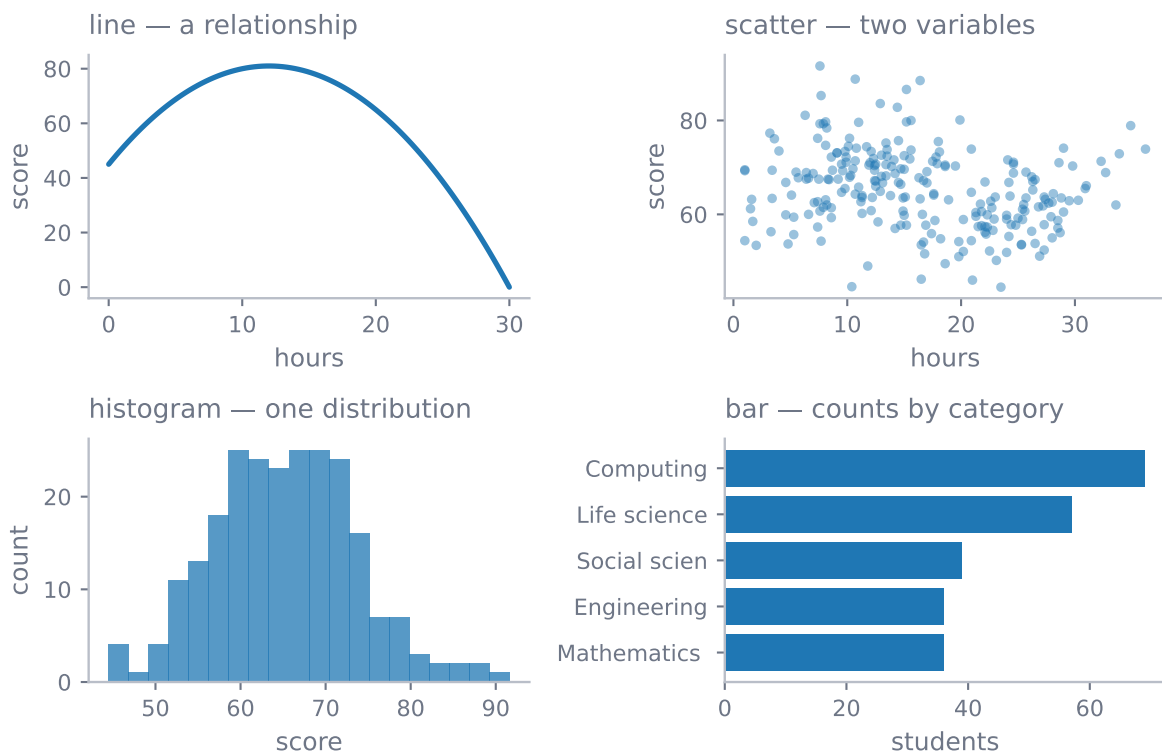
axes[0, 1].scatter(data["hours_practice"], data["final_score"], s=7,
                  ↪ alpha=0.45)
axes[0, 1].set(title="scatter - two variables", xlabel="hours",
               ↪ ylabel="score")

axes[1, 0].hist(data["final_score"], bins=20, alpha=0.75)
axes[1, 0].set(title="histogram - one distribution", xlabel="score",
               ↪ ylabel="count")

counts = data["background"].value_counts().sort_values()
axes[1, 1].barh(counts.index.str[:12], counts.values)
axes[1, 1].set(title="bar - counts by category", xlabel="students")

fig.tight_layout()
plt.show()

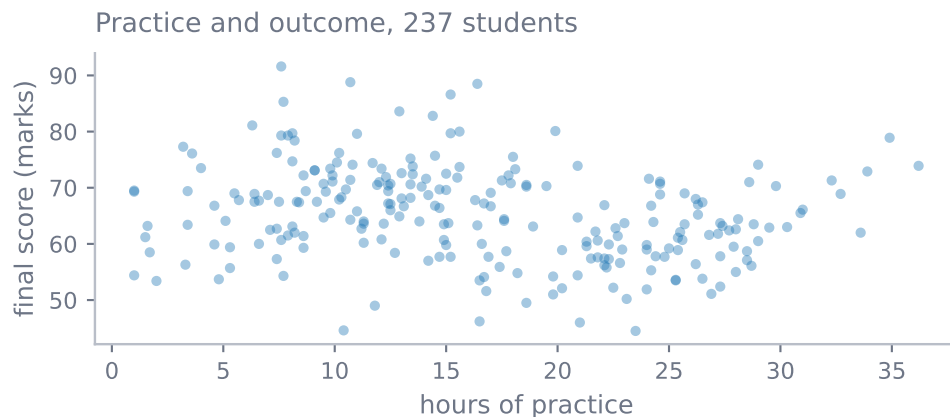
```



`fig.tight_layout()` stops labels from overlapping their neighbours. Call it before saving, every time.

14.3.2 Labels are not optional

```
fig, ax = plt.subplots(figsize=(5.2, 2.4))
ax.scatter(data["hours_practice"], data["final_score"], s=8, alpha=0.4)
ax.set(xlabel="hours of practice", ylabel="final score (marks)",
       title="Practice and outcome, 237 students")
fig.tight_layout()
plt.show()
```



Say what the variable is **and what unit it is in**. “score” is a label; “final score (marks)” is information. A reader who cannot tell whether your y-axis is marks, percentages or z-scores cannot check your conclusion.

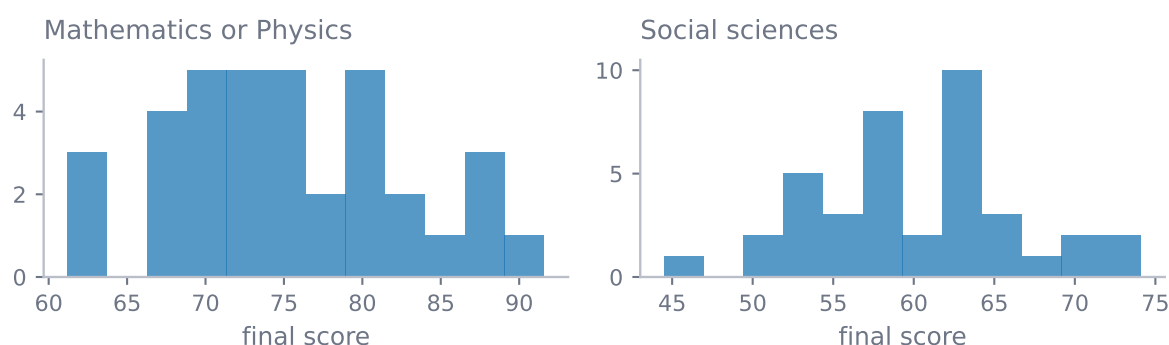
14.3.3 Comparisons need shared scales

This is the single most consequential honesty choice in the unit:

```
groups = ["Mathematics or Physics", "Social sciences"]

fig, axes = plt.subplots(1, 2, figsize=(6.4, 2.3))
for ax, name in zip(axes, groups):
    scores = data.loc[data["background"] == name, "final_score"]
    ax.hist(scores, bins=12, alpha=0.75)
    ax.set(title=name[:22], xlabel="final score")
fig.suptitle("Different bins and different axes - not comparable",
             ↵ fontsize=9)
fig.tight_layout()
plt.show()
```

Different bins and different axes — not comparable

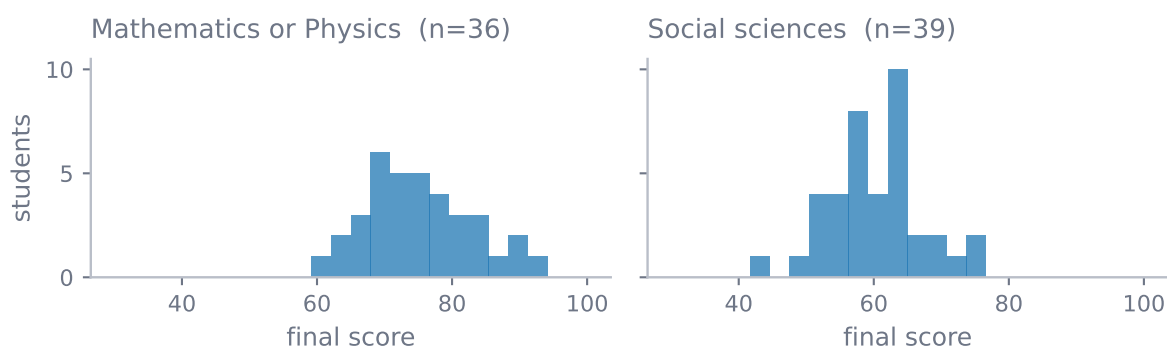


Those two panels look similar. They are not: each chose its own bins and its own axis limits, so the shapes are drawn to different rulers. Fix both:

```
bins = np.linspace(30, 100, 25)

fig, axes = plt.subplots(1, 2, figsize=(6.4, 2.3), sharex=True, sharey=True)
for ax, name in zip(axes, groups):
    scores = data.loc[data["background"] == name, "final_score"]
    ax.hist(scores, bins=bins, alpha=0.75)
    ax.set(title=f"{name[:22]} (n={len(scores)})", xlabel="final score")
axes[0].set(ylabel="students")
fig.suptitle("Same bins, same axes - now comparable", fontsize=9)
fig.tight_layout()
plt.show()
```

Same bins, same axes — now comparable



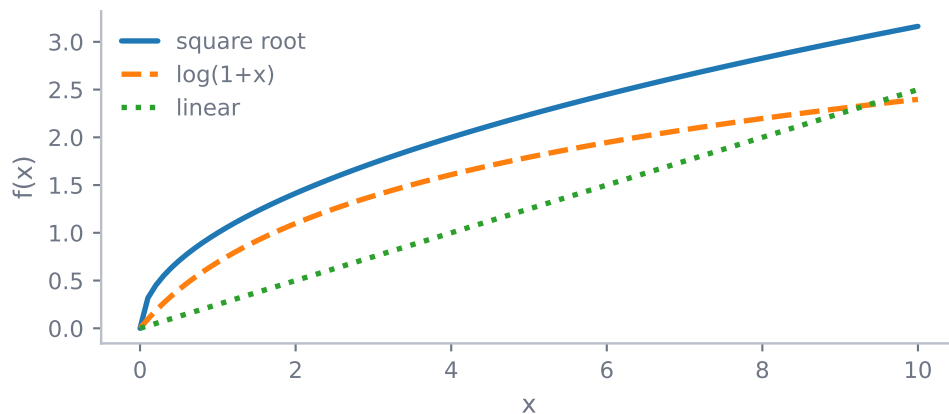
Same bins, `sharex`, `sharey`, and the group sizes stated. Now the difference you can see is a difference in the data.

14.3.4 Do not rely on colour alone

Roughly one man in twelve has some form of colour-vision deficiency, and plenty of figures get printed in greyscale anyway. Give every series a second cue — line style, marker shape, or a direct label:

```
x = np.linspace(0, 10, 100)

fig, ax = plt.subplots(figsize=(5.2, 2.4))
ax.plot(x, np.sqrt(x), linestyle="-", label="square root")
ax.plot(x, np.log1p(x), linestyle="--", label="log(1+x)")
ax.plot(x, x/4, linestyle=":", label="linear")
ax.set(xlabel="x", ylabel="f(x)")
ax.legend()
fig.tight_layout()
plt.show()
```



14.3.5 Saving

```
from pathlib import Path

OUT = Path("_outputs"); OUT.mkdir(exist_ok=True)

fig, ax = plt.subplots(figsize=(5.2, 2.4))
ax.hist(data["final_score"], bins=20, alpha=0.8)
ax.set(xlabel="final score (marks)", ylabel="students")
fig.tight_layout()
fig.savefig(OUT / "scores.svg")
plt.close(fig)

print("wrote", (OUT / "scores.svg").stat().st_size, "bytes")
```

wrote 24508 bytes

Use **SVG** or **PDF** for plots — they are lines and text, and stay sharp at any size. PNG or JPEG are for photographs. And `plt.close(fig)` after saving: a script that makes fifty figures without closing them will eat memory and warn you about it.

14.4 Worked example

One figure, made defensible.

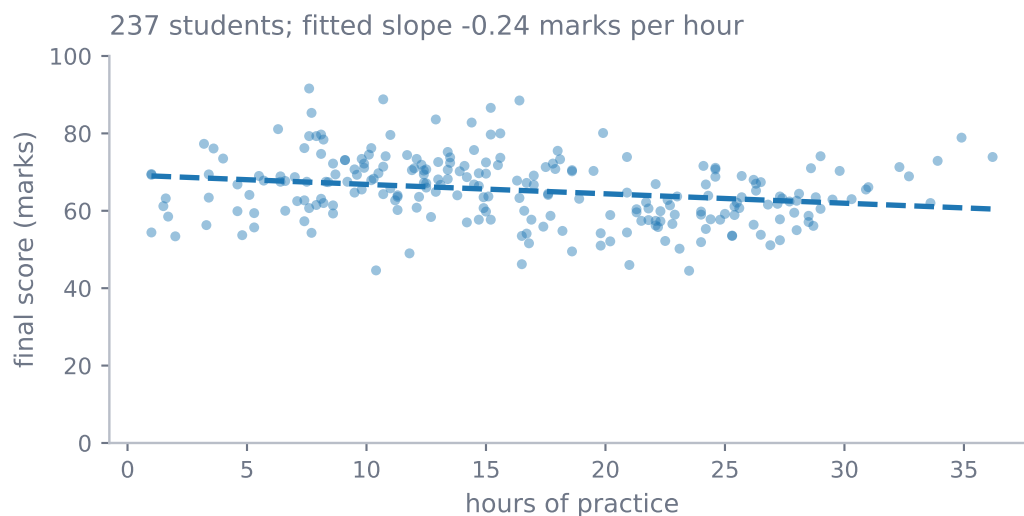
The cohort's overall relationship between practice and score is the dataset's headline oddity. Here it is, plotted carefully:

```
fig, ax = plt.subplots(figsize=(5.6, 2.9))

ax.scatter(data["hours_practice"], data["final_score"],
           s=14, alpha=0.45, edgecolor="none")

fit = np.polyfit(data["hours_practice"], data["final_score"], 1)
line_x = np.array([data["hours_practice"].min(),
                   data["hours_practice"].max()])
ax.plot(line_x, np.polyval(fit, line_x), linestyle="--", lw=2)

ax.set(xlabel="hours of practice", ylabel="final score (marks)",
       title=f"237 students; fitted slope {fit[0]:+.2f} marks per hour")
ax.set_ylim(0, 100)
fig.tight_layout()
plt.show()
```



Three decisions in that figure are worth naming.

The y-axis starts at zero and ends at 100, because the score is bounded there. Letting Matplotlib pick would have zoomed into the occupied range and made a modest slope look dramatic. Truncating an axis is the commonest way an honest number becomes a misleading picture.

The slope is in the title, with units. A reader can check the line against the number rather than eyeballing it.

Transparency is set to 0.45, because 237 points overlap. Without it, a dense region reads as one blob and you cannot see where the mass is.

And the thing the figure does *not* say: that practising less causes higher scores. The slope is negative, and it is real — but [S12](#) shows the relationship reverses inside every background

group. A plot can be drawn correctly and still invite the wrong conclusion, which is why the caption matters as much as the code.

14.5 Try it yourself

Exercise 1 Why is `plt.subplots()` preferable to calling `plt.plot()` directly?

💡 Show the solution

`plt.plot()` draws on whatever figure Matplotlib currently considers active, which is invisible state. In a notebook run out of order, or a script with several figures, “current” is not always what you think.

`fig, ax = plt.subplots()` names both objects, so every subsequent command says what it is acting on. The moment you have two panels it is the only workable option.

Exercise 2 Two histograms are drawn side by side, each with `bins=10` and no shared axes. Give two distinct reasons they cannot be compared.

💡 Show the solution

- **Different bin edges.** `bins=10` means “ten bins spanning *this* data”, so the two panels divide different ranges into different intervals. A bar in one is not the same width as a bar in the other.
- **Different axis limits.** Each panel scales to its own data, so equal bar heights represent different counts and equal horizontal distances represent different score differences.

Fix both: pass an explicit `bins=` array shared by both, and `sharex=True`, `sharey=True`. Then state each group’s `n`, because a taller bar may only mean a bigger group.

Exercise 3 A colleague’s bar chart of mean scores has a y-axis running from 58 to 62, and the difference between groups looks enormous. Is the chart wrong? What would you change?

💡 Show the solution

Nothing plotted is false — the bars are at the right heights for that axis. But the *impression* is wrong: a four-mark spread fills the whole panel and reads as a dramatic difference.

For bars, the length is the encoding, so the axis should start at zero; otherwise length no longer means magnitude. If the differences are genuinely small and genuinely interesting, plot the **difference** directly, with an interval showing how precisely it is known — which is **S8**’s subject.

The general rule: truncate an axis only when zero is meaningless for the quantity, and say so.

14.6 Check it in Python

Plots are easy to get subtly wrong, and three checks catch most of it.

Check the axis limits are what you intended:

```
fig, ax = plt.subplots(figsize=(4.6, 1.9))
ax.hist(data["final_score"], bins=20)
ax.set_ylim(0)
print("x limits:", tuple(round(v, 1) for v in ax.get_xlim()))
print("y limits:", tuple(round(v, 1) for v in ax.get_ylim()))
assert ax.get_ylim()[0] == 0, "count axis must start at zero"
plt.close(fig)
print("count axis starts at zero")
```

```
x limits: (42.1, 94.0)
y limits: (0.0, 26.2)
count axis starts at zero
```

Check nothing was silently dropped. A histogram outside its range simply does not draw the points:

```
scores = data["final_score"]
bins = np.linspace(40, 90, 26)          # a range chosen by eye
inside = scores.between(bins[0], bins[-1]).sum()
print(f"{inside} of {len(scores)} points fall inside the plotted range")
assert inside == len(scores), f"{len(scores) - inside} points would be
↳ invisible"
```

236 of 237 points fall inside the plotted range

AssertionError: 1 points would be invisible

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[11], line 5
      3 inside = scores.between(bins[0], bins[-1]).sum()
      4 print(f"{inside} of {len(scores)} points fall inside the plotted
↳ range")
----> 5 assert inside == len(scores), f"{len(scores) - inside} points would
↳ be invisible"
AssertionError: 1 points would be invisible
```

That assertion is not decoration — it just caught a real mistake. A range of 40 to 90 looked reasonable and quietly excluded a student. Derive the range from the data instead of choosing it by eye:

```
bins = np.linspace(scores.min(), scores.max(), 26)
inside = scores.between(bins[0], bins[-1]).sum()
print(f"{inside} of {len(scores)}; range {bins[0]:.1f} to {bins[-1]:.1f}")
assert inside == len(scores)
print("every point is inside the plotted range")
```

237 of 237; range 44.5 to 91.6
every point is inside the plotted range

Check it survives greyscale. If two series differ only in hue, converting to luminance makes them indistinguishable:

```
fig, ax = plt.subplots(figsize=(4.6, 1.9))
styles = ["-", "--", ":"]
for k, style in enumerate(styles):
    ax.plot(np.linspace(0, 5, 50), np.linspace(0, 5, 50)**(k+1)/5**k,
            linestyle=style, label=f"series {k+1}")
ax.legend()
handles, _ = ax.get_legend_handles_labels()
print("distinct line styles:", len({h.get_linestyle() for h in handles}),
      "of", len(handles), "series")
assert len({h.get_linestyle() for h in handles}) == len(handles)
plt.close(fig)
print("every series has its own line style")
```

distinct line styles: 3 of 3 series
every series has its own line style

14.7 Common wrong turns

Where this usually goes wrong

Truncating the axis on a bar chart The bar's length *is* the number. Starting the axis above zero breaks that and makes small differences look large.

Comparing panels with unshared bins or axes Each panel then uses its own ruler. Pass one explicit bins array and `sharex / sharey`, and state each group's `n`.

Relying on colour alone Add a line style, a marker shape, or a direct label. Some readers cannot distinguish your hues, and print often cannot either.

Unlabelled axes, or labels without units "score" could be marks, a percentage or a z-score. The unit belongs in the label.

Using `plt.plot()` in anything with more than one figure It draws on whichever figure is currently active — invisible state that breaks when cells run out of order.

Forgetting `plt.close(fig)` in a script Figures accumulate in memory, and Matplotlib will eventually warn you about it.

Overplotting without transparency Hundreds of opaque points become one shape. `alpha` restores the density.

14.8 Check yourself

Answers are at the foot of the page.

1. `fig, ax = plt.subplots()` — which object do you call `savefig` on?
 - a. `ax` b. `fig` c. either d. `plt`

2. Why should a bar chart's value axis start at zero when a scatter plot's need not?
3. Two histograms use `bins=10` each. Name the problem in one sentence.
4. When would PNG be the right format for a figure?
5. A scatter of 5,000 points shows one solid mass. What single argument helps most?

💡 Show the answers

1. **(b) fig.** The figure is the page being written out; the axes are drawn on it.
2. Because a bar encodes its value as **length from the baseline**, so moving the baseline breaks the encoding. A scatter encodes value as *position*, and a reader reads the position off the axis — so a zoomed range is legitimate, provided the axis is labelled.
3. `bins=10` means ten bins across *each panel's own range*, so the two panels use different bin widths and are not comparable.
4. For genuinely raster content — a photograph, a satellite image, a rendered heatmap of a very large grid. Line-and-text plots should be SVG or PDF.
5. **alpha** — transparency, so overlapping points accumulate visibly and you can see where the density actually is. (Reducing `s` helps too, and for very large `n` a 2-D histogram may be better than a scatter at all.)

Where this shows up next

P11 saves figures as checked outputs of a script rather than things you made once by hand. **S2** and **S3** choose which of these displays a question calls for. **S12** returns to the scatter in this unit's worked example and shows why its slope points the wrong way.

15 Work someone else can rerun

Python · Unit 11

P11 · Python Reading: 5 pages With practice: 2 hours Assumes: P9, P10

Why this unit is here

An analysis nobody can rerun is an opinion with numbers attached.

The someone who most often cannot rerun it is **you**, four months later, with a marker asking how you got a figure. Everything in this unit is aimed at that moment: a folder a stranger can open, run one command in, and get your numbers back.

It is also the unit where the habits of the whole Python track stop being separate tricks and become one way of working.

15.1 Before you start

You should be able to:

- load, clean, group and join a table — P9;
- produce and save a labelled figure — P10;
- write a function with an assertion — P5.

Quick check. Why did P9 write the excluded rows to a file rather than just counting them?

 Show the answer

Because “we dropped three rows” cannot be checked and “here are the three rows and why” can. Every exclusion is a decision, and a decision nobody can inspect is indistinguishable from a mistake.

15.2 The idea

Reproducible does not mean elaborate. It means someone else can get your numbers without asking you anything. That reduces to five questions, and if a folder answers all five it is reproducible enough:

The five questions: **what was the question, what data, what code, what environment, and how do I run it.**

project/	
README.md	the question, and the one command to run
requirements.txt	exact package versions
data/	source data — never edited
analysis.py	loads, cleans, summarises, plots
checks.py	tests of the functions above
outputs/	generated — safe to delete

Figure 15.1: A layout that answers the five questions. Source data is never edited; everything in outputs/ can be deleted and rebuilt by one command.

15.3 The mechanics

15.3.1 Paths that work on someone else's machine

```
from pathlib import Path

ROOT = Path.cwd()
DATA = ROOT / "../data"
print((DATA / "cohort.csv").exists())
```

True

Path joins with / and works identically on Windows, macOS and Linux. C:\Users\me\project\data.csv written as a string works on exactly one machine — yours, until you rename a folder.

Build paths relative to the script, never to wherever the terminal happened to be:

```
ROOT = Path(__file__).resolve().parent
```

15.3.2 Pin the environment

```
import sys, numpy, pandas, matplotlib, scipy
print("python", sys.version.split()[0])
for package in (numpy, pandas, matplotlib, scipy):
    print(f"{package.__name__:<11}", package.__version__)
```

```
python      3.11.14
numpy       1.26.4
pandas      2.3.3
matplotlib  3.10.8
scipy       1.17.0
```

Record those in `requirements.txt` with `==`, not `>=`:

```
numpy==1.26.4
pandas==2.3.3
matplotlib==3.10.8
scipy==1.17.0
```

`>=` means “whatever is current when someone installs it”, which is a promise that your results will change one day for reasons unconnected to your work.

15.3.3 Seed anything random, once

```
import numpy as np

rng = np.random.default_rng(2026)
print(rng.normal(size=3).round(4))

again = np.random.default_rng(2026)
print(again.normal(size=3).round(4))
```

```
[-0.7931  0.2406 -1.8963]
[-0.7931  0.2406 -1.8963]
```

Same seed, same numbers, on any machine. Create the generator **once** near the top and pass it where it is needed — creating a fresh one inside a loop restarts the sequence, as P7 showed.

A seed makes a simulation *repeatable*. It does not make it correct, and it does not make a sample representative.

15.3.4 Keep source data immutable

Read from `data/`, write to `outputs/`, and never the reverse. If a cleaning step overwrites its own input, the second run operates on different data from the first and you can never get back to the start.

15.3.5 Test the functions, not the script

```
def standardise(values):
    """Return sample-standardised values as a NumPy array."""
    values = np.asarray(values, dtype=float)
    if values.ndim != 1 or values.size < 2:
        raise ValueError("need a one-dimensional sample of at least 2
↪ values")
    sd = values.std(ddof=1)
```

```

    if not np.isfinite(sd) or sd == 0:
        raise ValueError("standard deviation must be finite and non-zero")
    return (values - values.mean()) / sd

z = standardise([2, 4, 4, 4, 5, 5, 7, 9])
assert np.isclose(z.mean(), 0)
assert np.isclose(z.std(ddof=1), 1)
print("mean", round(float(z.mean()), 12), " sd", round(float(z.std(ddof=1)),
↪ 12))

```

mean 0.0 sd 1.0

Written in a file called `checks.py`, those become a test suite:

```

# checks.py
import numpy as np
import pytest
from analysis import standardise

def test_standardised_has_mean_zero_and_sd_one():
    z = standardise([2, 4, 4, 4, 5, 5, 7, 9])
    assert np.isclose(z.mean(), 0)
    assert np.isclose(z.std(ddof=1), 1)

def test_refuses_constant_input():
    with pytest.raises(ValueError):
        standardise([3, 3, 3])

```

Run them with `pytest`. A test that a function *refuses* bad input is worth as much as one that it handles good input — often more, because that is the path nobody exercises by hand.

i A passing test says the behaviour matched your expectation. It does not say your expectation was right. Tests catch regressions and typos; they cannot tell you that you standardised the wrong column.

15.3.6 Notebooks: restart and run all

A notebook shows the *output* of cells, not the order they ran in. A cell you edited and never re-ran leaves a stale number on screen that no longer follows from the code above it.

Before you submit or share anything: **Restart Kernel and Run All Cells**, and check it still works from top to bottom. If it does not, it was never going to work for anyone else either.

When logic becomes reusable, move it into a `.py` file and import it. Keep the notebook for narrative.

15.4 Worked example

The pipeline as a script someone else can run.

```
from pathlib import Path
import pandas as pd

ROOT = Path.cwd()
DATA, OUT = ROOT / "../data", ROOT / "_outputs"
OUT.mkdir(exist_ok=True)
SEED = 2026

def load(path):
    """Load the cohort file and convert types deliberately."""
    frame = pd.read_csv(path)
    for column in ("diagnostic_score", "hours_practice", "final_score"):
        frame[column] = pd.to_numeric(frame[column], errors="raise")
    return frame

def split(frame):
    """Separate analysable rows from excluded ones, with reasons."""
    ok = (frame["final_score"].between(0, 100)
          & frame["final_score"].notna()
          & frame["background"].notna())
    kept, dropped = frame.loc[ok].copy(), frame.loc[~ok].copy()
    assert len(kept) + len(dropped) == len(frame)
    return kept, dropped

def summarise(frame):
    """One row per background, with counts and spread."""
    return (frame.groupby("background", as_index=False)
            .agg(n=("final_score", "size"),
                 mean_score=("final_score", "mean"),
                 sd_score=("final_score", "std"))
            .round(2))

cohort = load(DATA / "cohort.csv")
analysis, excluded = split(cohort)
summary = summarise(analysis)

summary.to_csv(OUT / "group_summary.csv", index=False)
excluded.to_csv(OUT / "excluded_rows.csv", index=False)

assert summary["n"].sum() == len(analysis)
print(f"{len(cohort)} rows in, {len(analysis)} analysed, {len(excluded)}
↳ excluded")
```

```
print(f"wrote {len(list(OUT.glob('*.csv')))} files to {OUT.name}/")
summary
```

240 rows in, 237 analysed, 3 excluded
wrote 2 files to _outputs/

	background	n	mean_score	sd_score
0	Computing	69	65.85	6.60
1	Engineering	36	67.15	5.33
2	Life sciences	57	60.51	7.29
3	Mathematics or Physics	36	75.28	7.63
4	Social sciences	39	60.21	6.58

Four named functions, each testable on its own; every path built from ROOT; every exclusion written out; two assertions guarding the arithmetic. Nothing here is clever, and that is rather the point.

15.4.1 The README that goes with it

```
# Cohort outcomes by prior background

**Question.** Does prior background relate to final score, and how do the
groups differ in practice hours?

**Data.** `data/cohort.csv` - 240 synthetic records. Generated by
`data/make_cohort.py` with seed 2026. Not real students.

**Setup.**
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

**Run.**
python analysis.py

**Outputs.** `outputs/group_summary.csv`, `outputs/excluded_rows.csv`.
Three rows are excluded: one missing score, one missing background, one
↪ score
of 187 (impossible). See `outputs/excluded_rows.csv`.
```

Ten lines. It answers all five questions, and it is the difference between a folder and a piece of work.

15.5 Try it yourself

Exercise 1 A notebook produces a figure you cannot reproduce after restarting it. Name two likely causes.

💡 Show the solution

- **Cells were run out of order.** A name still held a value from a cell that has since been edited or deleted, so the figure depended on state not visible in the notebook as written.
- **An unseeded random step.** Anything using `np.random` without a fixed seed gives different numbers each run.

A third, less often noticed: the notebook read a file that has since been overwritten by a later cell — the immutable-source-data rule exists for exactly this.

Exercise 2 Why is `pandas>=2.0` in a requirements file worse than `pandas==2.3.3`?

💡 Show the solution

`>=` installs whatever is current on the day someone runs it. A default changes, a function is deprecated, and your numbers move — for reasons unconnected to your analysis, with nothing in your project recording what changed.

`==` fixes it. When you *choose* to upgrade, you change one line, rerun the tests, and see what moved. That is a decision with evidence rather than a surprise.

Exercise 3 Your analysis reads `data/cohort.csv`, cleans it, and writes the cleaned version back to the same path. Give two things that go wrong.

💡 Show the solution

- **The second run cleans already-cleaned data.** If cleaning is not exactly idempotent — dropping rows, rescaling, recoding — run two gives a different answer from run one, and neither is reproducible.
- **The original is gone.** You cannot check what the raw values were, cannot rerun with a different rule, and cannot show anyone what you excluded.

Write to `outputs/`. Treat `data/` as read-only, and if you can, make it literally read-only so the mistake becomes an error.

15.6 Check it in Python

Record the environment inside the output, so a result carries its own provenance:

```
import sys, json, numpy, pandas
provenance = {
    "python": sys.version.split()[0],
    "numpy": numpy.__version__,
    "pandas": pandas.__version__,
}
```



```

    "seed": SEED,
    "rows_in": int(len(cohort)),
    "rows_analysed": int(len(analysis)),
    "rows_excluded": int(len(excluded)),
}
(OUT / "provenance.json").write_text(json.dumps(provenance, indent=2))
print(json.dumps(provenance, indent=2))

```

```

{
  "python": "3.11.14",
  "numpy": "1.26.4",
  "pandas": "2.3.3",
  "seed": 2026,
  "rows_in": 240,
  "rows_analysed": 237,
  "rows_excluded": 3
}

```

Six months on, that file answers “which versions produced this?” without anyone having to remember.

Assert the invariants that must hold, so a broken run stops rather than producing a plausible wrong answer:

```

assert len(analysis) + len(excluded) == len(cohort), "rows lost in the
↳ split"
assert summary["n"].sum() == len(analysis), "group sizes do not match"
assert analysis["final_score"].between(0, 100).all(), "impossible score
↳ survived"
assert not analysis["background"].isna().any(), "missing background
↳ survived"
print("all four invariants hold")

```

all four invariants hold

Confirm the outputs are actually there, and that nothing was written into the source directory:

```

for name in ("group_summary.csv", "excluded_rows.csv", "provenance.json"):
    path = OUT / name
    print(f" {name:<22} {path.stat().st_size:>6} bytes")
print("\nsource data untouched:", sorted(p.name for p in (ROOT /
↳ '../data').glob('*.csv'))))

```

group_summary.csv	177 bytes
excluded_rows.csv	220 bytes
provenance.json	147 bytes

source data untouched: ['background_meta.csv', 'cohort.csv']

15.7 Getting help, and using an assistant

You are working alone, so how you ask for help matters.

A good request contains four things: **a minimal example** that reproduces the problem, **what you expected**, **what happened** including the full final line of the traceback, and **your versions** from the check above. Assembling that often solves the problem before you send it — which is itself a reason to do it.

On AI assistants, plainly. They are useful and you will use them. Two things worth deciding now, while nothing is at stake.

Use one as a tutor, not an answer key. “Explain why this raises” and “what does `ddof` change here” leave you able to do the next one. “Write this for me” does not, and the gap shows up in the first thing you have to do unaided.

Never paste anything you would not email to a stranger — unpublished data, personal records, credentials, assessed work belonging to someone else.

And when you do keep generated code: read every line, and be able to say what it does. Code you cannot explain is code you cannot debug, defend, or fix at eleven at night before a deadline. The academic-integrity line at your university will be written down somewhere; find it and read it before you need it, not after.

15.8 Common wrong turns

Where this usually goes wrong

Absolute paths `/Users/me/project/data.csv` works on exactly one machine. Build from `Path(__file__).resolve().parent`.

Writing cleaned data over the source The second run then cleans already-cleaned data, and the original is gone. Source data is read-only; generated files go in `outputs/`.

`>=` in requirements Your results change when an unrelated package updates. Pin with `==` and upgrade deliberately.

Seeding inside a loop Restarts the sequence every iteration, so every draw is identical. Create the generator once.

Trusting notebook output The visible numbers may come from cells that no longer exist. Restart and run all before believing anything.

Treating a passing test as proof of correctness It shows the behaviour matched your expectation. Your expectation can be wrong, and no test will tell you.

Pasting code you cannot explain It works until it does not, and then you cannot fix it. Read every line you keep.

15.9 Check yourself

Answers are at the foot of the page.

1. Which belongs in version control?

a. `outputs/figure.svg` b. `requirements.txt` c. `.venv/` d. all three

2. Why build paths with `pathlib` rather than string concatenation?
3. A colleague's notebook gives different numbers each run. Two likely causes?
4. Your tests pass but a reviewer finds the analysis answers the wrong question. What does that say about tests?
5. Name two things that belong in a README and one thing that does not belong in the repository at all.

 Show the answers

1. **(b) requirements.txt.** It is source: small, hand-written, and needed to rebuild everything else. `outputs/` is generated and should be reproducible from the code; `.venv/` is large, machine-specific, and rebuilt from `requirements.txt`.
2. Because separators differ between operating systems, and `pathlib` joins correctly on all of them. It also makes relative-to-the-script paths natural, which is what stops a project from working on one machine only.
3. An unseeded random step, or cells run out of order so a value came from code that no longer exists. A third: reading a file that a later cell overwrites.
4. That tests check **internal consistency**, not scientific validity. They confirm the code does what you told it to; whether that was the right thing to tell it is a question no test can answer.
5. In the README: the question, the data and where it came from, setup steps, the exact run command, what outputs appear, and known limitations. Not in the repository at all: credentials, API keys, personal data, or anyone else's assessed work.

Where this shows up next

That completes the Python track. Everything from here uses it.

M1 starts the Mathematics track, and every unit in it ends with a *Check it in Python* block built on these habits. **S1** opens Statistics by asking what question an analysis is answering — the same question this unit's README begins with.

Part III

Mathematics

16 Reading mathematics

Mathematics · Unit 1

M1 · Mathematics Reading: 4 pages With practice: 1.5 hours Assumes: nothing — start here

Why this unit is here

Most people who find mathematics hard do not find the *ideas* hard. They find the **notation** hard, and then never get to the ideas.

A formula is a compressed sentence. Once you can read it aloud and say what every symbol stands for, the rest is arithmetic you already know. This unit teaches the reading, and nothing else — there is no new mathematics in it at all.

16.1 Before you start

You should be able to rearrange something like $2x + 3 = 11$, work with fractions, and read a pair of coordinates off a graph. That is genuinely all.

Quick check. Solve $3x - 4 = 11$.

 Show the answer

$x = 5$. Add 4 to both sides to get $3x = 15$, then divide both sides by 3.

If that felt slow rather than wrong, you will be fine — it comes back quickly. If it felt genuinely unfamiliar, spend an hour on a GCSE algebra refresher before going on, because everything after this assumes it.

16.2 The idea

Take a formula that will appear later in these notes:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

It looks like four unfamiliar things. It is one familiar thing: **add up all the values and divide by how many there are**. It is the average.

Read it aloud, left to right:

- $\bar{x}$ — “x bar”, a single number, the average of the x values.
- $=$ — is defined to be.
- $\frac{1}{n}$ — one over n , where n is how many values there are.
- $\sum$ — “sum”, meaning add up what follows.
- $i = 1$ underneath and n on top — starting at the first value and going to the last.
- x_i — “x sub i”, the i -th value.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad \text{mean} = \text{sum}(x) / \text{len}(x)$$

↓
/ len(x)
sum(...)
each x
the whole thing

Figure 16.1: The same instruction written twice. Every part of the formula corresponds to a part of the code; neither is more real than the other.

```
values = [68.4, 55.1, 72.0, 61.3]
mean = sum(values) / len(values)
print(mean)
```

64.2

The formula and the code say the same thing. Neither is the “real” version — they are two notations for one instruction, and being fluent in both is the point of this course.

16.3 The mechanics

16.3.1 Sets of numbers

Symbol	Name	Means
$\mathbb{N}$	naturals	1, 2, 3, ...
$\mathbb{Z}$	integers	..., -2, -1, 0, 1, 2, ...
$\mathbb{R}$	reals	every number on the line, including π and $\sqrt{2}$

$x \in \mathbb{R}$ reads “ x is a real number”. The $\in$ means “is in”.

16.3.2 Intervals

Round brackets exclude the endpoint; square brackets include it.

- $[0, 1]$ — from 0 to 1, **including** both.
- $(0, 1)$ — from 0 to 1, **excluding** both.
- $[0, 1)$ — includes 0, excludes 1.

The distinction matters when the endpoint is impossible: a probability lies in $[0, 1]$, but a value you are about to divide by lies in $(0, \infty)$ — zero excluded, deliberately.

16.3.3 Summation

$$\sum_{i=1}^4 x_i = x_1 + x_2 + x_3 + x_4$$

The subscript says where to start, the superscript where to stop, and the body says what to add. It is a **for** loop:

```
x = [10, 20, 30, 40]

total = 0
for i in range(len(x)):
    total = total + x[i]

print(total, sum(x))
```

100 100

Two conventions worth knowing. Mathematics counts from 1 and includes the last index; Python counts from 0 and excludes the stop value. They describe the same four numbers by different bookkeeping, and mixing them up is the commonest off-by-one error there is.

A sum with something more elaborate inside works identically:

$$\sum_{i=1}^n (x_i - \bar{x})^2$$

“For each value, subtract the mean, square it, and add all of those up.” You will meet this exact expression in [S2](#) — it is the heart of the variance.

```
import numpy as np

x = np.array([68.4, 55.1, 72.0, 61.3])
print(sum((value - x.mean())**2 for value in x))
print((x - x.mean())**2).sum()
```

169.70000000000005
169.70000000000005

16.3.4 Subscripts and function notation

x_i is not x multiplied by i . The subscript is a **label** — the i -th item, exactly like `x[i]`.

$f(x)$ is not f times x either. It is “the function f , applied to x ”, and it is `f(x)` in code too.

16.3.5 Powers, roots and logarithms

$$x^n = \underbrace{x \times x \times \cdots \times x}_{n \text{ times}} \quad x^{1/2} = \sqrt{x} \quad x^{-1} = \frac{1}{x}$$

```
print(2**10, 9**0.5, 2**-1)
```

1024 3.0 0.5

The three rules you will use constantly:

$$x^a x^b = x^{a+b} \quad \frac{x^a}{x^b} = x^{a-b} \quad (x^a)^b = x^{ab}$$

16.3.6 Greek letters

They are just letters, and the convention is stable enough to be worth learning:

	Name	Usually means
μ	mu	a population mean
σ	sigma	a population standard deviation
$\bar{x}$	x-bar	a sample mean
α, β	alpha, beta	parameters to be estimated
λ	lambda	a rate
Δ	delta	a change in something

Note σ and Σ : lower case is a standard deviation, upper case is the summation sign. Same letter, different jobs, decided by case alone.

16.4 Worked example

Reading a formula you have not met, entirely from its parts.

Here is the sample standard deviation, which arrives properly in [S2](#):

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Work outward from the middle, which is how these are built.

Innermost: $x_i - \bar{x}$ — each value minus the mean. How far this observation sits from the centre.

Then: $(x_i - \bar{x})^2$ — squared. Squaring makes everything positive, so distances above and below the mean do not cancel out.

Then: $\sum_{i=1}^n$ — add all those squared distances up.

Then: $\frac{1}{n-1}$ — divide by one less than the count. (Why $n-1$ rather than n is a real question with a real answer, and it is S2's, not this unit's.)

Finally: $\sqrt{}$ — square root, which undoes the squaring and returns the answer to the original units. If x was in marks, s is in marks.

Now build it up in code, one layer at a time:

```
x = np.array([68.4, 55.1, 72.0, 61.3])

deviations = x - x.mean()
squared = deviations**2
total = squared.sum()
divided = total / (len(x) - 1)
s = divided**0.5

print("deviations", deviations.round(2))
print("squared    ", squared.round(2))
print("sum        ", round(total, 2))
print("/(n-1)     ", round(divided, 2))
print("sqrt       ", round(s, 3))
print("numpy      ", round(x.std(ddof=1), 3))
```

```
deviations [ 4.2 -9.1  7.8 -2.9]
squared    [17.64 82.81 60.84  8.41]
sum        169.7
/(n-1)     56.57
sqrt       7.521
numpy      7.521
```

Five lines, five layers, and the last one confirms it against NumPy. A formula you can take apart like this is a formula you can check — and checking is what makes it yours rather than something you copied.

16.5 Try it yourself

Exercise 1 Write $\sum_{i=1}^5 i^2$ out in full, then compute it.

 Show the solution

$$1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55.$$

```
sum(i**2 for i in range(1, 6))    # range stops before 6
```

Note the `range(1, 6)`: mathematics says “to 5 inclusive”, Python says “stop before 6”. Same five numbers.

Exercise 2 Say in words what each means, and give a value that is in the first but not the second.

- a. $x \in [0, 10]$ b. $x \in (0, 10)$

💡 Show the solution

- a. x is a real number from 0 to 10, **including** both ends.
b. x is strictly between 0 and 10, **excluding** both.

Either 0 or 10 is in the first and not the second.

Where this bites: if x is about to appear in a denominator, you need $(0, 10]$, not $[0, 10]$ — and the bracket is the only thing saying so.

Exercise 3 Translate into a single line of Python:

$$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})$$

Then predict its value before running it, for any x at all.

💡 Show the solution

```
(x - x.mean()).mean()
```

It is **zero**, always, for any set of numbers.

The mean is the balance point: the distances above it exactly cancel the distances below. That is precisely why the variance squares the deviations first — without squaring, the whole thing collapses to nothing and tells you no measure of spread at all.

Try it and you will get something like 10^{-15} rather than exactly 0 — the floating-point noise from P2.

16.6 Check it in Python

The habit this unit is really teaching: **translate a formula into code, then check it against a library that already implements it**. Agreement is strong evidence you read the notation correctly.

```
x = np.array([68.4, 55.1, 72.0, 61.3])

by_hand = sum(x) / len(x)
print(f"by hand {by_hand:.6f}    numpy {x.mean():.6f}    agree:
↪ {np.isclose(by_hand, x.mean())}")
```

```
by hand 64.200000    numpy 64.200000    agree: True
```

And the deviations-sum-to-zero fact from Exercise 3, which is worth seeing:

```
print("sum of deviations:", (x - x.mean()).sum())
print("close to zero:      ", np.isclose((x - x.mean()).sum(), 0))
```

```
sum of deviations: -7.105427357601002e-15
close to zero:      True
```

Not exactly zero, and it should not be — that residue is floating-point arithmetic, not a mistake. `np.isclose` is the right question to ask.

16.7 Common wrong turns

Where this usually goes wrong

Reading x_i as x times i The subscript is a label — the i -th value, like `x[i]`. Likewise $f(x)$ is a function applied to x , not f multiplied by x .

Mixing the two counting conventions Mathematics counts from 1 and includes the last index; Python counts from 0 and excludes the stop. $\sum_{i=1}^n$ is `range(0, n)` or just `range(n)`.

Confusing σ with Σ Lower case is a standard deviation, upper case is a summation. The case is the only difference and it changes everything.

Ignoring which bracket an interval uses $[0, 1]$ and $(0, 1)$ differ exactly where it matters — at the endpoints, which is usually where a formula would break.

Trying to absorb a formula whole Work outward from the innermost bracket. Every formula in these notes was built in layers and comes apart the same way.

Skipping the check because the algebra “looks right” Three lines of code will tell you whether you read the notation correctly, and it takes less time than staring at it.

16.8 Check yourself

Answers are at the foot of the page.

1. What is $\sum_{i=1}^3 2i$?
a. 6 b. 12 c. 8 d. 2
2. What is the difference between σ and Σ ?
3. Is 0 in the interval $(0, 5]$?
4. Write $\sqrt{x}$ using a power.
5. Why does the variance square the deviations before adding them?

 Show the answers

1. **(b) 12.** $2(1) + 2(2) + 2(3) = 2 + 4 + 6$.
2. Lower-case σ is a standard deviation — a number. Upper-case Σ is the instruction to add up. Case is the only distinction.
3. **No.** The round bracket excludes 0. The interval is everything strictly greater than 0, up to and including 5.
4. $x^{1/2}$, or equivalently $x^{0.5}$.

5. Because without squaring they sum to exactly zero, for any data at all — the mean is the balance point. Squaring makes every deviation positive so they accumulate into a genuine measure of spread.

Where this shows up next

M2 uses this notation to describe functions, and adds the handful of curves that recur everywhere. **S2** returns to the standard deviation formula from the worked example and explains the $n - 1$. Every formula in the rest of these notes is built from the pieces in this unit.

17 Functions, exponentials and logarithms

Mathematics · Unit 2

M2 · Mathematics Reading: 4 pages With practice: 1.5 hours Assumes: M1

Why this unit is here

A **function** is the object every model in data science is. A regression is a function from predictors to a prediction; a loss is a function from parameters to a number you want small; a probability density is a function from an outcome to a height.

And a small number of shapes recur endlessly. Recognise those five curves and you can read most of what follows — including why nearly every quantity in data science gets logged at some point.

17.1 Before you start

You should be able to:

- read $f(x)$, subscripts and $\sum$ — M1;
- work with powers and roots — M1.

Quick check. If $f(x) = 3x + 1$, what is $f(4)$?

 Show the answer

13. Substitute 4 wherever x appears: $3(4) + 1$.
`f(4)` in code does exactly the same thing.

17.2 The idea

A function is a rule with a **domain**: what you are allowed to put in.

$$f : \mathbb{R} \rightarrow \mathbb{R}, \quad f(x) = x^2$$

Read as “ f takes a real number and gives back a real number, by squaring it”. The rule alone is not the whole story — the domain is part of the definition, and leaving it out is where errors come from:

```
import numpy as np
print(np.sqrt(-4))
```

nan

$\sqrt{x}$ is only defined for $x \geq 0$ in the reals. Its domain is $[0, \infty)$, and NumPy warns rather than pretending otherwise.

The three that will catch you:

- **division** needs a denominator that is not zero;
- **square root** needs a non-negative argument;
- **logarithm** needs a *strictly positive* argument.

17.3 The mechanics

17.3.1 The five shapes

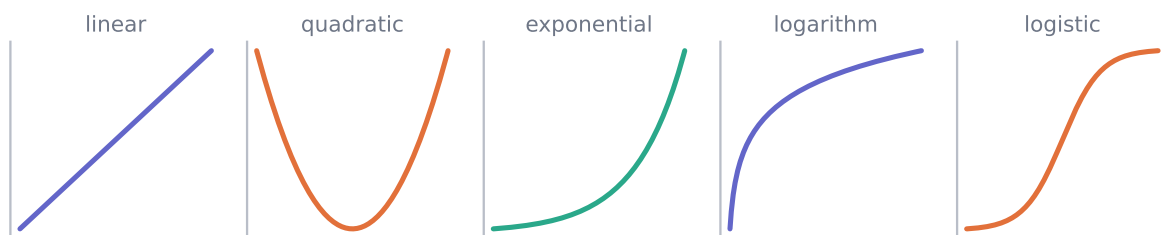


Figure 17.1: The five curves that recur. Each is defined by how it *changes*: constant slope, turning, accelerating, decelerating, saturating.

Linear, $f(x) = a + bx$. Constant rate of change: every extra unit of x adds the same b . The slope b is the whole content.

Quadratic, $f(x) = ax^2 + bx + c$. Turns exactly once — a single maximum or minimum. Loss functions are often quadratic near their best point, which is why they are worth recognising.

Exponential, $f(x) = e^x$. Constant *proportional* growth: every extra unit multiplies. Doubling times, compound interest, unchecked epidemics.

Logarithmic, $f(x) = \ln x$. The inverse of the exponential. Grows without limit but ever more slowly.

Logistic, $f(x) = 1/(1 + e^{-x})$. Starts flat, accelerates, saturates. It maps anything at all into $(0, 1)$, which is exactly what you want when the output must be a probability.

17.3.2 Exponentials and logs are inverses

$$\ln(e^x) = x \quad e^{\ln x} = x \quad (x > 0)$$

```
print(np.log(np.exp(3.7)))
print(np.exp(np.log(3.7)))
```

```
3.7
3.7000000000000006
```

Each undoes the other, which is what makes the logarithm the tool for getting a variable out of an exponent.

17.3.3 The three log rules

$$\ln(ab) = \ln a + \ln b \quad \ln \frac{a}{b} = \ln a - \ln b \quad \ln(a^b) = b \ln a$$

```
a, b = 6.0, 2.5
print(np.isclose(np.log(a*b), np.log(a) + np.log(b)))
print(np.isclose(np.log(a/b), np.log(a) - np.log(b)))
print(np.isclose(np.log(a**b), b*np.log(a)))
```

```
True
True
True
```

That third rule is the important one. It **turns a power into a multiplication**, which is how you solve for something stuck in an exponent:

$$2^x = 100 \implies x \ln 2 = \ln 100 \implies x = \frac{\ln 100}{\ln 2}$$

```
x = np.log(100) / np.log(2)
print(f"x = {x:.4f}, check: 2**x = {2**x:.4f}")
```

```
x = 6.6439, check: 2**x = 100.0000
```

i `log` means the **natural** logarithm, base e , in essentially all mathematics and in NumPy. Base 10 is `np.log10`, base 2 is `np.log2`. Calculators and spreadsheets often use `log` for base 10, which causes real confusion — check which you have.

17.3.4 Why data gets logged

Two reasons, both practical.

It turns multiplicative structure into additive structure. If a quantity grows by a constant *percentage*, its logarithm grows by a constant *amount* — a straight line, which is far easier to model and to read.

```
weeks = np.arange(0, 11)
users = 100 * 1.4**weeks          # 40% growth per week

print("users:      ", users.round(0)[:5], "...")
print("log users:", np.log(users).round(3)[:5], "...")
print("differences in log:", np.diff(np.log(users)).round(4)[:4])
```

```
users:      [100. 140. 196. 274. 384.] ...
log users: [4.605 4.942 5.278 5.615 5.951] ...
differences in log: [0.3365 0.3365 0.3365 0.3365]
```

The raw numbers accelerate away; the differences in the log are constant at $\ln 1.4 \approx 0.336$. Exponential growth has become a straight line.

It compresses a long right tail. Incomes, city sizes, file sizes and gene expression levels all have a few enormous values that dominate everything.

```
rng = np.random.default_rng(2026)
income = rng.lognormal(mean=10, sigma=1.0, size=1000)

print(f"raw:      median {np.median(income):>10,.0f}    mean
↳ {income.mean():>10,.0f}")
print(f"logged:   median {np.median(np.log(income)):>10.2f}    mean
↳ {np.log(income).mean():>10.2f}")
```

```
raw:      median      22,840    mean      38,177
logged:   median       10.04    mean       10.02
```

On the raw scale the mean sits well above the median, dragged by the tail. On the log scale they very nearly coincide.

But: a log changes what a difference *means*. A gap of 0.7 in logs is a factor of two, whatever the starting point. If you log, say so, and report back on the original scale where you can.

17.4 Worked example

Reading the cohort's diagnostic scores, and whether logging helps.


```
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
hours = cohort["hours_practice"].to_numpy()

print(f"n {hours.size}")
print(f"min {hours.min():.1f}    median {np.median(hours):.1f}    "
      f"mean {hours.mean():.1f}    max {hours.max():.1f}")
```

```
n 240
min 1.0    median 15.2    mean 16.4    max 36.2
```

Mean and median are close, so there is no long tail here. Logging would achieve nothing, and that is worth checking rather than doing reflexively:

```
ratio = hours.mean() / np.median(hours)
print(f"mean / median = {ratio:.3f}")
print("strong right tail?" , ratio > 1.2)
```

```
mean / median = 1.082
strong right tail? False
```

Now a quantity where it does help — the same lognormal incomes:

```
for name, values in [("hours_practice", hours), ("simulated income",
↪ income)]:
    r = values.mean() / np.median(values)
    print(f"{name:<18} mean/median {r:5.2f}    log-scale ratio "
          f"{np.log(values).mean()/np.median(np.log(values)):5.2f}")
```

```
hours_practice      mean/median  1.08    log-scale ratio  0.97
simulated income    mean/median  1.67    log-scale ratio  1.00
```

The income data has a mean 65% above its median; after logging, the two agree to within a couple of per cent. The hours data was already symmetric and logging does nothing for it.

The lesson is the check, not the transformation: **compare mean to median before deciding**. A log is a tool for a specific shape, not a routine step.

17.5 Try it yourself

Exercise 1 State the domain of each.

a. $f(x) = \frac{1}{x-3}$ b. $g(x) = \sqrt{x+2}$ c. $h(x) = \ln(x-1)$

💡 Show the solution

- a. Everything except 3: $\{x \in \mathbb{R} : x \neq 3\}$. Division by zero.
- b. $x \geq -2$, that is $[-2, \infty)$. The argument of the root must be non-negative.
- c. $x > 1$, that is $(1, \infty)$. **Strictly** greater — $\ln 0$ is undefined, so 1 is excluded. That is the one people get wrong; the square root allows its endpoint and the logarithm does not.

Exercise 2 Solve $3^x = 50$ for x , and check your answer.

💡 Show the solution

Take logs of both sides and use $\ln(a^b) = b \ln a$:

$$x \ln 3 = \ln 50 \implies x = \frac{\ln 50}{\ln 3} \approx 3.561$$

```
x = np.log(50) / np.log(3)
print(x, 3**x)           # 3.5609..., 50.0000...
```

Any base works as long as you use the same one on both sides — $\log_{10}$ gives the identical answer, because the two logs differ by a constant factor that cancels in the ratio.

Exercise 3 A quantity doubles every 3 years. Write it as a function of years t , then find how long until it is ten times its starting value.

💡 Show the solution

$$N(t) = N_0 \cdot 2^{t/3}$$

Ten times the start means $2^{t/3} = 10$, so

$$\frac{t}{3} \ln 2 = \ln 10 \implies t = \frac{3 \ln 10}{\ln 2} \approx 9.97 \text{ years}$$

```
t = 3 * np.log(10) / np.log(2)
print(t, 2**(t/3))       # 9.966..., 10.0
```

Worth noticing: ten-fold takes only about 3.3 doublings, not ten. Compounding is consistently faster than intuition expects, which is exactly why exponentials are worth recognising on sight.

17.6 Check it in Python

Check an inverse actually inverts, over a range rather than at one point:

```
values = np.array([0.1, 1.0, 3.7, 250.0])
print(np.allclose(np.log(np.exp(values)), values))
print(np.allclose(np.exp(np.log(values)), values))
```

True
True

Check a log rule rather than trusting your memory of it:

```
a, b = 6.0, 2.5
checks = {
    "ln(ab) = ln a + ln b": (np.log(a*b), np.log(a) + np.log(b)),
    "ln(a/b) = ln a - ln b": (np.log(a/b), np.log(a) - np.log(b)),
    "ln(a^b) = b ln a": (np.log(a**b), b*np.log(a)),
    "ln(a+b) = ln a + ln b": (np.log(a+b), np.log(a) + np.log(b)),
}
for name, (left, right) in checks.items():
    print(f" {name:<26} {np.isclose(left, right)}")
```

$\ln(ab) = \ln a + \ln b$	True
$\ln(a/b) = \ln a - \ln b$	True
$\ln(a^b) = b \ln a$	True
$\ln(a+b) = \ln a + \ln b$	False

The last one is **false**, and deliberately included. There is no rule for the log of a sum, and inventing one is a standard error — the check catches it in one line.

Check the domain before applying a function to real data:

```
values = np.array([12.0, 0.0, 5.5, -3.0])
loggable = values > 0
print(f"{loggable.sum()} of {values.size} values can be logged")
print("logged:", np.log(values[loggable]).round(3))
```

2 of 4 values can be logged
logged: [2.485 1.705]

Filtering first is better than logging and then hunting for the `-inf` and `nan` that appear.

17.7 Common wrong turns

Where this usually goes wrong

Inventing a rule for $\ln(a + b)$ There isn't one. The rules cover products, quotients and powers — never sums.

Forgetting that $\ln$ needs a *strictly* positive argument $\sqrt{0}$ is fine; $\ln 0$ is not. Zero is a genuine value in plenty of datasets, and it is the one that breaks a log transform.

Assuming log means base 10 In NumPy and in mathematics it is base e . Calculators and spreadsheets often differ. Use `np.log10` when you mean base 10.

Logging by reflex A log transform is for right-skewed, multiplicative quantities. Compare mean to median first; if they agree, logging buys nothing and costs interpretability.

Reporting results on the log scale without saying so A difference of 0.7 in logs is a factor of two, not “0.7 more”. State the scale, and back-transform where you can.

Stating a rule without its domain “ $f(x) = 1/x$ ” is incomplete. The domain is part of the function, and it is where the errors live.

17.8 Check yourself

Answers are at the foot of the page.

1. What is the domain of $\ln(x - 2)$?
a. $x \geq 2$ b. $x > 2$ c. $x \neq 2$ d. all reals
2. Simplify $\ln(x^3) - \ln(x)$.
3. Why does logging turn constant-percentage growth into a straight line?
4. A variable has mean 4,200 and median 900. What does that suggest, and what would you try?
5. Is $\ln(a + b) = \ln a + \ln b$ ever true for positive a and b ?

Show the answers

1. **(b)** $x > 2$. Strictly greater — the logarithm is undefined at zero, so $x = 2$ is excluded.
2. $\ln(x^3) - \ln x = 3 \ln x - \ln x = 2 \ln x$. (Equivalently $\ln(x^3/x) = \ln(x^2)$.) Valid for $x > 0$.
3. Because constant-percentage growth means multiplying by the same factor each period, and the log of a product is a sum of logs — so equal multiplications become equal additions, which is a straight line.
4. A strong right skew: a few very large values dragging the mean far above the median. Try a log transform, then check whether mean and median come into agreement on the new scale.
5. **Essentially never.** It would need $a + b = ab$, which for positive numbers holds only on a curve (for instance $a = b = 2$). It is not a rule and treating it as one is a standard error.

Where this shows up next

M3 asks how fast a function changes, which is the start of everything that follows. **M7** integrates the exponential, and **S6** meets it again inside the normal density. The logistic curve returns whenever a model must output a probability.

18 Rates of change

Mathematics · Unit 3

M3 · Mathematics Reading: 5 pages With practice: 2 hours Assumes: M1, M2

Why this unit is here

Every model you will ever fit is chosen by asking one question over and over: *if I nudge this number slightly, does the model get better or worse?*

That question is a derivative. It is the same question whether you are tuning a single parameter by hand or letting a neural network tune ten million of them, and it is the idea that the next four units are built on. Before any of that, though, a derivative is something much more ordinary: a way of saying how quickly one quantity responds to a change in another.

18.1 Before you start

You should be able to:

- read a graph and find the gradient of a straight line — M1 ;
- substitute a number into a formula — M1 ;
- say what the domain of a function is — M2 .

Quick check. What is the gradient of the straight line through (1, 3) and (3, 11)?

💡 Show the answer

$$\text{gradient} = \frac{11 - 3}{3 - 1} = \frac{8}{2} = 4.$$

If that took real effort, reread M1 before going on. Everything below is this calculation, applied to curves instead of lines.

18.2 The idea

Throughout these notes we use one dataset: a synthetic record of a previous cohort who worked through this material. Here are three of its columns.

```
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
cohort[["background", "hours_practice", "final_score"]].head()
```

	background	hours_practice	final_score
0	Mathematics or Physics	12.9	83.6
1	Life sciences	22.8	56.6
2	Engineering	1.7	58.5
3	Computing	7.5	67.5
4	Computing	13.5	73.8

Suppose we model how a student's final score responds to hours of practice. For one particular group of students, a reasonable-looking model is

$$s(h) = 45 + 6h - 0.25h^2$$

where h is hours of practice and s is the final score in marks.

Now ask: **how fast is the score improving?**

That question has no single answer until you say *where*. Between $h = 0$ and $h = 4$ the score climbs from 45 to 65, so it is improving at

$$\frac{s(4) - s(0)}{4 - 0} = \frac{65 - 45}{4} = 5 \text{ marks per hour.}$$

Between $h = 20$ and $h = 24$ the same model gives $s(20) = 65$ and $s(24) = 45$, an *average* of -5 marks per hour. The model says the same student, practising the same amount, is now going backwards. Same curve, two opposite answers, because we asked about two different places on it.

This quantity — change in output divided by change in input — is the **average rate of change**, and it always refers to an interval, never to a point.

18.2.1 Shrinking the interval

To talk about the rate at a *single* value of h , take shorter and shorter intervals around it. Both sides matter, so we shrink towards $h = 4$ from above and from below:

```
def s(h):
    return 45 + 6*h - 0.25*h**2

print(f"{'interval':>9}    {'from the right':>14}    {'from the left':>13}")
for step in [4, 2, 1, 0.5, 0.1, 0.01]:
    right = (s(4 + step) - s(4)) / step
    left = (s(4 - step) - s(4)) / (-step)
    print(f"{'step':>9}    {'right':>14.4f}    {'left':>13.4f}")
```

interval	from the right	from the left
4	3.0000	5.0000
2	3.5000	4.5000
1	3.7500	4.2500
0.5	3.8750	4.1250
0.1	3.9750	4.0250
0.01	3.9975	4.0025

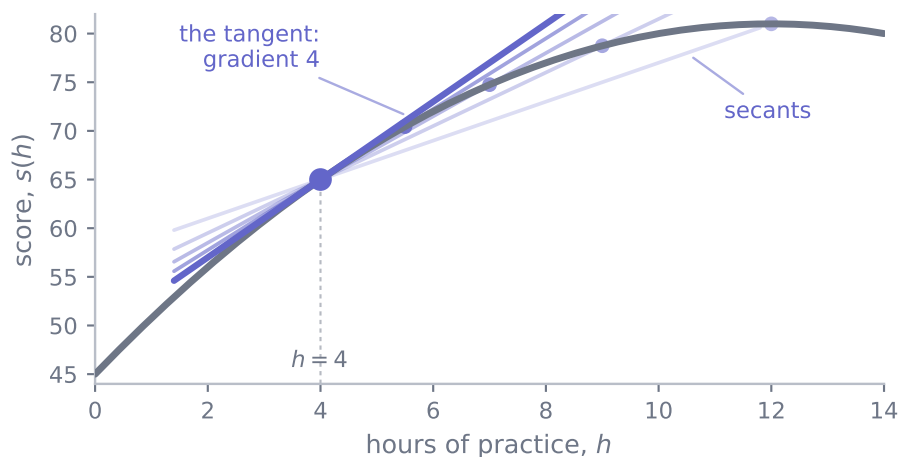


Figure 18.1: Each faint line joins $h = 4$ to a point further along the curve; its gradient is one row of the table above. As that second point slides in, the lines settle onto the single bold line — the tangent, whose gradient is 4.

Both columns are settling on 4 Both columns are settling on 4, one from below and one from above. Neither ever *reaches* 4 — every row uses a genuine interval of some width — but both get and stay as close to 4 as you like, by taking the interval short enough. That the two sides agree is not a formality; later in this unit you will meet a function where they do not.

That settled-on value is the **derivative** of s at $h = 4$, written $s'(4)$. We say $s'(4) = 4$ marks per hour.

i Two readings of the same number, both worth carrying:

Geometric. $s'(4)$ is the gradient of the *tangent* to the curve at $h = 4$. Each row of the table above is the gradient of a line through two points on the curve; as the second point slides towards the first, those lines settle into one limiting position, and that is the tangent. (It is tempting to define a tangent as the line that touches the curve once and does not cross it. That is a useful picture here but not a definition — some tangents do cross their curve at the point of contact.)

Practical. $s'(4)$ is the *sensitivity* of the score to practice at that point: one more hour buys roughly 4 more marks. This is the reading you will use constantly from M6 onwards, and it is the reason a derivative carries the units of the output divided by the units of the input — here, marks per hour.

18.3 The mechanics

18.3.1 The difference quotient

For a function f and a point a , the average rate of change over an interval of width h is

$$\frac{f(a+h) - f(a)}{h}$$

which is called the **difference quotient**. The derivative is the value it settles on as h shrinks towards zero, written

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

Read $\lim_{h \rightarrow 0}$ as “the value this settles on as h gets arbitrarily small”. You cannot simply put $h = 0$: that gives $0/0$, which is not a number. The whole point of the notation is to say *approach* zero without arriving.

! The symbol h is doing two different jobs on this page: it is hours of practice in the worked model, and interval width in the general definition. That collision is normal in mathematics and worth noticing rather than tripping over. In the general definition, h is always the interval width.

18.3.2 Notation

All of these mean the same thing:

$$f'(x) \quad \frac{df}{dx} \quad \frac{d}{dx}f(x)$$

The first is compact. The second is useful because it names the input variable, which matters when there is more than one — as there will be in [M6](#).

18.3.3 Doing it from first principles

Take $f(x) = x^2$. Then

$$\begin{aligned} \frac{f(x+h) - f(x)}{h} &= \frac{(x+h)^2 - x^2}{h} \\ &= \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \frac{2xh + h^2}{h} \\ &= 2x + h. \end{aligned}$$

That last step is legitimate because h is small but *not* zero, so dividing by it is allowed. Now let h shrink: the expression settles on $2x$. So

$$f(x) = x^2 \implies f'(x) = 2x.$$

The same three moves — expand, cancel the h in the denominator, let what remains shrink — handle every first-principles derivative you will be asked for. In [M4](#) you will get rules that make this unnecessary, but doing it by hand two or three times first is what makes those rules mean something.

18.3.4 When there is no derivative

A derivative does not always exist. The clearest failure is a corner. Take $f(x) = |x|$ at $x = 0$: approaching from the right, the difference quotient is always $+1$; from the left it is always -1 . The two sides disagree, so nothing settles, so $f'(0)$ does not exist.

```
for step in [1, 0.1, 0.001]:
    right = (abs(0 + step) - abs(0)) / step
    left = (abs(0 - step) - abs(0)) / (-step)
    print(f"step {step:<7} from the right {right:+.0f}    from the left
          ↪ {left:+.0f}")
```

```
step 1      from the right +1    from the left -1
step 0.1    from the right +1    from the left -1
step 0.001  from the right +1    from the left -1
```

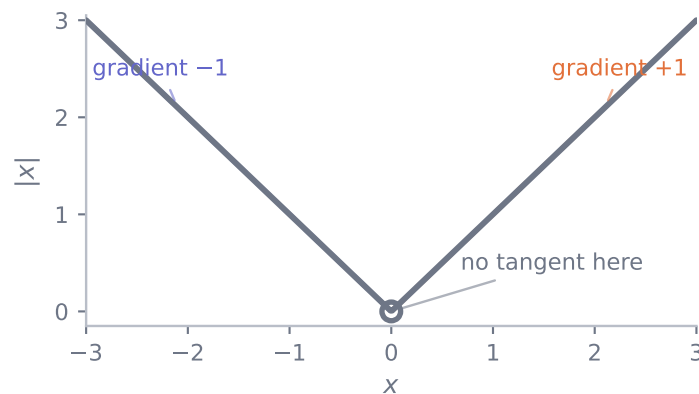


Figure 18.2: At every point except the origin, $|x|$ has an unambiguous tangent. At $x = 0$ the two sides disagree — one arrives with gradient -1 , the other with $+1$ — so no single line fits, and there is no derivative there.

The function is perfectly well behaved — continuous, defined everywhere, easy to draw. It simply has no single tangent at that point. Continuity is not enough for differentiability, and that gap matters later: some loss functions used in practice have exactly this kind of kink.

18.3.5 The tangent line

Once you have $f'(a)$, you have the straight line that best matches f near a :

$$f(x) \approx f(a) + f'(a)(x - a).$$

This is the **local linear approximation**. It says: start from the value at a , then move along at the rate you measured there. It is exact at $x = a$ and gets worse as you move away, and it is the single most useful approximation in applied mathematics — M6 is essentially this idea in many variables at once.

18.4 Worked example

The model above has a best value, and the derivative finds it.

Our model was $s(h) = 45 + 6h - 0.25h^2$. Differentiate from first principles.

$$\begin{aligned} \frac{s(h+t) - s(h)}{t} &= \frac{[45 + 6(h+t) - 0.25(h+t)^2] - [45 + 6h - 0.25h^2]}{t} \\ &= \frac{6t - 0.25(2ht + t^2)}{t} \\ &= 6 - 0.5h - 0.25t. \end{aligned}$$

(Here t is the interval width, since h is already in use.) Letting t shrink gives

$$s'(h) = 6 - 0.5h.$$

Now read it:

Table 18.1: Sensitivity of the model at four values of h

h	$s'(h)$	what it says
4	+4.0	an extra hour is worth about 4 marks
10	+1.0	still improving, but four times more slowly
12	0.0	the model has stopped improving at this instant
20	-4.0	the model now predicts the score <i>falling</i>

Two things are worth saying Two things are worth saying about that last row, and both are habits rather than calculations.

First, solving $s'(h) = 0$ is how you find *candidates* for a best value without plotting anything. A zero derivative marks a place where the curve is momentarily flat; whether that is a maximum, a minimum or neither is a further question, and so is whether the true best value sits at the edge of the allowed range instead. Here the model is a downward parabola, so $h = 12$ really is its peak. Doing that properly is the whole of M5.

Note also what $s'(12) = 0$ does *not* say. It is an instantaneous rate, not the value of one more hour: $s(13) - s(12) = -0.25$ marks. The distinction between a rate at a point and a

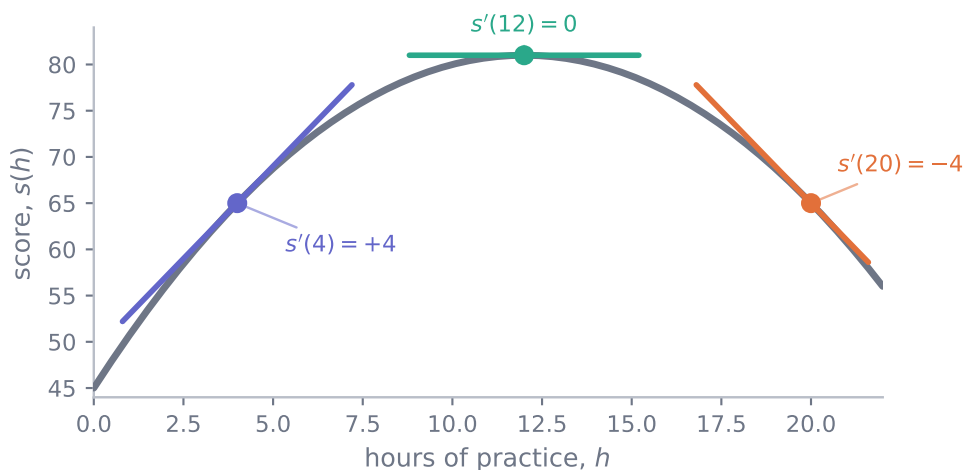


Figure 18.3: The same curve with its tangent drawn at three places. The gradient falls steadily, reaches zero at $h = 12$ where the curve is momentarily flat, and is negative beyond it.

change over an interval is the same one this unit opened with, and it does not go away once you have a derivative.

Second, the model predicts that practising for 20 hours is worse than practising for 12. Nothing in the data forces that; it is an artefact of having chosen a downward parabola, which must eventually come down. A derivative tells you what your *model* does. Whether the model deserves to be believed out there is a separate question, and one that [S11](#) takes seriously.

18.5 Try it yourself

Exercise 1 From first principles, differentiate $f(x) = 5x - 2$.

💡 Show the solution

$$\frac{f(x+h) - f(x)}{h} = \frac{[5(x+h) - 2] - [5x - 2]}{h} = \frac{5h}{h} = 5.$$

There is no h left to shrink, so $f'(x) = 5$ for every x . That is what you should expect: the graph is a straight line of gradient 5, and its tangent is itself.

Exercise 2 From first principles, differentiate $f(x) = x^2 - 3x$.

💡 Show the solution

$$\begin{aligned} \frac{f(x+h) - f(x)}{h} &= \frac{[(x+h)^2 - 3(x+h)] - [x^2 - 3x]}{h} \\ &= \frac{2xh + h^2 - 3h}{h} \\ &= 2x + h - 3. \end{aligned}$$

As h shrinks this settles on $f'(x) = 2x - 3$.

Sanity check: at $x = 1.5$ the derivative is zero, and $x = 1.5$ is exactly halfway between the roots $x = 0$ and $x = 3$ — the bottom of the parabola.

Exercise 3 A university models the cost of running a module as a smooth function $C(n)$ of the number of students enrolled, in pounds. (Enrolment is a whole number, so C is a continuous stand-in for something discrete — a very common move, and one worth noticing.) You are told that $C'(100) = 42$.

Write down, in one sentence and with units, what that means. Then say what it does **not** tell you.

💡 Show the solution

It means that at an enrolment of 100 students, the modelled cost is rising at **£42 per additional student**. Using the local linear approximation, the 101st student is expected to add roughly £42 — *roughly*, because the derivative describes the rate at exactly $n = 100$, while the 101st student is a whole unit away.

It does not tell you the total cost $C(100)$: a rate says nothing about a level. It also says nothing about the 200th student, because the rate can change — indeed it usually does, which is the entire reason we need derivatives rather than a single number.

18.6 Check it in Python

You will rarely differentiate by hand in practice, but you should always be able to check that you did it right. The tool is a **centred difference**: evaluate a small step either side and take the gradient between them.

```
import numpy as np

def s(h):
    return 45 + 6*h - 0.25*h**2

def s_prime(h):          # our by-hand answer
    return 6 - 0.5*h

def centred_difference(f, x, step):
    return (f(x + step) - f(x - step)) / (2 * step)

for h in [4, 10, 12, 20]:
    approx = centred_difference(s, h, 1e-6)
    exact = s_prime(h)
    print(f"h={h:>3}    numerical {approx:+.6f}    by hand {exact:+.6f}"
          ↪ agree: {np.isclose(approx, exact)}")
```

h= 4	numerical +4.000000	by hand +4.000000	agree: True
h= 10	numerical +1.000000	by hand +1.000000	agree: True
h= 12	numerical +0.000000	by hand +0.000000	agree: True
h= 20	numerical -4.000000	by hand -4.000000	agree: True

Agreement at four points is evidence, not proof. A disagreement is worth taking seriously, but it is not by itself proof that the algebra is wrong — as the next block shows, a perfectly correct derivative can disagree with a badly chosen numerical check.

One trap. It is tempting to think that a smaller step is always better. It is not:

```
for step in [1e-1, 1e-3, 1e-6, 1e-10, 1e-14]:
    approx = centred_difference(s, 4.0, step)
    print(f"step {step:<8.0e} estimate {approx:.10f}    error {abs(approx -
        ↪ 4.0):.3e}")
```

step 1e-01	estimate 4.0000000000	error 8.527e-14
step 1e-03	estimate 4.0000000000	error 4.889e-12
step 1e-06	estimate 3.9999999970	error 2.994e-09
step 1e-10	estimate 3.9999292767	error 7.072e-05
step 1e-14	estimate 4.2632564146	error 2.633e-01

The error gets *worse* as the step shrinks, and by 10^{-14} the estimate is wrong in the first decimal place. The reason is that you are asking the computer to subtract two numbers that agree to fourteen digits; almost all of the digits cancel, and what survives is mostly noise. P2 explains why computers do this.

There is a second effect that this particular model happens to hide. Our s is a parabola, and for a parabola the two sides of a centred difference cancel exactly, so the *method* contributes no error at all whatever step you choose. Everything in that table is the arithmetic, not the mathematics.

On a curve that bends more sharply the cancellation is only partial, and a large step then measures the wrong thing because the curve changes direction across the interval. In general both ends are bad and the usable range sits in the middle — around 10^{-6} when x is of a size near 1. For much larger or smaller values, scale the step to x rather than using a fixed number, and check that the answer is stable across a few step sizes before trusting it.

18.7 Common wrong turns

Where this usually goes wrong

Reading $f'(a)$ as a value of f $f(4) = 65$ marks. $f'(4) = 4$ marks per hour. Different quantities, different units. The rule is that a derivative carries the units of the output divided by the units of the input — so if the input is a plain number with no units of its own, the derivative does share the output’s units, and only then.

Reading the derivative off the height of the graph A curve can be very high and perfectly flat — high f , zero f' — or close to zero and plunging. The derivative is about *steepness*, and the two are unrelated.

Assuming every function has a derivative everywhere $|x|$ is continuous at 0 and has no derivative there. Continuity is necessary but not sufficient.

Dropping the units “The derivative is 4” is almost useless. “4 marks per hour of practice” can be checked against reality, argued with, and acted on. Carry the units through every line.

Believing the model because the calculus was correct $s'(20) = -4$ is a flawless statement about the parabola we chose. Whether practice really harms scores after 12 hours is a question about the world, and the derivative has no opinion on it.

18.8 Check yourself

Answers are at the foot of the page. Try to give a reason as well as an answer — if you cannot say *why*, treat it as wrong.

1. A function has $f(2) = 10$ and $f'(2) = -3$. Using the local linear approximation, what is your estimate of $f(2.1)$?
a. 10.3 b. 9.7 c. -3 d. The approximation cannot be used here
2. Water is draining from a tank. $V(t)$ is the volume in litres after t minutes. What are the units of $V'(t)$, and what sign would you expect?
3. True or false: if f is continuous at $x = 0$ then $f'(0)$ exists.
4. For $f(x) = x^2$, the difference quotient simplified to $2x + h$. Why is it wrong to say “set $h = 0$ in the original quotient” rather than “let h shrink”?
5. Your by-hand derivative and a centred difference disagree in the fourth decimal place. Name two quite different explanations.

Show the answers

1. **(b), 9.7.** The approximation gives $f(2) + f'(2)(0.1) = 10 - 0.3 = 9.7$. Option (a) gets the sign of the rate wrong; (c) confuses a rate with a value. Note that the question asks for an *estimate*: two facts about f at a single point cannot determine its value anywhere else. A sharply curving f could pass through $(2, 10)$ with gradient -3 and still be nowhere near 9.7 at $x = 2.1$.
2. **Litres per minute, and negative.** The tank is emptying, so volume is decreasing, so the rate of change is below zero.
3. **False.** $f(x) = |x|$ is the standard counterexample: continuous at 0 , no derivative there.
4. Because at $h = 0$ the original quotient reads $0/0$, which is not a number — you cannot divide by zero. The cancellation that produced $2x + h$ is only valid *because* $h \neq 0$. The limit describes where the values head, and deliberately never evaluates at the forbidden point.
5. Either the algebra is wrong, or the numerical check is: a step size too large (the curve bends across the interval) or too small (floating-point cancellation). Rerun the check at several step sizes before suspecting the algebra — the plot of error against step size will tell you which it is.

Where this shows up next

M4 gives you rules — power, product, chain — so you never have to do first principles again. **M5** sets $f'(x) = 0$ to find best values, which is the calculation behind fitting any model. **M6**

does all of this with several variables at once and arrives at gradient descent. **S11** uses it to fit a straight line to the cohort data, at which point the mathematics, the statistics and the Python all turn out to be the same piece of work.

19 The differentiation toolkit

Mathematics · Unit 4

M4 · Mathematics Reading: 5 pages With practice: 2 hours Assumes: M3

Why this unit is here

M3 defined the derivative and made you compute a few from first principles. That was necessary once and is unbearable as a habit.

This unit replaces it with six rules. After them you will be able to differentiate essentially any function you meet in this course in a line or two — including the ones that appear inside loss functions, which is where all of this is heading.

19.1 Before you start

You should be able to:

- state what $f'(a)$ means, in words and in units — M3;
- compute a derivative from the difference quotient for a quadratic — M3;
- work with powers, roots and logarithms — M2.

Quick check. In M3 we found from first principles that $f(x) = x^2$ has $f'(x) = 2x$. Without doing any work, what do you expect $f'(x)$ to be for $f(x) = x^3$?

💡 Show the answer

$3x^2$. If you spotted the pattern — bring the power down, then reduce it by one — you have already guessed the first and most useful rule below.
If you did not, no matter. That is what this unit is for.

19.2 The idea

Every complicated function is built from simple ones by a small number of moves: adding them, multiplying them, dividing them, and feeding one into another.

The rules below mirror those moves exactly. There is a rule for a sum, a rule for a product, a rule for a quotient, and a rule for a composition. So differentiating a complicated function is a matter of noticing **how it was built**, and then applying the matching rule.

That is the whole method, and it is worth saying plainly because students often try to memorise the rules as a list of unrelated facts. They are not a list. They are a set of instructions for taking a function apart.

So before differentiating anything, ask one question: *what is the outermost thing being done here?* Is this fundamentally a product of two things, or a quotient, or one function applied to another? Answer that, and the rule chooses itself.

19.3 The mechanics

19.3.1 The six rules

Let f and g be differentiable, and let c and n be constants.

	Rule	
Constant	$(c)' = 0$	a flat line has no slope
Power	$(x^n)' = n x^{n-1}$	any real n , wherever x^n is defined and differentiable
Sum	$(f + g)' = f' + g'$	and $(cf)' = cf'$
Product	$(fg)' = f'g + fg'$	not $f'g'$
Quotient	$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$	order matters in the numerator
Chain	$(f(g(x)))' = f'(g(x)) \cdot g'(x)$	differentiate the outside, times the inside

Two more you will need constantly, which we take as given here:

$$(e^x)' = e^x \qquad (\ln x)' = \frac{1}{x} \quad (x > 0)$$

The first is the defining property of e^x : it is the function that is its own rate of change. That is the entire reason e appears everywhere in this subject.

19.3.2 The chain rule deserves a paragraph to itself

It is the one people get wrong, and it is by far the most used — every neural network in existence is trained by applying it repeatedly.

The idea is that a composition is a two-stage process. In $(2x + 1)^5$, the inside stage turns x into $2x + 1$, and the outside stage raises that to the fifth power. If x moves a little, the inside moves 2 times as much, and the outside moves $5(\text{inside})^4$ times as much as the inside did. The two sensitivities **multiply**:

$$((2x + 1)^5)' = \underbrace{5(2x + 1)^4}_{\text{outside}} \times \underbrace{2}_{\text{inside}} = 10(2x + 1)^4$$

A reliable procedure, if you find it slippery: name the inside. Let $u = 2x + 1$. Then the function is u^5 , and

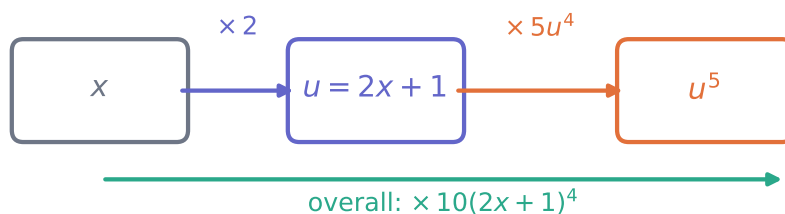


Figure 19.1: The chain rule as two stages in series. A nudge of 1 at the input becomes a nudge of 2 after the inside stage, and each unit of that becomes $5u^4$ after the outside stage. The sensitivities compound, so they multiply.

$$\frac{d}{dx}u^5 = 5u^4 \cdot \frac{du}{dx} = 5(2x + 1)^4 \cdot 2.$$

Forgetting the $\cdot du/dx$ is the single commonest error in first-year calculus.

19.3.3 Second derivatives

Differentiating f' gives f'' , the rate at which the *rate* is changing. It measures curvature: $f'' > 0$ means the slope is increasing, so the curve bends upwards. You will use it in [M5](#) to tell a minimum from a maximum.

19.4 Worked example

One function, taken apart.

$$f(x) = e^{-x}(x^3 + 2x + 2)$$

Ask the question first: what is the outermost operation? It is a **product** — e^{-x} times a polynomial. So the product rule chooses itself, and we will need the chain rule to handle e^{-x} along the way.

The two pieces and their derivatives:

$$\begin{array}{ll} u = e^{-x} & u' = e^{-x} \cdot (-1) = -e^{-x} \quad (\text{chain rule: inside is } -x) \\ v = x^3 + 2x + 2 & v' = 3x^2 + 2 \quad (\text{power and sum rules}) \end{array}$$

Now assemble with $(uv)' = u'v + uv'$:

$$\begin{aligned} f'(x) &= -e^{-x}(x^3 + 2x + 2) + e^{-x}(3x^2 + 2) \\ &= e^{-x}[-(x^3 + 2x + 2) + 3x^2 + 2] \\ &= e^{-x}(-x^3 + 3x^2 - 2x) \\ &= -e^{-x}(x^3 - 3x^2 + 2x) \\ &= -e^{-x}x(x - 1)(x - 2). \end{aligned}$$

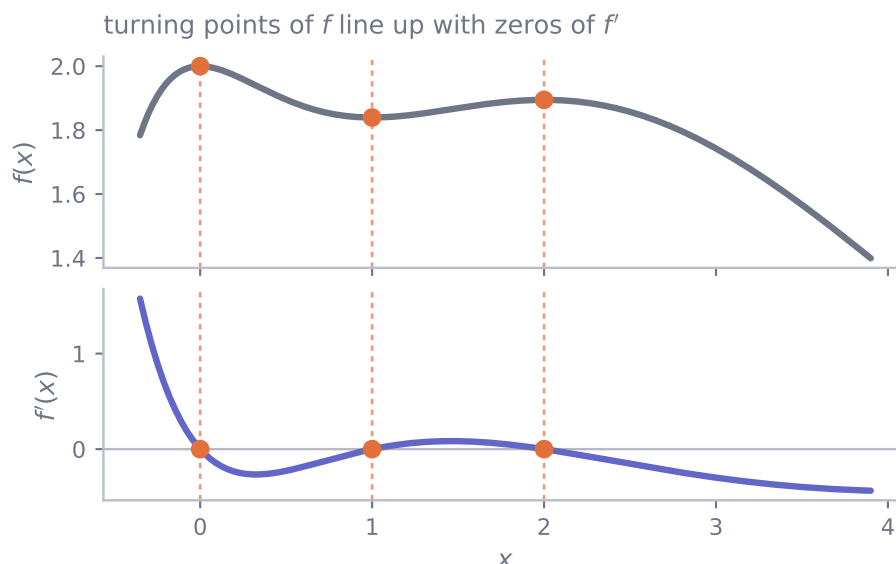


Figure 19.2: The function above its own derivative. Wherever f turns — at $x = 0, 1$ and 2 — the derivative crosses zero. That is the entire content of the next unit, visible in one picture.

That last factorisation is worth the extra line. It says at a glance that $f'(x) = 0$ exactly when x is 0, 1 or 2, because e^{-x} is never zero. Those three values are precisely what M5 will need. A derivative left as an unfactored mess hides that; a factored one hands it to you.

i And the model from M3. We laboured through $s(h) = 45 + 6h - 0.25h^2$ from first principles over most of a page. With the rules it is one line: the constant goes to 0, $6h$ goes to 6, and $-0.25h^2$ goes to $-0.5h$, giving $s'(h) = 6 - 0.5h$. Same answer, about thirty seconds.

19.5 Try it yourself

Exercise 1 Differentiate $f(x) = \frac{x}{3x+2}$.

Show the solution

Outermost operation: a quotient. With $u = x$ and $v = 3x + 2$, so $u' = 1$ and $v' = 3$:

$$f'(x) = \frac{u'v - uv'}{v^2} = \frac{1 \cdot (3x + 2) - x \cdot 3}{(3x + 2)^2} = \frac{3x + 2 - 3x}{(3x + 2)^2} = \frac{2}{(3x + 2)^2}.$$

Worth noticing: the answer is positive everywhere it is defined. Be careful how you say that, though. The function is undefined at $x = -2/3$, so its domain comes in two separate pieces, and it is increasing *on each piece* — not across the gap. Comparing $f(-1) = 1$ with $f(0) = 0$ shows why: the value drops as x rises, because the two points sit either side of the break.

Exercise 2 Differentiate $g(x) = \ln(x^2 + 1)$.

💡 Show the solution

Outermost operation: a composition — a logarithm applied to something. Let $u = x^2 + 1$, so $g = \ln u$ and $u' = 2x$:

$$g'(x) = \frac{1}{u} \cdot u' = \frac{2x}{x^2 + 1}.$$

The commonest wrong answer is $1/(x^2 + 1)$, which drops the $\cdot u'$. If you made it, that is the error to watch for from now on — it is the chain rule's signature failure.

Exercise 3 A model's squared error for one observation is $L(a) = (a - 7)^2$, where a is a parameter you get to choose. Differentiate L with respect to a , then find where $L'(a) = 0$ and say in words what that value of a is doing.

💡 Show the solution

Chain rule, with inside $u = a - 7$ and $u' = 1$:

$$L'(a) = 2(a - 7) \cdot 1 = 2(a - 7).$$

Setting $L'(a) = 0$ gives $a = 7$.

In words: $a = 7$ is the choice of parameter that makes the squared error stop changing — and since squaring makes L non-negative and $L(7) = 0$, it is the choice that makes the error as small as it can possibly be. You have just fitted a one-parameter model by calculus, which is the whole idea that [M5](#), [M6](#) and [M11](#) develop.

19.6 Check it in Python

Differentiating by hand is error-prone, and a numerical check costs two lines. This is the same centred difference from [M3](#), now used as a routine tool.

```
import numpy as np

def centred_difference(f, x, step=1e-6):
    return (f(x + step) - f(x - step)) / (2 * step)

def f(x):
    # e-x (x3 + 2x + 2)
    return np.exp(-x) * (x**3 + 2*x + 2)

def f_prime(x):
    # our by-hand answer
    return -np.exp(-x) * x * (x - 1) * (x - 2)

for x in [0.5, 1.5, 3.0]:
    numeric, symbolic = centred_difference(f, x), f_prime(x)
    print(f"x={x:<4} numeric {numeric:+.8f} by hand {symbolic:+.8f} "
          f"agree: {np.isclose(numeric, symbolic)}")
```

```
x=0.5  numeric -0.22744900  by hand -0.22744900  agree: True
```

```
x=1.5  numeric +0.08367381    by hand +0.08367381    agree: True
x=3.0  numeric -0.29872241    by hand -0.29872241    agree: True
```

Now use it on the three exercises at once — the same two lines, three times:

```
checks = [
    ("x/(3x+2)",      lambda x: x/(3*x + 2),      lambda x: 2/(3*x + 2)**2,
    ↪ 1.0),
    ("ln(x^2+1)",     lambda x: np.log(x**2 + 1), lambda x: 2*x/(x**2 + 1),
    ↪ 3.0),
    ("(a-7)^2",       lambda a: (a - 7)**2,        lambda a: 2*(a - 7),
    ↪ 4.0),
]

for name, fn, derivative, at in checks:
    numeric, symbolic = centred_difference(fn, at), derivative(at)
    print(f"{name:<12} at x={at}:  {numeric:+.6f} vs {symbolic:+.6f}    "
          f"agree: {np.isclose(numeric, symbolic)}")
```

```
x/(3x+2)      at x=1.0:  +0.080000 vs +0.080000    agree: True
ln(x^2+1)     at x=3.0:  +0.600000 vs +0.600000    agree: True
(a-7)^2       at x=4.0:  -6.000000 vs -6.000000    agree: True
```

Try deliberately breaking one. Change the $\ln(x^2+1)$ derivative to `lambda x: 1/(x**2 + 1)` — the chain-rule mistake from Exercise 2 — and watch the check fail. That is what it is for: it will not tell you the right answer, but it will tell you, in one second, that you do not yet have it.

19.7 Common wrong turns

Where this usually goes wrong

Forgetting the inside derivative $(\ln(x^2+1))'$ is $\frac{2x}{x^2+1}$, not $\frac{1}{x^2+1}$. Every chain-rule application ends in $\times u'$. If your answer does not contain a factor that came from the inside, you have dropped it.

Assuming $(fg)' = f'g'$ It is $f'g + fg'$. Test it on $f = g = x$: the product is x^2 with derivative $2x$, while $f'g' = 1$. Not the same.

Getting the quotient rule backwards The numerator is $f'g - fg'$, in that order. Swapping the terms flips the sign of every answer. If you struggle to keep it straight, rewrite f/g as $f \cdot g^{-1}$ and use the product and chain rules instead.

Applying the power rule to e^x $(e^x)' = e^x$, not xe^{x-1} . The power rule is for a *variable base and constant power*; e^x is the other way round.

Reaching for a rule before looking at the structure Ask what the outermost operation is first. Choosing the rule is most of the work; applying it is bookkeeping.

Leaving the answer unfactored $e^{-x}(-x^3 + 3x^2 - 2x)$ and $-e^{-x}x(x-1)(x-2)$ are the same function. Only the second one tells you where the derivative is zero, which is usually the reason you differentiated.

19.8 Check yourself

Answers are at the foot of the page.

1. What is the derivative of $f(x) = \sqrt{x}$?
a. $\frac{1}{2\sqrt{x}}$ b. $2\sqrt{x}$ c. $\frac{1}{2}x^{1/2}$ d. $\sqrt{x}$
2. Differentiate $y = (3x - 4)^7$.
3. A student writes $(x^2 e^x)' = 2x e^x$. What did they do wrong, and what is the correct answer?
4. Explain, in terms of *sensitivity*, why the chain rule multiplies rather than adds.
5. Your numerical check disagrees with your algebra at $x = 2$ but agrees at $x = 1$ and $x = 5$. Is that more likely to be an algebra error or a numerical one?

Show the answers

1. **(a)**, for $x > 0$. Write $\sqrt{x} = x^{1/2}$ and apply the power rule with $n = \frac{1}{2}$: $\frac{1}{2}x^{-1/2} = \frac{1}{2\sqrt{x}}$. The restriction matters — the expression is undefined at $x = 0$, where $\sqrt{x}$ is defined but has no finite slope. Option (c) is the common slip of forgetting to reduce the power.
2. $y' = 7(3x - 4)^6 \cdot 3 = 21(3x - 4)^6$. The $\cdot 3$ is the inside derivative.
3. They applied the power rule to x^2 and left e^x alone, as though it were a constant. It is not — it is a **product** of two functions of x , so: $(x^2 e^x)' = 2x e^x + x^2 e^x = x e^x (2 + x)$.
4. Because the two stages compound. If the inside responds twice as fast as the input, and the outside responds five times as fast as the inside, then the whole thing responds ten times as fast as the input — not seven.
5. There is not enough evidence to say, and that is the honest answer. Algebra is the first thing to suspect: a factorisation that divided by something, or a cancelled term that happens to vanish at $x = 2$. But a numerical check can also fail at a single point — if the function is not differentiable there, if the two evaluations nearly cancel, or if $x = 2$ sits close to a domain boundary. Test several step sizes at $x = 2$ before deciding: if the disagreement shrinks as the step changes it is numerical, and if it holds steady it is algebra.

Where this shows up next

M5 sets $f'(x) = 0$ and works out which of the resulting candidates is actually a best value — the factorised derivative from this unit's worked example is exactly what it starts from. **M6** applies the same rules one variable at a time, which is all a partial derivative is, and arrives at gradient descent. **S11** uses Exercise 3's calculation, with two parameters instead of one, to fit a line to data.

20 Finding the best value

Mathematics · Unit 5

M5 · Mathematics Reading: 4 pages With practice: 2 hours Assumes: M4

Why this unit is here

Fitting a model means choosing the parameter values that make it *least wrong*. That is an optimisation problem, and this unit is where you learn to solve one.

The central fact is small and easy to misuse: at a peak or a trough of a smooth curve, the derivative is zero. What takes care is everything around it — because a zero derivative is a *candidate*, not an answer, and the best value is sometimes nowhere near one.

20.1 Before you start

You should be able to:

- differentiate using the rules, and factorise the result — [M4](#);
- read $f'(x)$ as a rate of change — [M3](#).

Quick check. Differentiate $f(x) = x^2 - 6x + 5$ and find where $f'(x) = 0$.

 Show the answer

$f'(x) = 2x - 6$, which is zero at $x = 3$.

That is the whole method in one line. The rest of this unit is about what $x = 3$ actually *is*, and when this method misses the answer entirely.

20.2 The idea

At the top of a hill the ground is momentarily level. At the bottom of a valley it is level too. So if a smooth curve has a highest or lowest point somewhere in the middle of its range, the tangent there is horizontal — the derivative is zero.

A point where $f'(x) = 0$ is called a **stationary point**. Finding them is easy. Deciding what they are is the work:

The middle one is the warning. Its tangent is horizontal, so $f'(x) = 0$, and yet it is neither a peak nor a trough — the curve flattens and carries on in the same direction.

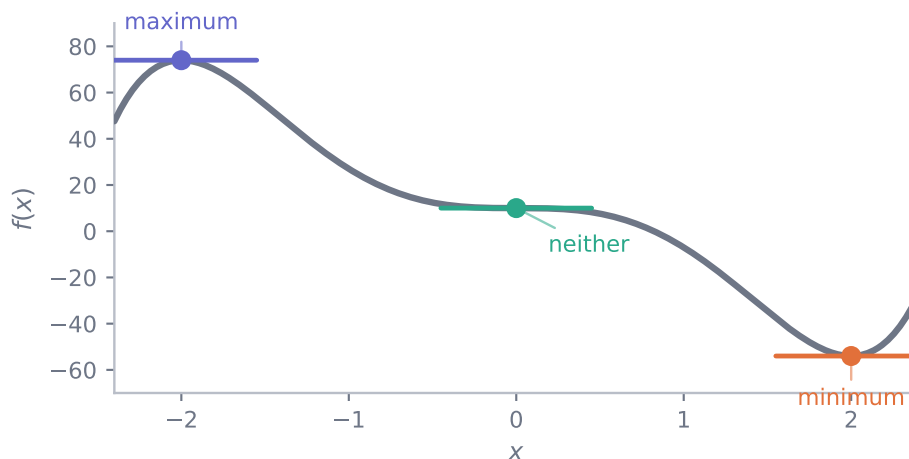


Figure 20.1: Three stationary points, all with $f'(x) = 0$. Only two are turning points, and the third is neither a maximum nor a minimum — which is why a zero derivative is a candidate rather than an answer.

20.3 The mechanics

20.3.1 The second-derivative test

f'' measures how the slope is changing, which is exactly what distinguishes a peak from a trough. If $f'(c) = 0$ and f'' is continuous near c :

$f''(c)$	curve bends	c is a
> 0	upwards	local minimum
< 0	downwards	local maximum
$= 0$	—	test says nothing

```
def f(x): return x**2 - 6*x + 5
def f1(x): return 2*x - 6
def f2(x): return 2.0

print(f"f'(3) = {f1(3)}, f''(3) = {f2(3)} > 0 → minimum")
print(f"f(3) = {f(3)}")
```

```
f'(3) = 0, f''(3) = 2.0 > 0 → minimum
f(3) = -4
```

When $f''(c) = 0$ the test is genuinely inconclusive — not “probably a minimum”. Both x^4 (a minimum) and x^3 (neither) have $f'(0) = f''(0) = 0$:

```
for name, g in [("x^4", lambda x: x**4), ("x^3", lambda x: x**3)]:
    left, centre, right = g(-0.1), g(0.0), g(0.1)
    print(f"{name}: f(-0.1)={left:+.5f} f(0)={centre:+.5f}
        ↪ f(0.1)={right:+.5f}")
```


x^4 : $f(-0.1)=+0.00010$ $f(0)=+0.00000$ $f(0.1)=+0.00010$
 x^3 : $f(-0.1)=-0.00100$ $f(0)=+0.00000$ $f(0.1)=+0.00100$

For x^4 both neighbours are above the centre — a minimum. For x^3 one is above and one below — neither. When the test fails, look at the values either side.

20.3.2 Local is not global

A **local** minimum is lower than its immediate neighbours. A **global** minimum is the lowest anywhere in the range you care about. They need not coincide, and the global one may sit at an **endpoint**, where the derivative is not zero at all:

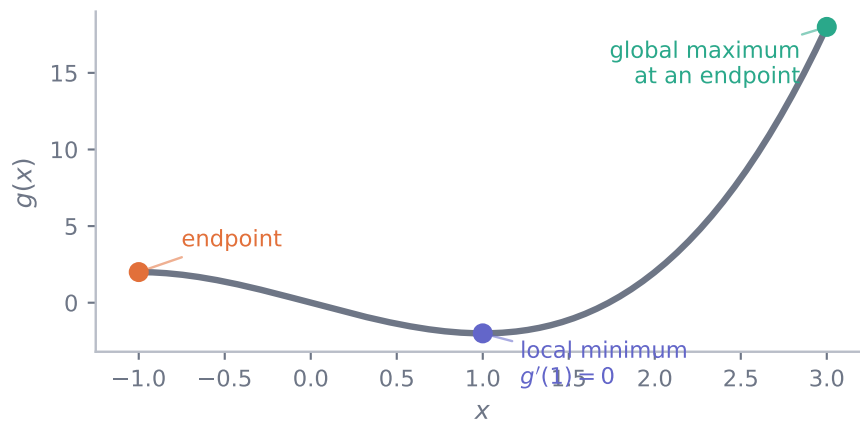


Figure 20.2: On $[-1, 3]$ the *highest* point is the right-hand endpoint, where the derivative is not zero at all. Checking stationary points alone would have missed it entirely — the only interior stationary point is the local minimum.

20.3.3 The full procedure

To find the best value of f on an interval $[a, b]$, collect **every** candidate and compare:

1. stationary points — where $f'(x) = 0$;
2. points where f' does not exist — corners, like $|x|$ at 0;
3. the endpoints a and b .

Then evaluate f at all of them and pick the largest or smallest. That is the whole method, and steps 2 and 3 are the ones people skip.

```
def g(x): return x**3 - 3*x
def g1(x): return 3*x**2 - 3

candidates = [-1.0, 1.0, 3.0]          # endpoints and the stationary
↪ point in range
for c in sorted(candidates):
    print(f" x={c:>4}   g(x)={g(c):>7.2f}   g'(x)={g1(c):>6.2f}")

best = min(candidates, key=g)
print(f"\nlowest on [-1, 3]: x={best}, g={g(best):.2f}")
```

x=-1.0	g(x)=	2.00	g'(x)=	0.00
x= 1.0	g(x)=	-2.00	g'(x)=	0.00
x= 3.0	g(x)=	18.00	g'(x)=	24.00

lowest on $[-1, 3]$: $x=1.0$, $g=-2.00$

A detail worth noticing. $g'(x) = 3x^2 - 3$ is zero at $x = \pm 1$, so *both* are stationary points — and $x = -1$ happens also to be the left endpoint, so it arrives on the list twice over. Only $x = 1$ is a genuinely interior stationary point, and it is not the answer: the largest value sits at $x = 3$, where the derivative is 24.

20.4 Worked example

Fitting a one-parameter model by calculus.

Suppose you must summarise a set of scores with a single number a , and you choose a to make the total squared error as small as possible:

$$L(a) = \sum_{i=1}^n (x_i - a)^2$$

Differentiate with respect to a , using the chain rule on each term:

$$L'(a) = \sum_{i=1}^n 2(x_i - a)(-1) = -2 \sum_{i=1}^n (x_i - a)$$

Set it to zero:

$$\sum_{i=1}^n (x_i - a) = 0 \implies \sum x_i = na \implies a = \frac{1}{n} \sum x_i = \bar{x}$$

The best single summary, in the squared-error sense, is **the mean**. That is not a definition — it is a result, and this is where it comes from.

Confirm it is a minimum rather than a maximum:

$$L''(a) = 2n > 0$$

Positive for any n , so it curves upwards everywhere: one stationary point, and it is the global minimum.

```
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
scores = cohort["final_score"].dropna()
scores = scores[scores.between(0, 100)].to_numpy()

def loss(a):
    return ((scores - a)**2).sum()
```

```

grid = np.linspace(scores.mean() - 6, scores.mean() + 6, 400)
best = grid[np.argmin([loss(a) for a in grid])]

print(f"mean of the data      {scores.mean():.4f}")
print(f"grid search minimum   {best:.4f}")
print(f"agree to 2 dp:        {np.isclose(scores.mean(), best, atol=0.05)}")

```

```

mean of the data      65.2718
grid search minimum   65.2568
agree to 2 dp:        True

```

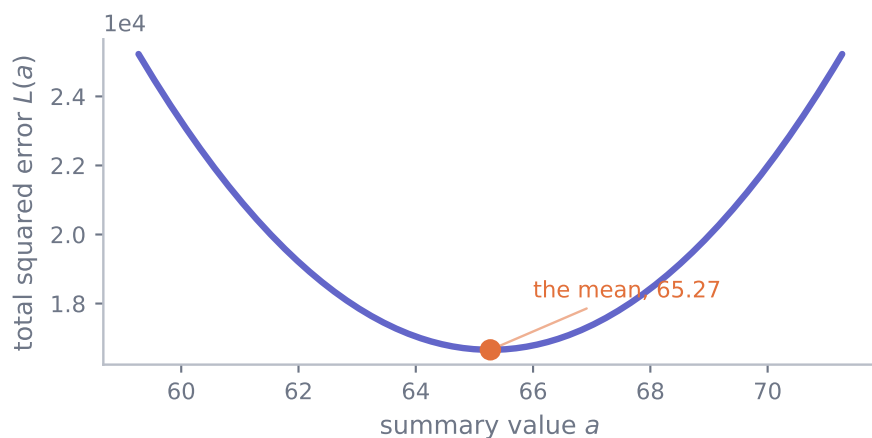


Figure 20.3: The squared-error loss as the summary value varies. One trough, at the mean — and $L'' > 0$ everywhere, so there is nowhere else it could be.

Change the loss and you change the answer. Minimising the sum of *absolute* errors gives the **median** instead — a different summary, from a different choice about what “wrong” means. The mean is not neutral; it is the answer to one specific question.

20.5 Try it yourself

Exercise 1 Find and classify the stationary points of $f(x) = x^3 - 3x^2 + 4$.

💡 Show the solution

$f'(x) = 3x^2 - 6x = 3x(x - 2)$, so stationary points at $x = 0$ and $x = 2$.
 $f''(x) = 6x - 6$:

- $f''(0) = -6 < 0 \rightarrow$ local **maximum**, $f(0) = 4$.
- $f''(2) = +6 > 0 \rightarrow$ local **minimum**, $f(2) = 0$.

Note neither is global over all of $\mathbb{R}$: x^3 dominates, so $f \rightarrow +\infty$ and $-\infty$ at the ends. “Global” needs an interval to be global over.

Exercise 2 Find the global minimum of $f(x) = x^2$ on $[1, 4]$. Why is the method of setting $f'(x) = 0$ misleading here?

💡 Show the solution

$f'(x) = 2x = 0$ gives $x = 0$ — which is **not in** $[1, 4]$, so it is not a candidate at all. The candidates are the endpoints: $f(1) = 1$ and $f(4) = 16$. The global minimum is 1, at $x = 1$. The trap is reporting $x = 0$ because the algebra produced it. Always check that a stationary point lies inside the interval before treating it as a candidate.

Exercise 3 $f(x) = x^4$ has $f'(0) = 0$ and $f''(0) = 0$. What kind of point is $x = 0$, and how would you decide without the second-derivative test?

💡 Show the solution

A **minimum** — in fact the global one, since $x^4 \geq 0$ with equality only at zero. Two ways to decide when the test is silent:

- **Evaluate either side.** $f(-0.1) = f(0.1) = 0.0001 > 0 = f(0)$, so the centre is lower than both neighbours.
- **Check the sign of f' .** $f'(x) = 4x^3$ is negative for $x < 0$ and positive for $x > 0$: the function falls then rises, which is a minimum.

The second is more reliable, because it examines a whole neighbourhood rather than two arbitrary points.

20.6 Check it in Python

Check a stationary point numerically before trusting your algebra:

```
def f(x): return x**3 - 3*x**2 + 4
def f1(x): return 3*x**2 - 6*x

def centred_difference(g, x, h=1e-6):
    return (g(x + h) - g(x - h)) / (2*h)

for c in (0.0, 2.0):
    print(f"x={c}: symbolic f'={f1(c):+.6f}    numerical
      ↪ {centred_difference(f, c):+.6f}")
```

```
x=0.0: symbolic f'="+0.000000    numerical +0.000000
x=2.0: symbolic f'="+0.000000    numerical +0.000000
```

Check the classification by looking either side — the method that still works when the second-derivative test is silent:

```
def classify(g, c, h=1e-3):
    left, centre, right = g(c - h), g(c), g(c + h)
    if left > centre < right:
        return "minimum"
    if left < centre > right:
        return "maximum"
    return "neither"

for name, g, c in [("x^3-3x^2+4 at 0", f, 0.0), ("x^3-3x^2+4 at 2", f, 2.0),
                  ("x^4 at 0", lambda x: x**4, 0.0),
                  ("x^3 at 0", lambda x: x**3, 0.0)]:
    print(f" {name:<20} {classify(g, c)}")
```

```
x^3-3x^2+4 at 0      maximum
x^3-3x^2+4 at 2      minimum
x^4 at 0             minimum
x^3 at 0             neither
```

Both cases the second-derivative test cannot resolve are settled in one line.

And search the whole interval, so an endpoint answer cannot be missed:

```
def g(x): return x**3 - 3*x
grid = np.linspace(-1, 3, 2001)
values = g(grid)
print(f"lowest {values.min():.3f} at x={grid[values.argmin():.3f}")
print(f"highest {values.max():.3f} at x={grid[values.argmax():.3f}")
```

```
lowest -2.000 at x=1.000
highest 18.000 at x=3.000
```

A grid search is not a substitute for the calculus — it cannot prove anything and it misses features between grid points. But it is an excellent way to find out that your algebra disagrees with the function.

20.7 Common wrong turns

Where this usually goes wrong

Treating $f'(c) = 0$ as proof of a minimum It is a candidate. It may be a maximum, or neither, and the global answer may be somewhere else entirely.

Forgetting the endpoints On a closed interval the best value is very often at an end, where the derivative is not zero. It never appears in the algebra; you must add it.

Using a stationary point outside the interval $f'(x) = 0$ at $x = 0$ is irrelevant if you are working on $[1, 4]$. Check membership before treating it as a candidate.

Reading $f''(c) = 0$ as “probably a minimum” It says nothing at all. Look at the values or the sign of f' either side.

Missing points where f' does not exist $|x|$ has its minimum at a corner, where there is no derivative to set to zero. Non-differentiable points are candidates too.

Forgetting that the loss defines the answer Minimising squared error gives the mean; minimising absolute error gives the median. The summary you get is a consequence of the loss you chose.

20.8 Check yourself

Answers are at the foot of the page.

1. $f'(c) = 0$ and $f''(c) < 0$. What is c ?
 - a. local minimum
 - b. local maximum
 - c. neither
 - d. cannot tell
2. Name the three kinds of candidate for a global extremum on $[a, b]$.
3. Why does $f''(c) = 0$ leave the question open? Give two functions that show it.
4. Minimising $\sum (x_i - a)^2$ gives the mean. What does minimising $\sum |x_i - a|$ give?
5. You are told the minimum of $f(x) = x^2$ on $[2, 5]$ is at $x = 0$. What went wrong?

 Show the answers

1. **(b) a local maximum.** Negative second derivative means the curve bends downwards, so the stationary point is a peak.
2. Stationary points inside the interval; points where f' does not exist; and the two endpoints.
3. Because both a genuine extremum and a non-extremum can have zero second derivative. x^4 has $f'(0) = f''(0) = 0$ and a minimum at 0; x^3 has the same derivatives at 0 and no extremum there.
4. The **median**. Different loss, different summary — which is the point: the mean is the answer to one specific question about what “wrong” means.
5. $x = 0$ is not in $[2, 5]$, so it was never a candidate. The minimum is at the endpoint $x = 2$, giving $f(2) = 4$.

Where this shows up next

M6 does all of this with several parameters at once, and arrives at gradient descent — the method used when there is no formula for the answer. **M11** minimises exactly this unit’s squared-error loss with many parameters, and gets a closed-form solution. **S11** applies it to fit a line to the cohort data.

21 Several variables and the gradient

Mathematics · Unit 6

M6 · Mathematics Reading: 6 pages With practice: 2.5 hours Assumes: M5

Why this unit is here

Real models have more than one number to choose. A straight line has two; a logistic regression has one per column of your data; a language model has billions. Fitting any of them means adjusting all of those numbers *together* until the model is least wrong.

This unit is where the one-variable calculus of [M3–M5](#) becomes usable on real models. The tool is the **gradient**: a list of sensitivities, one per parameter, that answers “if I nudge this one, does the model get better or worse?” for every parameter at once. Following it downhill is **gradient descent**, and that loop — with no changes beyond the number of parameters — is how essentially every model in modern machine learning is trained.

21.1 Before you start

You should be able to:

- differentiate with the rules, especially the chain rule — [M4](#);
- set a derivative to zero to find a candidate best value, and remember it is only a candidate — [M5](#);
- read a derivative as a sensitivity carrying units — [M3](#);
- do arithmetic on a whole NumPy array at once — [P6](#).

Quick check. Differentiate $f(x) = (3x + 2c)^2$ with respect to x , treating c as a constant.

 Show the answer

Chain rule: outside is $()^2$, inside is $3x + 2c$ whose derivative is 3. So

$$f'(x) = 2(3x + 2c) \cdot 3 = 6(3x + 2c).$$

The $2c$ came along for the ride and contributed nothing of its own, because it does not move when x moves. Holding one symbol still while differentiating with respect to another is the entire mechanical content of this unit; you have just done it.

21.2 The idea

In M5 you summarised the cohort’s scores with a single number a , chose it to minimise $L(a) = \sum (x_i - a)^2$, and found the mean. One knob, one derivative, one equation.

Now use practice hours as well, and predict a score with the straight line $\hat{s} = a + bh$. Two knobs. The total squared error is a function of both,

$$L(a, b) = \frac{1}{n} \sum_{i=1}^n (s_i - a - bh_i)^2,$$

and asking for “the derivative of L ” no longer has a single answer — for the same reason that in M3 asking for “the rate of change” had no answer until you said *where*. Here you must say *in which direction*.

Picture yourself on a hillside in thick fog. Your height depends on two coordinates and you want to get down, but you cannot see the valley. What you *can* do is feel the ground: step a little east and notice whether you rose or fell, step a little north and do the same. Those two numbers are the **partial derivatives**, and the fact this whole unit rests on is that on a smooth surface they are enough — from the slope due east and the slope due north you can work out the slope in *every* direction, including the steepest. (The word *smooth* is doing real work: a surface can have both partial derivatives at a point and still misbehave in a diagonal direction there. Everything in these notes is well behaved.)

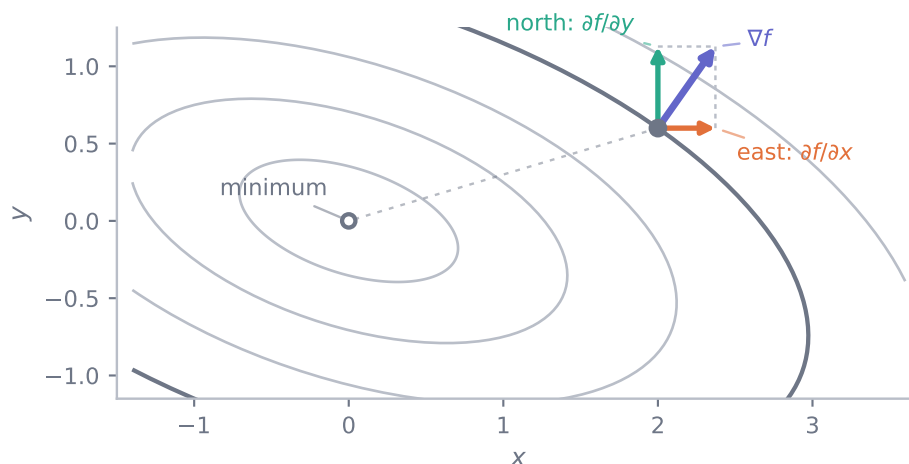


Figure 21.1: Contours of a two-variable function: each ring is a set of points of equal height, and the minimum is the bullseye. At the marked point, the slope due east and the slope due north combine into one arrow — the gradient — which points straight uphill and crosses the contour at right angles. Note that its reverse does *not* point at the minimum.

That arrow is the gradient. Everything else in this unit is either how to compute it or what to do once you have it.

21.3 The mechanics

21.3.1 Partial derivatives

The **partial derivative** of f with respect to x is the ordinary derivative of M3, taken with every other input frozen:

$$\frac{\partial f}{\partial x}(a, b) = \lim_{t \rightarrow 0} \frac{f(a + t, b) - f(a, b)}{t}.$$

Only the first slot moves: this is the difference quotient of M3, with b sitting there unchanged throughout.

So the working rule is: **to differentiate with respect to x , treat every other variable as a constant** and use the rules of M4 exactly as they stand. No new rules. For $f(x, y) = x^2y + 3y - 5$:

$$\frac{\partial f}{\partial x} = 2xy, \quad \frac{\partial f}{\partial y} = x^2 + 3.$$

In the first, y is a constant multiplier and $3y - 5$ is a constant, so it differentiates to zero. In the second, x^2 is the constant multiplier of y .

The curly ∂ is not decoration. It says: other variables exist, and they are being held still. Writing d here would claim there is only one input. Where space is short the same two quantities are written f_x and f_y .

21.3.2 The gradient

Collect the partials into one object, in a fixed order:

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right).$$

Read ∇f as “grad f ”. It is not a number — it is a list with one entry per input, and it has a direction as well as a size. (M8 gives that kind of object a proper name and an arithmetic.) Three facts make it useful.

It gives you every direction. Move a small step $(\Delta x, \Delta y)$ from (a, b) and the height changes by roughly

$$f(a + \Delta x, b + \Delta y) \approx f(a, b) + \frac{\partial f}{\partial x} \Delta x + \frac{\partial f}{\partial y} \Delta y.$$

This is the local linear approximation of M3 with one term per input, and it is the reason two numbers suffice: the two partials, weighted by how far you step in each direction, give the slope along any step at all.

It points uphill, steepest. Among all directions of the same small length, the one that increases f fastest is the direction of ∇f itself. So $-\nabla f$ is the direction of steepest *decrease* — the way down. That claim is checked numerically at the end of this unit rather than asserted.

Its length says how steep. $\|\nabla f\| = \sqrt{f_x^2 + f_y^2}$ is the slope in that steepest direction. Where the ground is level, it is zero.

21.3.3 Stationary points, and a new way to be fooled

A stationary point now needs *all* the partials to vanish at once: $\nabla f = (0, 0)$ means $\partial f/\partial x = 0$ **and** $\partial f/\partial y = 0$ at the same point. That is a system of simultaneous equations rather than a single one, and as in M5 a solution is a *candidate*, not an answer. With two variables there is a new way for a candidate to disappoint, with no one-variable analogue: a **saddle**, which is a minimum along one direction and a maximum along another.

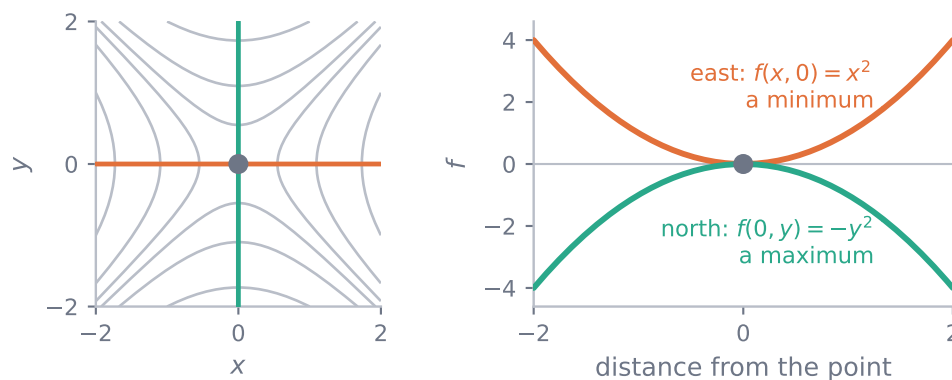


Figure 21.2: The saddle $f(x, y) = x^2 - y^2$ at the origin. Left: its contours, with the flat point marked. Right: the same point, seen along two directions — walk east and it is the bottom of a valley; walk north and it is the top of a ridge. Both partials are zero, and it is neither a maximum nor a minimum.

At the origin, $\partial f/\partial x = 2x = 0$ and $\partial f/\partial y = -2y = 0$. Both partials vanish and the point is neither best nor worst. In two variables a saddle is a curiosity; with a thousand parameters it is the *typical* stationary point, and recognising one is a large part of why training a big model is hard.

21.3.4 Gradient descent

For the models in this unit you could solve $\nabla L = 0$ by algebra, and M11 does exactly that. For most models you cannot: there is no formula, only a function you can evaluate and differentiate. So walk downhill instead. Repeat, from any starting guess:

$$a \leftarrow a - \eta \frac{\partial L}{\partial a}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

Both updates use the partials computed at the *old* values, and are applied together. The number η (“eta”) is the **learning rate** or step size. It is yours to choose, and the choice matters:

η	what happens
too small	every step is safe and the loss crawls down; you run out of patience
about right	the loss falls quickly and flattens out

η	what happens
too large	each step overshoots the bottom and lands further up the far side; the loss grows without limit

There is no universally correct η , because the safe range depends on how sharply the surface curves — and, as the next section shows, on something as mundane as the units your columns are measured in.

21.4 Worked example

Fit a line to the cohort by gradient descent.

We model final score from hours of practice, on the rows where the score is usable. That is the 238-row filter used throughout these notes.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h, s = usable["hours_practice"].to_numpy(), usable["final_score"].to_numpy()
print(f"{len(usable)} rows; hours run from {h.min():.1f} to {h.max():.1f}")
```

238 rows; hours run from 1.0 to 36.2

The loss is $L(a, b) = \frac{1}{n} \sum (s_i - a - bh_i)^2$. Differentiate it with respect to each parameter in turn, treating the other as a constant and using the chain rule on the bracket. Write $r_i = s_i - a - bh_i$ for the residual:

$$\frac{\partial L}{\partial a} = \frac{1}{n} \sum 2r_i \cdot (-1) = -\frac{2}{n} \sum r_i, \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum 2r_i \cdot (-h_i) = -\frac{2}{n} \sum r_i h_i.$$

Now descend. First attempt, straight from the raw columns, starting at $a = b = 0$ with $\eta = 0.01$:

```
def gradient(a, b, x, y):
    r = y - a - b*x
    return -2*r.mean(), -2*(r*x).mean()

a, b = 0.0, 0.0
for step in range(1, 7):
    ga, gb = gradient(a, b, h, s)
    a, b = a - 0.01*ga, b - 0.01*gb
    print(f"step {step}    a = {a:>12.4g}    b = {b:>12.4g}    loss = {(s - a
        ↪ - b*h)**2).mean():>10.3g}")
```

step 1	a =	1.305	b =	21.12	loss =	1.1e+05
step 2	a =	-4.352	b =	-99.95	loss =	3.58e+06
step 3	a =	29.87	b =	593.4	loss =	1.18e+08
step 4	a =	-164.3	b =	-3378	loss =	3.86e+09
step 5	a =	949.9	b =	1.937e+04	loss =	1.27e+11
step 6	a =	-5431	b =	-1.109e+05	loss =	4.15e+12

The loss multiplies by about 32 every step. Nothing is wrong with the calculus — the partials are right and the direction is genuinely downhill. The step *length* is wrong. Practice hours run up to 36, so $\partial L/\partial b$ carries a factor of h and comes out about sixteen times the size of $\partial L/\partial a$ at the start, as the first line of output shows. The surface is a long narrow valley, and $\eta = 0.01$ is safe for a and lethal for b .

The standard fix is to put the input on a scale of its own size — subtract its mean and divide by its spread, so it runs over a few units either side of zero:

```
z = (h - h.mean()) / h.std()

a, b = 45.0, 10.0          # any starting guess will do
for step in range(1, 41):
    ga, gb = gradient(a, b, z, s)
    a, b = a - 0.3*ga, b - 0.3*gb
    if step in (1, 2, 3, 5, 40):
        print(f"step {step:>2}    a = {a:8.4f}    b = {b:8.4f}    loss = {(s -
            ↪ a - b*z)**2).mean():9.4f}")
```

step 1	a =	57.1631	b =	2.8270	loss =	154.7954
step 2	a =	62.0284	b =	-0.0423	loss =	80.3551
step 3	a =	63.9745	b =	-1.1899	loss =	68.4446
step 5	a =	65.0643	b =	-1.8327	loss =	66.2340
step 40	a =	65.2718	b =	-1.9551	loss =	66.1759

Same loss, same two partial derivatives, same descent loop — only the scale of one column changed, and $\eta = 0.3$ is now comfortable where 0.01 was fatal. (The two runs start from different points, which makes no difference here: the surface has one minimum and every starting point reaches it.)

Finally, translate back. A slope measured per standard deviation of practice becomes a slope per hour by dividing by that standard deviation, and the intercept shifts to match:

```
slope, intercept = b / h.std(), a - b*h.mean()/h.std()
exact_slope, exact_intercept = np.polyfit(h, s, 1)
print(f"gradient descent    intercept {intercept:8.4f}    slope {slope:+.4f}
    ↪ marks per hour")
print(f"closed form        intercept {exact_intercept:8.4f}    slope
    ↪ {exact_slope:+.4f} marks per hour")
print(f"agree to 4 dp: {np.allclose([slope, intercept], [exact_slope,
    ↪ exact_intercept], atol=1e-4)}")
```

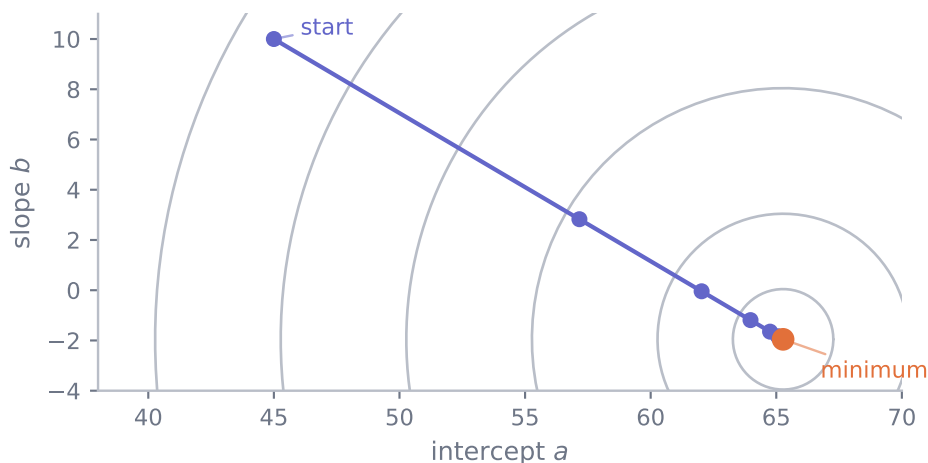


Figure 21.3: The descent above, drawn on the contours of $L(a, b)$, whose minimum is at $a = 65.27$, $b = -1.96$. On this scaling the contours are circles, so downhill points straight at the minimum and the path is a straight run. The steps shorten as it arrives, because the gradient itself shrinks towards zero — the method slows down on its own.

```
gradient descent    intercept    69.2261    slope -0.2408 marks per hour
closed form        intercept    69.2261    slope -0.2408 marks per hour
agree to 4 dp: True
```

Two readings, and the second matters more. The mathematics: an iterative walk downhill has landed on the answer that algebra gives directly, which is the evidence that the walk is doing what it claims. The data: the fitted slope is about -0.24 marks per hour of practice — the model says practising *more* goes with scoring *less*. That is a real feature of this dataset and not an arithmetic slip, and it reverses completely once you look within each background group. [S12](#) is about why.

21.5 Try it yourself

Exercise 1 For $f(x, y) = x^2y - 4y^3 + 7$, find both partial derivatives and evaluate ∇f at $(2, 1)$. Which direction from that point increases f fastest?

💡 Show the solution

Holding y fixed, $-4y^3 + 7$ is a constant and y is a multiplier:

$$\frac{\partial f}{\partial x} = 2xy.$$

Holding x fixed, x^2 is a multiplier and 7 is a constant:

$$\frac{\partial f}{\partial y} = x^2 - 12y^2.$$

At $(2, 1)$: $\nabla f = (2 \cdot 2 \cdot 1, 4 - 12) = (4, -8)$.

The fastest increase is in the direction of ∇f itself: for every 4 you move in x , move -8 in y . The tempting error is to answer “ $(4, -8)$ ” to the question *how much does f*

increase — that is a direction, not an amount. The steepest slope there is its length, $\sqrt{4^2 + 8^2} \approx 8.94$.

Exercise 2 Find the stationary point of $g(x, y) = x^2 + xy + y^2 - 3y$.

💡 Show the solution

Both partials must vanish at once:

$$\frac{\partial g}{\partial x} = 2x + y = 0, \quad \frac{\partial g}{\partial y} = x + 2y - 3 = 0.$$

From the first, $y = -2x$. Substituting: $x - 4x - 3 = 0$, so $x = -1$ and $y = 2$. The only stationary point is $(-1, 2)$, where $g = -3$.

It is a minimum, though nothing above proves that. Checking is easy: $g(-1.1, 2) = g(-0.9, 2) = -2.99$ and $g(-1, 1.9) = g(-1, 2.1) = -2.99$, all higher than -3 . Four neighbours are not a proof either — a saddle can hide in a diagonal direction that neither axis explores — but they are the right first move, and they are what the code in the next block automates.

Exercise 3 Take $L(a, b) = (a - 3)^2 + 4(b + 1)^2$, starting at $(0, 0)$ with $\eta = 0.1$. Do one step of gradient descent by hand. Then work out how large η can be before the b updates start to grow instead of shrink.

💡 Show the solution

$\nabla L = (2(a - 3), 8(b + 1))$, so at $(0, 0)$ it is $(-6, 8)$. One step gives

$$(a, b) \leftarrow (0, 0) - 0.1 \cdot (-6, 8) = (0.6, -0.8).$$

Both moved towards the minimum at $(3, -1)$.

For the second part, follow one parameter through a general step. The b update is $b \leftarrow b - 8\eta(b + 1)$, so the distance to the target $b = -1$ obeys

$$(b + 1) \leftarrow (1 - 8\eta)(b + 1).$$

The distance shrinks only when $|1 - 8\eta| < 1$, that is when $\eta < 0.25$. The same argument for a gives $\eta < 1$. So b sets the limit, and it is the more sharply curved direction — the coefficient 4 in front of $(b + 1)^2$ is exactly what makes it so. This is the worked example in miniature: one badly scaled parameter caps the step size for all of them.

21.6 Check it in Python

First, that the partials are right. A centred difference in one coordinate at a time is the two-variable version of the check from M3:

```
def f(x, y):  
    return x**2 * y - 4*y**3 + 7  
  
def grad_by_hand(x, y):
```

```

    return np.array([2*x*y, x**2 - 12*y**2])

def grad_numeric(g, x, y, step=1e-6):
    return np.array([(g(x + step, y) - g(x - step, y)) / (2*step),
                     (g(x, y + step) - g(x, y - step)) / (2*step)])

print("by hand ", grad_by_hand(2.0, 1.0))
print("numeric ", grad_numeric(f, 2.0, 1.0))
print("agree: ", np.allclose(grad_by_hand(2.0, 1.0), grad_numeric(f, 2.0,
↪ 1.0)))

```

```

by hand  [ 4. -8.]
numeric  [ 4. -8.]
agree:    True

```

Break it on purpose and watch the check fire — a check you have never seen fail is a check you have no reason to trust:

```

def grad_wrong(x, y):
    return np.array([2*x*y, x**2 - 12*y])      # forgot to square the y

for point in [(2.0, 1.0), (2.0, 3.0)]:
    print(f"at {point}:  broken {grad_wrong(*point)}    numeric
↪   {grad_numeric(f, *point)}"
          f"    agree: {np.allclose(grad_wrong(*point), grad_numeric(f,
↪   *point))}")

```

```

at (2.0, 1.0):  broken [ 4. -8.]    numeric [ 4. -8.]    agree: True
at (2.0, 3.0):  broken [ 12. -32.]   numeric [ 12.          -104.00000001]    agree: False

```

The first row is the more instructive one. At $y = 1$ the mistake is invisible, because y and y^2 agree there — a check run only at that point would have passed a wrong derivative. Test where nothing is special.

Then the unit's main claim: that the gradient points in the direction of steepest increase. Nothing so far has demonstrated this. Try every direction and see which wins:

```

angles = np.linspace(0, 2*np.pi, 3601)      # every 0.1 degree
step = 1e-6
rise = np.array([(f(2 + step*np.cos(t), 1 + step*np.sin(t)) - f(2, 1)) /
↪   step
                  for t in angles])

best = angles[rise.argmax()]
gx, gy = grad_by_hand(2.0, 1.0)
predicted = np.arctan2(gy, gx) % (2*np.pi)

print(f"steepest direction found by search : {np.degrees(best):8.3f}
↪   degrees")

```

```

print(f"direction of the gradient      : {np.degrees(predicted):8.3f}
↪ degrees")
print(f"steepest rise found           : {rise.max():.4f}")
print(f"length of the gradient        : {np.hypot(gx, gy):.4f}")

```

```

steepest direction found by search : 296.600 degrees
direction of the gradient          : 296.565 degrees
steepest rise found                : 8.9443
length of the gradient             : 8.9443

```

Both pairs agree — the two angles to within the 0.1° resolution of the search, and the steepest rise to four decimal places against the gradient’s length. That is the claim, tested rather than asserted.

Finally, that descent converges — and when it does not. Everything above is about one point; this is about the loop:

```

def descend(eta, steps=200):
    a, b = 0.0, 0.0
    for _ in range(steps):
        ga, gb = gradient(a, b, z, s)
        a, b = a - eta*ga, b - eta*gb
    return a, b, ((s - a - b*z)**2).mean()

for eta in (0.5, 0.3, 0.05, 1.05):
    a_e, b_e, loss = descend(eta)
    verdict = "converged" if np.isfinite(loss) and loss < 67 else "FAILED"
    print(f"eta={eta:<5} a={a_e:>12.4g} b={b_e:>12.4g} loss={loss:>12.4g}
    ↪ {verdict}")

```

eta=0.5	a=	65.27	b=	-1.955	loss=	66.18	converged
eta=0.3	a=	65.27	b=	-1.955	loss=	66.18	converged
eta=0.05	a=	65.27	b=	-1.955	loss=	66.18	converged
eta=1.05	a=	-1.24e+10	b=	3.713e+08	loss=	1.538e+20	FAILED

Three converge to the same place from the same start and the fourth runs away, so “gradient descent finds the minimum” is a claim about the method *and* the step size together. Note also that $\eta = 0.5$ lands on the answer in a single step: on this scaling the surface is a perfectly circular bowl, and half the gradient is exactly the distance to the bottom. That is luck arranged by the standardising — on any real loss surface it will not happen.

21.7 Common wrong turns

Where this usually goes wrong

Treating ∇f as a number It has one component per input. “The gradient is 8.94” confuses the gradient with its length, and loses the only part you need in order to take a step — which way to go.

Letting the frozen variable thaw In $\partial(x^2y)/\partial x = 2xy$, the y is a constant multiplier. It is not a function of x , so no product rule and no chain rule apply to it. If you find yourself differentiating y with respect to x , you have stopped taking a partial derivative.

Reading $\nabla f = 0$ as “minimum” It is a candidate, exactly as in M5, plus one new possibility: a saddle, which is a minimum along one direction and a maximum along another. In many dimensions saddles outnumber genuine minima.

Expecting $-\nabla f$ to point at the minimum It points down the steepest slope *here*. On an elongated bowl that is across the valley rather than along it, which is why descent zigzags and why scaling the inputs helps so much. The gradient is local information and knows nothing about where the bottom is.

Updating the parameters one at a time Compute all the partials at the current point, *then* apply all the updates. If you overwrite a and then use the new a to compute $\partial L/\partial b$, you are no longer stepping along the gradient. It often still converges, which is what makes the bug expensive: nothing visibly breaks.

Blaming the calculus when the loss explodes A diverging loss almost always means the step size is too large for the curvature — very often because one column is measured in units hundreds of times bigger than another. Check the scaling before you check the algebra.

21.8 Check yourself

Answers are at the foot of the page. Try to give a reason as well as an answer.

1. For $f(x, y) = 3x^2y$, what is $\partial f/\partial y$ at $(2, 5)$?
a. 12 b. 60 c. $30x$ d. 0
2. At a point P , both partial derivatives of f are zero; walking east from P the ground rises both ways, walking north it falls both ways. What is P ?
3. A loss L is measured in marks² and the parameter a in marks. What are the units of $\partial L/\partial a$, and what does that make the units of η ?
4. Gradient descent on your model diverges. Name two changes that could fix it without touching a single derivative.
5. Your update loop sets $\mathbf{a} = \mathbf{a} - \text{eta} \cdot \text{grad_a}$, then computes grad_b from the new $\mathbf{a}$, then sets $\mathbf{b} = \mathbf{b} - \text{eta} \cdot \text{grad_b}$. Why is this not gradient descent, and why is the bug hard to notice?

Show the answers

1. **(a) 12.** Holding x fixed, $f = (3x^2)y$ is a constant times y , so $\partial f/\partial y = 3x^2 = 12$, and the $y = 5$ is never used. Option (b) is $\partial f/\partial x = 6xy$, the other partial — the most common slip. Option (c) is the right idea left un-evaluated; (d) treats *both* variables as constants.
2. A **saddle**. East is a minimum, north is a maximum, and the point is neither — so $\nabla f = 0$ has told you nothing about whether it is a good place to be.
3. **Marks** — marks² divided by marks. Since $\eta \cdot \partial L/\partial a$ is subtracted from a , it must also be in marks, so here η is a plain number. That is a coincidence of this

particular loss, not a general fact: in general η carries units of (parameter)² per unit of loss, which is why it is not a free-floating knob and why a rate that works on one dataset can be catastrophic on the same data rescaled.

4. Reduce η ; and standardise the input columns so that no parameter's curvature is enormously larger than another's. Both change the step, not the mathematics. (A third: start from a different point — occasionally decisive, but it will not rescue a step size that is simply too big.)
5. Gradient descent steps along ∇L evaluated at **one** point. Mixing an updated a with an old b means the two components come from two different points, so the step is along no gradient at all. It is hard to notice because the mixed step is usually still downhill, so the loss still falls and the fit still looks fine — it just is not the algorithm you wrote down, and it will diverge under conditions the real one survives.

Where this shows up next

M11 minimises this unit's loss with many parameters and finds that, for a linear model, $\nabla L = 0$ can be solved exactly — no descent, no learning rate. **M12** explains the long narrow valley properly: its shape is set by directions in the data, and that is what standardising was fixing. **S11** fits this same line to the cohort and asks the question this unit deliberately did not: whether to believe it. **S12** takes up the negative slope, and why it turns positive inside every background group.

22 Accumulation and area

Mathematics · Unit 7

M7 · Mathematics Reading: 6 pages With practice: 2 hours Assumes: M4

Why this unit is here

A derivative answers *how fast is this changing?* This unit answers the opposite question: *how much has accumulated?* Given a rate, what is the total; given a curve, what is the area underneath it.

You need this for one reason above all others. In the Statistics track a continuous quantity is described by a **density**, and a density on its own is not a probability — you get a probability by taking the area under it between two values. Every statement of the form “about 43% of the cohort scored between 60 and 70” is an area, and from S5 onwards you will be reading them constantly. The other half of the unit is the fact that makes areas computable at all: accumulation and differentiation are inverse operations.

22.1 Before you start

You should be able to:

- differentiate with the rules — M4;
- read a derivative as a rate carrying units — M3;
- work with e^x and $\ln x$ — M2;
- add up a NumPy array with `.sum()` — P7.

Quick check. Differentiate $F(x) = \frac{1}{3}x^3 - 2x$.

💡 Show the answer

$$F'(x) = x^2 - 2.$$

Now read that backwards: you have just found a function whose derivative is $x^2 - 2$. Going in that direction — from a rate back to the thing that accumulated it — is what this unit is about, and by the end of it you will see that $\frac{1}{3}x^3 - 2x$ is exactly what computes areas under $x^2 - 2$.

22.2 The idea

Start with a rate that does not change. If water runs into a tank at a steady 5 litres per minute for 12 minutes, the total is $5 \times 12 = 60$ litres. Draw the rate against time and that total is the area of a rectangle: height times width.

Now let the rate vary, and the trick is to pretend for a moment that it does not. Chop the 12 minutes into short strips. Across one short strip the rate barely changes, so treat it as constant, and that strip contributes about (rate) $\times$ (width) — a rectangle. Add the rectangles up and you have an estimate of the total. Take narrower strips and the estimate improves.

Here is that estimate for the score model of M3, $s(h) = 45 + 6h - 0.25h^2$, over $0 \leq h \leq 12$, using six strips and then twenty-four:

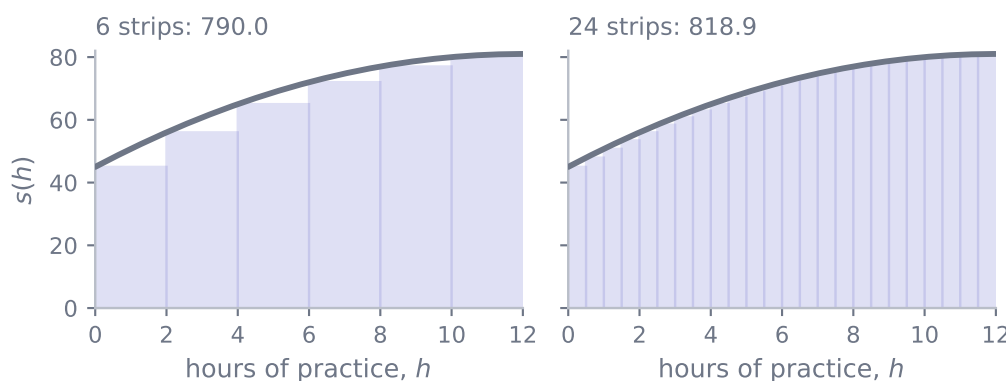


Figure 22.1: The same area estimated with six strips and with twenty-four. Each rectangle takes the height of the curve at its left edge, so every one of them undershoots on a rising curve; the visible gap between the rectangles and the curve is exactly the error, and it shrinks as the strips narrow.

```
def s(h):  
    return 45 + 6*h - 0.25*h**2  
  
import numpy as np  
for n in [6, 24, 100, 1000, 100_000]:  
    left_edges = np.linspace(0, 12, n + 1)[: -1]  
    print(f"{n:>7} strips    estimate {s(left_edges).sum() * 12/n:12.6f}")
```

6 strips	estimate	790.000000
24 strips	estimate	818.875000
100 strips	estimate	825.832800
1000 strips	estimate	827.783928
100000 strips	estimate	827.997840

The estimates settle on 828, in the same way that the difference quotients of M3 settled on 4. There the interval shrank towards a point; here the strips shrink towards zero width and there are ever more of them. The value they settle on is the **integral** of s from 0 to 12, written

$$\int_0^{12} s(h) dh = 828.$$

Two readings, as with the derivative. **Geometrically** it is the area between the curve and the horizontal axis, between those two limits. **Practically** it is a total accumulated from a rate: here, marks-per-student summed over an interval of hours, so the answer carries units of marks $\times$ hours. An integral always carries the units of the thing being integrated multiplied by the units of the variable you integrate over.

22.3 The mechanics

22.3.1 Notation

$$\int_a^b f(x) dx$$

The $\int$ is an elongated S, for *sum*. The dx is the width of a strip, and it also tells you which variable you are accumulating over — which matters as soon as there is more than one symbol in play. a and b are the **limits**. This is a *definite* integral: limits attached, and the answer is a number.

Two properties fall straight out of the picture:

- **Area below the axis counts as negative.** A strip where f is negative has negative height, so it subtracts. The integral is a *signed* area, and it can be zero for a function that is nowhere zero.
- **Swapping the limits flips the sign**, so $\int_b^a f = -\int_a^b f$. Strips of negative width.

22.3.2 The fundamental theorem

Adding up ever more rectangles is a terrible way to get an exact answer. The theorem that rescues it is the most useful result in this half of the subject.

Fix the left limit and let the right one move. Define the **accumulation function**

$$A(x) = \int_a^x f(t) dt,$$

the area collected so far when you have got as far as x . (The variable inside had to be renamed: t runs across the strips while x marks where you stopped. They cannot both be x .)

Now nudge x by a small amount h . The extra area is a sliver of width h and height about $f(x)$, so $A(x+h) - A(x) \approx f(x)h$, and therefore

$$A'(x) = \lim_{h \rightarrow 0} \frac{A(x+h) - A(x)}{h} = f(x).$$

The sliver argument needs f to be continuous at x : if f jumps there, the sliver has no single height and A has a corner instead of a tangent. Every f in this unit is continuous, and the result is stated for that case.

Accumulating and differentiating undo each other. That is the fundamental theorem of calculus, and its practical form is: if you can find any F with $F' = f$, then

$$\int_a^b f(x) dx = F(b) - F(a),$$

usually written $[F(x)]_a^b$. Such an F is called an **antiderivative** of f .

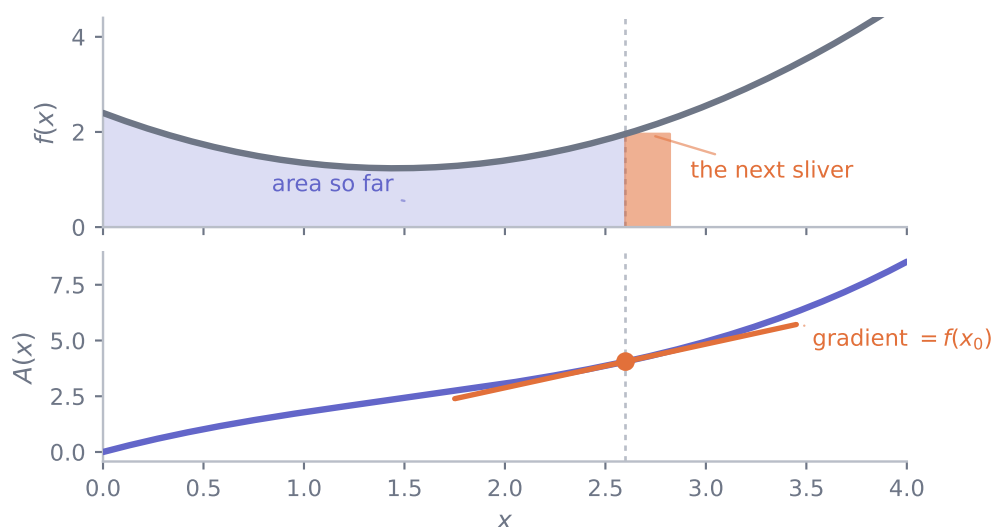


Figure 22.2: The same function twice. Above: f , with the area accumulated up to x_0 shaded and the next thin sliver picked out — its area is about $f(x_0)$ times its width. Below: the accumulated area $A(x)$ as a function of where you stop. The gradient of A at x_0 is the *height* of f at x_0 , which is what the fundamental theorem says.

22.3.3 The antiderivative table

Every line of it is a line of M4's table read from right to left.

$f(x)$	an antiderivative $F(x)$	
c	cx	check: $(cx)' = c$
$x^n, n \neq -1$	$\frac{x^{n+1}}{n+1}$	raise the power, divide by the new one
$\frac{1}{x}$	$\ln x $	the case the power rule cannot do
e^x	e^x	its own antiderivative, as it is its own derivative

Sums and constant multiples pass through unchanged: $\int(f + cg) = \int f + c \int g$. There is nothing so convenient for products or quotients — the chain rule and product rule run backwards into methods (substitution, integration by parts) that these notes do not need.

Antiderivatives are not unique: $F(x) + C$ works for any constant C , because a constant differentiates to zero. Hence the $+C$ you may remember. For a *definite* integral it cancels — $(F(b) + C) - (F(a) + C)$ — so you may drop it, and this unit does.

22.3.4 Where this stops working

Plenty of ordinary functions have no antiderivative expressible in the usual symbols. The one that matters most is e^{-x^2} , whose area is the whole business of the normal distribution in [S6](#). No amount of cleverness will produce a formula for it. Such integrals are computed numerically, by a better-organised version of the strips above, and the last section of this unit does exactly that.

22.4 Worked example

Part 1: an average, from an integral. Over the first 12 hours of practice, what score does the model of [M3](#) predict *on average*? Not the average of a few sampled points — the average over the whole continuous range, which is the accumulated total divided by the width:

$$\text{average} = \frac{1}{b-a} \int_a^b f(x) dx.$$

An antiderivative of $s(h) = 45 + 6h - 0.25h^2$, term by term from the table:

$$S(h) = 45h + 3h^2 - \frac{h^3}{12}.$$

Evaluate at both limits:

$$\int_0^{12} s(h) dh = \left[45h + 3h^2 - \frac{1}{12}h^3 \right]_0^{12} = (540 + 432 - 144) - 0 = 828,$$

which is the number the strips were converging on. The average is $828/12 = 69.0$ marks — and note the units working: marks $\times$ hours, divided by hours, gives marks.

Part 2: a proportion, from an area. Now the cohort itself. A histogram drawn with `density=True` has bar *heights* in **proportion of the cohort per mark**, chosen so that the total **area** of the bars is exactly 1. An area under it is then a proportion.

```
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
scores = cohort[cohort["final_score"].between(0,
↪ 100)]["final_score"].to_numpy()

edges = np.arange(42.5, 100.1, 2.5)
heights, edges = np.histogram(scores, bins=edges, density=True)
widths = np.diff(edges)
```

```

within = (edges[:-1] >= 60) & (edges[:-1] < 70)
print(f"{len(scores)} scores, {len(widths)} bars {widths[0]} marks wide")
print(f"total area under the bars {(heights * widths).sum():.6f}")
print(f"area of the bars from 60-70 {(heights[within] *
↪ widths[within]).sum():.6f}")
print(f"proportion scoring 60-70 {((scores >= 60) & (scores <
↪ 70)).mean():.6f}")

```

```

238 scores, 23 bars 2.5 marks wide
total area under the bars 1.000000
area of the bars from 60-70 0.432773
proportion scoring 60-70 0.432773

```

The last two numbers are equal, and that is the whole point: the area under a density between two values *is* the proportion of the data between them. The tallest bar here is only about 0.054, and reading that as a count or a probability is the standard mistake: it is a proportion *per mark*, and nothing stops such a height rising above 1. Express the same scores as fractions of full marks rather than in marks and the horizontal axis compresses by a factor of 100, so every height multiplies by 100.

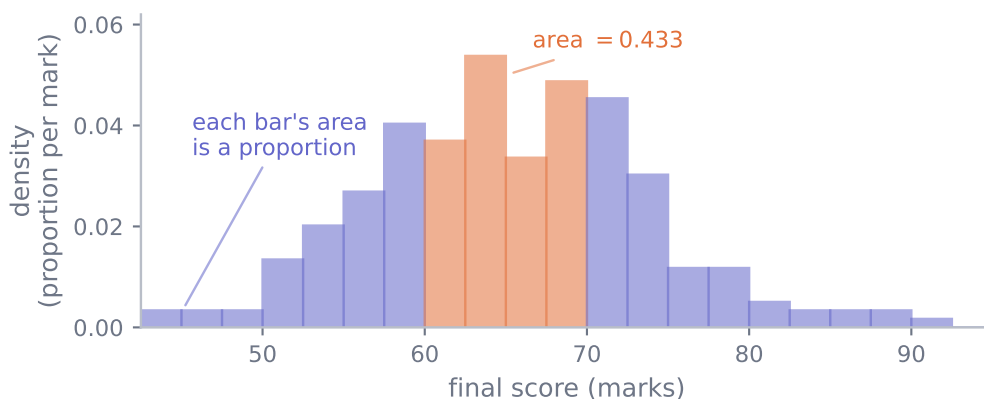


Figure 22.3: Final scores as a density: bar heights are proportions of the cohort per mark, and every bar's area is the proportion of the cohort in that range. The shaded bars are the range 60 to 70, whose total area is 0.433 — which is exactly the fraction of the 238 students who scored there.

22.5 Try it yourself

Exercise 1 Evaluate $\int_1^3 (2x + 1) dx$, then check the answer without calculus.

💡 Show the solution

An antiderivative is $F(x) = x^2 + x$, so

$$\int_1^3 (2x + 1) dx = [x^2 + x]_1^3 = (9 + 3) - (1 + 1) = 10.$$

The check: $y = 2x + 1$ is a straight line, so the region under it between $x = 1$ and $x = 3$ is a trapezium with parallel sides $f(1) = 3$ and $f(3) = 7$ and width 2. Its area is $\frac{1}{2}(3 + 7) \times 2 = 10$.

Do this whenever the shape is one you know. Geometry and the antiderivative are computing the same thing, and disagreement means one of them is wrong.

Exercise 2 Evaluate $\int_0^2 (x - 1) dx$. Then explain the answer.

💡 Show the solution

$$\int_0^2 (x - 1) dx = \left[\frac{1}{2}x^2 - x \right]_0^2 = (2 - 2) - 0 = 0.$$

Zero, although $x - 1$ is zero only at the single point $x = 1$. Between 0 and 1 the function is negative and contributes $-\frac{1}{2}$; between 1 and 2 it is positive and contributes $+\frac{1}{2}$; they cancel exactly.

The tempting reading is “the area under the curve is zero”, which is false: the region between the line and the axis has area 1. The integral is a *signed* area. If you want the geometric area, integrate $|x - 1|$, or split at the crossing point and add the pieces without their signs.

Exercise 3 Water flows into a tank at a rate $r(t)$ litres per minute, where t is minutes since you started watching. You are told that $\int_0^{30} r(t) dt = 450$.

Say in one sentence, with units, what that means — and then say what it does **not** tell you.

💡 Show the solution

450 litres flowed in during the first 30 minutes. The units come out of the integral itself: litres per minute, multiplied by minutes.

What it does not tell you:

- **the rate at any particular moment.** The average was 15 litres per minute, but the flow could have been steady, or a torrent for two minutes and a trickle after. Totals do not determine rates, exactly as in [M3](#) a rate did not determine a total.
- **how much is in the tank.** The integral measures what flowed *in* over that window. Without the starting volume — and without knowing whether anything drained out — you cannot get a level from it.

This is the mirror image of Exercise 3 in [M3](#), and worth holding side by side with it: a rate says nothing about a level, and a total says nothing about an instant.

22.6 Check it in Python

The claim to test is the fundamental theorem: that evaluating an antiderivative at the two limits gives the same number as adding up strips. Two ways of adding up strips, in fact —

the crude left-edge rectangles from earlier, and the **trapezoid rule**, which joins consecutive points with a straight line instead of a flat one, so each strip follows the slope of the curve rather than ignoring it.

```
def left_rectangles(f, a, b, n):
    edges = np.linspace(a, b, n + 1)
    return f(edges[:-1]).sum() * (b - a)/n

def trapezoid(f, a, b, n):
    y = f(np.linspace(a, b, n + 1))
    return (y[0]/2 + y[1:-1].sum() + y[-1]/2) * (b - a)/n

def S(h):
    # our antiderivative, by hand
    return 45*h + 3*h**2 - h**3/12

exact = S(12) - S(0)
print(f"antiderivative: S(12) - S(0) = {exact}")
for n in (4, 16, 64, 1024):
    print(f"  n={n:>5}    rectangles {left_rectangles(s, 0, 12, n):11.5f}"
          f"    trapezoid {trapezoid(s, 0, 12, n):11.5f}")
```

```
antiderivative: S(12) - S(0) = 828.0
n=      4    rectangles    769.50000    trapezoid    823.50000
n=     16    rectangles    814.21875    trapezoid    827.71875
n=     64    rectangles    824.60742    trapezoid    827.98242
n=    1024    rectangles    827.78899    trapezoid    827.99993
```

Both converge on 828, and the trapezoid rule with 64 strips is already closer than the rectangles with 1024 — a sixteenth of the work for a tenth of the error. The method matters as much as the number of strips, which is the lesson of M3's numerical derivative arriving from the other direction.

Now the integral that has no antiderivative. Over the whole real line, $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$, a fact you cannot get to by any manipulation of the table above:

```
gauss = lambda x: np.exp(-x**2)

for limit, n in [(3, 1_000), (6, 100), (6, 1_000), (6, 100_000)]:
    estimate = trapezoid(gauss, -limit, limit, n)
    print(f"over [-{limit}, {limit}] with n={n:>7}    {estimate:.10f}"
          f"    matches sqrt(pi): {np.isclose(estimate, np.sqrt(np.pi))}")
```

```
over [-3, 3] with n=    1000    1.7724146921    matches sqrt(pi): False
over [-6, 6] with n=     100    1.7724538509    matches sqrt(pi): True
over [-6, 6] with n=    1000    1.7724538509    matches sqrt(pi): True
over [-6, 6] with n= 100000    1.7724538509    matches sqrt(pi): True
```

The first line is the one to study. It uses a thousand strips — ten times as many as the line below it, which lands on the right answer — and it is still wrong in the fifth decimal

place, and adding strips will never fix it. The error is not in the step size but in the *interval*: cutting the curve off at ± 3 throws away real area in the tails. Two different things can go wrong with a numerical integral, and they need opposite responses.

22.7 Common wrong turns

Where this usually goes wrong

Confusing the integral with the integrand $r(t)$ is a rate in litres per minute; $\int_0^{30} r(t) dt$ is a total in litres. They are different quantities in different units, exactly as f and f' are in M3.

Assuming an integral cannot be negative It is a signed area. $\int_0^2 (x-1) dx = 0$ even though the region between the line and the axis plainly has area 1.

Reading a density as a probability A density has units — here, proportion of the cohort per mark — and its height can be any positive number at all, including numbers far above 1. Only its *area* is a probability, which is why this unit is a prerequisite for S5.

Losing the constant of integration where it matters It cancels in a definite integral and you should drop it there. It does not cancel when you are reconstructing a quantity from its rate: knowing the flow into a tank gives you the change in level, and you still need one measured level to pin down C .

Expecting every function to have an antiderivative e^{-x^2} has none in elementary terms, and it is the single most important function in statistics. Failing to find a formula is often not a failure.

Blaming the strip count for every numerical error Too few strips and too narrow an interval look alike — both give an answer that is a bit wrong. Only the first is cured by more strips. Change one thing at a time and watch which one moves the answer.

22.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

- What is $\int_0^2 3 dx$?
a. 3 b. 6 c. 0 d. $3x$
- $\int_1^e \frac{1}{x} dx$ — evaluate it, and say which line of the antiderivative table you used and why the power rule is no help.
- A density curve for waiting times, measured in minutes, reaches a height of 1.8 at its peak. Is that a mistake? What would the height become if the same waiting times were measured in hours?
- F and G are both antiderivatives of the same f , and $F \neq G$. What can you say about $F - G$, and why does the difference not affect $\int_a^b f$?
- Your trapezoid estimate of $\int_{-3}^3 e^{-x^2} dx$ sticks at 1.77241 no matter how many strips you use, while $\sqrt{\pi} = 1.77245$. What is wrong, and what fixes it?

💡 Show the answers

1. **(b) 6.** The region is a rectangle of height 3 and width 2. By the table, $[3x]_0^2 = 6 - 0$. Option (a) forgets the width; (d) forgets that a definite integral is a number, not a function — the limits have been substituted in and x is gone.
2. **1.** $\int \frac{1}{x} dx = \ln|x|$, so the answer is $\ln e - \ln 1 = 1 - 0 = 1$. The power rule would want $\frac{1}{x} = x^{-1}$ to become $\frac{x^0}{0}$, which divides by zero. That single excluded case is the reason logarithms appear in the table at all.
3. **Not a mistake.** A density is a *proportion per unit of the measurement* — here, proportion of waiting times per minute — and nothing bounds its height — only its total area is fixed at
 1. Re-expressed in hours the horizontal axis compresses by a factor of 60, so heights multiply by 60 to keep the area at 1: the peak becomes about 108.
4. $F - G$ is a **constant**. Both have derivative f , so $(F - G)' = 0$, and only a constant has zero derivative everywhere on an interval. It does not affect the definite integral because it is added at b and subtracted at a , cancelling.
5. The **interval**, not the strip count. Cutting off at ± 3 discards the tails of e^{-x^2} , and that missing area is a fixed amount which no refinement of the strips can recover — it is why the estimate stops improving rather than converging slowly. Widening to ± 6 fixes it. The diagnostic habit worth taking away: if refining the mesh stops helping, the error is in the problem you set up, not in how finely you chopped it.

Where this shows up next

S5 is the unit this one exists for: a continuous random variable is described by a density, and every probability it gives you is an area computed by integration. **S6** meets e^{-x^2} properly, as the normal distribution, and lives with the fact that its integral has no formula. **S7** and **S8** read areas in the tails of a distribution to say how far an estimate might be from the truth.

23 Data as vectors

Mathematics · Unit 8

M8 · Mathematics Reading: 5 pages With practice: 1.5 hours Assumes: M1

Why this unit is here

A dataset is a grid of numbers, and so far you have treated it one number at a time. This unit treats a whole row or a whole column as a **single object** — a vector — with an arithmetic of its own.

That change of view pays for itself immediately. The distance between two students becomes the length of a vector. The correlation between two variables turns out to be the cosine of the angle between two vectors, which is why it can never leave the range -1 to 1 . The gradient of [M6](#) was a vector all along. And every unit from here to the end of the Mathematics track — matrices, solving $A\mathbf{x} = \mathbf{b}$, least squares, the SVD — is built on the two operations defined below.

23.1 Before you start

You should be able to:

- read subscript and $\sum$ notation — [M1](#);
- find a point on a pair of axes — [M1](#);
- make and index a NumPy array — [P6](#);
- use `.sum()`, and recall that `*` and `@` are different — [P7](#).

Quick check. How far is the point $(3, 4)$ from the origin?

 Show the answer

$$\sqrt{3^2 + 4^2} = \sqrt{25} = 5.$$

Pythagoras. Everything this unit says about length in 238 dimensions is that same calculation with more terms under the square root, and nothing else.

23.2 The idea

Take two students from the cohort and record two numbers for each — hours of practice and final score:

	hours	score
S0001	12.9	83.6
S0002	22.8	56.6

Each row is a pair of numbers, so each is a point on a pair of axes. Draw an arrow to it from the origin and you have a **vector**: an ordered list of numbers, which you can also think of as a position, or as a step of a certain size in a certain direction.

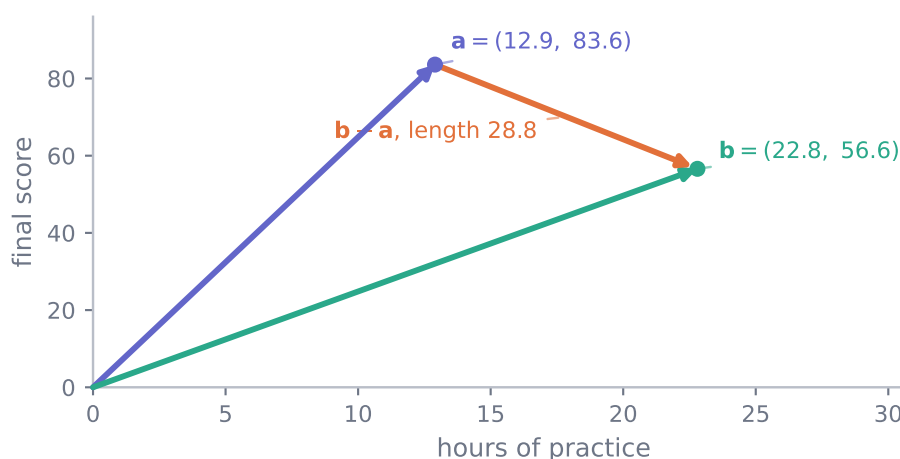


Figure 23.1: Two students as vectors, and the step from one to the other. The difference vector $\mathbf{b} - \mathbf{a}$ is the arrow joining their tips, and its length, 28.8, is the distance between the two students. The two axes are on different scales — hours span 34, marks span 96 — so the arrows’ apparent directions are not the real angles. That distortion is not a drawing error but the thing Exercise 2 is about: geometry on raw data depends on the units each column happens to use.

Now the move that matters. A student’s row is a vector; so is a *column*. The 238 usable final scores in the cohort form one vector with 238 components, and the 238 practice hours form another. You cannot draw those, and you do not need to: the arithmetic below is written with $\sum$ and works for any number of components at all. The picture is a two-dimensional aid to intuition. The formulas are the thing.

23.3 The mechanics

23.3.1 Notation

A vector is an ordered list of n real numbers, written in bold: $\mathbf{v} = (v_1, v_2, \dots, v_n)$. The set of all of them is written $\mathbb{R}^n$, and n is the **dimension**. A student’s three measurements live in $\mathbb{R}^3$; a column of 238 scores lives in $\mathbb{R}^{238}$.

Note what n counts. It is the number of *components in this vector*, not the number of variables in the dataset. A column vector of 238 scores has dimension 238, however few columns the table has.

23.3.2 Adding and scaling

Both work component by component:

$$\mathbf{a} + \mathbf{b} = (a_1 + b_1, \dots, a_n + b_n), \quad c\mathbf{a} = (ca_1, \dots, ca_n).$$

Both mean something on data. Adding the vector $(5, 5, \dots, 5)$ to a column of marks gives everyone five more marks. Multiplying a column of hours by 60 converts it to a column of minutes — a change of units is a scalar multiple. Vectors of *different* lengths cannot be added at all, which is a feature: it is the arithmetic refusing to combine 238 scores with 12 of something else.

23.3.3 Length and distance

The length of a vector, written $\|\mathbf{v}\|$, is Pythagoras with n terms:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2} = \sqrt{\sum_{i=1}^n v_i^2}.$$

The distance between two vectors is the length of the step between them, $\|\mathbf{a} - \mathbf{b}\|$. That single formula is what lets an algorithm ask “which two students are most alike?” — and it is worth noticing straight away that it takes the columns’ units at face value, so a variable measured in large numbers dominates it. That is the same scaling problem that sank the first attempt in [M6](#), arriving in a new place.

23.3.4 The dot product

Multiply matching components and add up the results:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \dots + a_n b_n = \sum_{i=1}^n a_i b_i.$$

The answer is a single number, not a vector. Two immediate consequences: taking the dot product of a vector with itself gives $\mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|^2$, and in NumPy this is `a @ b` — `a * b` is the componentwise product from [P7](#), a vector, and a different thing entirely.

The reason the dot product is worth having a name is that it is also a statement about *angle*:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta, \quad \text{so} \quad \cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|},$$

where θ is the angle between the two vectors. One formula, two readings — a sum of products on the left, a geometric fact on the right. The identity is checked numerically at the end of the unit rather than proved here.

Since $\cos \theta$ never leaves $[-1, 1]$, the dot product can never exceed $\|\mathbf{a}\| \|\mathbf{b}\|$ in size. And $\theta = 90^\circ$ gives $\cos \theta = 0$: two nonzero vectors are **orthogonal** — at right angles — exactly when their dot product is zero. (The zero vector has zero dot product with everything and no direction at all, so it is the one case where the angle is simply undefined.) That is the definition used in $\mathbb{R}^{238}$, where “at right angles” cannot be checked by looking.

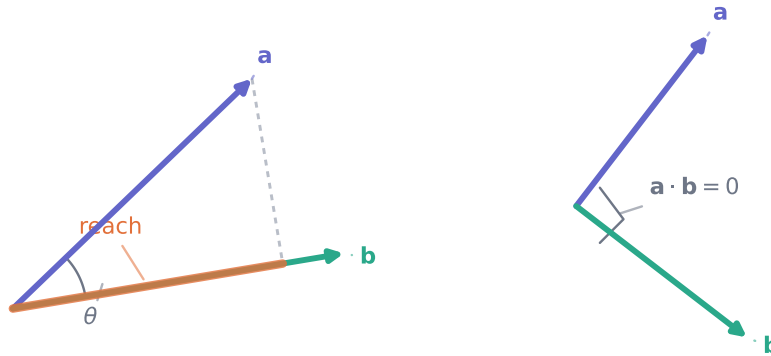


Figure 23.2: Left: the dot product as reach. Drop $\mathbf{a}$ perpendicularly onto the direction of $\mathbf{b}$; the dot product is the length of $\mathbf{b}$ times how far $\mathbf{a}$ reaches along it. Right: when the angle is a right angle the reach is zero, so the dot product is zero however long the two vectors are.

23.4 Worked example

Correlation is the cosine of an angle.

Take the two cohort columns as vectors in $\mathbb{R}^{238}$ and **centre** them: subtract each one's mean, so that each has mean zero. Write $\tilde{h}$ and $\tilde{s}$ for the centred hours and centred scores.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h, s = usable["hours_practice"].to_numpy(), usable["final_score"].to_numpy()
hc, sc = h - h.mean(), s - s.mean()

cos_theta = (hc @ sc) / (np.linalg.norm(hc) * np.linalg.norm(sc))
print(f"dimension          {hc.shape[0]}")
print(f"hc . sc              {hc @ sc:.4f}")
print(f"||hc||, ||sc||        {np.linalg.norm(hc):.4f},
↪ {np.linalg.norm(sc):.4f}")
print(f"cos(theta)            {cos_theta:.6f}")
print(f"np.corrcoef            {np.corrcoef(h, s)[0, 1]:.6f}")
print(f"theta                  {np.degrees(np.arccos(cos_theta)):.2f} degrees")
```

```
dimension          238
hc . sc            -3777.9636
||hc||, ||sc||     125.2584, 129.0720
cos(theta)         -0.233679
np.corrcoef        -0.233679
theta              103.51 degrees
```

The correlation coefficient and the cosine are not merely close; they are the same calculation. That is what correlation *is*: the cosine of the angle between two centred columns. Three things follow at once, none of which you have to memorise separately.

- It cannot exceed 1 or fall below -1 , because no cosine does.
- $+1$ means angle zero — the two centred columns point the same way, so one is a positive multiple of the other and the scatter is a perfect rising line.
- 0 means orthogonal.

Here the angle is 103.5° : slightly more than a right angle, which is the geometric version of “a weak negative relationship”.

Why the centring is not optional: without it the same calculation gives 0.876 , which would announce a strong positive relationship in data that has a negative one. Uncentred, both columns are entirely positive, so every term of the sum is positive and the angle between them is forced to be small. Centring removes that artefact — and it has a tidy vector meaning, which is Exercise 3.

The dot product is a sum over students, so you can ask which students it is made of. Each contributes $\tilde{h}_i \times \tilde{s}_i$, which is positive when the student is on the same side of average in both columns and negative when they are on opposite sides:

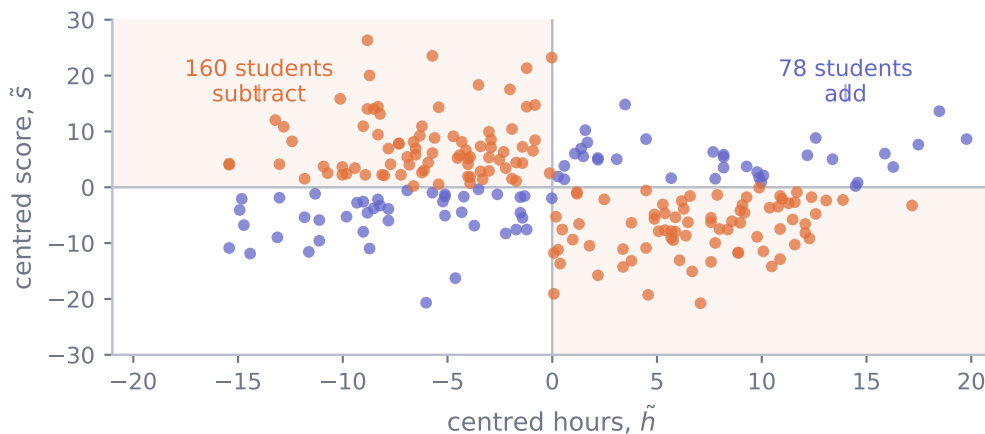


Figure 23.3: The 238 students, each plotted at its centred hours and centred score. A student in a shaded quadrant is above average in one column and below in the other, so contributes a negative term to the dot product. There are 160 of those against 78 of the others, and they outweigh them: the sum comes to -3778 , and dividing by the two lengths turns it into the correlation -0.234 .

That picture is worth holding on to, because it is what a correlation coefficient is doing under the arithmetic: counting students for and against, weighted by how far from average they are. It also shows how little separates the two groups here — 160 against 78 is not a landslide, and -0.234 is not a strong number. S12 shows that even this weak negative reverses inside every background group.

23.5 Try it yourself

Exercise 1 For $\mathbf{a} = (3, -1, 2)$ and $\mathbf{b} = (1, 4, 0)$, find $\mathbf{a} + \mathbf{b}$, $2\mathbf{a}$, $\mathbf{a} \cdot \mathbf{b}$ and $\|\mathbf{a}\|$.

💡 Show the solution

$$\mathbf{a} + \mathbf{b} = (4, 3, 2), \quad 2\mathbf{a} = (6, -2, 4).$$

$$\mathbf{a} \cdot \mathbf{b} = (3)(1) + (-1)(4) + (2)(0) = 3 - 4 + 0 = -1.$$

$$\|\mathbf{a}\| = \sqrt{9 + 1 + 4} = \sqrt{14} \approx 3.742.$$

The one to check yourself on is the dot product: the answer is the number -1 , not the vector $(3, -4, 0)$. That vector is the componentwise product — what $\mathbf{a} * \mathbf{b}$ computes in NumPy — and summing it is the last step you must not forget.

Exercise 2 Two students are recorded as $(89.1, 12.9, 83.6)$ and $(42.4, 22.8, 56.6)$, in the order (diagnostic score, hours, final score). How far apart are they? Which of the three measurements is doing most of the work, and why is that a problem?

 Show the solution

The difference is $(89.1 - 42.4, 12.9 - 22.8, 83.6 - 56.6) = (46.7, -9.9, 27.0)$, so

$$\|\mathbf{a} - \mathbf{b}\| = \sqrt{46.7^2 + 9.9^2 + 27^2} = \sqrt{2180.89 + 98.01 + 729} = \sqrt{3007.9} \approx 54.84.$$

The diagnostic score contributes 2180.89 of that 3007.9 — about 72% — not because it matters more but because it is measured on a scale that produces bigger gaps. Hours of practice, which run from 1 to 36, can never contribute much against a variable running from 27 to 100.

This is why distances are usually computed on standardised columns. It is the same lesson as M6’s diverging descent: a formula that adds squared differences treats “one unit” as meaning the same thing in every column, and it is your job to make that true before you use it.

Exercise 3 Let $\mathbf{1} = (1, 1, \dots, 1)$ and let $\tilde{\mathbf{v}}$ be a centred vector — one whose components have mean zero. Show that $\tilde{\mathbf{v}} \cdot \mathbf{1} = 0$, and say what that means in words.

 Show the solution

$$\tilde{\mathbf{v}} \cdot \mathbf{1} = \sum_{i=1}^n \tilde{v}_i \times 1 = \sum_{i=1}^n \tilde{v}_i = n \times \text{mean}(\tilde{\mathbf{v}}) = 0.$$

In words: **a centred vector is orthogonal to the all-ones vector**. Centring is not just a convenience for making the arithmetic tidy — it is the geometric operation of removing everything the column has in common with “the same value for everybody”, leaving only the variation about that value.

That is the honest reason the worked example had to centre before measuring an angle. Correlation is meant to be about variation, and the uncentred vectors still carry their overall level, which is what dragged the answer to 0.876.

23.6 Check it in Python

The claim to test is the identity $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$, which the unit asserted without proof. Nothing in the definition of the dot product mentions an angle, so this needs checking rather than believing.

In two dimensions the angle can be measured independently, straight off the axes, with `arctan2`:

```
a2, b2 = np.array([2.6, 2.5]), np.array([3.6, 0.6])

angle_a = np.arctan2(a2[1], a2[0])          # each vector's own direction
angle_b = np.arctan2(b2[1], b2[0])
theta_geometry = abs(angle_a - angle_b)

theta_dot = np.arccos((a2 @ b2) / (np.linalg.norm(a2) * np.linalg.norm(b2)))

print(f"angle measured from the axes {np.degrees(theta_geometry):.6f}
↪ degrees")
print(f"angle from the dot product {np.degrees(theta_dot):.6f} degrees")
print(f"agree: {np.isclose(theta_geometry, theta_dot)}")
```

```
angle measured from the axes 34.414375 degrees
angle from the dot product 34.414375 degrees
agree: True
```

That works only because two dimensions can be drawn. The formula, though, has no such restriction, and the consequence worth testing in high dimensions is that $|\mathbf{a} \cdot \mathbf{b}|$ can never exceed $\|\mathbf{a}\|\|\mathbf{b}\|$ — because a cosine cannot exceed 1. Try to break it:

```
rng = np.random.default_rng(2026)
ratios = []
for _ in range(10_000):
    x, y = rng.normal(size=50), rng.normal(size=50)
    ratios.append(abs(x @ y) / (np.linalg.norm(x) * np.linalg.norm(y)))
ratios = np.array(ratios)

parallel = rng.normal(size=50)
tripled = abs(parallel @ (3*parallel)) / (np.linalg.norm(parallel) *
↪ np.linalg.norm(3*parallel))
print(f"10,000 random pairs in R^50: largest |cos| {ratios.max():.6f}"
      f" typical |cos| {np.median(ratios):.6f}")
print(f"a vector against 3x itself: |cos| {tripled:.6f}")
```

```
10,000 random pairs in R^50: largest |cos| 0.515970 typical |cos| 0.094704
a vector against 3x itself: |cos| 1.000000
```

Ten thousand attempts get nowhere near 1, and the bound is reached only when the two vectors are parallel. The typical value is worth a second look too. The code takes absolute values, so what the median says is that half of all random pairs have $|\cos \theta|$ below 0.09 — an angle within about 5° of a right angle, on one side or the other, since positive and negative are equally likely. In fifty dimensions, two directions picked at random are nearly orthogonal — a fact with no analogue in the plane, and one to remember before reading much into a small correlation.

Note also the last line: tripling a vector does not change the angle at all. **Angles are scale-free and dot products are not**, which is exactly why correlation is reported instead of the raw sum of products.

Finally, the trap that costs the most time in practice:

```
x, y = np.array([3.0, -1.0, 2.0]), np.array([1.0, 4.0, 0.0])
print("x * y =", x * y, " -> a vector, the componentwise product")
print("x @ y =", x @ y, " -> a number, the dot product")
print("(x * y).sum() =", (x * y).sum(), " -> the same number, the long way")
```

```
x * y = [ 3. -4.  0.] -> a vector, the componentwise product
x @ y = -1.0 -> a number, the dot product
(x * y).sum() = -1.0 -> the same number, the long way
```

`*` and `@` both run without error and return different kinds of thing. Nothing will tell you which one you meant.

23.7 Common wrong turns

Where this usually goes wrong

Writing `a * b` for the dot product In NumPy `*` multiplies component by component and gives a *vector*; `@` gives the single number. Both are legal, so the mistake surfaces later as a shape error somewhere unrelated, or as a silently wrong answer.

Reading a large dot product as a strong relationship The dot product grows if you simply rescale a column — measure hours in minutes and it multiplies by 60. Only after dividing by both lengths is the number comparable with anything, and that quotient is the cosine.

Correlating without centring On these two columns it turns -0.234 into $+0.876$, which is not a small error but the opposite conclusion. Uncentred columns of positive numbers all point roughly the same way.

Trusting distance across columns with different scales $\|a - b\|$ adds squared differences as though one unit meant the same thing in every column. A diagnostic score out of 100 will drown a column of hours whatever the two variables actually mean.

Confusing the dimension with the number of variables A column of 238 scores is one vector in $\mathbb{R}^{238}$. It is not “a 238-variable problem”, and n here counts rows.

Expecting the pictures to keep working The formulas hold in any dimension; two- and three-dimensional *intuition* does not always. Two directions drawn at random in $\mathbb{R}^{50}$ are nearly at right angles almost every time, as the check above measures — nothing in the plane prepares you for that.

23.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. $a = (1, 2, 2)$. What is $\|a\|$?

- a. 5 b. 3 c. $\sqrt{5}$ d. 9
- Two centred columns have $\cos \theta = 0$. What does that say about the vectors, and what does it say about the two variables?
 - You double every entry of $\mathbf{b}$. What happens to $\mathbf{a} \cdot \mathbf{b}$, and what happens to the angle between them?
 - Someone reports a correlation of 1.4. Without seeing their data, how do you know it is wrong?
 - $\mathbf{x} * \mathbf{y}$ and $\mathbf{x} @ \mathbf{y}$ both run and neither raises an error. What is different about what they return, and how would you catch the mistake?

 Show the answers

- (b)** 3. $\sqrt{1+4+4} = \sqrt{9} = 3$. Option (a) adds the components; (c) forgets to square them; (d) is $\|\mathbf{a}\|^2$, which is $\mathbf{a} \cdot \mathbf{a}$ — a related quantity, and a common slip when the square root gets dropped.
- The vectors are **orthogonal**, at right angles. For the variables it means the correlation is zero: no *linear* relationship. It does not mean the two are unrelated — a strong curved relationship can have correlation zero, which is why a scatter plot is not optional.
- The dot product **doubles**; the angle is **unchanged**. Scaling changes a vector's length, not its direction, and the two extra factors of 2 cancel in $\mathbf{a} \cdot \mathbf{b} / (\|\mathbf{a}\| \|\mathbf{b}\|)$. This is the whole reason correlation is scale-free.
- Correlation is a cosine, and no cosine exceeds 1. So the number cannot be a correlation of anything — the mistake is arithmetic, not data, and no amount of looking at the dataset will explain it.
- $\mathbf{x} * \mathbf{y}$ returns a **vector** of componentwise products; $\mathbf{x} @ \mathbf{y}$ returns a single **number**. The reliable catch is to check the shape, or to assert it: `assert np.isscalar(result) or result.shape == ()`. Left unchecked it propagates — a vector where a number was expected often broadcasts happily into something with a plausible shape and no meaning.

Where this shows up next

M9 stacks vectors into a matrix, and matrix multiplication turns out to be a grid of dot products. **M11** solves least squares by finding the vector in a subspace closest to the data — “closest” being the length defined here, and the answer being characterised by an orthogonality condition. **M12** looks for the directions in which a dataset varies most, which is a question about angles. **S3** and **S10** use correlation as a summary and worry about what it does and does not license you to say.

24 Matrices as operations on data

Mathematics · Unit 9

M9 · Mathematics Reading: 6 pages With practice: 2 hours Assumes: M8

Why this unit is here

Your dataset already is a matrix: one row per student, one column per variable — 240 by 7 as the cohort file sits on disk. So the first thing a matrix can be is a table, and you knew that.

The useful idea is the second one. A matrix is also an **operation** — feed it a vector and it hands back another vector — and almost everything done to data in practice is one of those operations written down. Making a *linear* prediction for every row at once, rotating a cloud of points, re-weighting a set of columns: each of those is a single multiplication. Once you can read Ax two ways, the whole of the rest of this track opens up, because [M10](#), [M11](#) and [M12](#) are all questions about what a particular matrix does.

24.1 Before you start

You should be able to:

- take a dot product, and say what it means — [M8](#);
- compute a vector's length, and recognise orthogonality — [M8](#);
- read subscript notation like a_{ij} — [M1](#);
- index a 2-D NumPy array and check its `.shape` — [P6](#).

Quick check. What is $(2, 5) \cdot (3, -1)$?

 Show the answer

$$(2)(3) + (5)(-1) = 6 - 5 = 1.$$

Every entry of every matrix product in this unit is one of these. If the dot product is not automatic yet, reread [M8](#) first — the rest of this unit is bookkeeping about *which* dot products to take.

24.2 The idea

Write the cohort's practice hours and a column of ones side by side:

$$X = \begin{pmatrix} 1 & h_1 \\ 1 & h_2 \\ \vdots & \vdots \\ 1 & h_{238} \end{pmatrix}$$

That is a **matrix**: a rectangular grid of numbers, here with 238 rows and 2 columns, which we call its **shape**, 238×2 . Rows first, always.

Now take the fitted line from M6, whose two parameters were an intercept and a slope, and put them in a vector $\theta = (69.2261, -0.2408)$. The prediction for student i is

$$\hat{y}_i = 69.2261 \times 1 + (-0.2408) \times h_i,$$

which is exactly the dot product of row i of X with θ . Do it for every row and you have all 238 predictions. **That** is what a matrix times a vector means:

$$X\theta = \begin{pmatrix} \text{row}_1 \cdot \theta \\ \text{row}_2 \cdot \theta \\ \vdots \\ \text{row}_{238} \cdot \theta \end{pmatrix}$$

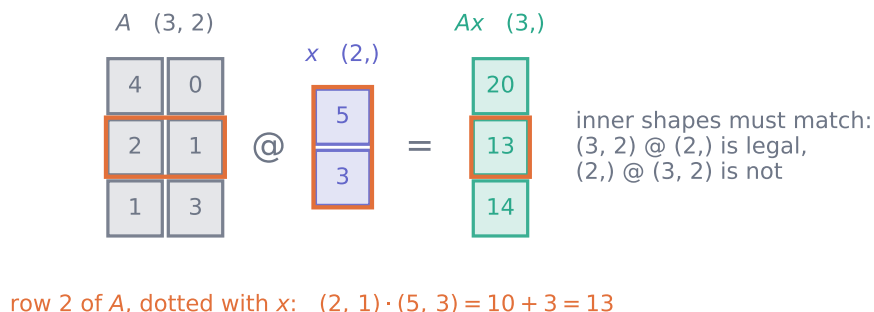


Figure 24.1: A matrix times a vector, with the shapes that make it legal. Each entry of the answer is one dot product: a whole row of the matrix against the whole input vector. The highlighted row gives the highlighted output entry, $2 \times 5 + 1 \times 3 = 13$. The matrix has as many columns as the input has entries — that is the rule — and as many rows as the answer.

Two readings of Ax , and you need both.

Rows. Each output entry is one dot product of a row with the input. This is the reading for “apply a rule to every observation” — one prediction per student.

Columns. Equivalently, Ax is the columns of A added together after being scaled by the entries of x :

$$\begin{pmatrix} 4 & 0 \\ 2 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 5 \\ 3 \end{pmatrix} = 5 \begin{pmatrix} 4 \\ 2 \\ 1 \end{pmatrix} + 3 \begin{pmatrix} 0 \\ 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 20 \\ 13 \\ 14 \end{pmatrix}$$

Same answer, different story. The column reading says that whatever $\mathbf{x}$ you choose, $A\mathbf{x}$ can only ever be a combination of A 's columns — it cannot escape them. That single observation is what [M10](#) and [M11](#) are about, so it is worth a picture.

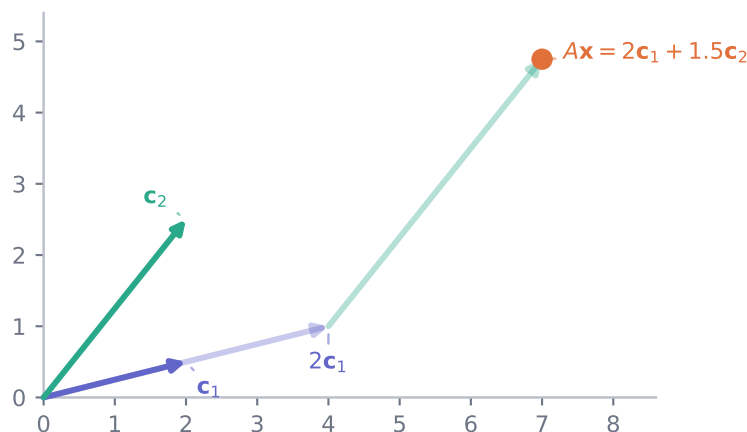


Figure 24.2: The column reading in the plane. Whatever the two weights are, $A\mathbf{x}$ is built by walking some multiple of the first column and then some multiple of the second. Here $2\mathbf{c}_1 + 1.5\mathbf{c}_2$ lands on $(7, 4.75)$. With two columns pointing in different directions every point in the plane is reachable; if they pointed the same way, only one line would be.

24.3 The mechanics

24.3.1 Shape, and the one rule

A matrix with m rows and n columns has shape $m \times n$; the entry in row i , column j is a_{ij} , row index first. A vector of n numbers behaves like an $n \times 1$ matrix.

For AB to exist, **the number of columns of A must equal the number of rows of B** . Then

$$(m \times n)(n \times p) = (m \times p) :$$

the inner dimensions must match and they disappear; the outer ones survive. Most of the errors you will make in this subject are this rule being broken, and the error message will say so in exactly these numbers.

24.3.2 The product

Entry (i, j) of AB is the dot product of **row i of A** with **column j of B** :

$$(AB)_{ij} = \sum_{k=1}^n a_{ik} b_{kj}.$$

A matrix product is a grid of dot products, one per (row, column) pair, and that is the whole definition. Everything else is consequence.

Order matters. AB and BA are usually different, and often only one of them even has legal shapes:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}, \quad \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}$$

The second matrix swaps columns when it multiplies from the right and swaps rows when it multiplies from the left. So “multiply by B ” is not one instruction — which side you multiply on is part of the instruction.

Matrix multiplication *is* associative, $(AB)C = A(BC)$, which is what lets you combine a chain of operations into one matrix and apply it once.

24.3.3 The identity

I has ones down the diagonal and zeros elsewhere, and $IA = AI = A$. It is the operation that does nothing, and it is the yardstick for M10’s question: given A , is there a matrix that undoes it?

24.3.4 The transpose

$A^\top$ is A with rows and columns exchanged: $(A^\top)_{ij} = a_{ji}$, so an $m \times n$ matrix becomes $n \times m$. Two facts earn their keep:

- $(AB)^\top = B^\top A^\top$ — the order reverses.
- For a data matrix X , the product $X^\top X$ is small, square, and full of the sums that summarise the columns. It is the object every least-squares calculation is built out of, and the worked example below takes it apart.

The transpose is also how a dot product is written in matrix notation: $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b}$, a $(1 \times n)(n \times 1) = (1 \times 1)$ product — a single number, as M8 said.

24.4 Worked example

One multiplication predicts the whole cohort.

Build the 238×2 design matrix — a column of ones for the intercept and the practice hours — and take the fit from M6 at full precision.

```

import numpy as np
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h, y = usable["hours_practice"].to_numpy(), usable["final_score"].to_numpy()

X = np.column_stack([np.ones(len(h)), h])
slope, intercept = np.polyfit(h, y, 1)
theta = np.array([intercept, slope])

predictions = X @ theta
print(f"X {X.shape} @ theta {theta.shape} -> predictions
      ↪ {predictions.shape}")
print(f"theta = {theta}")
print(f"first three predictions {predictions[:3]}")
print(f"by hand, student 1:      {theta[0] * 1 + theta[1] * h[0]:.4f}")

```

```

X (238, 2) @ theta (2,) -> predictions (238,)
theta = [69.22611682 -0.24079311]
first three predictions [66.11988566 63.73603384 68.81676853]
by hand, student 1:      66.1199

```

No loop over students appears anywhere. The shapes did the work: $(238, 2)$ against $(2,)$ can only combine one way, and it produces one number per row.

Now the small square matrix. $X^T X$ has shape $(2, 238) \times (238, 2) = (2, 2)$ — four numbers, whatever the number of students:

```

print("X.T @ X =\n", X.T @ X)
print("\nX.T @ y =", X.T @ y)
print(f"\nn = {len(h)},    sum h = {h.sum():.1f},    sum h^2 =
      ↪ {(h**2).sum():.2f}")
print(f"sum y = {y.sum():.1f},    sum h*y = {(h*y).sum():.2f}")

```

```

X.T @ X =
[[ 238.    3908.4 ]
 [ 3908.4 79872.82]]

```

```

X.T @ y = [ 15534.7 251330.53]

```

```

n = 238,    sum h = 3908.4,    sum h^2 = 79872.82
sum y = 15534.7,    sum h*y = 251330.53

```

Read the entries. $X^T X$ is $\begin{pmatrix} n & \sum h \\ \sum h & \sum h^2 \end{pmatrix}$ and $X^T y$ is $(\sum y, \sum hy)$: the 476 numbers of the design matrix have been compressed into four, and the 238 outcomes into two. Those six numbers are all that a straight-line fit needs, which is why fitting one to a billion rows is not a billion times harder than fitting it to a thousand.

And the property that makes it the *best* line. Take the residuals $\mathbf{r} = \mathbf{y} - X\theta$ and multiply by X^T :

```
r = y - X @ theta
print("X.T @ r          =", X.T @ r)
print(f"sum of residuals      {r.sum():+.3e}")
print(f"sum of residual x hours {(r * h).sum():+.3e}")
print("orthogonal to both columns of X:", np.allclose(X.T @ r, 0))
```

```
X.T @ r          = [7.89412979e-12 1.21630706e-10]
sum of residuals      +7.887e-12
sum of residual x hours +1.216e-10
orthogonal to both columns of X: True
```

Both entries are zero to within floating-point noise — note that this needed `np.allclose` rather than `== 0`, for the reasons in P2. Written out, $X^T \mathbf{r} = \mathbf{0}$ says the residual vector is **orthogonal to every column of X** , in exactly the sense of M8: its dot product with the ones column is zero, and its dot product with the hours column is zero.

That is not a coincidence of this dataset. It is the defining property of the least-squares fit, and M11 turns it round — imposes $X^T \mathbf{r} = \mathbf{0}$ and solves for θ — to get the answer without any descent at all.

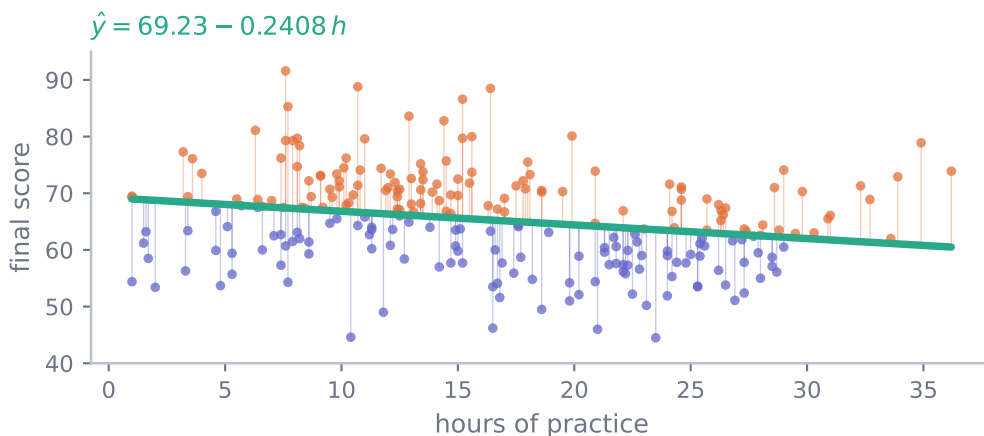


Figure 24.3: The fitted line, with each student's residual drawn as the vertical gap to it. Orange gaps are above the line, indigo below. $X^T \mathbf{r} = \mathbf{0}$ is the statement that these gaps balance twice over: they sum to zero, and they still sum to zero after each is weighted by that student's hours of practice.

24.5 Try it yourself

Exercise 1 Let $A = \begin{pmatrix} 1 & 2 \\ 0 & 3 \\ 4 & 1 \end{pmatrix}$ and $\mathbf{x} = (2, -1)$. Compute $A\mathbf{x}$ two ways — by rows and by columns — and confirm they agree.

💡 Show the solution

By rows, each entry is a dot product:

$$A\mathbf{x} = \begin{pmatrix} (1)(2) + (2)(-1) \\ (0)(2) + (3)(-1) \\ (4)(2) + (1)(-1) \end{pmatrix} = \begin{pmatrix} 0 \\ -3 \\ 7 \end{pmatrix}.$$

By columns, scale and add:

$$2 \begin{pmatrix} 1 \\ 0 \\ 4 \end{pmatrix} - 1 \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \\ 8 \end{pmatrix} - \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -3 \\ 7 \end{pmatrix}.$$

Shapes: $(3, 2)$ times $(2,)$ gives $(3,)$. The answer has one entry per *row* of A , and $\mathbf{x}$ needs one entry per *column* — get those the wrong way round and NumPy raises the mismatch rather than guessing.

Exercise 2 Let $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $B = \begin{pmatrix} 1 & 0 \\ 5 & 1 \end{pmatrix}$. Compute AB and BA .

💡 Show the solution

$$AB = \begin{pmatrix} 1+10 & 0+2 \\ 3+20 & 0+4 \end{pmatrix} = \begin{pmatrix} 11 & 2 \\ 23 & 4 \end{pmatrix}, \quad BA = \begin{pmatrix} 1 & 2 \\ 5+3 & 10+4 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 8 & 14 \end{pmatrix}.$$

Different — and not in some subtle way. Both products are legal here because both matrices are square and the same size; for non-square matrices one order is usually illegal outright, which at least tells you immediately.

Worth checking a habit: for AB , entry $(2, 1)$ used row 2 of A and column 1 of B , giving $(3)(1) + (4)(5) = 23$. If you found yourself using column 2 of A , you have the convention backwards.

Exercise 3 X is a data matrix with 500 rows and 4 columns, and $\mathbf{y}$ has 500 entries. Which of these are legal, and what shape does each result have?

$X\mathbf{y}$, $X^\top\mathbf{y}$, $X^\top X$, $XX^\top$, $\mathbf{y}^\top X$.

💡 Show the solution

expression	shapes	verdict
$X\mathbf{y}$	$(500, 4)(500,)$	illegal — 4 columns against 500 entries
$X^\top\mathbf{y}$	$(4, 500)(500,)$	legal, gives $(4,)$
$X^\top X$	$(4, 500)(500, 4)$	legal, gives $(4, 4)$
$XX^\top$	$(500, 4)(4, 500)$	legal, gives $(500, 500)$
$\mathbf{y}^\top X$	$(1, 500)(500, 4)$	legal on paper, gives $(1, 4)$

That last row carries a warning. On paper $\mathbf{y}$ is a 500×1 column and $\mathbf{y}^\top$ is a 1×500 row, so the product is 1×4 . NumPy makes no such distinction for a one-dimensional array: $\mathbf{y.T}$ is the same object as $\mathbf{y}$, and $\mathbf{y.T} @ \mathbf{X}$ comes back with shape $(4,)$. The mathematics and the library agree about the four numbers and disagree about how to wrap them — worth knowing before it surprises you.

The pair worth staring at is the last two of the square ones. $\mathbf{X}^\top \mathbf{X}$ is 4×4 — sixteen numbers, one per pair of columns, and it is what least squares needs. $\mathbf{X} \mathbf{X}^\top$ is 500×500 — a quarter of a million numbers, one per pair of *students*. Both are legal, both are meaningful, and writing one when you meant the other is a mistake whose only symptom may be that your program has become extremely slow.

24.6 Check it in Python

The unit's central claim is that a matrix product is a grid of dot products. Test it by building one from nothing but the dot product of M8 and comparing against @:

```
rng = np.random.default_rng(2026)
A = rng.normal(size=(4, 3))
B = rng.normal(size=(3, 5))

by_hand = np.zeros((A.shape[0], B.shape[1]))
for i in range(A.shape[0]):
    for j in range(B.shape[1]):
        by_hand[i, j] = A[i, :] @ B[:, j]      # row i of A against column j
        ↪ of B

print("shapes:", A.shape, "@", B.shape, "->", (A @ B).shape)
print("built from dot products matches @ :", np.allclose(by_hand, A @ B))
```

```
shapes: (4, 3) @ (3, 5) -> (4, 5)
built from dot products matches @ : True
```

Then the claims about shape and order, including the ones that are supposed to fail:

```
print("(AB)^T == B^T A^T :", np.allclose((A @ B).T, B.T @ A.T))
print("(AB)^T == A^T B^T :", np.allclose((A @ B).T, A.T @ B.T))
    if A.T.shape[1] == B.T.shape[0] else "not even a legal shape")

try:
    B @ A
except ValueError as exc:
    print("B @ A raises:", exc)
```

```
(AB)^T == B^T A^T : True
(AB)^T == A^T B^T : not even a legal shape
B @ A raises: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc
```

The middle line is the one to look at: reversing the order in a transpose is not a stylistic choice you can get away with — here the shapes do not even permit it. And the error message from the last block names the two dimensions that disagree, which is almost always enough to find the mistake.

One more, because it is the single most common bug in code that uses matrices:

```
M = np.array([[1.0, 2.0], [3.0, 4.0]])
print("M * M =\n", M * M)
print("M @ M =\n", M @ M)
print("same?", np.array_equal(M * M, M @ M))
```

```
M * M =
[[ 1.  4.]
 [ 9. 16.]]
M @ M =
[[ 7. 10.]
 [15. 22.]]
same? False
```

`*` squares each entry where it stands. `@` takes the four dot products. Both return a 2×2 array, both run without complaint, and only one of them is matrix multiplication — the point [P7](#) made before you had the definition to attach it to.

24.7 Common wrong turns

Where this usually goes wrong

Using `*` when you mean `@` On two square matrices of the same size, both are legal and both return the right shape. Nothing will fail. The answer is simply wrong, and it can travel a long way before anything notices.

Assuming $AB = BA$ They differ in general, and frequently only one is even legal. “Multiply by B ” is an incomplete instruction until you say which side.

Getting rows and columns the wrong way round a_{ij} is row i , column j ; shape is (rows, columns); entry (i, j) of a product uses row i of the *left* matrix and column j of the *right* one. Every one of those is row-first. Mix the convention up — column i of the left against row j of the right — and you get $(BA)^T$, which for square matrices has exactly the right shape and is neither AB nor its transpose. The shape check will not save you.

Distributing a transpose without reversing $(AB)^T = B^T A^T$, not $A^T B^T$. In the usual case the second is not even a legal product, which is a mercy.

Confusing $X^T X$ with XX^T For a 500×4 data matrix these are 4×4 and 500×500 . Both compute; one of them will exhaust your memory on a real dataset.

Reading a matrix as *only* a table The table view says what is stored. The operation view says what it does, and that is the view [M10](#), [M11](#) and [M12](#) need — a matrix takes a vector and returns a vector, and the columns are the only directions it can return.

24.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A is 3×5 and B is 5×2 . What is the shape of AB ?
a. 3×2 b. 5×5 c. 2×3 d. illegal
2. In one sentence, what is entry (i, j) of AB ?
3. X is your 238×2 design matrix. Give the shapes of $X^T X$ and XX^T , and say which one you want.
4. $A\mathbf{x}$ can never be anything other than a combination of the columns of A . Why, and what does that imply if the two columns of a 3×2 matrix are parallel?
5. Your code runs, the shapes are right, and the numbers are wrong. Name two matrix-specific mistakes that behave exactly like that.

Show the answers

1. **(a)** 3×2 . The inner dimensions (5 and 5) match and vanish; the outer ones survive. Option (b) keeps the inner dimension, which is the one that disappears; (c) has the factors the wrong way round — that would be the shape of $B^T A^T$.
2. The **dot product of row i of A with column j of B** . Row from the left matrix, column from the right one.
3. $X^T X$ is 2×2 ; XX^T is 238×238 . You want $X^T X$ — four numbers summarising the columns, which is what fitting a line needs. The other is one number per pair of students, and it grows with the square of your sample size.
4. Because $A\mathbf{x}$ is the columns scaled by the entries of $\mathbf{x}$ and added — that is the column reading of the product, so no choice of $\mathbf{x}$ can produce a direction the columns do not already span. If the two columns of a 3×2 matrix are parallel, they span only a single line, so every possible $A\mathbf{x}$ lies on that line and almost every target is unreachable. [M10](#) is about what to do then.
5. Writing `*` for `@` on two square matrices of the same size, and a stray transpose — using A^T where A was meant, on a square matrix. Both keep the shape and change the answer, so neither raises anything. (A third: taking entry (i, j) from column i of the left matrix and row j of the right, which yields $(BA)^T$ — right shape, wrong matrix. On the pair in Exercise 2 it returns $\begin{pmatrix} 1 & 8 \\ 2 & 14 \end{pmatrix}$ while $AB = \begin{pmatrix} 11 & 2 \\ 23 & 4 \end{pmatrix}$.)

Where this shows up next

[M10](#) asks the reverse question: given A and a target $\mathbf{b}$, which $\mathbf{x}$ satisfies $A\mathbf{x} = \mathbf{b}$ — and what to do when none does. [M11](#) imposes the orthogonality condition $X^T \mathbf{r} = \mathbf{0}$ found in the worked example and solves it, which is least squares in closed form. [M12](#) asks which directions a matrix stretches most, and finds them in $X^T X$. [S11](#) fits this exact design matrix to the cohort and asks what the fitted slope licenses you to say.

25 Solving $Ax = b$

Mathematics · Unit 10

M10 · Mathematics Reading: 6 pages With practice: 2 hours Assumes: M9

Why this unit is here

M9 asked what a matrix *does* to a vector. This unit asks the reverse: given the matrix and the answer you want, which input produces it? That question is what “solving” means, and it is where fitting a model ends up once you stop walking downhill and write the condition for the best fit directly.

The part that matters most is not the arithmetic — a library does that in one call. It is knowing which of three situations you are in: exactly one solution, none at all, or infinitely many. With real data the honest answer is usually **none**, and recognising that rather than fighting it is precisely what M11 is for.

25.1 Before you start

You should be able to:

- multiply a matrix by a vector, by rows and by columns — M9;
- check that two shapes can legally be multiplied — M9;
- recognise when two vectors point along the same line — M8;
- call a NumPy function and read what it returns — P6.

Quick check. Solve $2x + y = 7$ and $x - y = 2$.

💡 Show the answer

Add the two equations: $3x = 9$, so $x = 3$, and then $y = 1$.

You have just done Gaussian elimination on a 2×2 system. Everything below is that, plus a vocabulary for the cases where it does not work.

25.2 The idea

A system of linear equations is a matrix equation. These two,

$$\begin{array}{l} 2x + y = 7 \\ x - y = 2 \end{array} \quad \text{are} \quad \underbrace{\begin{pmatrix} 2 & 1 \\ 1 & -1 \end{pmatrix}}_A \underbrace{\begin{pmatrix} x \\ y \end{pmatrix}}_{\mathbf{x}} = \underbrace{\begin{pmatrix} 7 \\ 2 \end{pmatrix}}_{\mathbf{b}},$$

and as in M9 there are two ways to look at that, both worth having.

The row picture. Each row is one equation, and in two variables each equation is a line. A solution is a point on every line at once — an intersection. Two lines in a plane can cross once, never (parallel), or everywhere (the same line drawn twice), and those are the only possibilities:

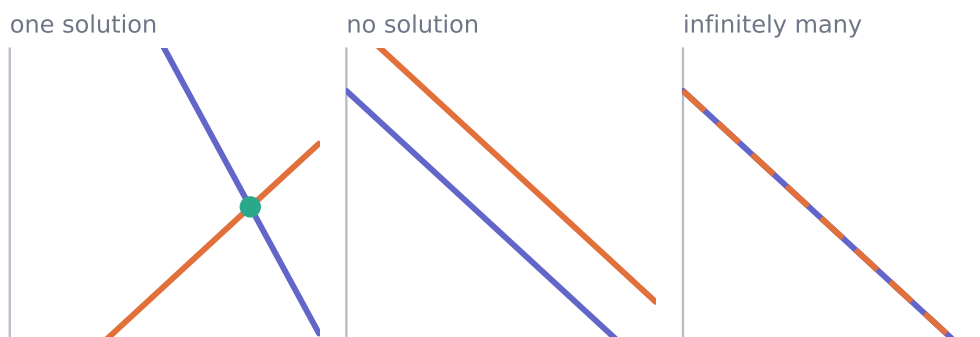


Figure 25.1: The three things a system can do, as lines. Left: one crossing, one solution. Middle: parallel lines, no point lies on both, and the system is *inconsistent*. Right: the two equations describe the same line, so every point on it solves the system and there are infinitely many solutions. Nothing else can happen.

The column picture. By the column reading of M9, $A\mathbf{x}$ is the columns of A scaled by the entries of $\mathbf{x}$ and added. So the question “does $A\mathbf{x} = \mathbf{b}$ have a solution?” becomes

can I build $\mathbf{b}$ out of the columns of A ?

The set of everything you can build that way is called the **span** of the columns — the reachable set. A solution exists exactly when $\mathbf{b}$ lies inside it, and that is a far more useful way to think about it than the lines, because it survives into dimensions you cannot draw.

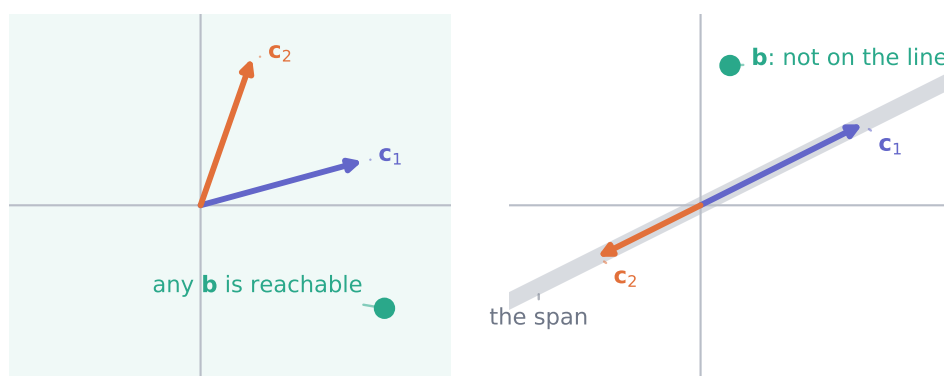


Figure 25.2: The same three cases seen from the columns. Left: two columns pointing in different directions span the whole plane, so every target is reachable and there is one recipe for each. Right: two columns along the same line span only that line — targets on it are reachable in infinitely many ways, and everything off it, including the $\mathbf{b}$ marked here, is unreachable no matter what $\mathbf{x}$ you choose.

25.3 The mechanics

25.3.1 Elimination

The method is the one from the quick check, written down carefully: add multiples of one equation to another until the system is triangular, then substitute back. On

$$\begin{aligned}2x + y &= 7 \\ x - y &= 2\end{aligned}$$

subtract $\frac{1}{2} \times$ (row 1) from row 2 to remove x there, leaving $-\frac{3}{2}y = -\frac{3}{2}$, so $y = 1$; substitute into row 1 to get $x = 3$.

Three operations are allowed, because none of them changes the solution set: swap two rows, multiply a row by a nonzero number, and add a multiple of one row to another. That is the whole algorithm, and `numpy.linalg.solve` is a careful version of it — careful mainly about *which* row it uses to eliminate with, because dividing by a tiny pivot destroys accuracy.

25.3.2 Singular matrices

Elimination breaks down when a row eliminates to nothing. For a square A that happens exactly when the columns are dependent — one is a multiple of the other, or more generally one is a combination of the rest — and such an A is called **singular**. A singular matrix has a span smaller than the space it lives in, so most targets are unreachable, and the few that are reachable are reachable in infinitely many ways. Singular is the boundary between the second and third panels of both figures.

For a 2×2 matrix there is a single number that detects it, the **determinant**:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc, \quad \text{singular} \iff \det A = 0.$$

For the system above, $\det = (2)(-1) - (1)(1) = -3 \neq 0$, so one solution, which is what we found.

25.3.3 The inverse, and why not to use it

If A is square and non-singular there is a matrix A^{-1} with $A^{-1}A = AA^{-1} = I$, and then

$$Ax = b \implies x = A^{-1}b.$$

That is the right way to *think* and the wrong way to *compute*. Forming A^{-1} does more arithmetic than solving the system, and every extra operation is another chance to lose digits. **Write `np.linalg.solve(A, b)`, not `np.linalg.inv(A) @ b`** — the two agree on well-behaved problems and diverge exactly when you most need the answer, as the check at the end of this unit shows.

25.3.4 When “solvable” is too crude a question

$\det A \neq 0$ says a unique solution exists. It says nothing about whether you can compute it reliably. If the columns are nearly dependent — the lines nearly parallel — the crossing point is exquisitely sensitive to the right-hand side:

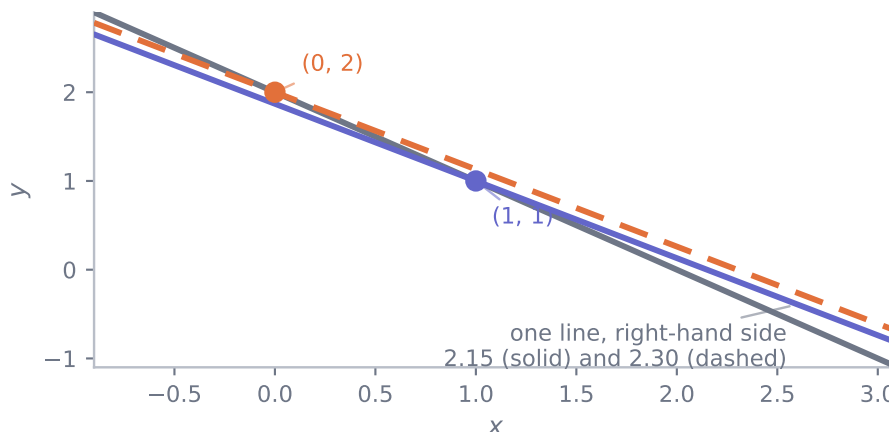


Figure 25.3: Two nearly parallel lines. Moving the second line’s right-hand side from 2.15 to 2.30 — a shift you can barely see — slides the crossing point from (1, 1) to (0, 2). The system is not singular, so a unique solution exists in both cases; it is just that where the solution *is* depends on the data far more strongly than you might expect.

The number that measures this is the **condition number**, `np.linalg.cond(A)`. Roughly: a condition number of 10^k means you can expect to lose about k digits of accuracy between the data and the answer. It is the same phenomenon that made gradient descent diverge in M6 — a long narrow valley and a nearly singular matrix are two descriptions of one thing.

25.3.5 More equations than unknowns

A dataset gives you one equation per row and one unknown per parameter, so a model with two parameters fitted to 238 students is 238 equations in 2 unknowns. Such a system is **overdetermined**, and unless the data is exactly linear it has no solution at all. That is not a failure of the data or the method; it is the normal case, and it is the whole reason the next unit exists.

25.4 Worked example

The cohort system has no solution — and the one you can solve instead.

Take the design matrix of M9 and ask for an *exact* fit: values of θ with $X\theta = \mathbf{y}$, every prediction landing on every student’s actual score.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
```

```

usable = cohort[cohort["final_score"].between(0, 100)]
h, y = usable["hours_practice"].to_numpy(), usable["final_score"].to_numpy()
X = np.column_stack([np.ones(len(h)), h])

print(f"X is {X.shape}: {X.shape[0]} equations in {X.shape[1]} unknowns")
print(f"rank of X           {np.linalg.matrix_rank(X)}")
print(f"rank of [X | y]     {np.linalg.matrix_rank(np.column_stack([X,
↪ y]))}")

```

```

X is (238, 2): 238 equations in 2 unknowns
rank of X           2
rank of [X | y]     3

```

Those two ranks are the test. The rank of X is the dimension of the span of its columns; here it is 2, so the reachable set is a plane inside a 238-dimensional space. Attaching $\mathbf{y}$ raises the rank to 3, which says $\mathbf{y}$ adds a direction the columns could not reach: $\mathbf{y}$ is **outside the span**, and $X\theta = \mathbf{y}$ has no solution. The right panel of the span figure, in 238 dimensions.

So ask a different question — not *hit $\mathbf{y}$ exactly*, but *get as close as possible* — and the condition for that was already found in M9: the residual must be orthogonal to every column of X , that is $X^T(\mathbf{y} - X\theta) = \mathbf{0}$. Rearranged, that is a system small enough to solve by hand:

$$X^T X \theta = X^T \mathbf{y} \quad \Leftrightarrow \quad \begin{pmatrix} 238 & 3908.4 \\ 3908.4 & 79872.82 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 15534.7 \\ 251330.53 \end{pmatrix}$$

238 equations have become 2. Check it is solvable, then solve it:

$$\det = (238)(79872.82) - (3908.4)^2 = 19,009,731.16 - 15,275,590.56 = 3,734,140.6,$$

which is not zero, so there is exactly one answer. Eliminate: multiply row 1 by 3908.4/238 and subtract it from row 2, leaving a single equation in b ; then substitute back for a . The arithmetic is unpleasant enough to be worth handing over:

```

A = X.T @ X
rhs = X.T @ y
theta = np.linalg.solve(A, rhs)

print(f"det(X'X)    {A[0,0]*A[1,1] - A[0,1]*A[1,0]:.1f}")
print(f"cond(X'X)   {np.linalg.cond(A):.1f}")
print(f"theta        intercept {theta[0]:.6f}    slope {theta[1]:.6f}")
print(f"residual of the solved system {np.linalg.norm(A @ theta -
↪ rhs):.3e}")
print(f"but of the original system    {np.linalg.norm(X @ theta - y):.4f}")

```

```

det(X'X)    3,734,140.6
cond(X'X)   1716.7
theta        intercept 69.226117    slope -0.240793
residual of the solved system  5.846e-11
but of the original system    125.4985

```

Two residuals, and the difference between them is the whole point of this unit. The 2×2 system is solved essentially exactly — a residual of 6×10^{-11} against right-hand sides of order 10^5 — because that system genuinely has an answer. The original 238-equation system is still missed by 125.5, and no choice of θ does better, because $\mathbf{y}$ is not in the span and nothing can put it there.

The answer itself is the one you have seen twice already: intercept 69.226117, slope -0.240793 , the same numbers gradient descent walked to in [M6](#) and `np.polyfit` returned in [M9](#). Three routes — an iterative walk, a library call and a 2×2 system solved by elimination — and one answer, which is the reassurance that all three are computing the same thing.

25.5 Try it yourself

Exercise 1 Solve $\begin{matrix} x + 2y = 5 \\ 3x - y = 1 \end{matrix}$ by elimination, and check your answer by substitution.

 Show the solution

Subtract $3 \times$ (row 1) from row 2: $(3x - y) - 3(x + 2y) = 1 - 15$, so $-7y = -14$ and $y = 2$. Then row 1 gives $x = 5 - 4 = 1$.

Check both equations, not just the one you substituted into: $1 + 4 = 5$ and $3 - 2 = 1$. Checking only the equation you used to back-substitute will confirm an arithmetic slip rather than catch it.

$\det = (1)(-1) - (2)(3) = -7 \neq 0$, consistent with there being exactly one solution.

Exercise 2 For each system, say how many solutions it has and why.

$$\begin{array}{ll} \text{(a)} & \begin{matrix} x + y = 3 \\ 2x + 2y = 6 \end{matrix} \\ \text{(b)} & \begin{matrix} x + y = 3 \\ 2x + 2y = 7 \end{matrix} \end{array}$$

 Show the solution

Both have $\det = (1)(2) - (1)(2) = 0$, so neither has a unique solution — but they differ, and the determinant alone cannot tell you how.

(a) infinitely many. The second equation is exactly twice the first, so it adds nothing. Every point on $x + y = 3$ is a solution.

(b) none. The left-hand sides are still proportional but the right-hand sides are not: $x + y$ cannot be both 3 and 3.5. The lines are parallel and distinct.

In column language: the columns $(1, 2)$ and $(1, 2)$ span only the line through $(1, 2)$. In (a), $\mathbf{b} = (3, 6)$ lies on that line and is reachable in infinitely many ways; in (b), $\mathbf{b} = (3, 7)$ does not and is unreachable.

Exercise 3 You fit a model with 5 parameters to 300 rows of data, giving a 300×5 matrix X . Someone asks you to solve $X\theta = \mathbf{y}$ exactly. What do you tell them, and what do you compute instead?

💡 Show the solution

There will almost certainly be **no exact solution**. The columns of X span at most a 5-dimensional set inside a 300-dimensional space, and a real $\mathbf{y}$ — with measurement noise, and a model that is at best approximately right — will not lie in it. Only a suspiciously perfect dataset would.

What you compute instead is the θ minimising $\|\mathbf{y} - X\theta\|$, by solving the 5×5 system $X^T X \theta = X^T \mathbf{y}$.

And the honest footnote: if there *were* an exact solution with 5 parameters and 300 rows, that would be a reason for suspicion rather than celebration. It would mean $\mathbf{y}$ landed exactly inside a 5-dimensional slice of a 300-dimensional space, which does not happen by accident. The usual explanation is that one of your columns is secretly a copy of the thing you are predicting. (It does not strictly follow that the data is noise-free — noise that happens to lie along the columns leaves no residual either — but that is a coincidence you should not expect, and looking for a leaked column is the right first move.)

25.6 Check it in Python

Solve, and then check the solution rather than trusting it:

```
for A_small, b_small in [(np.array([[2.0, 1.0], [1.0, -1.0]]),
    ↪ np.array([7.0, 2.0])),
    (np.array([[7.0, 2.0], [3.0, 5.0]]), np.array([1.0,
    ↪ 1.0]))]:
    x = np.linalg.solve(A_small, b_small)
    print(f"solution {x}")
    print(f"  A @ x == b ? {np.array_equal(A_small @ x, b_small)}"
          f"  allclose ? {np.allclose(A_small @ x, b_small)}"
          f"  residual {np.abs(A_small @ x - b_small).max():.2e}")
```

```
solution [3. 1.]
  A @ x == b ? True    allclose ? True    residual 0.00e+00
solution [0.10344828 0.13793103]
  A @ x == b ? False   allclose ? True    residual 1.11e-16
```

Both solutions are correct. The first system happens to have a whole-number answer that survives the round trip exactly, so `==` says `True`; the second has an answer that does not terminate in binary, and `==` says `False` while the residual is 10^{-16} . Which one you get is an accident of the arithmetic, so `==` is never the question to ask. Substitute back and measure the residual.

A singular system refuses, which is the behaviour you want:

```
singular = np.array([[1.0, 1.0], [2.0, 2.0]])
for target in ([3.0, 6.0], [3.0, 7.0]):
    try:
        print(f"b={target} ->", np.linalg.solve(singular,
    ↪ np.array(target)))
```

```
except np.linalg.LinAlgError as exc:
    print(f"b={target} -> LinAlgError: {exc}")
```

```
b=[3.0, 6.0] -> LinAlgError: Singular matrix
b=[3.0, 7.0] -> LinAlgError: Singular matrix
```

Note that *both* fail with the same error, although mathematically one has infinitely many solutions and the other has none. `solve` promises a unique answer and declines whenever it cannot give one; distinguishing the two cases is `matrix_rank`'s job, as in the worked example.

And the reason not to invert. The Hilbert matrices below are famous for being very nearly singular, which makes them a stress test rather than a realistic case — but the ordering they reveal holds generally:

```
def hilbert(n):
    return np.array([[1.0/(i + j + 1) for j in range(n)] for i in range(n)])

print(f"{n:>3} {'condition':>10} {'solve error':>12} {'inv @ b
↪ error':>14}")
for n in (6, 10, 14):
    H = hilbert(n)
    x_true = np.ones(n)
    b = H @ x_true
    err_solve = np.abs(np.linalg.solve(H, b) - x_true).max()
    err_inv = np.abs(np.linalg.inv(H) @ b - x_true).max()
    print(f"{n:>3} {np.linalg.cond(H):>10.1e} {err_solve:>12.2e}
↪ {err_inv:>14.2e}")
```

n	condition	solve error	inv @ b error
6	1.5e+07	2.39e-10	1.05e-09
10	1.6e+13	1.69e-04	5.55e-03
14	6.2e+17	8.90e+00	5.41e+02

Two lessons in one table. `inv` is consistently the worse of the two — by a factor of four at $n = 6$, thirty at $n = 10$ and sixty at $n = 14$ — so prefer `solve` as a matter of habit. And by $n = 14$ *both* are useless: the condition number has passed 10^{16} and the answer carries no correct digits at all. That last part is a statement about *double precision*, which carries about 16 digits — the condition number says how many of them you lose, so a problem this bad exhausts the supply. Eighty-digit arithmetic would solve the same system perfectly well. What you cannot do is rescue it with a cleverer algorithm at the same precision: the fix is more digits, or a better-posed question.

25.7 Common wrong turns

Where this usually goes wrong

Computing the inverse to solve a system `np.linalg.inv(A) @ b` does more work than `np.linalg.solve(A, b)` and is reliably less accurate. There is almost no reason to form an inverse numerically; if you find yourself typing `inv`, ask what you actually want.

Assuming a solution exists because the algebra is legal With more equations than unknowns the normal outcome is no solution at all. That is a fact about the data, not a bug, and it is why least squares exists.

Reading $\det A \neq 0$ as “this will work” A determinant is a yes/no on uniqueness and says nothing about accuracy. A matrix can be comfortably non-singular and still have a condition number of 10^{10} , at which point the answer you get is mostly noise. Check `np.linalg.cond`, not just the determinant.

Using a tiny determinant as a measure of near-singularity Scale every entry of a 2×2 matrix by 0.001 and the determinant falls by a factor of a million while the matrix is exactly as well behaved as before. The determinant has units; the condition number does not.

Checking a solution with `==` `A @ x == b` will be `False` for reasons that have nothing to do with your mathematics. Use `np.allclose`, and look at the size of `A @ x - b`.

Expecting more data to make a system easier More rows means more equations against the same unknowns, which makes an exact solution *less* likely, not more. What more rows buy you is a better approximate answer, which is a different thing.

25.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A is 2×2 with $\det A = 0$ and $A\mathbf{x} = \mathbf{b}$ has at least one solution. How many does it have?
 - a. exactly one
 - b. none
 - c. infinitely many
 - d. cannot tell
2. In column language, what exactly is the condition for $A\mathbf{x} = \mathbf{b}$ to have a solution?
3. X is 300×5 . What are the shapes of $X^T X$ and $X^T \mathbf{y}$, and why is turning the problem into that system worth doing?
4. Two colleagues solve the same system. One writes `inv(A) @ b`, the other `solve(A, b)`, and they disagree in the sixth decimal place. Who is right, and what should you check next?
5. `np.linalg.solve` raises `LinAlgError` on your system. Name the two quite different situations that produce that, and how you would tell them apart.

 Show the answers

1. **(c) infinitely many.** A zero determinant rules out a unique solution, so the only options are none or infinitely many — and we were told at least one exists. Option (d) is the tempting one, but the extra fact given closes it off.
2. **$\mathbf{b}$ must lie in the span of the columns of A** — it must be expressible as a combination of them. That is the same statement as $\text{rank}(A) = \text{rank}([A \mid \mathbf{b}])$, which is how you check it in code.

3. $X^T X$ is 5×5 and $X^T \mathbf{y}$ has 5 entries. It is worth doing because 300 equations that have no solution become 5 that do, and the size of the system you actually solve stops depending on how much data you have.
4. First establish whether the disagreement is even large. Sixth *decimal place* is an absolute measure and tells you nothing on its own — if the solution is of size 10^9 , a gap of 10^{-6} is the last bit of double precision and both answers are excellent. What matters is the disagreement *relative* to the size of the answer. If that is around 10^{-6} , roughly ten digits have been lost, and the thing to check next is `np.linalg.cond(A)`. `solve` is the better bet either way, but when conditioning is the problem the response is to reformulate, not to pick a winner.
5. Either the matrix is **singular** — columns dependent, so no unique solution exists — or it is **not square**, so `solve` does not apply at all. The shape distinguishes them immediately. If it is square and singular, compare `matrix_rank(A)` with `matrix_rank([A | b])` to find out whether you are in the “no solutions” case or the “infinitely many” one.

Where this shows up next

M11 takes the unsolvable system of the worked example seriously: it derives $X^T X \theta = X^T \mathbf{y}$ rather than pulling it out of **M9**, and explains why the closest reachable point is the one whose residual is orthogonal to the span. **M12** explains the condition number properly, in terms of how much a matrix stretches in its most and least favoured directions. **S11** fits this system to the cohort and asks what the answer means, which no amount of linear algebra can settle.

26 When there is no solution: least squares

Mathematics · Unit 11

M11 · Mathematics Reading: 5 pages With practice: 2 hours Assumes: M5, M10

Why this unit is here

M10 ended on an unsolvable system, and said that with real data this is the normal case rather than a mishap. This unit is the answer to *then what*.

If you cannot hit $\mathbf{y}$, get as close to it as you can. That single sentence, made precise, produces the most-used calculation in applied statistics — every fitted line, every regression coefficient, every “controlling for” in a results table. It arrives here twice over: once from the calculus of M5 and once from the geometry of M8, and the two derivations meet at the same small system of equations. Meeting it from both sides is what makes the formula memorable instead of arbitrary.

26.1 Before you start

You should be able to:

- say when $A\mathbf{x} = \mathbf{b}$ has a solution, in terms of the span — M10;
- set a derivative to zero and check you have a minimum — M5;
- take a dot product and recognise orthogonality — M8;
- form $X^T X$ and read its shape — M9.

Quick check. Which value of a makes $(3 - a)^2 + (7 - a)^2$ smallest?

💡 Show the answer

Differentiate: $-2(3 - a) - 2(7 - a) = 0$, so $4a = 20$ and $a = 5$ — the mean of 3 and 7, exactly as M5 found in general.

That is least squares with one parameter and no predictors. Everything in this unit is that calculation with more columns.

26.2 The idea

We want $X\theta = \mathbf{y}$ and cannot have it: by the column reading of M9, $X\theta$ can only ever land inside the span of X 's columns, and M10 showed that the cohort's $\mathbf{y}$ is outside it.

So change the question. Among all the points the span *can* reach, which is nearest to $\mathbf{y}$? “Nearest” means smallest distance, and distance is the length of M8, so we are choosing θ to minimise

$$\|\mathbf{y} - X\theta\|^2 = \sum_{i=1}^n (y_i - (X\theta)_i)^2,$$

the sum of squared residuals. Hence **least squares**.

Geometry answers this before any algebra does. The nearest point of a plane to a point off it is the foot of the perpendicular — any other point in the plane is further away, by Pythagoras. So the winning θ is the one whose residual sticks straight out of the span:

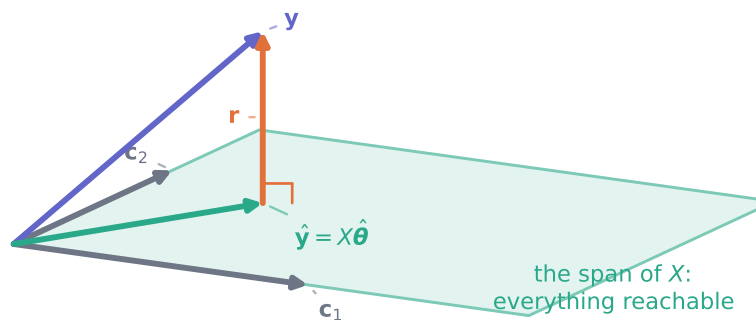


Figure 26.1: Least squares in one picture — a perspective drawing, in which the shaded parallelogram is a flat span seen edge-on from above rather than a region of the page, so the right angle at the foot of $\mathbf{r}$ is a fact about the vectors and not about the printed picture. The shaded region is everything the columns of X can reach; $\mathbf{y}$ is not in it. The closest reachable point $\hat{\mathbf{y}} = X\hat{\theta}$ is directly below $\mathbf{y}$, and the residual $\mathbf{r}$ joining them meets the span at a right angle. Any other point in the shaded region is further from $\mathbf{y}$, because moving within the span adds a second perpendicular leg to the triangle.

That right angle is the whole of least squares. Turning it into equations takes three lines, and the next section does it twice.

26.3 The mechanics

26.3.1 Route one: geometry

The residual $\mathbf{r} = \mathbf{y} - X\hat{\theta}$ must be perpendicular to the span, and the span is built from the columns of X , so it is enough for $\mathbf{r}$ to be orthogonal to every column. By M8, orthogonal means dot product zero; by M9, all those dot products at once are $X^T\mathbf{r}$. So

$$X^T(\mathbf{y} - X\hat{\theta}) = \mathbf{0} \quad \Longleftrightarrow \quad X^T X \hat{\theta} = X^T \mathbf{y}.$$

These are the **normal equations** — “normal” in its geometric sense of *at right angles*, nothing to do with the normal distribution. M9 found $X^T \mathbf{r} = \mathbf{0}$ by computing it on the cohort and M10 solved the resulting system; this is where it comes from.

26.3.2 Route two: calculus

Take the same quantity and minimise it as a function of the parameters, exactly as in M6. With $r_i = y_i - \sum_j x_{ij} \theta_j$,

$$L(\theta) = \sum_i r_i^2, \quad \frac{\partial L}{\partial \theta_j} = \sum_i 2r_i \frac{\partial r_i}{\partial \theta_j} = -2 \sum_i x_{ij} r_i = -2 (\text{column } j) \cdot \mathbf{r}.$$

Setting every partial derivative to zero says every column has zero dot product with $\mathbf{r}$ — which is $X^T \mathbf{r} = \mathbf{0}$ again. The two routes are the same statement in different languages.

And any stationary point really is a minimum, not just a candidate. L is a sum of squares, so it is never negative and it has no maximum; it is a bowl, and every flat place on a bowl is at the bottom. That is the M5 check, done once for the whole family of problems rather than case by case.

What is *not* guaranteed is that the flat place is a single point. If two columns of X are identical the bowl has a level trough running along it, every point of which is an equally good answer, and L does not grow at all as you slide along it. That is the next section.

26.3.3 When the answer is unique

$X^T X$ is invertible exactly when the columns of X are linearly independent — when no column is a combination of the others. Then

$$\hat{\theta} = (X^T X)^{-1} X^T \mathbf{y},$$

a formula worth recognising and, as M10 insisted, not worth computing that way.

If two columns are identical, the span is unchanged by trading one for the other and there are infinitely many equally good $\hat{\theta}$. The interesting case is the near miss: columns that are nearly dependent leave a long flat valley in the loss, along which the fit barely changes while the coefficients move a great deal.

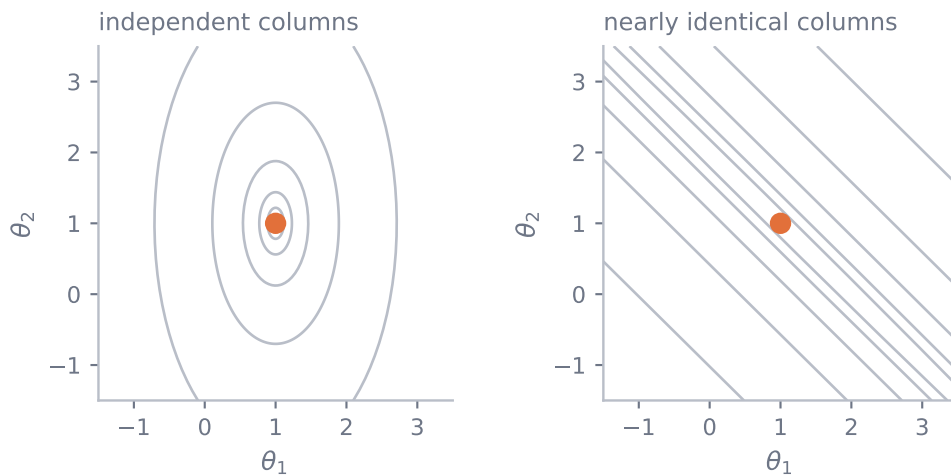


Figure 26.2: The loss surface for two independent predictors (left) and two nearly identical ones (right), on the same synthetic data. Both have exactly one minimum, marked. But on the right the floor of the valley is almost level for a long way, so the fit can barely tell one point along it from another — the *predictions* are pinned down and the individual *coefficients* are not.

26.3.4 Why libraries do not form $X^T X$

Multiplying X by its own transpose **squares the condition number** of M10: $\text{cond}(X^T X) = \text{cond}(X)^2$. On the cohort's two-column design that is a jump from 41.4 to 1717, which costs nothing; on a badly scaled design it is the difference between an answer and noise. So write

```
theta, *_ = np.linalg.lstsq(X, y, rcond=None)
```

which works on X directly and never forms $X^T X$ at all. The normal equations are how you *understand* least squares; `lstsq` is how you compute it.

26.4 Worked example

Adding one column reverses the answer.

Fit the cohort two ways: score on practice hours alone, then score on practice hours *and* the diagnostic score taken on arrival.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
h = usable["hours_practice"].to_numpy()
g = usable["diagnostic_score"].to_numpy()
y = usable["final_score"].to_numpy()
ones = np.ones(len(y))

for name, X in [("1 + hours", np.column_stack([ones, h])),
```

```

        ("1 + hours + diagnostic", np.column_stack([ones, h, g]))]:
theta, *_ = np.linalg.lstsq(X, y, rcond=None)
residual = np.linalg.norm(y - X @ theta)
print(f"{name:<24} theta = {np.round(theta, 4)}")
print(f"{'':<24} ||r|| = {residual:.4f}    cond(X) =
↪ {np.linalg.cond(X):.1f}")

```

```

1 + hours                theta = [69.2261 -0.2408]
                        ||r|| = 125.4985    cond(X) = 41.4
1 + hours + diagnostic   theta = [8.9129 0.9156 0.6619]
                        ||r|| = 70.6309    cond(X) = 614.1

```

The coefficient on hours of practice goes from -0.2408 to $+0.9156$. Not smaller, not larger — **the opposite sign**. Nothing was done to the data between the two lines, and neither fit is an error; they are answers to different questions.

The explanation is in one number:

```

print(f"corr(hours, diagnostic) {np.corrcoef(h, g)[0, 1]:+.4f}")
print(f"corr(diagnostic, score) {np.corrcoef(g, y)[0, 1]:+.4f}")

```

```

corr(hours, diagnostic) -0.8130
corr(diagnostic, score) +0.6579

```

Students who arrive stronger practise less. So “hours of practice”, on its own, is partly acting as a marker for *arriving weak* — and arriving weak predicts a lower final score. The first fit measures that whole tangle. The second holds the diagnostic score fixed, so its hours coefficient answers a narrower and much more useful question: **among students who arrived equally prepared**, what does an extra hour of practice go with? Just under a mark.

You can see both facts at once by splitting the cohort into three bands of roughly equal size by diagnostic score:

The residual norm also falls, from 125.50 to 70.63, so the second model predicts better. That proves nothing at all. Adding a column can never *increase* the sum of squares, because the old fit remains available by giving the new column a coefficient of zero — and for any column that is not already a combination of the existing ones, the fall is strict. A column of pure random noise qualifies, as the check at the end of this unit demonstrates. A smaller residual is not evidence that a column belongs in the model.

What this example does show is that a regression coefficient has no meaning on its own. “The effect of practice is -0.24 ” and “the effect of practice is $+0.92$ ” are both true statements about the same 238 students, and the thing that distinguishes them is what else was in the matrix. [S12](#) returns to this with the background groups, where it has a name.

26.5 Try it yourself

Exercise 1 Take X to be a single column of ones, so the model is “predict the same value for everybody”. Write down $X^T X$ and $X^T y$, and solve the normal equations.

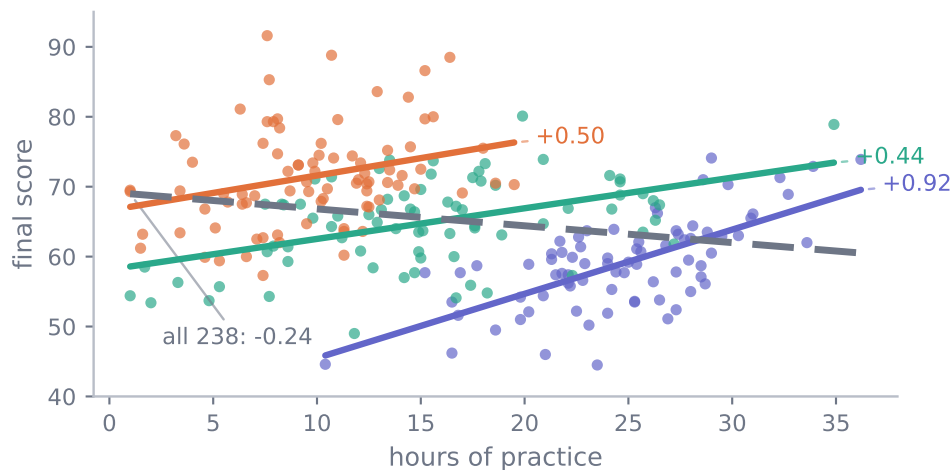


Figure 26.3: Practice against final score, with the cohort split by diagnostic score into three groups of roughly equal size (78, 76 and 84 — tied scores at the boundaries stop them being exactly equal) — indigo the third who scored lowest on arrival, orange the third who scored highest. Each group’s own fitted line rises. The overall line, in grey, falls — because the groups sit in different places: students who arrived stronger practised less and still scored higher. The three within-group slopes, $+0.92$, $+0.44$ and $+0.50$, are what the multiple-regression coefficient of $+0.92$ is summarising.

💡 Show the solution

X is $n \times 1$, so $X^T X$ is 1×1 :

$$X^T X = \sum_{i=1}^n 1 \times 1 = n, \quad X^T \mathbf{y} = \sum_{i=1}^n 1 \times y_i = \sum_i y_i.$$

So $n\hat{\theta} = \sum y_i$ and $\hat{\theta} = \bar{y}$, the mean.

This is M5’s result arriving by a different road, and it is worth noticing what the geometry says about it: the span of a single ones column is the set of vectors with all entries equal, and the closest such vector to $\mathbf{y}$ is the one at $\mathbf{y}$ ’s mean. The residual is then $\mathbf{y} - \bar{y}$, which is orthogonal to the ones column — the fact you proved in M8, Exercise 3.

Exercise 2 A fit gives $X^T X = \begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix}$ and $X^T \mathbf{y} = \begin{pmatrix} 10 \\ 22 \end{pmatrix}$. Find $\hat{\theta}$, and confirm it is the unique answer.

💡 Show the solution

$\det = (4)(14) - (6)(6) = 56 - 36 = 20 \neq 0$, so $X^T X$ is invertible: the columns of X are independent and the answer is unique.

Eliminate: from $4\theta_1 + 6\theta_2 = 10$ and $6\theta_1 + 14\theta_2 = 22$, subtract $1.5 \times$ the first from the second to get $5\theta_2 = 7$, so $\theta_2 = 1.4$ and then $\theta_1 = (10 - 8.4)/4 = 0.4$.

Check both equations: $4(0.4) + 6(1.4) = 1.6 + 8.4 = 10$ and $6(0.4) + 14(1.4) = 2.4 + 19.6 = 22$.

Exercise 3 Someone builds a design matrix with a column of hours of practice and a second

column of the same hours expressed in minutes. What goes wrong, and what happens to the *predictions*?

💡 Show the solution

The second column is $60\times$ the first, so the columns are dependent, the span is no bigger than it was with one of them, and $X^T X$ is singular. There are **infinitely many** $\hat{\theta}$: whatever (θ_1, θ_2) works, so does $(\theta_1 + 60c, \theta_2 - c)$ for any c , since the two changes cancel exactly.

For **this** X the predictions are unaffected. Every one of those parameter vectors produces the same $X\hat{\theta}$, because the closest point in the span is unique even when the recipe for reaching it is not. That is the distinction to carry away: collinearity destroys the interpretation of individual coefficients while the fitted values of a given fit stay put.

Be careful how far you take that. It says the projection is unique for a fixed design matrix; it does not say a nearly collinear fit is robust. Two designs differing in the fourth decimal place can span quite different sets and produce quite different fitted values. In practice you rarely get exact duplication — you get two columns measuring almost the same thing, which is the right-hand panel of the valley figure. `lstsq` returns an answer without complaint, and the coefficients are the first thing to distrust.

26.6 Check it in Python

First, that the two derivations and the library agree:

```
X = np.column_stack([ones, h])

normal = np.linalg.solve(X.T @ X, X.T @ y)      # the normal equations
library, *_ = np.linalg.lstsq(X, y, rcond=None)
polyfit = np.polyfit(h, y, 1)[::-1]            # intercept, slope

print(f"normal equations {normal}")
print(f"np.linalg.lstsq  {library}")
print(f"np.polyfit       {polyfit}")
print("all three agree:", np.allclose(normal, library) and
      ↪ np.allclose(normal, polyfit))
```

```
normal equations [69.22611682 -0.24079311]
np.linalg.lstsq  [69.22611682 -0.24079311]
np.polyfit       [69.22611682 -0.24079311]
all three agree: True
```

Then the claim that gives the method its name — that no other θ has a smaller sum of squares. Nudge the answer in every direction and watch the loss rise:

```
def loss(theta):
    r = y - X @ theta
    return r @ r
```



```

rng = np.random.default_rng(2026)
best = loss(library)
beaten = 0
for _ in range(20_000):
    nudge = library + rng.normal(scale=0.05, size=2)
    if loss(nudge) < best:
        beaten = beaten + 1
print(f"loss at the least-squares answer {best:.6f}")
print(f"nudges that beat it, out of 20000: {beaten}")
print(f"and the residual is orthogonal to X: {np.allclose(X.T @ (y - X @
↪ library), 0)}")

```

```

loss at the least-squares answer 15749.873767
nudges that beat it, out of 20000: 0
and the residual is orthogonal to X: True

```

Twenty thousand attempts and none of them wins, which is what “least” means. The orthogonality check on the last line is the same statement — the two are not independent pieces of evidence, they are the geometry and the arithmetic of one fact.

Next, that a falling residual means nothing. Add a column of pure noise — information that cannot possibly help — and watch the sum of squares drop anyway:

```

X0 = np.column_stack([ones, h])
theta0, *_ = np.linalg.lstsq(X0, y, rcond=None)
rss0 = ((y - X0 @ theta0)**2).sum()
print(f"1 + hours                                RSS {rss0:.4f}")

for trial in range(3):
    noise = rng.normal(size=len(y))
    Xn = np.column_stack([ones, h, noise])
    thetan, *_ = np.linalg.lstsq(Xn, y, rcond=None)
    rssn = ((y - Xn @ thetan)**2).sum()
    print(f" + a column of random noise RSS {rssn:.4f}    (fell by {rss0 -
↪ rssn:.4f})")

Xd = np.column_stack([ones, h, h])
thetad, *_ = np.linalg.lstsq(Xd, y, rcond=None)
print(f" + a duplicate of hours          RSS {(y - Xd @ thetad)**2).sum():.4f}
↪ (unchanged)")

```

```

1 + hours                                RSS 15749.8738
+ a column of random noise RSS 15671.2007    (fell by 78.6731)
+ a column of random noise RSS 15489.4165    (fell by 260.4573)
+ a column of random noise RSS 15734.5969    (fell by 15.2768)
+ a duplicate of hours          RSS 15749.8738    (unchanged)

```

Three columns of nonsense, three improvements. The only column that fails to improve the fit is the duplicate — and it fails because it adds nothing to the span, which is exactly the

case the guarantee is silent about. Judging a model by whether the residual fell is judging it by something that was going to happen regardless.

Finally, why `lstsq` rather than the normal equations. Fit a polynomial of rising degree, where the columns $1, t, t^2, \dots$ become steadily more alike, and compare the two routes against an answer we know:

```
t = np.linspace(0, 1, 120)

print(f"{'degree':>7} {'cond(X)':>10} {'cond(X.T X)':>12} {'normal eqs':>12}
↪ {'lstsq':>10}")
for degree in (3, 7, 11):
    Xp = np.column_stack([t**k for k in range(degree + 1)])
    truth = np.ones(degree + 1)
    yp = Xp @ truth                                     # exactly solvable by
↪ construction
    err_normal = np.abs(np.linalg.solve(Xp.T @ Xp, Xp.T @ yp) - truth).max()
    err_lstsq = np.abs(np.linalg.lstsq(Xp, yp, rcond=None)[0] - truth).max()
    print(f"{'degree':>7} {np.linalg.cond(Xp):>10.1e} {np.linalg.cond(Xp.T @
↪ Xp):>12.1e}"
          f" {'err_normal':>12.1e} {'err_lstsq':>10.1e}")
```

degree	cond(X)	cond(X.T X)	normal eqs	lstsq
3	1.2e+02	1.5e+04	1.6e-12	7.5e-15
7	1.2e+05	1.4e+10	2.9e-07	6.0e-12
11	1.2e+08	1.5e+16	6.0e-01	3.3e-09

Two things to read off. `cond(X.T X)` really is `cond(X)` squared, in every row. And by degree 11 the normal equations return an answer wrong in the *first* decimal place while `lstsq`, working on the same data, is still correct to nine. The mathematics is identical; only the route differs.

26.7 Common wrong turns

Where this usually goes wrong

Reading a coefficient without saying what else is in the model The cohort’s practice coefficient is -0.24 alone and $+0.92$ alongside the diagnostic score. Neither number means anything until you say which matrix it came out of.

Treating a smaller residual as evidence Adding any column can only reduce the sum of squares, since the previous fit survives by giving the new column a zero coefficient. Improvement is automatic and therefore not informative.

Computing $(X^T X)^{-1} X^T y$ literally It is the right formula and the wrong algorithm — it inverts a matrix (M10) whose condition number has already been squared. `np.linalg.lstsq` does neither.

Confusing “the projection is unique” with “the predictions are stable” For a *fixed* X , the closest point in the span is unique even when the $\hat{\theta}$ reaching it is not — so duplicate columns leave the fitted values of that fit alone while making the coefficients arbitrary. That is a statement about one design matrix, not a promise of robustness: nudge

the data slightly and a nearly collinear fit can move its fitted values a long way too. Coefficients are the first casualty, not the only possible one.

Thinking least squares requires a straight-line relationship The model must be linear in the **parameters**, not in the data. Fitting $y = \theta_0 + \theta_1 t + \theta_2 t^2$ is a least-squares problem with three columns, and its curve is a parabola.

Believing the minimum is the truth Least squares finds the closest point in the span of the columns you supplied. If the right column is not in X , the answer is the best available lie, and the arithmetic will not mention it.

26.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. What condition does least squares impose on the residual $\mathbf{r}$?
 - a. $\mathbf{r} = \mathbf{0}$
 - b. $\mathbf{r}$ is orthogonal to every column of X
 - c. $\mathbf{r}$ is as small as the data allows in every entry
 - d. $\sum r_i = 0$
2. Why is $L(\theta) = \|\mathbf{y} - X\theta\|^2$ guaranteed to have a minimum rather than a maximum at its stationary point?
3. A colleague adds a column and reports that the residual sum of squares fell. What have they demonstrated?
4. $\text{cond}(X) = 10^4$. What is $\text{cond}(X^\top X)$, and roughly how many digits does solving the normal equations cost you?
5. Two columns of X are nearly identical. Which of these become unreliable — the fitted values, the individual coefficients, or both?

Show the answers

1. **(b).** Orthogonality to every column is the defining condition, and it is what $X^\top \mathbf{r} = \mathbf{0}$ says. (a) is what we wanted and could not have. (d) is *implied* by (b) whenever X contains a column of ones — the ones column's dot product with $\mathbf{r}$ is $\sum r_i$ — but it is one consequence, not the condition. (c) is not a coherent requirement: residuals trade off against one another and cannot all be minimised at once.
2. Because L is a sum of squares: never negative, and with no maximum. So any flat place has to be a lowest point — a bowl has no summit. Note what this does *not* claim: if the columns of X are dependent the bottom is a level trough rather than a point, and there are infinitely many equally good answers. Guaranteed minimum, not guaranteed unique.
3. **Nothing.** The residual sum of squares cannot rise when a column is added, because the previous fit is still reachable by setting the new coefficient to zero. They have demonstrated that their arithmetic works.
4. $\text{cond}(X^\top X) = 10^8$, so roughly **eight** of the sixteen digits double precision offers, against about four if you work with X directly. That is the entire argument for `lstsq`.

5. **The coefficients first and worst.** For the design matrix in front of you the closest point in the span is unique, so $X\hat{\theta}$ is pinned down while the division of credit between two near-identical columns is not. But uniqueness is not stability: perturb the data a little and a nearly collinear fit can shift its fitted values as well. The safe summary is that the coefficients become untrustworthy immediately and the predictions are merely better behaved, not safe.

Where this shows up next

M12 explains the condition number properly — as a ratio of how much a matrix stretches its most and least favoured directions — and gives the decomposition that `lstsq` actually uses. **S11** takes this exact fit and asks the statistical questions: how uncertain is that slope, and what would license calling it an effect. **S12** takes up the sign reversal in the worked example and shows how routinely it happens.

27 Directions in data: eigenvectors and the SVD

Mathematics · Unit 12

M12 · Mathematics Reading: 6 pages With practice: 2.5 hours Assumes: M11

Why this unit is here

A matrix does not treat all directions alike. It stretches some, shrinks others, and leaves a few pointing exactly where they started. This unit asks which directions those are and by how much — one question, which turns out to answer four you have already met.

Why $\text{cond}(X^T X)$ is $\text{cond}(X)^2$ (M11). Why a nearly collinear design leaves a long flat valley in the loss instead of a bowl (M11). What `np.linalg.lstsq` is actually doing (M10). And what a *principal component* is, which is where a dataset with forty columns gets turned into a picture you can look at. It is also the last unit of the Mathematics track.

27.1 Before you start

You should be able to:

- read $A\mathbf{x}$ as an operation on a vector — M9;
- form $X^T X$ and say what shape it is — M9;
- say what the condition number of a matrix is *for* — M11;
- take a length and recognise orthogonality — M8.

Quick check. What does $\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$ do to the vectors $(1, 0)$ and $(0, 1)$?

💡 Show the answer

$(1, 0) \mapsto (2, 0)$ and $(0, 1) \mapsto (0, 3)$: each is stretched, neither is turned. Those two directions are **eigenvectors** of that matrix, with stretch factors 2 and 3. This unit is about finding such directions when the matrix is not conveniently diagonal — and about what they tell you when the matrix is made out of data.

27.2 The idea

Take every unit vector — a whole circle of directions — and apply a matrix to all of them. What comes back is an **ellipse**. Not sometimes: always, for any 2×2 matrix, and the higher-dimensional version is an ellipsoid.

That is a strong statement and it is the whole geometry of the subject. It says a matrix does exactly three things: it picks out a set of perpendicular input directions, stretches each by its own factor, and lays the results down along a (possibly different) set of perpendicular output directions.

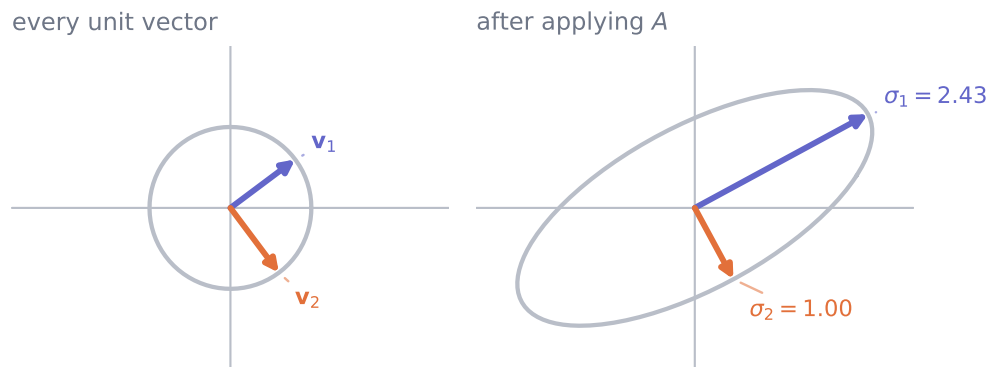


Figure 27.1: Left: every unit vector, with two particular ones picked out. Right: what the matrix $A = \begin{pmatrix} 2 & 0.9 \\ 0.4 & 1.4 \end{pmatrix}$ does to all of them. The circle becomes an ellipse, and the two marked directions are the pair that is still perpendicular after the journey: they come out along the ellipse's axes, stretched by 2.43 and 1.00. No other perpendicular pair survives as one, and no direction is stretched by more than 2.43 or less than 1.00.

Once you see that ellipse, several loose ends from earlier units tie themselves off. Its longest and shortest half-widths are the biggest and smallest stretch the matrix can apply; their ratio is the condition number. A nearly singular matrix is one whose ellipse is nearly flat.

27.3 The mechanics

27.3.1 Eigenvectors

For a **square** matrix A , a nonzero $\mathbf{v}$ with

$$A\mathbf{v} = \lambda\mathbf{v}$$

is an **eigenvector**, and the number λ is its **eigenvalue**. In words: the matrix does not turn this direction at all, it only scales it. Note that $\mathbf{v}$ is a *direction* — any nonzero multiple of an eigenvector is one too, with the same eigenvalue, so libraries return a unit-length representative and the sign is arbitrary.

Not every real matrix has real eigenvectors: a rotation by 90° turns every direction, so nothing survives unturned. But the matrices that arise from data are of a much better-behaved kind.

27.3.2 Symmetric matrices, which are the ones you get

A is **symmetric** if $A^\top = A$. Both matrices this course cares about are: $X^\top X$ from M11, and the covariance and correlation matrices of a dataset. For those, a theorem guarantees the good case:

A real symmetric matrix has real eigenvalues, and its eigenvectors can be chosen to be mutually orthogonal.

So a symmetric matrix has a full set of perpendicular directions it merely scales — its ellipse's axes *are* its eigenvectors. This is why data analysis keeps producing orthogonal directions without anyone asking for them.

27.3.3 The singular value decomposition

Eigenvectors need a square matrix, and a data matrix is 238×3 . The **SVD** removes that restriction. Any matrix at all factorises as

$$X = U\Sigma V^\top,$$

where Σ is diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \dots$ — the **singular values** — and U and V have orthogonal unit columns. Read left to right it is exactly the ellipse: $V^\top$ names the input directions, Σ stretches each by its own σ , and U says where the results point.

The bridge to eigenvalues is the fact that pays the debts listed at the top:

$$X^\top X = V\Sigma^2 V^\top,$$

so the eigenvalues of $X^\top X$ are the squares of the singular values of X , with the same directions.

27.3.4 The condition number, at last

$$\text{cond}(X) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

— the ratio between the ellipse's longest and shortest half-widths, which is the most and the least a matrix stretches anything. (Not its *eccentricity*, which is a different number: $\sqrt{1 - \sigma_2^2/\sigma_1^2}$.) And now the M11 claim is a one-liner: the singular values of $X^\top X$ are σ_i^2 , so their ratio is the square of the ratio, and $\text{cond}(X^\top X) = \text{cond}(X)^2$ — for a design matrix with independent columns, which is the case that matters here. (If the columns are dependent, $X^\top X$ has a zero eigenvalue and its condition number is infinite, which is the same warning said louder.) Forming $X^\top X$ squashes the ellipse, which is why `lstsq` — which runs on the SVD of X itself — does not form it.

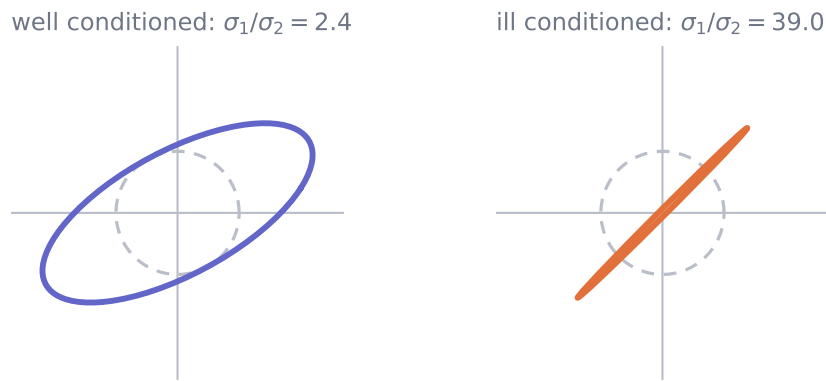


Figure 27.2: The unit circle under two matrices, drawn to the same scale. Left, $\sigma_1/\sigma_2 = 2.4$: a mild ellipse, and a system you can solve happily. Right, $\sigma_1/\sigma_2 = 39$: nearly flat, so directions that were far apart on the circle arrive almost on top of one another — which is what it means for information to be lost, and what a large condition number is measuring.

27.3.5 Principal components

Now point the same machinery at the data rather than at the design matrix. Centre every column, and take the covariance matrix (or, if the columns are on different scales, standardise first and take the correlation matrix). It is symmetric, so it has orthogonal eigenvectors. Those directions are the **principal components**, and each eigenvalue is the variance of the data along its direction.

The first principal component is the single direction along which the cloud is most spread out. This is a change of *coordinates*, not a change of data: you are looking at the same cloud from a better angle.

27.4 Worked example

The cohort has two stories in it, and PCA finds both.

Take the three numeric columns, standardise them so no column dominates through its units alone, and eigen-decompose the correlation matrix.

```
import numpy as np
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
usable = cohort[cohort["final_score"].between(0, 100)]
names = ["diagnostic_score", "hours_practice", "final_score"]
Z = usable[names].to_numpy()

C = np.corrcoef(Z.T)
values, vectors = np.linalg.eigh(C)
order = np.argsort(values)[::-1]
values, vectors = values[order], vectors[:, order]

print("variance along each principal direction:")
```



```

for k in range(3):
    axis = vectors[:, k]
    axis = axis * np.sign(axis[np.argmax(abs(axis))]) # sign is arbitrary;
    ↪ fix it
    loads = " ".join(f"{n.split('_')[0]:>10} {c:+.3f}" for n, c in
    ↪ zip(names, axis))
    print(f" PC{k+1} eigenvalue {values[k]:.4f} ({100*values[k]/3:4.1f}%)
    ↪ {loads}")
print(f"\neigenvalues sum to {values.sum():.4f}, one per standardised
    ↪ column")

```

variance along each principal direction:

PC1	eigenvalue	2.1673	(72.2%)	diagnostic	+0.667	hours	-0.562	final	+0.48
PC2	eigenvalue	0.7720	(25.7%)	diagnostic	-0.051	hours	+0.619	final	+0.78
PC3	eigenvalue	0.0607	(2.0%)	diagnostic	+0.743	hours	+0.548	final	-

0.384

eigenvalues sum to 3.0000, one per standardised column

Read the loadings as recipes. **PC1**, which carries 72% of the variation, is *arrived prepared, practised little, scored well* — the three columns moving together in the pattern that has haunted these notes since [M6](#). It is a single axis along which the cohort mostly lies, and it is a confound rather than an effect.

PC2, with a further 26%, has almost no diagnostic score in it and loads practice and final score in the *same* direction: *practised a lot, scored well, regardless of how you arrived*. That is the real effect, the one [M11](#) recovered as a coefficient of +0.92 once the diagnostic score was held fixed. Nobody told the decomposition to look for it.

PC3 is worth 2% and is nearly nothing, which is itself informative: it says the three columns are close to satisfying one linear relation, so they carry about two dimensions of information rather than three.

Finally, the promised connection to conditioning. The singular values of the two design matrices from [M11](#) give their condition numbers directly:

```

h = usable["hours_practice"].to_numpy()
g = usable["diagnostic_score"].to_numpy()
ones = np.ones(len(h))

for name, X in [("1 + hours", np.column_stack([ones, h])),
                ("1 + hours + diagnostic", np.column_stack([ones, h, g]))]:
    sigma = np.linalg.svd(X, compute_uv=False)
    print(f"{name:<24} sigma = {np.round(sigma, 3)}")
    print(f"{'':24} sigma_max/sigma_min = {sigma.max()/sigma.min():8.2f}"
          f" np.linalg.cond = {np.linalg.cond(X):8.2f}")
    print(f"{'':24} cond(X'X) = {np.linalg.cond(X.T @ X):.1f}"
          f" and cond(X)^2 = {np.linalg.cond(X)**2:.1f}")

```

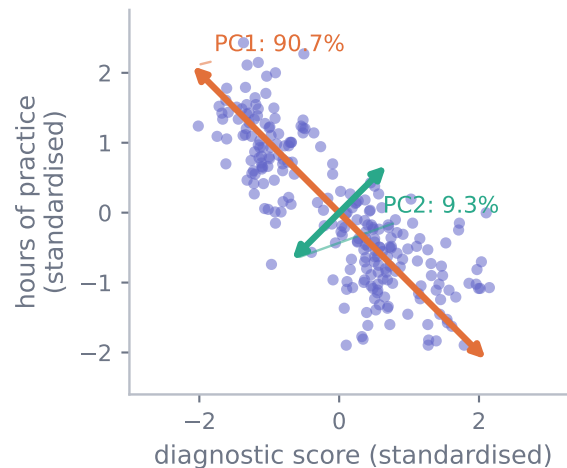


Figure 27.3: The cohort in two of its standardised columns, with the principal axes of that two-column problem drawn to lengths proportional to $\sqrt{\lambda}$. The cloud is an ellipse and the axes are its principal directions: 90.7% of the variation lies along the long one, which runs down and to the right — students who arrived stronger practised less. The other 9.3% is all that remains once you know where a student sits along it.

```
1 + hours          sigma = [282.956  6.829]
                    sigma_max/sigma_min =    41.43  np.linalg.cond =    41.43
                    cond(X'X) = 1716.7   and cond(X)^2 = 1716.7
1 + hours + diagnostic sigma = [1024.104  177.347  1.668]
                    sigma_max/sigma_min =   614.14  np.linalg.cond =   614.14
                    cond(X'X) = 377170.7   and cond(X)^2 = 377170.7
```

The third design's smallest singular value is 1.67 against a largest of 1024, and that near-zero direction is the collinearity of `M11` measured rather than described: the columns very nearly satisfy a linear relation, so there is a direction in parameter space that the design barely responds to. It is the long flat valley, given a number.

27.5 Try it yourself

Exercise 1 Find the eigenvalues and eigenvectors of $A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$.

💡 Show the solution

Eigenvalues solve $\det(A - \lambda I) = 0$:

$$(3 - \lambda)^2 - 1 = 0 \implies 3 - \lambda = \pm 1 \implies \lambda = 4 \text{ or } 2.$$

For $\lambda = 4$: $(A - 4I)\mathbf{v} = 0$ gives $-v_1 + v_2 = 0$, so $\mathbf{v} \propto (1, 1)$. For $\lambda = 2$: $v_1 + v_2 = 0$, so $\mathbf{v} \propto (1, -1)$.

Two things to notice. The eigenvectors are **orthogonal** — $(1, 1) \cdot (1, -1) = 0$ — as the theorem promises for a symmetric matrix. And they are directions: $(2, 2)$ and $(-1, -1)$ are the same eigenvector as $(1, 1)$, so a library returning $(-0.707, -0.707)$ has not given you a different answer.

Exercise 2 A 500×8 design matrix has singular values $\sigma = (40, 12, 9, 5, 3, 2, 1, 0.004)$. What is $\text{cond}(X)$, what is $\text{cond}(X^T X)$, and what should you do?

💡 Show the solution

$$\text{cond}(X) = \frac{40}{0.004} = 10,000, \quad \text{cond}(X^T X) = 10,000^2 = 10^8.$$

Working through the normal equations would cost about eight of the sixteen digits double precision offers; working from the SVD of X , about four. So first: use `lstsq` and not $(X^T X)^{-1}$.

But the deeper answer is that $\sigma_8 = 0.004$ against $\sigma_1 = 40$ is a message about the *data*, not the arithmetic. It says the eight columns very nearly satisfy a linear relation — that they carry about seven columns' worth of information. The coefficient attached to that near-null direction is whatever noise happens to say. Look for the relation, and consider dropping a column or combining two.

Exercise 3 You run PCA on a dataset with heights in metres and incomes in pounds, without standardising. PC1 comes out as almost exactly the income axis. Is that a discovery?

💡 Show the solution

No. Incomes vary over tens of thousands and heights over fractions of a metre, so income's variance dwarfs height's by a factor of billions before anything interesting has been considered. PCA maximises variance, and variance has units.

Re-express the incomes in thousands of pounds and heights in millimetres and PC1 will change, from data that is identical in every respect that matters. That is the test: a result that moves when you change a unit is a result about the units.

Standardise each column first — subtract the mean and divide by the standard deviation — which is the same as working from the correlation matrix rather than the covariance matrix, and is what the worked example above does. The exception is when the columns are already in the same units and their relative sizes are genuinely meaningful, in which case leaving them alone is the informed choice.

27.6 Check it in Python

First, the definition:

```
A2 = np.array([[3.0, 1.0], [1.0, 3.0]])
values2, vectors2 = np.linalg.eigh(A2)

for k in range(2):
    v = vectors2[:, k]
    print(f"lambda = {values2[k]:.4f}    A @ v = {np.round(A2 @ v, 4)}"
          f"    lambda * v = {np.round(values2[k]*v, 4)}"
          f"    equal: {np.allclose(A2 @ v, values2[k]*v)}")
print("eigenvectors orthogonal:", np.isclose(vectors2[:, 0] @ vectors2[:,
↪ 1], 0))
```

```

lambda = 2.0000    A @ v = [-1.4142  1.4142]    lambda * v = [-1.4142  1.4142]    equal: True
lambda = 4.0000    A @ v = [2.8284 2.8284]    lambda * v = [2.8284 2.8284]    equal: True
eigenvectors orthogonal: True

```

Then the bridge between the SVD and the eigenvalues, which is the claim the condition-number result rests on:

```

X = np.column_stack([ones, h, g])
U, sigma, Vt = np.linalg.svd(X, full_matrices=False)

eig_of_XtX = np.sort(np.linalg.eigvalsh(X.T @ X))[:-1]
print("singular values of X      ", np.round(sigma, 4))
print("their squares              ", np.round(sigma**2, 4))
print("eigenvalues of X.T @ X     ", np.round(eig_of_XtX, 4))
print("match:", np.allclose(sigma**2, eig_of_XtX))
print("U S V.T rebuilds X:", np.allclose(U @ np.diag(sigma) @ Vt, X))

```

```

singular values of X      [1024.1041  177.3473    1.6675]
their squares             [1.04878930e+06  3.14520663e+04  2.78070000e+00]
eigenvalues of X.T @ X   [1.04878930e+06  3.14520663e+04  2.78070000e+00]
match: True
U S V.T rebuilds X: True

```

And a check that fails, on purpose. It is tempting to compare eigenvectors entry by entry; the sign is arbitrary, so that comparison is unreliable even when nothing is wrong:

```

mine = vectors2[:, 0]
theirs = -vectors2[:, 0]          # an equally valid answer to the same
    ↪ problem

print("mine   ", np.round(mine, 4))
print("theirs", np.round(theirs, 4))
print("allclose on the entries:      ", np.allclose(mine, theirs))
print("same direction (|dot| == 1):  ", np.isclose(abs(mine @ theirs),
    ↪ 1.0))

```

```

mine   [-0.7071  0.7071]
theirs [ 0.7071 -0.7071]
allclose on the entries:      False
same direction (|dot| == 1):  True

```

The first test says the two disagree and the second says they do not. The second is the one to trust: compare eigenvectors by the absolute value of their dot product *after* scaling both to unit length — which is what `eigh` returns, so the check above is safe as written — or by fixing a sign convention yourself, as the worked example did.

27.7 Common wrong turns

Where this usually goes wrong

Treating an eigenvector's sign or length as meaningful $\mathbf{v}$ and $-2\mathbf{v}$ are the same eigenvector. Two libraries, or the same library on two days, may hand you opposite signs, and neither is wrong. Never compare them with `==` or `allclose` on the raw entries.

Running PCA on unstandardised columns Variance carries units, so the column with the largest *variance* wins — which is not the same as the column with the largest numbers, since PCA centres first. A column of values near a billion that barely moves contributes almost nothing. Unless every column is already in the same units for a good reason, standardise first — that is, decompose the correlation matrix rather than the covariance matrix.

Calling PC1 “the most important variable” It is not a variable at all; it is a weighted combination of all of them. In the cohort, PC1 is *arrived prepared, practised little, scored well* — three columns at once, which is precisely why it was worth computing.

Reading a small eigenvalue as a useless direction It says the data barely varies along there, which usually means the columns nearly satisfy a linear relation. That is a finding about your variables, and often the most actionable thing in the output.

Expecting eigenvectors from a non-square matrix $A\mathbf{v} = \lambda\mathbf{v}$ needs input and output in the same space. A 238×3 matrix has no eigenvectors; it has singular values and vectors, which is what the SVD exists to provide.

Assuming a large condition number means you made a mistake It usually means your columns overlap. The remedy is in the data — drop a column, combine two, collect a design that separates them — not in the solver.

27.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. X is a design matrix with two independent columns and singular values 10 and 2. What is $\text{cond}(X^T X)$?
a. 5 b. 25 c. 10 d. 100
2. Why does a correlation matrix always have orthogonal eigenvectors, while a general square matrix need not?
3. The eigenvalues of a 4×4 correlation matrix are 2.9, 0.9, 0.2, 0.0. What does the last one tell you, and what do the four sum to?
4. Your PCA gives PC1 loadings (0.71, -0.71); a colleague's gives (-0.71, 0.71) on the same data. Who is wrong?
5. In one sentence each: what do U , Σ and V contribute in $X = U\Sigma V^T$?

 Show the answers

1. **(b)** 25. $\text{cond}(X) = 10/2 = 5$, and the eigenvalues of $X^T X$ are the squares of the singular values, so the ratio squares: 25. Option (a) is $\text{cond}(X)$, the most tempting wrong answer; (d) squares the wrong thing.

2. Because a correlation matrix is **symmetric**, and the spectral theorem guarantees real eigenvalues and orthogonal eigenvectors for real symmetric matrices. A general square matrix has no such guarantee — a 90° rotation of the plane has no real eigenvectors at all, because it turns every direction. (Not every rotation does: a 180° rotation is $-I$ and leaves every direction unturned, and a rotation in three dimensions always fixes its axis.)
3. A zero eigenvalue means there is a direction along which the data does not vary at all: the four columns satisfy an exact linear relation, so one is redundant and the correlation matrix is singular. The four sum to **4** — the eigenvalues of a correlation matrix sum to the number of columns, because each standardised column contributes variance 1.
4. **Neither.** The sign of an eigenvector is arbitrary, and $(0.71, -0.71)$ and $(-0.71, 0.71)$ are the same direction. If they disagreed by anything other than an overall sign there would be something to investigate.
5. V names the perpendicular input directions; Σ says how much each is stretched; U names the perpendicular output directions those stretched pieces are laid along.

Where this shows up next

That completes the Mathematics track. What it was for becomes visible in Statistics.

S3 uses correlation as a summary of two columns; this unit showed what a whole matrix of them contains. **S11** fits the design matrix of **M11** and asks how uncertain its coefficients are — a question whose answer is governed by the singular values above. **S12** returns to PC1, the *arrived prepared, practised little, scored well* direction, and gives it its proper name.

Part IV

Statistics

28 What is the question?

Statistics · Unit 1

S1 · Statistics Reading: 8 pages With practice: 1.5 hours Assumes: nothing — you can start here

Why this unit is here

Statistics is usually taught as a menu of procedures, and it is easy to come away thinking the skill is picking the right one. It is not. The procedures are the easy part; a library computes them for you in one line.

The hard part is knowing *what you are asking, about whom, and what the answer entitles you to say*. Get that wrong and no amount of correct arithmetic saves you — you get a confident, reproducible, well-presented answer to a question nobody asked. This unit gives you the vocabulary for stating a statistical question properly, and one habit: **write the question down before you touch the data**.

28.1 Before you start

Nothing statistical. You should be able to read a table of data and say what one row of it means, work out an average, and read a percentage.

If you also want to *run* the code rather than read it, [P8](#) is as much pandas as you need. Nothing in the explanations depends on it.

Quick check. A table has one row per student, with columns for their prior background, hours of practice and final score. Five students scored 58, 62, 71, 64 and 65. What is their mean score, and what does one *column* of that table represent?

💡 Show the answer

The mean is $(58 + 62 + 71 + 64 + 65)/5 = 320/5 = 64$.

A column is one **variable** — one thing measured on every student. A row is one **unit** — everything measured on one student. Almost every confusion later in this track is really a confusion about which of those two you are talking about.

28.2 The idea

Here is a question of the kind people actually ask:

Does practice help?

It sounds answerable. It is not, and why not is the whole of this unit.

Help whom? The students in this table, or students in general? **Help how much, measured how?** Final score, pass rate, something else? **Help in what sense** — do people who practise more *tend to* score higher, or would a given person score higher *if they practised more*? Only the second of those is about cause.

Underneath all three is the gap statistics exists to manage: you have a table, and the claim you want to make is almost never about the table.

Throughout these notes we use one dataset: a synthetic record of a previous cohort who worked through this material.

```
import pandas as pd

cohort = pd.read_csv("../data/cohort.csv")
print(f"{len(cohort)} rows, {cohort.shape[1]} columns")
cohort.head(3)
```

240 rows, 7 columns

	student_id	background	diagnostic_score	hours_practice	final_score	completed	st
0	S0001	Mathematics or Physics	89.1	12.9	83.6	True	20
1	S0002	Life sciences	42.4	22.8	56.6	True	20
2	S0003	Engineering	68.4	1.7	58.5	True	20

Two hundred and forty students — and not one of them is the point. They have finished, and nothing you compute will change their marks. What you care about is students *like* them, including the ones arriving next year. Those rows are interesting only as evidence about a larger group you cannot see.

The gap has a standard vocabulary, worth learning properly.

- The **population** is the entire group your question is about. Here: something like *students starting an MSc in data science from a mixed set of prior backgrounds*. Note that you had to *decide* that. It was not in the file.
- The **sample** is the set of units you actually measured — the 240 rows.
- An **estimand** is the number you want, defined on the population: say μ , the mean final score across that whole population. It is one fixed number, and short of measuring every single unit you never see it.
- An **estimate** is the number you compute from the sample — here $\bar{x} = 65.3$, the mean of the 237 rows the worked example keeps. You do see it, and it is not μ .

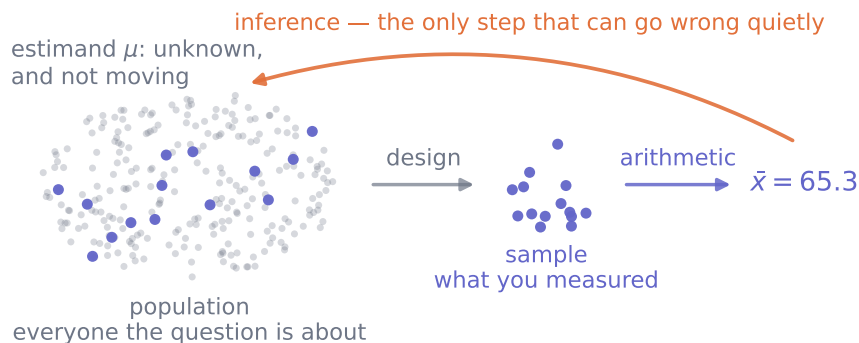


Figure 28.1: The shape of every statistical question. Arithmetic on a sample is safe and mechanical. The arrow that goes back — from the number you computed to a claim about everyone — is the one that can be wrong while everything on the page looks fine.

That picture splits the subject in two. **Describing** your sample is arithmetic: the mean of the 237 usable scores is 65.27, and no assumption is needed to say so — S2 and S3. **Inferring** something about the population is a *claim*, it depends on how the sample was collected, and it is only ever true with a margin attached. Nearly everything from S4 onwards exists to make that margin honest.

28.3 The mechanics

28.3.1 Four things a usable question names

A question is ready to work on when you can name all four of these. Until then you cannot choose a method, and choosing one anyway wastes a week.

1. **The unit.** What does one row represent — a student, a submission, a week-of-a-student? Get this wrong and every count you report is wrong.
2. **The population.** Which units is the answer about? Not “the data”: the group you intend to speak about.
3. **The estimand.** A number defined on that population, stated precisely enough that someone else would compute the same thing. “How well students do” is not one; “the mean final score” is.
4. **The design.** How did units get from the population into your table? This is the part people skip, and it decides which conclusions are defensible.

One legitimate sharpening of our vague question:

Among students arriving from a *life sciences* background (population), taking a student as the unit, what is the **difference in mean final score** between those who practised *more than* the group median and those who practised *no more than* it (estimand)? The data are one past cohort’s recorded outcomes, with nobody assigned to practise (design).

Even the tie rule had to be stated: one life-sciences student practised for exactly the group median, and a definition saying “more” and “less” leaves them in neither group.

Longer, uglier, answerable — and it makes visible, in that last clause, something the original question hid.

28.3.2 Parameters and estimates

The convention is Greek letters for the population and Latin for the sample.

Table 28.1: The same quantity, in the two places it can live

Quantity	Population (parameter)	Sample (estimate)
mean	μ	$\bar{x}$
standard deviation	σ	s
proportion	p	$\hat{p}$

A convention rather than a law — you will meet $\hat{\mu}$ and $\hat{\theta}$ too, where the hat means “estimated from data” — but the distinction is real and load-bearing. **The parameter is fixed and, short of a census, unknown; the estimate is known and varies.** Draw a different 25 students and you will almost certainly compute a different mean; the population mean does not move, because it never depended on who you happened to ask.

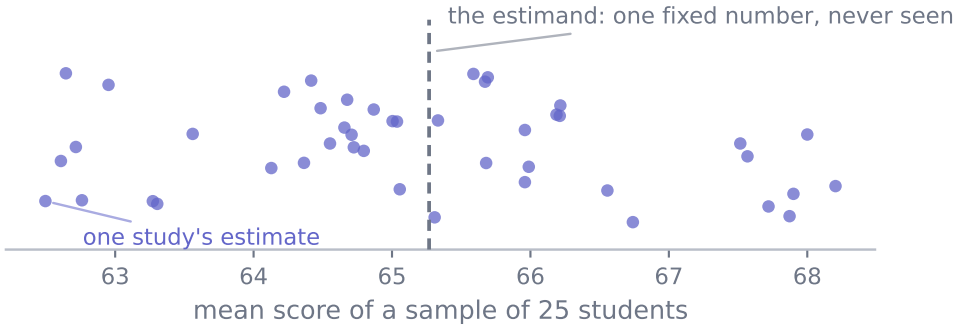


Figure 28.2: Forty-five imaginary studies, each measuring 25 students from the same population and reporting their mean score. The dashed line is the population mean. It does not move; the dots do. Every real study you will ever read is one dot, reported without the rest of the picture.

That spread is not a defect in anyone’s work. It is what sampling *is*, and S7 puts a number on it.

28.3.3 What kind of variable is it?

The type of a variable decides which arithmetic on it means anything.

Table 28.2: Five of the cohort’s columns by type

Column	Type	What that permits
final_score	quantitative, continuous	mean, differences, spread
background	categorical, nominal	counts, proportions, mode
completed	categorical, binary	counts, proportions
started_on	date	ordering, gaps, counts per day

Column	Type	What that permits
<code>student_id</code>	label	nothing — it is a name, not a number

Quantitative splits into continuous and discrete. A count — submissions, faults, siblings — is discrete: nothing lies between 3 and 4. Time, mass or a score on a fine scale is continuous, and the distinction drives the choice of model in [S6](#). It is about the quantity, not the file: `final_score` is recorded to one decimal, so the stored values are strictly discrete, but the thing measured varies continuously.

Categorical splits into nominal and ordinal. Nominal categories have no order — asking whether Computing is “more than” Engineering is meaningless. Ordinal ones are ordered but not evenly spaced: a satisfaction rating, a degree classification. A median is safe there; a *mean* assumes the gap from 1 to 2 matches the gap from 4 to 5, which nobody has checked and which is usually false. It is done constantly anyway; do it knowing that you are.

28.3.4 Where the sample came from

Three designs, in decreasing order of how much they let you say. A **census** measures every unit, so there is no inference left to do — rare, and usually impossible. A **simple random sample** gives every unit the same chance of selection, and is what the rest of this track assumes when it says “random”. A **convenience sample** is whoever was available: the people who replied, the users who opted in, the cohort whose records were kept.

“Random” alone is not enough. A design that samples one subgroup with probability 0.9 and another with probability 0.1 is perfectly random and systematically skewed; such designs are legitimate, and the analysis then has to weight each unit by the reciprocal of its selection probability. And note what even equal chances promise: not that any one sample resembles the population, but that the *procedure* has no systematic preference and that the variability is quantifiable.

Two consequences, both denied in practice constantly.

Sample size does not fix a bad design. If the mechanism that put units in your table is related to what you are measuring, more data makes the estimate more precisely wrong. A 1936 US election poll collected 2.4 million responses and called the wrong winner, beaten by a survey of a few thousand: two million of the wrong people is still the wrong people.

Observational is not experimental. In an experiment the investigator assigns the treatment — who practises more is decided by a coin, not by the student — so a difference in outcome can be laid at the treatment’s door. Our cohort is observational, and students who chose more hours differ from those who chose fewer in every other way too.

28.4 Worked example

One vague question, three precise ones, two opposite answers.

Every analysis on this dataset starts by deciding which rows are usable. The file has three planted defects, described in [P9](#).

```
print(f"rows in the file {len(cohort)}    usable {len(usable)}    "
      f"excluded {len(cohort) - len(usable)}")
```

```
rows in the file 240    usable 237    excluded 3
```

One row has no recorded score, one has no background, and one scores 187 out of 100. Every unit in these notes that filters on score *and* group analyses these same 237 students.

Now three sharpenings of *does practice help?*

Question A — across the whole cohort. Split the 237 at the median number of hours — above it against at-or-below it — and compare mean final scores.

```
split = usable["hours_practice"].median()
more = usable[usable["hours_practice"] > split]["final_score"]
less = usable[usable["hours_practice"] <= split]["final_score"]
print(f"median hours of practice : {split}")
print(f"practised less  n={len(less):>3}    mean {less.mean():.2f}")
print(f"practised more  n={len(more):>3}    mean {more.mean():.2f}")
print(f"    difference {more.mean() - less.mean():+.2f} marks")
```

```
median hours of practice : 15.2
practised less  n=121    mean 68.01
practised more  n=116    mean 62.40    difference -5.61 marks
```

The students who practised more scored **5.61 marks lower** on average. On this reading, practice looks harmful.

Question B — within each background. The same comparison, and the same tie rule, but each group split at its own median, so students are only ever compared with others who arrived in a similar position.

```
def practice_gap(frame):
    """Above-median students' mean score minus the rest's."""
    cut = frame["hours_practice"].median()
    above = frame["hours_practice"] > cut
    return frame[above]["final_score"].mean() -
        ↪ frame[~above]["final_score"].mean()

by_group = usable.groupby("background").apply(practice_gap,
        ↪ include_groups=False)
print(by_group.round(2).to_string())
```

```
background
Computing          4.10
Engineering        5.14
Life sciences      7.63
Mathematics or Physics 5.24
Social sciences    7.61
```

In **every one of the five groups** the above-median students scored higher, by between 4.10 and 7.63 marks. On this reading, practice helps, and helps consistently.

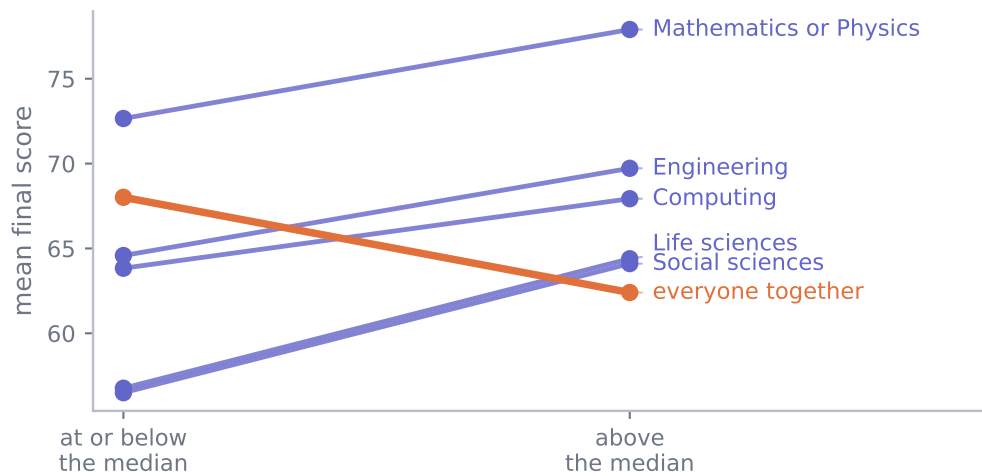


Figure 28.3: The same 237 students, summarised two ways. Each line joins the mean score of a set's at-or-below-median students to that of its above-median students. The five indigo lines split their own background group at that group's median; the orange line splits the whole cohort at the overall median (121 students against 116, the three exactly on 15.2 hours counting as at-or-below). Five lines up, one line down.

Both answers are correct arithmetic on the same 237 rows. They point in opposite directions because they answer different questions, and the reason is in the data.

```
columns = ["diagnostic_score", "hours_practice", "final_score"]
print(usable.groupby("background")[columns].mean()
      .sort_values("hours_practice").round(1).to_string())
```

	diagnostic_score	hours_practice	final_score
background			
Mathematics or Physics	89.3	8.0	75.3
Engineering	72.4	10.5	67.2
Computing	68.9	12.5	65.9
Life sciences	47.3	22.6	60.5
Social sciences	39.5	27.4	60.2

Students from Mathematics or Physics start furthest ahead and practise least; those from life and social sciences start furthest behind and practise most. So Question A compares a mostly-life-and-social-sciences half against a mostly-quantitative half: a comparison of *backgrounds* wearing the costume of a comparison of *practice*. Question B holds background fixed and asks the question you meant. The reversal has a name and a large literature, and S12 gives it both.

Question C — would practising more raise *your* score? This is what a student actually wants to know, and the data cannot answer it. Nobody assigned hours of practice, and whatever made a student choose more of them — anxiety, spare time, having further to go — is unmeasured. Question B is closer to the causal question than Question A, and it is still an association among students who chose their own hours.

Saying so is not a failure of the analysis. It is the analysis.

28.5 Try it yourself

Exercise 1 Classify each of these variables as quantitative continuous, quantitative discrete, categorical nominal, categorical ordinal, or a label:

- (a) a student's postcode;
- (b) the number of assignments a student submitted;
- (c) a satisfaction rating on a scale of 1 (poor) to 5 (excellent);
- (d) time taken to complete an exam, in minutes;
- (e) degree classification: third, 2:2, 2:1, first.

Show the solution

- (a) **Nominal categorical.** A postcode contains digits, which tempts people to treat it as a number, but the arithmetic is meaningless — there is no sense in which AL10 is twice AL5.
- (b) **Quantitative discrete.** A count. Nothing lies between 3 and 4 submissions.
- (c) **Ordinal categorical.** Ordered — 4 really is better than 3 — but the gaps are not known to be equal. The median is safe. The mean is a convention that assumes equal spacing; it is reported everywhere, and it is worth knowing that you are making an assumption when you report it.
- (d) **Quantitative continuous.** If it is recorded to the nearest minute the stored values are whole numbers, but the underlying quantity is continuous and you would model it that way.
- (e) **Ordinal categorical,** for the same reason as (c). Whether the gap between a 2:2 and a 2:1 is the same as the gap between a 2:1 and a first is not a question the scale answers.

Exercise 2 A colleague asks: “*Do students who use the discussion forum do better?*”

Rewrite it as a question you could work on. Name the unit, the population, the estimand and the design, and then name one conclusion the design will not support.

Show the solution

There is no single right answer — that is the point of the exercise — but a usable one names all four. For example:

- **Unit:** one student.
- **Population:** students enrolled on this module in a given year. (If you want to speak about future cohorts, say so, and be ready to defend it.)
- **Estimand:** the difference in mean final score between students who posted at least once and students who never posted.
- **Design:** observational, from the module's own records; nobody was assigned to use the forum.

What it will not support: that using the forum *causes* better marks. The students who post are self-selected, and engagement, confidence and available time all plausibly drive both posting and marks. Also worth stating: “at least one post” is a decision you made, not a fact about the world, and the answer may depend on it. Write it down so the choice is visible rather than buried.

Exercise 3 A satisfaction survey is posted on a course forum. Of 900 enrolled students, 180 reply, and their mean rating is 4.2 out of 5.

- (a) What population is 4.2 an estimate for?
- (b) Name two mechanisms that could make it a poor estimate of the mean rating of all 900.
- (c) Would surveying twice as many students the same way fix the problem?

 Show the solution

- (a) Two different things are being run together, and separating them is the point. As a **description**, 4.2 is the mean rating of the 180 people who replied, and that needs no assumption at all. As an **estimate** it targets whichever population you nominate — nominate all 900 and 4.2 is an estimate of their mean rating, a biased one unless repliers resemble non-repliers *in respect of satisfaction*. Bias is a property of how good the estimate is, not of whether it is one. The population it describes for free and the population it is offered as evidence about are different, and that gap is the finding.
- (b) Any two of: **self-selection** — people with strong opinions, in either direction, are likelier to answer; **undercoverage** — the survey lives on a forum, so students who do not read the forum cannot reply, and forum readers are plausibly more engaged; **timing** — a survey posted in week 12 misses everyone who dropped out, who are unlikely to have been the satisfied ones.
- (c) **No**. Surveying more people the same way leaves the mechanism intact. The estimate would become more precise and no less biased: a tighter cluster around the wrong number. Only a change of *design* — chasing non-respondents, sampling from the enrolment register rather than the forum — helps. Note too that the ceiling here is 900: at some point you cannot buy precision with more people at all, and everything left is bias.

28.6 Check it in Python

Two claims in this unit are worth checking rather than believing: that the cleaning step removes three rows and no more, and that the reversal is real rather than a slip in the reading.

```
assert len(cohort) == 240, f"expected 240 rows, got {len(cohort)}"
assert len(usable) == 237, f"expected 237 usable, got {len(usable)}"

dropped = cohort.loc[~cohort.index.isin(usable.index)]
print(dropped[["student_id", "background", "hours_practice",
↪ "final_score"]])
```


	student_id	background	hours_practice	final_score
17	S0018	Computing	14.3	NaN
102	S0103	NaN	26.4	66.2
188	S0189	Life sciences	23.1	187.0

One missing score, one missing background, one impossible score. Exactly the three defects the dataset documents, and no accidental fourth.

Now the reversal. `practice_gap` is written once and applied to the whole cohort and to each group, so if the answers differ it is the data doing it rather than two slightly different calculations.

```
overall = practice_gap(usable)
print(f"whole cohort : {overall:+.2f}")
print(f"by group      : {by_group.min():+.2f} to {by_group.max():+.2f}")
```

```
whole cohort : -5.61
by group      : +4.10 to +7.63
```

A reasonable person, having seen the whole-cohort number, would assume the groups agree with it. Assert that, and watch it fail:

```
assert (by_group < 0).all(), (
    f"whole cohort gives {overall:+.2f}, but "
    f"{(by_group > 0).sum()} of {len(by_group)} groups disagree in sign"
)
```

AssertionError: whole cohort gives -5.61, but 5 of 5 groups disagree in sign

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[11], line 1
----> 1 assert (by_group < 0).all(), (
      2     f"whole cohort gives {overall:+.2f}, but "
      3     f"{(by_group > 0).sum()} of {len(by_group)} groups disagree in
    ↪  sign"
      4 )
```

AssertionError: whole cohort gives -5.61, but 5 of 5 groups disagree in sign

That is the check doing its job. The assertion is not wrong about the arithmetic; it is wrong about the world, and it says so loudly instead of letting a false assumption travel unnoticed into a report.

The statement that survives is the one with both halves in it:

```
assert (by_group > 0).all(), "not every group shows a positive gap"
assert overall < 0, "the whole-cohort gap is not negative"
print(f"all {len(by_group)} groups positive (smallest
    ↪ {by_group.min():+.2f}), "
      f"cohort as a whole {overall:+.2f}")
```

all 5 groups positive (smallest +4.10), cohort as a whole -5.61

Both facts are true at once, and any sentence you write about this dataset has to be compatible with both.

28.7 Common wrong turns

Where this usually goes wrong

Choosing the method before stating the question “Which test should I use?” is not a first question, it is a fourth. Until the unit, population, estimand and design are written down, no test is the right one — and the answer is often that you do not need a test at all, you need an estimate with an interval on it.

Reporting a sample statistic as if it were the population value “The mean score was 65.3” describes 237 people. “We estimate the mean score of students on this course at about 65” is a claim about a population. The second needs a margin attached and the first does not; sliding from one to the other is how a description quietly becomes an overclaim.

Believing that a large sample makes a sample representative Size controls how much your estimate bounces around; the design controls what it is bouncing around. A million self-selected responses estimate the views of self-selecting people very precisely.

Reading an association as a lever In the cohort, practising more goes with scoring higher within a background. That does not establish that a given student adding five hours gains five marks. For that you would need the hours to have been assigned, or an argument about what else differs — and the argument is the hard part.

Averaging things that do not average The mean of a satisfaction rating assumes the rungs of the ladder are evenly spaced. The mean of a category code is nonsense dressed as a number. If a variable is nominal, count it; if it is ordinal, prefer the median and say what you did.

Letting the data pick the question Slicing until something interesting appears and then reporting the slice as though it had been the plan is the commonest way honest people produce false findings. Writing the question down first is a defence precisely because it is dated evidence of what you meant to ask.

28.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer — if you cannot say *why*, treat it as wrong.

1. Which of these is an estimand?
 - a. $\bar{x} = 65.3$, the mean score of the 237 usable rows
 - b. The mean final score of all students on this kind of course
 - c. The `final_score` column
 - d. The decision to exclude three defective rows
2. A dataset records, for each of 60 patients, their blood pressure at four visits. A colleague computes a confidence interval for mean blood pressure and reports it as based on “ $n = 240$ independent observations”. What has gone wrong, and what is the unit?

3. True or false: taking a random sample guarantees the sample resembles the population.
4. In the worked example, Question A and Question B gave answers with opposite signs. Which one was calculated incorrectly?
5. You are given the mean of a variable coded 1 = Computing, 2 = Engineering, 3 = Life sciences, 4 = Mathematics or Physics, 5 = Social sciences. It is 2.87. What does that tell you?

 Show the answers

1. **(b).** An estimand is a number defined on the *population*. (a) is an estimate — computed, known, and liable to differ from one sample to the next. (c) is a variable, not a number. (d) is part of the design and analysis plan, which matters enormously but is not a quantity being estimated.
2. The unit has been confused with the measurement. There really are 240 rows and 240 measurements — that part is fine. What is wrong is *independent*: the **unit is the patient**, so there are only 60 independent units, and four readings from one person carry much less information than four readings from four people. Treating them as 240 independent observations overstates how much you know — [S8](#) shows exactly what it costs.
3. **False**, and the distinction is worth being precise about. Random sampling makes the *procedure* well behaved: over many repetitions it neither favours nor disfavors any part of the population, and the variability is quantifiable. Any single random sample can still be unlucky, and small ones often are. What random sampling rules out is *systematic* mismatch, not bad luck.
4. **Neither.** Both are correct arithmetic and you can rerun them. They answer different questions — one compares students against the whole cohort, the other only against students from the same background — and there is no rule that two different questions must have answers of the same sign. The mistake would be to report either one as “the effect of practice”.
5. **Nothing.** The codes label a nominal variable, so their mean depends entirely on the arbitrary order in which the names were numbered. Renumber the backgrounds in reverse alphabetical order and the “mean” changes while the data do not. Count the categories or report proportions instead.

Where this shows up next

[S2](#) takes one variable and describes it properly — centre, spread, shape — and shows why a mean on its own is rarely enough. [S3](#) does the same for two variables at once, which is where the comparison in this unit’s worked example gets its proper treatment. [S4](#) starts building the machinery that turns a description of a sample into a claim about a population. [S12](#) returns to the reversal above, names it, and shows how to spot the pattern before it embarrasses you.

29 Describing one variable

Statistics · Unit 2

S2 · Statistics Reading: 6 pages With practice: 2 hours Assumes: S1

Why this unit is here

You have a column of numbers and one sentence in which to describe it. Whatever you put in that sentence decides what your reader notices — and what they never find out.

This is also the foundation floor. The confidence intervals, tests and regression models you will meet first are all built out of a mean and a standard deviation, so the places where those two numbers mislead are the places where everything built on top of them misleads too. Knowing what a summary hides is the difference between using these tools and being used by them.

29.1 Before you start

You should be able to:

- say what a variable is and what one row of a table represents — S1;
- tell a quantitative variable from a categorical one — S1;
- work out an average by hand.

Quick check. Six values: 4, 8, 8, 9, 12, 100. What is the mean, and what is the median?

 Show the answer

The mean is $141/6 = 23.5$. The median is the average of the two middle values once sorted, $(8 + 9)/2 = 8.5$.

They disagree by a factor of nearly three, and neither is wrong. Five of the six values are below the mean. That gap — and what it is telling you — is most of this unit.

29.2 The idea

Describing one variable comes down to three questions, in this order:

1. **Where is it?** A typical value.
2. **How spread out is it?** How far from typical a value usually falls.
3. **What shape is it?** Symmetric or lopsided, one peak or several, with or without stragglers.

Most people answer the first, occasionally the second, and skip the third. That is a mistake, and the following picture is why.

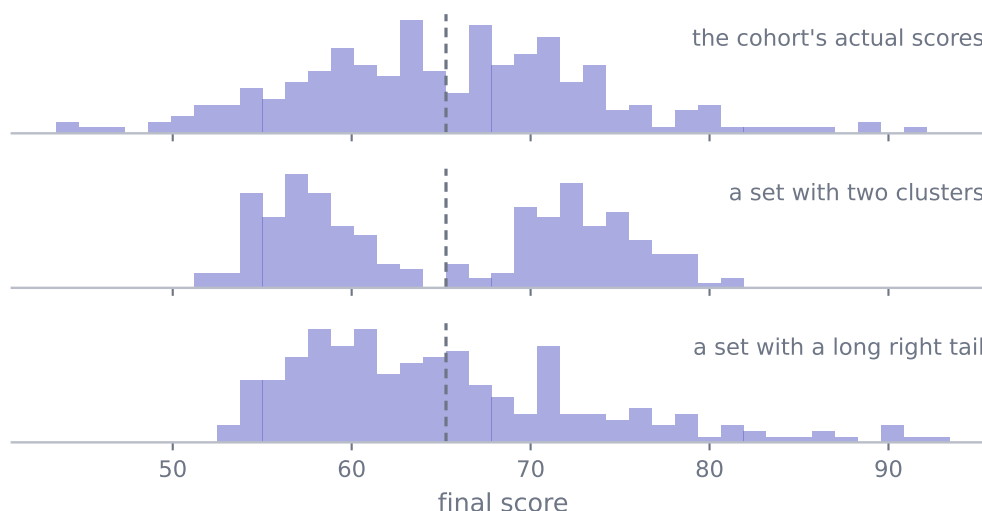


Figure 29.1: Three sets of 238 numbers with the same mean (65.3, dashed) and the same standard deviation (8.4). Only the top one is the cohort. A mean and a standard deviation are two numbers; the data is two hundred and thirty-eight. Two numbers cannot be the data.

Every summary is a compression, and compression throws things away. The skill is knowing what each one throws away, so that you can choose a summary that keeps the part that matters and can say what you have lost.

29.3 The mechanics

29.3.1 Where is it?

Three answers, and they answer slightly different questions.

The **mean** is the balance point:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

(If $\sum$ is unfamiliar notation, M1 unpacks it. It says “add these up”.) The mean uses every value, which is its strength and its weakness: every value gets a vote, including the ones that should not have one.

The **median** is the half-way point *by count*: sort the values, take the middle one, or the average of the two middle ones if n is even. Half the data is at or below it. Moving the largest observation from 92 to 920 does not move the median at all, because it is still the largest.

The **mode** is the most frequent value. It is the natural centre for a *nominal* variable, where neither the mean nor the median is available. For continuous data it is mostly an artefact of rounding — our scores are recorded to one decimal place, and the “most common score” is just whichever value happened to be recorded four times rather than three.

When the mean and the median disagree, the disagreement is information. It says the values on one side of the median sit further from it than the values on the other, so

the mean has been pulled that way — and that is usually the first sign of a long tail or a handful of extreme observations.

29.3.2 Quantiles

The median generalises. The ***p*-quantile** is a value at or below which approximately a proportion p of the observations lie; the median is the 0.5-quantile. The 0.25- and 0.75-quantiles are the **quartiles**, Q_1 and Q_3 , and together with the minimum, median and maximum they form the **five-number summary**.

“Approximately” is doing real work in that definition. With n observations there is generally no value with exactly a proportion p below it, so software interpolates between the two neighbouring sorted values — and there is more than one defensible way to do it. On $\{2, 4, 7, 9\}$, asking for Q_1 gives 2.0, 3.0, 3.5 or 4.0 depending on which of NumPy’s thirteen conventions you choose, and even on the cohort’s 238 scores they do not all agree: the default returns $Q_1 = 59.4$ and two of the others return 59.3 and 59.325. The differences shrink as n grows and are usually invisible, but when a quartile is load-bearing, record which convention produced it.

29.3.3 How spread out is it?

The **range**, $\max - \min$, uses exactly two of your observations and throws away the rest. It also grows with sample size — measure more people and you will eventually meet a more extreme one — so it is not comparable between samples of different sizes.

The **interquartile range**, $IQR = Q_3 - Q_1$, is the width of the middle half. It ignores the tails entirely, which is exactly what you want when the tails are unreliable and exactly what you do not want when they are the point.

The **sample variance** measures spread by average squared distance from the mean:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2,$$

and the **sample standard deviation** s is its square root.

Two things about that formula deserve more than a nod.

Why square? Because the deviations $x_i - \bar{x}$ sum to zero by construction — that is what makes $\bar{x}$ the balance point — so their average is always 0 and carries no information. Squaring removes the signs. It also weights a deviation of 10 four times as heavily as one of 5, which is a real modelling choice and part of why s is sensitive to extreme values.

Why $n - 1$? Because the deviations are measured from $\bar{x}$ rather than from the population mean μ , and $\bar{x}$ sits in the middle of *this particular sample* by construction. That makes the squared deviations systematically a little too small. Dividing by $n - 1$ instead of n corrects it exactly, in the sense that under independent sampling with finite variance the long-run average of s^2 equals σ^2 . Note what that does *not* say: it does **not** make s an unbiased estimate of σ , because a square root is not a linear operation and averaging does not survive it.

Units matter. If scores are in marks then s^2 is in marks *squared*, which is not a quantity anyone has intuition about. s is back in marks and can be compared to the data. That is the only reason the standard deviation is reported more often than the variance.

29.3.4 What shape is it?

Skew is asymmetry: which side has the longer tail. A common heuristic is that $\bar{x} > \text{median}$ indicates a right (positive) skew.

It is a useful prompt and it is not a theorem. Take $\{0, 1, 1, 2, 2\}$: the mean is 1.2, the median is 1, so the rule says right-skewed — but the standard skewness coefficient works out at -0.34 , which says the opposite. Treat a gap between mean and median as a reason to plot the data, not as a conclusion about its shape.

Modality is how many peaks there are. Two peaks usually means two populations have been stacked on top of each other, and the right response is almost never a better summary of the whole — it is to split them and describe each. The cohort's scores are a mixture of five backgrounds, which is worth remembering every time you look at them.

29.3.5 Seeing it

A **histogram** bins the values and draws the counts. The bin width is a choice you make, and it changes the picture:

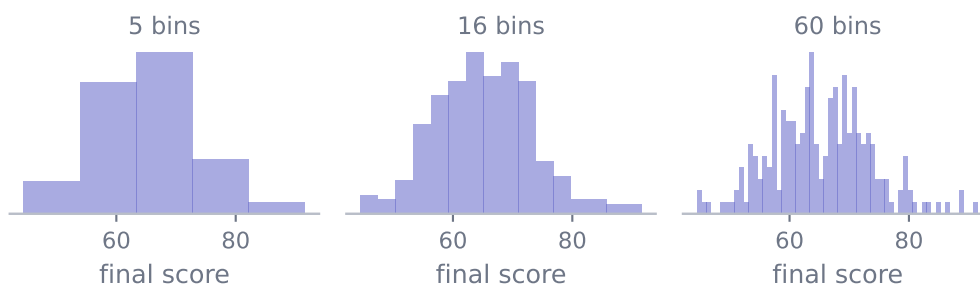


Figure 29.2: The same 238 scores at three bin widths. Too few bins and the shape is a rumour; too many and you are looking at which decimal each mark happened to land on. Nothing about the data changed between these three panels.

There is no correct answer, which is why you should look at more than one before believing any of them.

A **boxplot** draws the five-number summary. In the Tukey convention the box spans Q_1 to Q_3 with the median marked inside it, and the whiskers reach to the most extreme observation still within $1.5 \times \text{IQR}$ of the box. Anything further out is drawn as an individual point.

Two things people get wrong about that. The whiskers are **not** the minimum and maximum — they stop at the last point inside the fence. And a point drawn beyond a whisker is **not** thereby an error: it is a value that is far from the middle, which is a description, not a verdict. Other conventions put the whiskers at different places, so label which one you used.

29.4 Worked example

Describe the cohort's final scores, properly.

```

print(f"n          {scores.size}")
print(f"mean      {scores.mean():6.2f}      sd
↪ {scores.std(ddof=1):6.2f}")
print(f"median    {np.median(scores):6.2f}      IQR   "
      f"{np.subtract(*np.quantile(scores, [0.75, 0.25])):6.2f}")
print("five-number summary  " +
      " ".join(f"{q:.1f}" for q in np.quantile(scores, [0, .25, .5, .75,
↪ 1])))

```

```

n          238
mean      65.27      sd      8.38
median    64.80      IQR     11.28
five-number summary  44.5  59.4  64.8  70.7  91.6

```

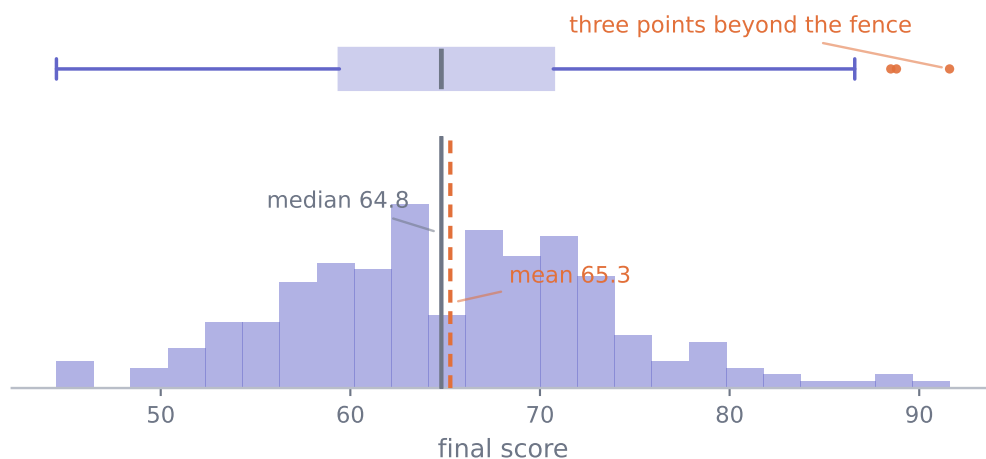


Figure 29.3: The 238 final scores as a histogram, with the same data as a Tukey boxplot above it on a shared scale. The box holds the middle half; the whiskers reach the furthest point within 1.5 IQR of it; the three orange points lie beyond that fence and are drawn individually.

A defensible sentence: *the 238 usable final scores have a mean of 65.3 marks and a standard deviation of 8.4; the middle half lies between 59.4 and 70.7, and the distribution is close to symmetric with a slight right lean.*

Now the point of insisting on the cleaning step in [S1](#). Here is the same description with the impossible score of 187 left in.

```

with_defect =
↪ cohort.dropna(subset=["final_score"])["final_score"].to_numpy()
for name, values in [("cleaned ", scores), ("defect in", with_defect)]:
    print(f"{name}  n={values.size}  mean {values.mean():6.2f}  "
          f"median {np.median(values):6.2f}  sd {values.std(ddof=1):6.2f}")

```

```

cleaned    n=238    mean  65.27    median  64.80    sd    8.38
defect in  n=239    mean  65.78    median  64.90    sd    11.49

```


One bad row in 239 moves the median by a tenth of a mark and inflates the standard deviation by **37%**. That asymmetry is the whole argument for looking at both: the median barely noticed, and if you had computed only the mean and the standard deviation you would have had no signal that anything was wrong.

Finally, remember what this variable actually is. One student has no recorded background, so the breakdown below describes 237 of the 238 — the filter follows the question, and this question needs a group label.

```
grouped = scored.dropna(subset=["background"])
print(f"all {len(grouped)} together: sd
      ↳ {grouped['final_score'].std():.2f}\n")
print(grouped.groupby("background")["final_score"]
      .agg(["count", "mean", "median", "std"]).round(2).to_string())
```

all 237 together: sd 8.40

	count	mean	median	std
background				
Computing	69	65.85	66.70	6.60
Engineering	36	67.15	68.65	5.33
Life sciences	57	60.51	59.80	7.29
Mathematics or Physics	36	75.28	74.40	7.63
Social sciences	39	60.21	60.50	6.58

The standard deviation of those 237 students is 8.40, and no single group's is above 7.63. The extra spread is not variability *within* any group — it is the distance between the group means, from 60.2 to 75.3, showing up as spread once they are poured into one column. Describing the mixture as though it were one population is correct arithmetic and a poor description, and [S10](#) is where that gets taken seriously.

29.5 Try it yourself

Exercise 1 For the sample $\{2, 3, 4, 5, 6, 6, 7\}$, compute by hand: the mean, the median, the mode, the sample variance and the sample standard deviation.

 Show the solution

The sum is 33 and $n = 7$, so $\bar{x} = 33/7 = 4.71$ (to 2 d.p.).

Sorted, the middle value is the fourth: the **median is 5**. The **mode is 6**, the only value appearing twice.

For the variance, square each deviation from 4.71:

$$(-2.71)^2 + (-1.71)^2 + (-0.71)^2 + (0.29)^2 + (1.29)^2 + (1.29)^2 + (2.29)^2 = 19.43$$

and divide by $n - 1 = 6$: $s^2 = 3.24$, so $s = 1.80$.

The commonest slip is dividing by 7. That gives 2.78, which is not wrong so much as a different quantity — it is the variance of these seven numbers *treated as the whole population*, and it is what `numpy.var` returns by default. (Its square root, 1.67, is what

`numpy.std` returns by default; both take a `ddof` argument that switches the divisor to $n - 1$.)

Exercise 2 A department reports that the mean starting salary of its graduates is £42,000. The median is £31,000.

- (a) What does the gap tell you about the distribution?
- (b) Which number should the department publish, and which would a prospective student rather know?
- (c) What would you need to see before trusting either?

💡 Show the solution

- (a) At least half the graduates earn £31,000 or less, and the mean sits £11,000 above that. So the salaries above the median must reach a very long way above it to drag the average up — a minority earning a great deal. For salaries that is nearly always a long right tail, and it is what you should expect before looking. Strictly, the gap tells you which way the mean has been pulled, not that the skewness coefficient is positive; the *What shape is it?* section has a five-value counterexample.
- (b) A prospective student wants the **median**, which answers “what does a typical graduate earn?”. The mean answers “what would everyone earn if the total were shared equally?”, which is not the question. The department has an incentive to publish the larger number; that incentive is exactly why you should ask which one you were given.
- (c) The five-number summary or a histogram, the sample size, and the response rate. A mean over 12 responses from 300 graduates is not a fact about the department, and — from [S1](#) — the people who reply are plausibly the ones with good news.

Exercise 3 A boxplot of exam marks has $Q_1 = 52$, median = 61, $Q_3 = 70$, whiskers at 38 and 86, and individual points drawn at 21, 98 and 99. Which of these follow?

- (a) The lowest mark was 38.
- (b) About a quarter of students scored above 70.
- (c) The students at 98 and 99 made a data-entry error.
- (d) The mean is 61.

💡 Show the solution

Only **(b)**, and even that is “about” — quartiles are approximate for finite samples, and ties can make the counts uneven.

- (a) is false. The lower whisker reaches the most extreme observation *within the fence*, and the fence sits at $52 - 1.5 \times 18 = 25$, so 38 is merely the lowest mark above 25. The lowest mark is the point drawn at **21**. Had there been no point below the whisker, 38 would have been the minimum — but you would know that from the absence of a drawn point, not from the whisker.
- (b) does not follow at all. A point beyond the fence is a value far from the middle. That is a description, not a diagnosis, and treating it as one is how genuine top

performers get deleted from datasets. (Check the arithmetic while you are here: the upper fence is $70 + 1.5 \times 18 = 97$, which is why 98 and 99 are drawn separately and 86 is not.)

- (c) does not follow. The median is 61; the mean could be anywhere, and the two high points suggest it sits somewhat above.

29.6 Check it in Python

Three claims from the mechanics, checked.

First, that the deviations from the mean really do cancel — this is what makes $\bar{x}$ the balance point, and why the variance has to square them.

```
deviations = scores - scores.mean()
print(f"sum of deviations from the mean : {deviations.sum():.3e}")
print(f"sum of deviations from the median: {(scores -
↪ np.median(scores)).sum():.2f}")
```

```
sum of deviations from the mean : 9.095e-13
sum of deviations from the median: 112.30
```

Not exactly zero, but 10^{-13} on numbers of size 65 is floating-point noise, not a real residue — P2 explains why. Never test a computed float with `==`. The deviations from the *median* do not cancel, and there is no reason they should.

Second, the variance formula, done by hand against the library:

```
by_hand = (deviations ** 2).sum() / (scores.size - 1)
library = scored["final_score"].var()
print(f"by hand {by_hand:.6f}    pandas {library:.6f}    "
      f"agree: {np.isclose(by_hand, library)}")
```

```
by hand 70.293592    pandas 70.293592    agree: True
```

Third — and this one is a genuine trap rather than a demonstration. NumPy and pandas disagree about the default divisor. Assert that they agree and watch it fail:

```
assert np.isclose(np.std(scores), scored["final_score"].std()), (
    f"numpy says {np.std(scores):.4f}, pandas says
    ↪ {scored['final_score'].std():.4f}"
)
```

AssertionError: numpy says 8.3665, pandas says 8.3841

```
-----  
AssertionError                                Traceback (most recent call last)  
Cell In[9], line 1  
----> 1 assert np.isclose(np.std(scores), scored["final_score"].std()), (  
      2     f"numpy says {np.std(scores):.4f}, pandas says  
      ↪ {scored['final_score'].std():.4f}"  
      3 )  
AssertionError: numpy says 8.3665, pandas says 8.3841
```

Neither library is wrong. `numpy.std` divides by n by default and `pandas.std` divides by $n - 1$, and both document it. With $n = 238$ the gap is 0.018 marks and invisible; with $n = 5$ it is nearly 12%, and a figure or a test built on it would be quietly wrong. Say which you meant:

```
assert np.isclose(np.std(scores, ddof=1), scored["final_score"].std())  
assert np.isclose(np.std(scores, ddof=0), scored["final_score"].std(ddof=0))  
print(f"n=238:  ddof=0 {np.std(scores, ddof=0):.4f}    ddof=1 {np.std(scores,  
      ↪ ddof=1):.4f}")  
small = scores[:5]  
print(f"n=5:    ddof=0 {np.std(small, ddof=0):.4f}    ddof=1 {np.std(small,  
      ↪ ddof=1):.4f}")
```

```
n=238:  ddof=0 8.3665    ddof=1 8.3841  
n=5:    ddof=0 9.9746    ddof=1 11.1519
```

Finally, robustness, which is easiest to believe once you have watched it:

```
damaged = scores.copy()  
damaged[damaged.argmax()] = 1000  
print(f"original  mean {scores.mean():7.2f}    median  
      ↪ {np.median(scores):.2f}")  
print(f"damaged   mean {damaged.mean():7.2f}    median  
      ↪ {np.median(damaged):.2f}")
```

```
original  mean    65.27    median 64.80  
damaged   mean    69.09    median 64.80
```

One value changed; the mean moves by nearly four marks and the median does not move at all.

29.7 Common wrong turns

Where this usually goes wrong

Reporting a mean with no measure of spread A reported mean of 65 is compatible with everyone scoring 65, and with half the cohort scoring 40 and half scoring 90. A centre without a spread is not a description. Report both, or report the five-number summary.

Reporting a mean for a strongly skewed variable Salaries, waiting times, house prices and counts of rare events are all right-skewed, and for all of them the mean answers a question nobody asked. Give the median, and give the mean too if you also give the shape.

Treating the range as a measure of spread It depends on exactly two observations and grows with sample size, so it is not comparable between samples. Use the IQR or the standard deviation.

Reading boxplot whiskers as the minimum and maximum They reach the most extreme point within 1.5 IQR of the box. If points are drawn beyond a whisker, the extreme is out there, not at the whisker's end.

Deleting points that fall beyond a fence The fence is a drawing convention, not a test for error. Investigate such a point — a score of 187 out of 100 is genuinely impossible and goes — but a score of 91.6 is a student who did well, and removing it is falsifying data.

Forgetting which divisor you used `numpy.std` uses n ; `pandas.std` and R use $n - 1$. On large samples nobody notices. On small ones the difference is large, and it propagates into every interval and test you compute afterwards.

Describing a mixture as though it were one population A histogram with two peaks is usually two groups. Splitting them and describing each is nearly always more informative than any single summary of the whole.

29.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. Two variables both have mean 50 and standard deviation 10. What can you conclude about how similar their distributions are?
 - a. They are the same distribution
 - b. They have the same shape but may be shifted
 - c. Almost nothing — they can look completely different
 - d. One must be normal
2. Every student's mark is increased by 5. What happens to the mean, the median and the standard deviation?
3. In the cohort, the standard deviation of `final_score` across all 237 students with a recorded background is 8.40, while no single background's exceeds 7.63. How can the whole be more variable than every part?
4. You compute a sample standard deviation of 12.0 using $n - 1$, then recompute using n and get 11.4. Which is "correct"?
5. A variable has mean 40 and median 40. Is it symmetric?

💡 Show the answers

1. (c). This is the first figure in the unit. Same mean and same standard deviation is compatible with one peak, two peaks, or a tight cluster plus a long tail. (b) is the tempting answer and is wrong: matching two numbers constrains location and scale, not shape.

2. The mean and median both increase by 5. The standard deviation is **unchanged** — every deviation $x_i - \bar{x}$ is unchanged, because the mean moved by exactly as much as the data did. Spread is about differences, and adding a constant changes no difference.
3. Because the groups sit at different places. Spread within a group is small (each group's students resemble each other); the extra spread in the combined column is the distance *between* the group means, 60.2 to 75.3. This is the idea behind comparing groups in [S10](#), and it is why a mixture almost always looks more variable than its parts.
4. **Both, for different questions.** The $n - 1$ version estimates the spread of the population these observations came from; the n version describes the spread of exactly these observations, treated as everything there is. If you are doing inference you want $n - 1$. The real error is not knowing which one your code produced.
5. **Not necessarily.** Equal mean and median is what symmetry produces, but it does not imply it. Take $\{0, 3, 4, 4, 4, 6, 7\}$: the mean is 4 and the median is 4, and the set is plainly not symmetric about 4 — its skewness coefficient is -0.48 . Equal mean and median is evidence, not proof. Plot it.

Where this shows up next

[S3](#) does all of this for two variables at once and asks how they move together. [S6](#) puts named shapes to the histograms in this unit, and says which mechanisms produce which shape. [S7](#) takes the mean you have just computed and asks how much it would have differed had you measured 238 other students. [S8](#) turns that into the interval that goes next to every mean you report.

30 Describing two variables

Statistics · Unit 3

S3 · Statistics Reading: 5 pages With practice: 1.5 hours Assumes: S2

Why this unit is here

Almost no question worth asking is about one variable. *Does practice help? Do the students who arrive strongest finish strongest? Does this treatment work?* Every one of them is about how two things move together.

This unit gives you the standard summary of that — the correlation coefficient — and then spends most of its length on what the number cannot see. A correlation is one number standing in for a whole cloud of points, and the ways it misleads are specific, common, and easy to spot once you know to look.

30.1 Before you start

You should be able to:

- compute a mean and a standard deviation, and say what each is for — S2;
- read a histogram and a boxplot — S2;
- tell a quantitative variable from a categorical one — S1.

Quick check. The cohort's final scores have mean 65.3 and standard deviation 8.4. A student scores 82. How many standard deviations above the mean is that?

💡 Show the answer

$(82 - 65.3)/8.4 = 1.99$, so almost exactly two standard deviations above the mean. That quantity has a name — the **standardised value**, or **z-score** — and this unit is built out of it. Subtracting the mean removes the location; dividing by the standard deviation removes the units and the scale. What is left is comparable across variables measured in completely different things.

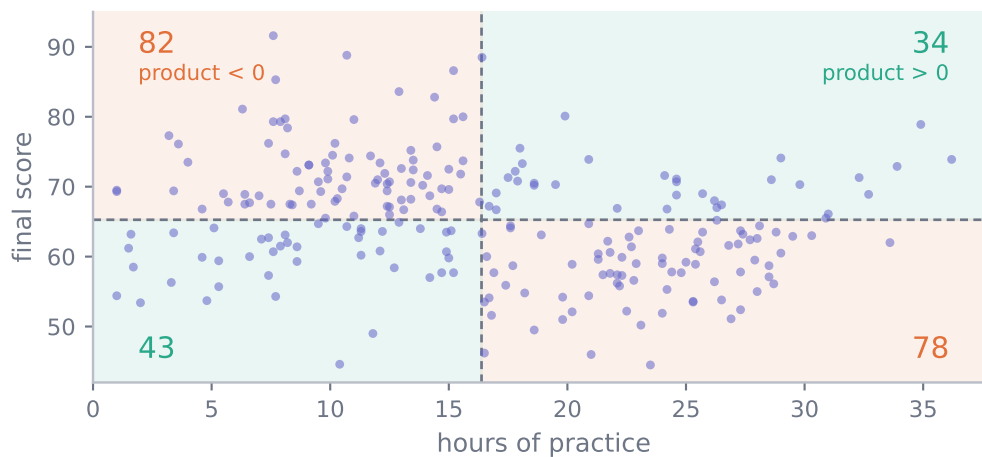


Figure 30.1: The 237 students, split at the mean hours of practice (16.4) and the mean final score (65.3). A point in the top-right or bottom-left agrees with both means in the same direction, and contributes a positive product; a point in the other two contributes a negative one. Here 160 points pull negative and 77 pull positive, so the total comes out below zero.

30.2 The idea

Put the two variables on a scatterplot, then draw a vertical line at the mean of one and a horizontal line at the mean of the other. That divides the cloud into four quadrants, and the quadrants are the whole idea.

For each student, take how far their hours are from the mean hours, take how far their score is from the mean score, and multiply. A student who is above average on both gets a positive product; so does a student below average on both. A student above average on one and below on the other gets a negative one.

Add up all 237 products. If the cloud leans upwards the positives dominate and the total is positive; if it leans downwards the total is negative; if there is no lean the products cancel. That total, divided by $n - 1$, is the **covariance**, and everything else in this unit is a repair to it.

30.3 The mechanics

30.3.1 Covariance, and why you cannot read it

$$\text{cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

The sign is informative. The size is not, because the covariance carries the units of both variables multiplied together — here marks $\times$ hours, a quantity nobody has intuition about. Worse, it depends on the units you happened to choose:

```
print(f"cov in marks x hours    {np.cov(hours, score, ddof=1)[0, 1]:10.4f}")
print(f"cov in marks x minutes {np.cov(hours * 60, score, ddof=1)[0,
↵ 1]:10.4f}")
```



```
cov in marks x hours      -16.0477
cov in marks x minutes    -962.8637
```

Nothing about the students changed. Recording practice in minutes rather than hours multiplied the covariance by 60. A summary that moves when you change your mind about units cannot be compared across studies.

30.3.2 Correlation is covariance made unitless

Divide by the standard deviation of each variable and the units cancel:

$$r = \frac{\text{cov}(x, y)}{s_x s_y} = \frac{1}{n-1} \sum_{i=1}^n z_{x,i} z_{y,i},$$

where $z_{x,i} = (x_i - \bar{x})/s_x$ is the standardised value from the quick check. The second form is worth carrying: **the correlation is the sum of the products of z-scores, divided by $n - 1$** . Standardise both variables, multiply them point by point, add up, divide by $n - 1$. (It is nearly the average of those products — the divisor is $n - 1$ rather than n , for the same reason the sample variance uses $n - 1$, and on the cohort the two differ by 0.001.)

This is **Pearson's correlation coefficient**, and it has four properties you should be able to state.

- It lies between -1 and $+1$, always.
- It is $+1$ exactly when the points lie on a straight line of positive slope, and -1 exactly when they lie on a straight line of negative slope.
- It is symmetric: $r(x, y) = r(y, x)$. It does not know which variable you think is the cause.
- It is unchanged by shifting or rescaling either variable — hours to minutes, Celsius to Fahrenheit, marks to percentages — except that multiplying by a negative number flips its sign.

30.3.3 What the number cannot see

r measures **linear** association and nothing else. That restriction is not a technicality; it is where most misreadings come from.

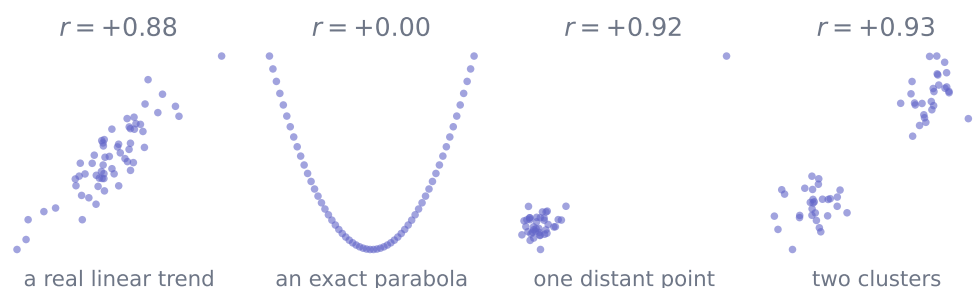


Figure 30.2: Four datasets. Three have correlations near $+0.9$ and nothing else in common; the fourth has a correlation of exactly zero and a perfect relationship — every y is determined by its x . One number cannot certify a shape.

Read those four again in order. The first is what people picture when they hear “ $r = 0.9$ ”. The second has a *perfect* relationship and a correlation of zero, because the parabola rises as fast on the right as it falls on the left and the products cancel exactly — $r = 0$ **does not mean the variables are unrelated**. The third is 39 points with no association and one point a long way off; delete that point and r collapses. The fourth is two clusters with no association inside either, and the correlation is measuring the gap between them, not anything about individual students.

The defence is the same in all four cases and takes five seconds: **plot it**.

30.3.4 When the relationship is monotone but not straight

Spearman’s rank correlation is Pearson’s r computed on the *ranks* rather than the values: replace the smallest x by 1, the next by 2, and so on, do the same for y , and correlate those. It measures whether the relationship is consistently increasing or decreasing, without insisting it be a straight line, and one distant point can only ever be *the largest rank* rather than a value seventeen standard deviations out.

Use it when the relationship looks monotone but curved, when one or both variables are ordinal, or when extreme values are doing more work than you are comfortable with. Report which one you used — “correlation” unqualified means Pearson.

30.3.5 The other two combinations

Two variables need not both be quantitative.

Categorical with quantitative — the case that dominates real work — is a comparison of groups: report a summary of the quantitative variable within each category, and draw side-by-side boxplots on a shared scale. [S10](#) makes this into inference.

Categorical with categorical is a **contingency table** of counts, read as *conditional proportions*. The direction matters and is a decision about the question, not about the data: the proportion of engineers who completed, and the proportion of completers who were engineers, are different numbers answering different questions, and confusing them is the error [S4](#) is largely about.

30.4 Worked example

Practice against score, one number at a time.

```
from scipy import stats

r = np.corrcoef(hours, score)[0, 1]
rho = stats.spearmanr(hours, score).statistic
print(f"n = {hours.size}")
print(f"Pearson   r = {r:+.4f}")
print(f"Spearman   = {rho:+.4f}")
```

```
n = 237
Pearson r = -0.2350
Spearman = -0.2585
```

So a weak negative association: students who practise more tend, across the cohort as a whole, to score slightly lower. Spearman agrees in sign and is slightly stronger, which tells you the relationship is not being manufactured by a handful of extreme points.

i M11 reports -0.2337 for the same pair of columns. It is not a different answer: that unit filters on `final_score` alone and keeps 238 rows, while this one also drops the student with no recorded background and keeps 237. One student out of 238 shifts the correlation by 0.0013 — enough to change the second decimal place, from -0.23 to -0.24 . If two numbers in these notes disagree, check the filter first.

Now split by background, which S1 established is the variable doing the damage.

```
print(f"{'background':<24}{'n':>5}{'Pearson':>10}{'Spearman':>10}")
print(f"{'everyone':<24}{hours.size:>5}{r:>+10.2f}{rho:>+10.2f}")
for name, group in usable.groupby("background"):
    g_h, g_y = group["hours_practice"], group["final_score"]
    print(f"{name:<24}{len(group):>5}"
          f"{np.corrcoef(g_h, g_y)[0, 1]:>+10.2f}"
          f"{stats.spearmanr(g_h, g_y).statistic:>+10.2f}")
```

background	n	Pearson	Spearman
everyone	237	-0.24	-0.26
Computing	69	+0.43	+0.38
Engineering	36	+0.64	+0.58
Life sciences	57	+0.68	+0.62
Mathematics or Physics	36	+0.54	+0.51
Social sciences	39	+0.66	+0.66

One correlation of -0.24 ; five correlations between $+0.43$ and $+0.68$. The overall number is not wrong, and it is not a useful description of any student in the dataset, because no student is a member of “everyone” without also being a member of exactly one background.

That is the fourth panel of the four-dataset figure, in real data: a correlation computed across clusters is largely a statement about where the clusters sit. S12 gives the pattern its name and its general treatment.

30.5 Try it yourself

Exercise 1 Two variables have $\text{cov}(x, y) = 24$, $s_x = 4$ and $s_y = 15$.

- What is r ?
- You now rescale x from metres to centimetres. What happens to the covariance, to s_x , and to r ?

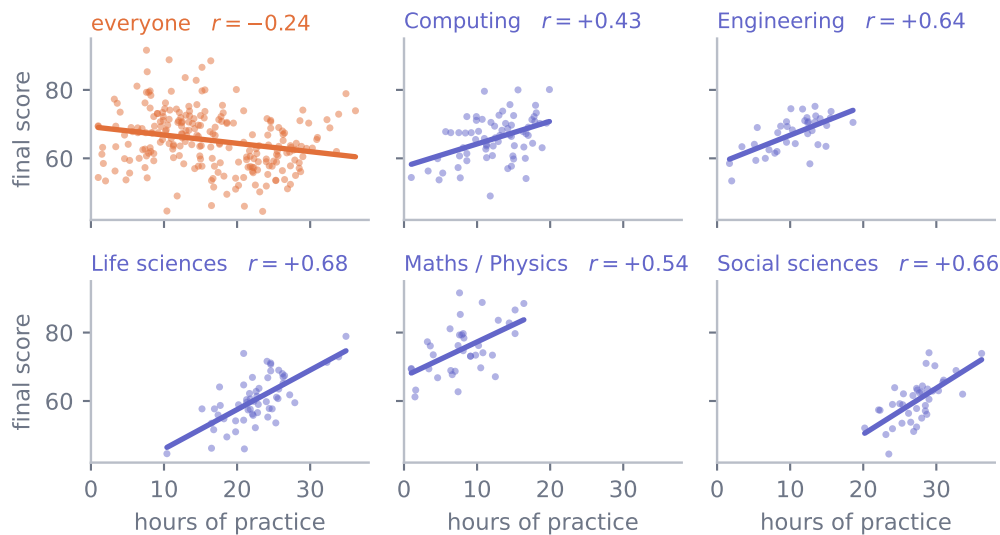


Figure 30.3: All 237 students in the first panel, then the same students split into their five backgrounds, on shared axes, with a straight line fitted to each panel. The first panel slopes down; the other five slope up. No observation was added, removed or altered — the five panels are a partition of the first.

💡 Show the solution

- (a) $r = 24 / (4 \times 15) = 0.4$.
- (b) Multiplying x by 100 multiplies the covariance by 100 (to 2400) and s_x by 100 (to 400). The correlation is $2400 / (400 \times 15) = 0.4$ — **unchanged**, because the same factor of 100 appears above and below the line. That invariance is the entire reason correlation is reported instead of covariance.

Exercise 2 A colleague finds $r = 0.02$ between two variables and concludes that they are unrelated. Give two distinct reasons the conclusion does not follow, and say what you would ask to see.

💡 Show the solution

- **The relationship may not be linear.** Any relation symmetric about the mean of x — a U-shape observed over a range centred on its vertex, for instance — produces $r \approx 0$ while y may be completely determined by x . The parabola panel in this unit has r exactly zero for that reason; observe the same parabola over an off-centre range and r climbs above 0.88.
- **The sample may be too small to see anything.** With $n = 8$, $r = 0.02$ is entirely consistent with a real and substantial association; r estimated from a small sample bounces around a great deal. S8 is how you find out.
- Or: **the population may be a mixture**, with opposite associations inside different groups that cancel when pooled — which is what happens in this unit's worked example, only in reverse.

What to ask for: **the scatterplot**, and the sample size. In that order.

Exercise 3 Ten students are ranked on two measures. Their x -ranks are 1 to 10 in order, and

their y values are 1, 4, 9, 16, 25, 36, 49, 64, 81, 100 — that is, $y = x^2$ exactly.

Without computing anything, say what Spearman's rank correlation must be, and whether Pearson's r will be larger, smaller or equal.

💡 Show the solution

Spearman is **exactly** +1. Ranking $y = x^2$ for positive x gives back the x ranks in the same order, and Pearson's r on two identical rank sequences is 1. Spearman detects *any* strictly increasing relationship, however curved.

Pearson's r will be **smaller** than 1 — about 0.97 — because the points do not lie on a straight line, and $r = 1$ requires exactly that. It is high because $y = x^2$ is not far from straight over this range; over a wider range it would fall further.

The general lesson: Spearman equal to 1 with Pearson below it is the signature of a curved but consistently increasing relationship.

30.6 Check it in Python

First, the claim that correlation is the average product of z-scores. Standardise both columns by hand and compare.

```
z_hours = (hours - hours.mean()) / hours.std(ddof=1)
z_score = (score - score.mean()) / score.std(ddof=1)
by_hand = (z_hours * z_score).sum() / (hours.size - 1)
print(f"average product of z-scores {by_hand:+.6f}")
print(f"numpy corrcoef          {np.corrcoef(hours, score)[0, 1]:+.6f}")
```

```
average product of z-scores -0.235010
numpy corrcoef              -0.235010
```

Second, that changing units leaves r alone but not the covariance:

```
minutes = hours * 60
print(f"covariance hours {np.cov(hours, score, ddof=1)[0, 1]:>10.4f}"
      f"    minutes {np.cov(minutes, score, ddof=1)[0, 1]:>10.4f}")
print(f"correlation hours {np.corrcoef(hours, score)[0, 1]:>10.4f}"
      f"    minutes {np.corrcoef(minutes, score)[0, 1]:>10.4f}")
```

```
covariance hours    -16.0477    minutes    -962.8637
correlation hours    -0.2350    minutes     -0.2350
```

Third — the one worth running yourself. It is tempting to write a check that says “if two variables are related, the correlation will notice”. Watch it fail on a relationship that is not merely strong but *exact*:

```
x = np.linspace(-2, 2, 61)
y = x ** 2                                # y is completely determined by x
assert abs(np.corrcoef(x, y)[0, 1]) > 0.5, (
    f"y is an exact function of x, yet r = {np.corrcoef(x, y)[0, 1]:.6f}"
)
```

AssertionError: y is an exact function of x, yet r = -0.000000

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[9], line 3
      1 x = np.linspace(-2, 2, 61)
      2 y = x ** 2                                # y is completely determined by x
----> 3 assert abs(np.corrcoef(x, y)[0, 1]) > 0.5, (
      4     f"y is an exact function of x, yet r = {np.corrcoef(x, y)[0,
    ↪ 1]:.6f}"
      5 )
AssertionError: y is an exact function of x, yet r = -0.000000
```

Every y here is fixed by its x with no noise at all, and the correlation is zero to machine precision. It is zero because the grid is symmetric about $x = 0$: for every point that contributes $+c$ to the sum of products there is a mirror point contributing $-c$. Shift the window off centre and the cancellation stops:

```
for lo, hi in [(-2, 2), (-1, 3), (0, 4)]:
    x = np.linspace(lo, hi, 61)
    print(f"x from {lo}>2} to {hi}: r = {np.corrcoef(x, x ** 2)[0,
    ↪ 1]:+.4f}")
```

```
x from -2 to 2: r = -0.0000
x from -1 to 3: r = +0.8855
x from 0 to 4: r = +0.9673
```

Same relationship, three different correlations, one of them zero. What r reports depends on the range you happened to observe, which is a good thing to know before quoting one.

30.7 Common wrong turns

Where this usually goes wrong

Reading $r = 0$ as “no relationship” It means no *linear* relationship. An exact parabola, observed symmetrically about its vertex, scores zero. Say “no linear association”, plot the data, and consider Spearman.

Quoting a covariance Its size depends on your choice of units, so it is not comparable with anyone else’s. Divide by the two standard deviations and quote r .

Treating r as a slope r is unitless and cannot tell you how many marks an hour of practice is worth. Two datasets with identical correlations can have wildly different slopes. The slope is S11’s business, and it carries units.

Computing r on a mixture If the data contain distinguishable groups, a pooled correlation partly measures the distance between the groups. Compute it within each group as well, always, and look at whether the answers agree.

Letting one point decide A single distant observation can create a correlation from nothing or destroy a real one. Plot the data; recompute without the point; if the answer changes, say so in the write-up rather than choosing the version you prefer.

Reading a correlation as a cause The coefficient is symmetric — it cannot even tell which variable came first. Everything S1 said about design applies here unchanged.

Reporting r without n A correlation from 8 observations and one from 8,000 are not the same kind of evidence, and the number alone does not distinguish them.

30.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. Which of these *can* change the correlation between height and weight in a sample?
 - a. Converting height from centimetres to inches
 - b. Converting weight from kilograms to pounds
 - c. Adding 5 kg to everyone's weight
 - d. Dropping the three tallest people from the sample
2. In the cohort, $r = -0.24$ overall and between $+0.43$ and $+0.68$ within each background. Which of those numbers is the “real” correlation between practice and score?
3. Spearman's rank correlation for a pair of variables is $+1$ but Pearson's is $+0.8$. What does that tell you about the relationship?
4. A study reports $r = 0.62$ between two variables measured on 6 people and describes it as a strong relationship. Is that reading defensible?
5. True or false: if $r = +0.9$, then a one-standard-deviation increase in x is associated with a 0.9-standard-deviation increase in y on average.

Show the answers

1. **(d).** Correlation is invariant to shifting and positive rescaling, so (a), (b) and (c) leave it exactly unchanged — the units cancel in the definition. Dropping the three tallest people changes *which* points are in the sample, which is a different operation entirely; it usually reduces r by removing the widest part of the range. Note the “can”: it is not guaranteed to change anything. If every point lay exactly on a straight line, r would be 1 before and after. It is the only option that can move the number at all.
2. **Both, and neither is “the” correlation** — they are answers to two different questions, exactly as in S1. -0.24 describes the cohort pooled, and mostly reflects that quantitative-background students start ahead and practise less. The within-group figures describe students compared with others like them. If you had to report one number for “does practice go with better marks”, the within-group ones are closer to what was meant — but the honest answer reports both.

3. That the relationship is **monotone but curved**. Spearman being exactly 1 means the ranks agree perfectly: y increases every time x does. Pearson below 1 means the points do not lie on a straight line. Fit a straight line to this and the residuals will show a systematic bend.
4. **No**. With six observations, $r = 0.62$ is unremarkable: correlations of that size arise easily between unrelated variables in samples that small, and the estimate itself bounces around enormously — S8 shows how to put a width on it. Ask for the scatterplot, and ask how many pairs of variables were examined before this one was reported.
5. **True**, and it is the most useful reading of r there is. Written in standardised units, the fitted straight line has slope exactly r : on average a point one standard deviation above the mean in x sits r standard deviations above the mean in y . Note “on average” and “associated with” — this is a description of the cloud, not a prediction for any individual and not a claim about what would happen if you intervened.

Where this shows up next

S10 turns the group comparisons here into inference, with intervals on the differences. **S11** replaces the unitless r with a slope in marks per hour, and attaches an interval to it. **S12** names the reversal in this unit’s worked example and shows how to look for it before it finds you. **M12** takes the correlation matrix of several variables at once and asks which directions in the data carry the most variation.

31 Probability, conditioning and Bayes

Statistics · Unit 4

S4 · Statistics Reading: 6 pages With practice: 2.5 hours Assumes: S1

Why this unit is here

From here on, every claim these notes make is a probability statement. A confidence level, a p -value, “the procedure captures the truth 95% of the time” — all of them are probabilities, and none of them mean what they appear to mean until you can handle conditioning.

There is one error in particular that this unit exists to prevent. It is swapping a conditional probability for its reverse: reading *the chance of a positive test given the disease* as *the chance of the disease given a positive test*, or *the chance of this data if the hypothesis were false* as *the chance the hypothesis is false*. It has sent people to prison. It is the same mistake every time, and once you can see it you cannot unsee it.

31.1 Before you start

You should be able to:

- turn a count into a proportion and back — S1;
- read a two-way table of counts — S1;
- work with fractions and percentages.

Quick check. Of the 237 students in the cohort, 69 scored 70 or more. What proportion is that, as a decimal and as a percentage?

💡 Show the answer

$69/237 = 0.291$, or 29.1%.

Keep hold of both the count and the total. Almost every confusion in this unit comes from losing track of what the denominator was.

31.2 The idea

A **probability** is a number between 0 and 1 attached to an event, measuring how often it happens. Take it, for now, as a long-run frequency: if you could repeat the situation many times, the probability is the fraction of times you would see the event. That is the reading frequentist statistics is built on, and it is why S8 can say things about *procedures* but not about individual results.

The whole subject then rests on one move: **conditioning**. Conditioning means restricting attention to a subset of the possibilities and asking the question again inside it. It changes the denominator, and changing the denominator changes the answer — often dramatically, and in a way people consistently fail to expect.

Here it is on the cohort. Take two facts about a student: they came from a Mathematics or Physics background, and they scored 70 or more. Twenty-four students have both.

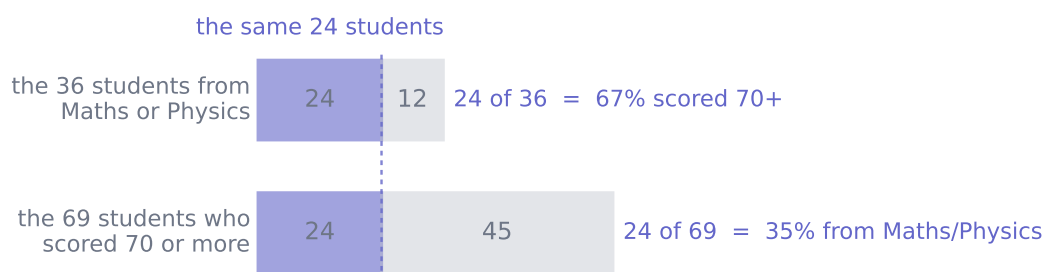


Figure 31.1: The same twenty-four students, counted against two different denominators. As a fraction of the 36 who came from Maths or Physics they are two thirds; as a fraction of the 69 who scored 70 or more they are about a third. Neither number is wrong, and they are not answers to the same question.

Two thirds and one third, from the same twenty-four people. The only thing that changed was which group we agreed to look at.

Write them properly and the difference is visible in the notation:

$$P(\text{scored } 70+ \mid \text{Maths/Physics}) = \frac{24}{36} = 0.67, \quad P(\text{Maths/Physics} \mid \text{scored } 70+) = \frac{24}{69} = 0.35.$$

The vertical bar is read “given”. **The thing after the bar is the denominator.** Swap the two sides and you have changed the question, not merely the phrasing.

31.3 The mechanics

31.3.1 The rules, all of them

An **event** is a set of outcomes; $P(A)$ is its probability. Everything follows from three facts: $0 \leq P(A) \leq 1$, the probability of “something happens” is 1, and the probabilities of outcomes that cannot both occur add up.

From those:

$$P(\text{not } A) = 1 - P(A)$$

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$P(A \cap B) = P(A | B) P(B)$$

The subtraction in the second is the whole content of it: add two events and you have counted their overlap twice, so take it off once. Read $\cup$ as “or” (inclusive) and $\cap$ as “and”.

31.3.2 Conditional probability

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad \text{defined when } P(B) > 0.$$

In counts, this is exactly what the figure did: of the things in B , what fraction are also in A . **To condition is to change the denominator.**

31.3.3 Independent is not the same as mutually exclusive

A and B are **independent** when

$$P(A \cap B) = P(A) P(B),$$

equivalently (when $P(B) > 0$) when $P(A | B) = P(A)$: learning that B happened tells you nothing new about A .

They are **mutually exclusive** when they cannot both happen, $P(A \cap B) = 0$.

These are close to opposites, and confusing them is one of the two commonest errors in the whole unit.

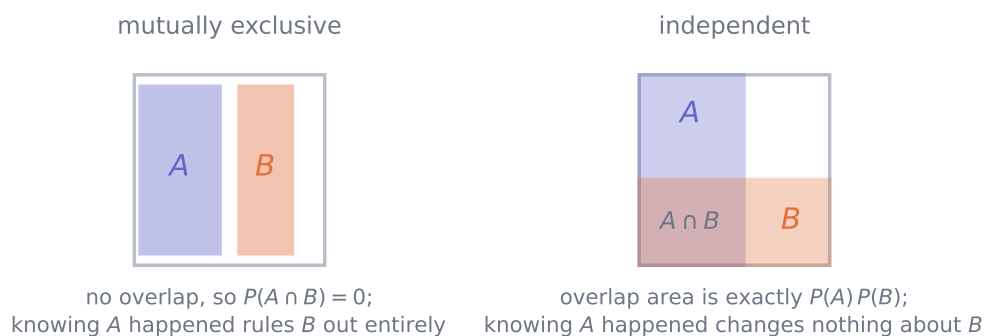


Figure 31.2: Two events drawn inside the space of all outcomes, with probability read as area. On the left they cannot both happen. On the right each is a strip across the whole space, so the overlap has area exactly $P(A)P(B)$ — which is what independence means.

If A and B are mutually exclusive and both have positive probability, they are as far from independent as two events can be: $P(A | B) = 0$, while $P(A) > 0$. Learning B has told you everything about A .

31.3.4 Splitting a probability up

If $B_1, \dots, B_k$ cover every possibility without overlapping — a **partition** — then

$$P(A) = \sum_{i=1}^k P(A | B_i) P(B_i).$$

This is the **law of total probability**, and in words it is unremarkable: an overall rate is the weighted average of the group rates, weighted by group size. It is the machinery behind S1's worked example, where a pooled answer disagreed with every group.

31.3.5 Bayes' theorem

Put the multiplication rule together both ways round — $P(A \cap B)$ equals both $P(A | B)P(B)$ and $P(B | A)P(A)$ — and rearrange:

$$P(B | A) = \frac{P(A | B) P(B)}{P(A)}.$$

That is **Bayes' theorem**. It is the machine for reversing a conditional, and the thing to notice is that you cannot reverse one without knowing $P(B)$ — the **base rate**. A conditional probability on its own does not determine its reverse.

That is not an abstraction, and the standard illustration is worth doing in counts rather than fractions, because in counts nobody gets it wrong. Take a disease that 1% of people have, and a test that is positive for 90% of people who have it and for 5% of people who do not. You test positive. What is the chance you have it?

Imagine 10,000 people.

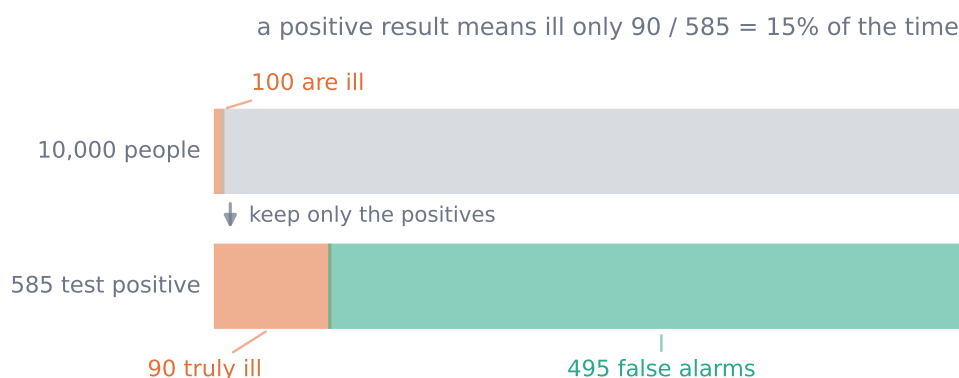


Figure 31.3: Ten thousand people, then only the 585 who test positive. The test is right nine times out of ten on people who are ill — but almost all the positives come from the enormous group who are not, so a positive result leaves you far more likely to be well than ill.

One hundred are ill and 90 of them test positive. Nine thousand nine hundred are well and 5% of them — 495 people — test positive anyway. So 585 people test positive and 90 of them are ill: $90/585 = 15\%$.

The test is good. The answer is 15% because the healthy group is a hundred times larger, so its small error rate still produces more positives than the ill group does in total.

Notice which dial matters. Push the sensitivity from 90% all the way to a perfect 100% and you gain ten true positives: $100/595 = 17\%$, barely a change. Cut the false-positive rate from 5% to 0.5% instead and the false alarms fall from 495 to 50, giving $90/140 = 64\%$. **When a condition is rare, the positive predictive value is governed by the false-positive rate and the base rate, and hardly at all by how well the test detects true cases.**

31.4 Worked example

The cohort's own screening test.

Every student sat a diagnostic on arrival. Treat “scored 70 or more on the diagnostic” as a test for “scored 70 or more at the end”, and ask how good a test it is.

```
cohort = pd.read_csv("../data/cohort.csv")
usable = cohort.dropna(subset=["background", "final_score"])
usable = usable[usable["final_score"].between(0, 100)]
usable = usable.assign(strong_start=usable["diagnostic_score"] >= 70,
                       strong_finish=usable["final_score"] >= 70)

print(pd.crosstab(usable["strong_start"], usable["strong_finish"],
  ↪ margins=True))
```

strong_finish	False	True	All
strong_start			
False	121	22	143
True	47	47	94
All	168	69	237

Four counts and two margins, and every probability in this unit is a ratio of two of them.

```
n = len(usable)
both = ((usable["strong_start"]) & (usable["strong_finish"])).sum()
start = usable["strong_start"].sum()
finish = usable["strong_finish"].sum()

print(f"P(strong finish)                {finish / n:.4f}")
print(f"P(strong start)                 {start / n:.4f}")
print(f"P(strong start AND strong finish) {both / n:.4f}")
print(f"P(strong finish | strong start)   {both / start:.4f}    <-
  ↪ {both}/{start}")
print(f"P(strong start | strong finish)   {both / finish:.4f}    <-
  ↪ {both}/{finish}")
```

P(strong finish)	0.2911
P(strong start)	0.3966

```

P(strong start AND strong finish)    0.1983
P(strong finish | strong start)      0.5000    <- 47/94
P(strong start | strong finish)      0.6812    <- 47/69

```

Read those last two lines slowly. The diagnostic flags 94 students, and exactly 47 of them go on to score 70 or more: **a positive flag is right half the time**. Yet the test is not useless, because 29% of students score 70+ overall, and among the flagged ones it is 50% — the flag has raised the odds considerably. It has simply not raised them to anything like certainty.

Now the same numbers as Bayes' theorem, to see the machine work:

$$P(\text{finish} \mid \text{start}) = \frac{P(\text{start} \mid \text{finish}) P(\text{finish})}{P(\text{start})} = \frac{0.6812 \times 0.2911}{0.3966} = 0.5000.$$

```

sensitivity = both / finish          # P(start | finish)
prior = finish / n                   # P(finish)
evidence = start / n                 # P(start)
print(f"Bayes gives {sensitivity * prior / evidence:.4f}")
print(f"direct ratio {both / start:.4f}")

```

```

Bayes gives 0.5000
direct ratio 0.5000

```

Finally, are the two events independent? If they were, the probability of both would be the product of the two separate probabilities:

```

p_both = both / n
p_product = (start / n) * (finish / n)
print(f"P(start AND finish) {p_both:.4f}")
print(f"P(start) x P(finish) {p_product:.4f}")
print(f"ratio {p_both / p_product:.2f} - a strong start makes a strong
↪ finish "
      f"{p_both / p_product:.2f} times likelier than chance alone")

```

```

P(start AND finish) 0.1983
P(start) x P(finish) 0.1155
ratio 1.72 - a strong start makes a strong finish 1.72 times likelier than chance alone

```

Not independent, and nobody expected them to be. What the numbers add is a size: students who start strong finish strong 1.7 times as often as independence would predict, and still only half the time.

31.5 Try it yourself

Exercise 1 $P(A) = 0.5$, $P(B) = 0.4$, and $P(A \cap B) = 0.2$.

- Are A and B independent?
- What is $P(A \cup B)$?
- What is $P(A \mid B)$, and what is $P(B \mid A)$?

💡 Show the solution

- (a) **Yes.** $P(A)P(B) = 0.5 \times 0.4 = 0.2$, which is exactly $P(A \cap B)$.
- (b) $P(A \cup B) = 0.5 + 0.4 - 0.2 = 0.7$. Forgetting to subtract the overlap gives 0.9, which is the commonest slip here.
- (c) $P(A | B) = 0.2/0.4 = 0.5$, and $P(B | A) = 0.2/0.5 = 0.4$. Both equal the unconditional probabilities, which is another way of saying the events are independent — and note that the two conditionals are *different numbers* even so. Independence does not make $P(A | B)$ and $P(B | A)$ equal; it makes each equal to its own unconditional probability.

Exercise 2 One in 200 people in a population has a certain condition. A screening test is positive for 99% of people who have it and for 2% of people who do not.

Work out, in counts, the probability that someone who tests positive has the condition. Then say what would have to change to push that probability above 50%.

💡 Show the solution

Take 20,000 people. 100 have the condition and 99 of them test positive. 19,900 do not, and 2% of them — 398 people — test positive anyway. So 497 test positive and 99 have the condition:

$$P(\text{condition} | \text{positive}) = \frac{99}{497} = 0.199.$$

About one in five, from a test that is right 99% of the time on people who have the condition.

To push it past 50% you need the false positives to fall below 99. The sensitivity cannot help — it is already at 99% and even perfection would only add one true positive. What matters is the **false-positive rate**: dropping it from 2% to 0.4% leaves about 80 false positives and a probability of $99/179 = 0.55$. The other route is a higher **base rate**: screening a group already known to be at risk, rather than everyone. This is precisely why population-wide screening for rare conditions is a harder problem than it sounds.

Exercise 3 A barrister tells a jury: “The chance of this DNA match occurring if the defendant were innocent is one in a million. So there is only a one in a million chance that the defendant is innocent.”

Name the error, write both probabilities in symbols, and explain what else the jury would need to know.

💡 Show the solution

The error is swapping a conditional for its reverse — often called the **prosecutor’s fallacy**. The barrister has been given

$$P(\text{match} | \text{innocent}) = 10^{-6}$$

and has reported it as

$$P(\text{innocent} \mid \text{match}) = 10^{-6}.$$

Bayes' theorem says those are equal only by coincidence, and to get from one to the other you need the **base rate**: how many people could plausibly have left that sample. Suppose the pool of possible sources is 10 million people, one of whom is guilty. Among the 10 million innocent people, about 10 would match by chance. So there are roughly 11 matching people, one guilty and ten not, and the probability of innocence given a match is around 10/11 — not one in a million. The number the jury was given is real; the conclusion drawn from it is off by a factor of about nine hundred thousand. This is not a hypothetical error. It has produced wrongful convictions.

31.6 Check it in Python

Every probability in this unit is a ratio of counts, so all of it is checkable directly. First, that conditioning really is division by the new denominator:

```
flagged = usable[usable["strong_start"]]
print(f"P(finish | start) by filtering
      ↪ {flagged['strong_finish'].mean():.4f}")
print(f"P(finish | start) by division {both / start:.4f}")
```

```
P(finish | start) by filtering  0.5000
P(finish | start) by division   0.5000
```

Second — the mistake this unit exists to prevent. It is tempting to think that if a flag predicts an outcome well, the outcome predicts the flag equally well. Assert it and watch it fail:

```
p_finish_given_start = both / start
p_start_given_finish = both / finish
assert np.isclose(p_finish_given_start, p_start_given_finish), (
    f"P(finish|start) = {p_finish_given_start:.4f} but "
    f"P(start|finish) = {p_start_given_finish:.4f}"
)
```

```
AssertionError: P(finish|start) = 0.5000 but P(start|finish) = 0.6812
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[9], line 3
      1 p_finish_given_start = both / start
      2 p_start_given_finish = both / finish
----> 3 assert np.isclose(p_finish_given_start, p_start_given_finish), (
      4     f"P(finish|start) = {p_finish_given_start:.4f} but "
      5     f"P(start|finish) = {p_start_given_finish:.4f}"
      6 )
AssertionError: P(finish|start) = 0.5000 but P(start|finish) = 0.6812
```


Both fractions have 47 on top. They differ because the denominators differ — 94 flagged students against 69 strong finishers. The two are equal only when the two events happen to be equally common, which is a coincidence rather than a rule:

```
print(f"P(start)  {start / n:.4f}    P(finish)  {finish / n:.4f}")
print(f"ratio of the two conditionals {(both / start) / (both /
↪  finish):.4f}")
print(f"ratio of the two base rates   {(finish / n) / (start / n):.4f}")
```

```
P(start)  0.3966    P(finish)  0.2911
ratio of the two conditionals 0.7340
ratio of the two base rates   0.7340
```

The two ratios agree, which is Bayes' theorem in one line: reversing a conditional multiplies it by the ratio of the base rates, and by nothing else.

Third, the addition rule, including the part everyone forgets:

```
either = ((usable["strong_start"]) | (usable["strong_finish"])).mean()
naive = start / n + finish / n
print(f"P(start OR finish) counted directly {either:.4f}")
print(f"P(start) + P(finish)                {naive:.4f}    too big by {naive
↪  - either:.4f}")
print(f"P(start AND finish)                {both / n:.4f}")
```

```
P(start OR finish) counted directly 0.4895
P(start) + P(finish)                0.6878    too big by 0.1983
P(start AND finish)                0.1983
```

The amount by which the naive sum overshoots is exactly the overlap, which is what the $-P(A \cap B)$ in the addition rule is there to remove.

31.7 Common wrong turns

Where this usually goes wrong

Reading $P(A | B)$ as $P(B | A)$ The single most expensive error in applied statistics. Two thirds of the Maths/Physics students scored 70+, but only a third of the students who scored 70+ were from Maths/Physics. Say the denominator out loud every time: “of the 36, how many...”.

Ignoring the base rate A test that is right 90% of the time on people with a rare disease still produces mostly false alarms, because the healthy group is so much bigger. Before believing a positive result, ask how many of the thing there were to begin with.

Treating mutually exclusive as independent They are nearly opposite. Two events that cannot both happen are maximally informative about each other: if one occurred, the other definitely did not. Independence means the overlap is exactly $P(A)P(B)$, not zero.

Multiplying probabilities that are not independent $P(A \cap B) = P(A)P(B)$ only under independence. In general it is $P(A | B)P(B)$. Multiplying dependent probabilities — several test results from the same faulty instrument, several children in the same family — has produced some of the most notorious statistical miscarriages of justice on record.

Expecting independent trials to even out A fair coin that has come up heads six times has probability exactly $1/2$ of heads next. The coin has no memory, and “it is due a tail” is a statement about the gambler, not the coin.

Forgetting that $P(A \cup B)$ double-counts Adding two probabilities counts the overlap twice. Subtract it once. If the events are mutually exclusive there is no overlap and the addition is safe, which is why the shortcut works often enough to be dangerous.

31.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. $P(A) = 0.3$ and $P(B) = 0.6$. Which is impossible?
 - a. $P(A \cap B) = 0$
 - b. $P(A \cap B) = 0.18$
 - c. $P(A \cap B) = 0.3$
 - d. $P(A \cap B) = 0.4$
2. In the worked example, $P(\text{strong finish} | \text{strong start}) = 0.50$ while $P(\text{strong finish}) = 0.29$. Does the diagnostic score predict the final score?
3. A test for a condition affecting 1 in 10,000 people is positive for 99% of those who have it and 1% of those who do not. Roughly what fraction of positive results are correct?
4. Two events are mutually exclusive and both have probability 0.3. Are they independent?
5. A colleague says: “There is a 5% chance of getting a result this extreme if the drug does nothing, so there is a 95% chance the drug works.” What has gone wrong?

Show the answers

1. **(d).** An intersection cannot be larger than either event on its own, and $0.4 > 0.3 = P(A)$. Note that (a), (b) and (c) are all possible: 0 makes them mutually exclusive, 0.18 makes them independent, and 0.3 means A is entirely contained in B .
2. **Yes, usefully, and not decisively.** It nearly doubles the probability — from 29% to 50% — which is a real gain. But half of the flagged students do not go on to score 70+, so treating the flag as a verdict about an individual would be wrong half the time. Both statements have to be made together.
3. **About 1%.** In a million people, 100 have the condition and 99 test positive; 999,900 do not and about 9,999 test positive anyway. That is roughly 10,098 positives of which 99 are correct — just under 1 in 100. If your first instinct was 99%, that is the base-rate error, and it is the normal instinct.
4. **No**, and this is the trap. $P(A \cap B) = 0$, while $P(A)P(B) = 0.09$. They are strongly *dependent*: knowing B happened tells you A certainly did not. Mutual exclusivity

and independence can hold at once only if one of the events has probability zero.

5. The conditional has been reversed. The 5% is $P(\text{result this extreme} \mid \text{drug does nothing})$, and the colleague has reported it as $1 - P(\text{drug works} \mid \text{result})$. Getting from the first to the second needs a base rate — how plausible the drug was before the study — which the p -value does not contain. [S9](#) is entirely about this distinction.

Where this shows up next

[S5](#) attaches probabilities to numbers rather than events, which is what makes them summable and averageable. [S6](#) names the standard families of those probabilities and says which mechanism produces which. [S9](#) is built on one conditional probability, and most of the trouble with p -values is people reading it backwards. [S12](#) returns to base rates, this time to explain why most published findings in a low-prior field are false.

32 Random variables

Statistics · Unit 5

S5 · Statistics Reading: 6 pages With practice: 2 hours Assumes: S4, M7

Why this unit is here

[S4](#) attached probabilities to *events*: things that either happen or do not. That is enough for coins and diagnoses and not much else, because you cannot average an event or add two of them together.

A **random variable** attaches probabilities to *numbers*, and numbers can be added, averaged and rescaled. Doing so gives you two quantities — the expected value and the variance — that turn out to obey simple rules, and those rules are what make the rest of statistics possible. Every standard error you will ever compute is one line of algebra away from the variance rules in this unit.

32.1 Before you start

You should be able to:

- compute and interpret a conditional probability — [S4](#);
- say why $P(A \cap B) = P(A)P(B)$ needs independence — [S4](#);
- read an integral as the area under a curve — [M7](#);
- compute a mean and a variance from data — [S2](#).

Quick check. A fair six-sided die is rolled. What is the probability of rolling a 3, and what is the probability of rolling *at most* 3?

 Show the answer

$P(X = 3) = 1/6$, and $P(X \leq 3) = 3/6 = 1/2$.

You have just used both of the functions this unit is about. The first is the **probability mass function**, evaluated at one value; the second is the **cumulative distribution function**, which accumulates everything up to a value.

32.2 The idea

A **random variable** is a rule that assigns a number to each outcome of a random process. Roll a die, record the face: that is a random variable. Pick a student from the cohort at random, record their final score: so is that.

The convention is a capital letter for the variable and a lower-case letter for a value it might take, so $P(X = x)$ reads “the probability that the random variable X turns out to equal the particular number x ”. The distinction is worth respecting: X is the *procedure*, x is *one result*.

Random variables come in two kinds, and the difference is not a technicality. (Strictly there is a third possibility — a variable can be part lump, part smooth, like a rainfall total that is exactly zero on dry days and continuous otherwise. Such mixtures are real and are handled by combining the two cases; nothing else in these notes needs them.)

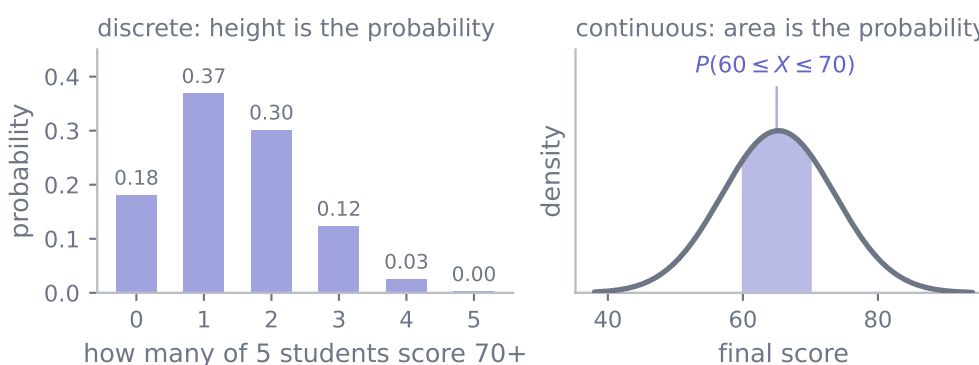


Figure 32.1: Two ways of carrying a probability. On the left each bar’s height is a probability and the six heights add to 1. On the right the curve’s height is not a probability at all — only the shaded area is, and the total area under the whole curve is 1.

A **discrete** random variable takes values you can list — 0, 1, 2, ... — and each one carries a probability of its own. The function $p(x) = P(X = x)$ is the **probability mass function**, and its values are genuine probabilities: between 0 and 1, adding to 1 across all the values.

A **continuous** random variable is one whose probabilities are given by a density, and for such a variable **the probability of any single exact value is zero**. That is a consequence of the definition rather than of having infinitely many values — a variable can have infinitely many possible values and still put a lump of probability on one of them, as the rainfall example does on zero. Under a density there are no lumps, so asking “what is the probability of a score of exactly 64.7?” has the answer 0, and that is not a defect in the question but a fact about the model.

What carries the probability instead is **area**. A continuous variable is described by a **probability density function** $f(x)$, and

$$P(a \leq X \leq b) = \int_a^b f(x) dx,$$

the area under the density between a and b . This is why M7 exists in these notes: a probability, for a continuous variable, is an integral, and it obeys the rules of integration because that is what it is.

Two consequences follow immediately. The total area under a density is 1. And $f(x)$ itself is *not* a probability — it can exceed 1 quite legitimately, as long as the width it applies over is small enough that the area does not.

32.3 The mechanics

32.3.1 The cumulative distribution function

One function works for both kinds:

$$F(x) = P(X \leq x).$$

It runs from 0 up to 1, never decreases, and for a continuous variable it is the accumulated area from the left — so F is an antiderivative of f , exactly as M7’s fundamental theorem describes. Every “probability of being below this value” that you will meet in S7 and S9 is a value of F , and every quantile is a value of F read backwards.

32.3.2 Expected value

$$E[X] = \sum_x x p(x) \quad (\text{discrete}) \quad E[X] = \int_{-\infty}^{\infty} x f(x) dx \quad (\text{continuous})$$

Read it as a weighted average: every value the variable can take, weighted by how likely it is. It is the long-run average of the results if you repeated the process for ever, and it is written μ when it belongs to a population — the μ of S1, which the sample mean $\bar{x}$ estimates.

each block's height is its probability; the pile balances at the mean

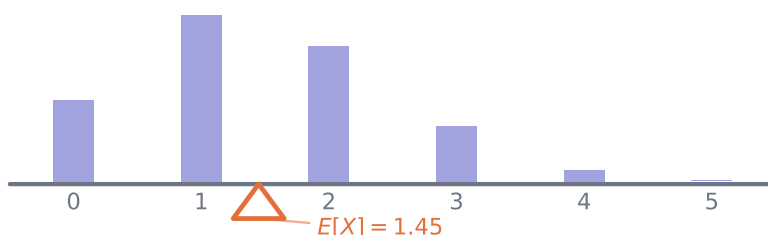


Figure 32.2: The left-hand distribution from the previous figure, treated as a pile of weights on a plank. The expected value is the point where it balances — which is why an expectation, unlike a median, is dragged about by values far from the centre.

Notice that $E[X] = 1.45$ is not a value the variable can take — you cannot have 1.45 students out of five. An expected value is a balance point, not a prediction.

Expectation is **linear**, and this is the most useful fact in the unit:

$$E[aX + b] = a E[X] + b, \quad E[X + Y] = E[X] + E[Y].$$

The second holds **whatever the relationship between X and Y** — no independence required. That is unusual and worth remembering, because the corresponding statement about variance is not free at all.

32.3.3 Variance

$$\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2,$$

with $\sigma = \sqrt{\text{Var}(X)}$ the standard deviation. Same idea as S2's s^2 , now attached to the distribution rather than to a sample. The second form is the one to compute with; the first is the one to understand.

The rules look similar to expectation's and are importantly different:

$$\text{Var}(aX+b) = a^2 \text{Var}(X), \quad \text{Var}(X+Y) = \text{Var}(X) + \text{Var}(Y) \text{ if } X \text{ and } Y \text{ are independent.}$$

Three things to take from that pair.

Adding a constant does nothing. Shifting every value by b moves the mean by b and leaves every deviation from the mean unchanged.

Scaling squares. Double a variable and its variance quadruples, so its standard deviation doubles. Variance lives in squared units; standard deviations are the ones that scale the way intuition expects.

The sum needs a condition, and it matters. The general statement is always $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y)$, with the covariance of S3 in it, and the two terms add cleanly exactly when that covariance is zero. Independence guarantees a zero covariance, which is why it is the condition usually quoted, but it is stronger than necessary: two variables can be uncorrelated and still dependent. Assuming a zero covariance you do not have is how a standard error comes out too small, and a confidence interval too narrow.

32.3.4 The consequence everything else is built on

Put n independent copies of X , each with mean μ and variance σ^2 , and average them. Expectation is linear, so the mean of the average is μ . Variance scales by a^2 with $a = 1/n$, and adds across independent terms, so

$$\text{Var}(\bar{X}) = \frac{1}{n^2} n\sigma^2 = \frac{\sigma^2}{n}, \quad \text{sd}(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

That quantity $\sigma/\sqrt{n}$ is the **standard error** of the mean, and the $\sqrt{n}$ in its denominator is the reason precision is expensive: to halve the standard error you need four times the data. S7 does nothing but explore this formula, and S8 puts it to work. It fell out of two lines of algebra here.

32.4 Worked example

A student drawn at random.

Define X to be the final score of one student chosen uniformly at random from the cohort's 238 usable records. That is a genuine random variable, and because we can see the whole population it is one whose μ and σ we can compute exactly rather than estimate.

```
mu = scores.mean()
sigma = scores.std(ddof=0)      # ddof=0: these 238 ARE the population here
print(f"E[X]      = {mu:.4f}")
print(f"Var(X)    = {sigma**2:.4f}")
print(f"sd(X)     = {sigma:.4f}")
print(f"check: E[X^2] - mu^2 = {(scores**2).mean() - mu**2:.4f}")
```

```
E[X]      = 65.2718
Var(X)    = 69.9982
sd(X)     = 8.3665
check: E[X^2] - mu^2 = 69.9982
```

Note the `ddof=0`. [S2](#) used $n - 1$ because it was *estimating* a population from a sample; here the 238 records are the entire population we are drawing from, so there is nothing to correct for.

Now the rules. Rescale to a 0–1 fraction, $Y = X/100$:

```
fractions = scores / 100
print(f"E[Y]    predicted {mu / 100:.4f}      actual {fractions.mean():.4f}")
print(f"Var(Y)  predicted {sigma**2 / 100**2:.6f}  actual
↳ {fractions.var(ddof=0):.6f}")
```

```
E[Y]    predicted 0.6527      actual 0.6527
Var(Y)  predicted 0.007000    actual 0.007000
```

The variance fell by a factor of 100^2 , not 100. Squared units.

Now the part people get wrong. Draw *two* students independently and add their scores; compare that with drawing *one* student and doubling.

```
rng = np.random.default_rng(2026)
two_students = rng.choice(scores, size=(120_000, 2)).sum(axis=1)
one_doubled = 2 * rng.choice(scores, size=120_000)

print(f"X1 + X2 : mean {two_students.mean():7.2f}    sd
↳ {two_students.std():6.2f}")
    f"    predicted sd {np.sqrt(2) * sigma:.2f}")
print(f"2X      : mean {one_doubled.mean():7.2f}    sd
↳ {one_doubled.std():6.2f}")
    f"    predicted sd {2 * sigma:.2f}")
```


$X_1 + X_2$	: mean	130.52	sd	11.81	predicted sd	11.83
$2X$	: mean	130.54	sd	16.74	predicted sd	16.73

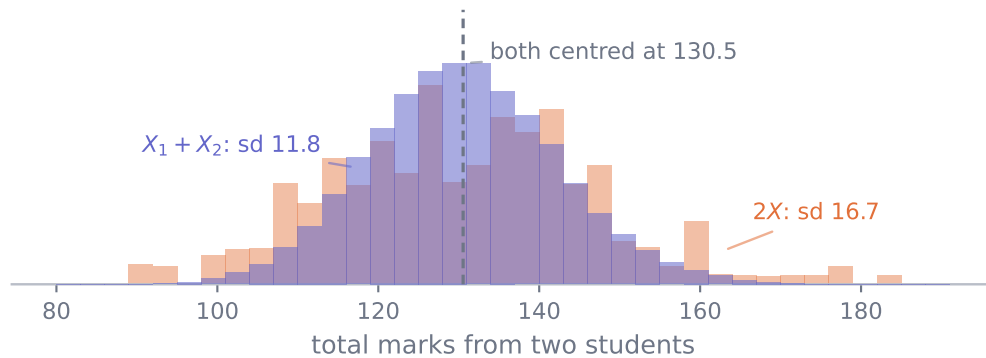


Figure 32.3: Two students' marks added together, against one student's mark doubled. Both have expected value 130.5, and the second is 40% more spread out. Independent errors partly cancel; a doubled error cannot cancel with anything.

Same expected total, and the doubled version is 40% wider. Both facts come straight from the rules: expectation is linear either way, but $\text{Var}(X_1 + X_2) = 2\sigma^2$ while $\text{Var}(2X) = 4\sigma^2$, and $\sqrt{4}/\sqrt{2} = \sqrt{2} = 1.41$.

The intuition behind the algebra: two independent students can be unlucky in opposite directions and partly cancel. One student, counted twice, cannot cancel with anything — whatever they did, it happens twice over.

32.5 Try it yourself

Exercise 1 A random variable X takes the value 0 with probability 0.7 and 1 with probability 0.3.

- Find $E[X]$ and $\text{Var}(X)$.
- Now let $S = X_1 + X_2 + \dots + X_{10}$ be the sum of ten independent copies. Find $E[S]$ and $\text{Var}(S)$.

💡 Show the solution

- $E[X] = 0(0.7) + 1(0.3) = 0.3$. For the variance, note that $X^2 = X$ here because $0^2 = 0$ and $1^2 = 1$, so $E[X^2] = 0.3$ and

$$\text{Var}(X) = 0.3 - 0.3^2 = 0.3 \times 0.7 = 0.21.$$

That shortcut generalises: a variable that is 1 with probability p and 0 otherwise has mean p and variance $p(1 - p)$.

- $E[S] = 10 \times 0.3 = 3$, and by independence $\text{Var}(S) = 10 \times 0.21 = 2.1$, so $\text{sd}(S) = 1.45$.

A common wrong answer is $\text{sd}(S) = 10 \times 0.458 = 4.58$ — multiplying the standard deviation by 10 rather than the variance. Standard deviations do not add; variances do.

Exercise 2 A continuous random variable has density $f(x) = 2x$ for $0 \leq x \leq 1$, and zero elsewhere.

- (a) Check that it really is a density.
- (b) Find $P(X \leq 0.5)$.
- (c) Find $E[X]$.

 Show the solution

- (a) A density must be non-negative — $2x \geq 0$ on $[0, 1]$ — and must enclose a total area of 1:

$$\int_0^1 2x \, dx = [x^2]_0^1 = 1. \checkmark$$

Note in passing that $f(1) = 2 > 1$. A density is allowed to exceed 1; only its integral is constrained.

- (b) $P(X \leq 0.5) = \int_0^{0.5} 2x \, dx = [x^2]_0^{0.5} = 0.25$. Not 0.5 — the density puts more weight on the right, so the left half of the range holds less than half the probability.
- (c) $E[X] = \int_0^1 x \cdot 2x \, dx = \int_0^1 2x^2 \, dx = [\frac{2}{3}x^3]_0^1 = \frac{2}{3}$.

Sanity check: $2/3$ is to the right of the middle of the range, which matches a density that rises towards 1.

Exercise 3 Two exam papers are each marked out of 50. A student's marks X and Y each have mean 30 and standard deviation 8.

- (a) What is the expected total, $E[X + Y]$?
- (b) What is $\text{sd}(X + Y)$ if the two marks are independent?
- (c) In reality a strong student tends to do well on both. Does the true standard deviation of the total go up or down relative to (b)?

 Show the solution

- (a) $E[X + Y] = 30 + 30 = 60$, and this needs **no** assumption about independence. Linearity of expectation is unconditional.
- (b) $\text{Var}(X + Y) = 64 + 64 = 128$, so $\text{sd} = \sqrt{128} = 11.3$. Note this is well below $8 + 8 = 16$.
- (c) **Up.** With positive correlation the covariance term is positive:

$$\text{Var}(X + Y) = 64 + 64 + 2\text{Cov}(X, Y) > 128.$$

At the extreme of a perfect positive relationship, $Y = X$, the total is $2X$ with standard deviation 16 — the case in this unit's worked example. Independence is what lets the two marks partly cancel; correlation removes that cancellation. Assuming independence when it does not hold makes your stated uncertainty too small, which is the dangerous direction to be wrong in.

32.6 Check it in Python

The expected value is a long-run average, and that is a claim you can watch come true. Draw students one at a time and keep a running mean.

```
rng = np.random.default_rng(7)
draws = rng.choice(scores, size=2000)
running = np.cumsum(draws) / np.arange(1, draws.size + 1)

for k in [10, 100, 1000, 2000]:
    print(f"after {k:>4} draws: running mean {running[k-1]:.3f}    "
          f"gap from E[X] {running[k-1] - mu:+.3f}")
```

```
after   10 draws: running mean 61.170    gap from E[X] -4.102
after  100 draws: running mean 63.972    gap from E[X] -1.300
after 1000 draws: running mean 64.993    gap from E[X] -0.279
after 2000 draws: running mean 65.120    gap from E[X] -0.152
```

The gap shrinks, unsteadily — this is the **law of large numbers**, and note what it does and does not promise. The running mean converges to μ ; it does not promise the gap shrinks at every step, and it says nothing at all about any individual draw.

Next, the variance rules, all three at once:

```
pairs = rng.choice(scores, size=(200_000, 2))
X, Y = pairs[:, 0], pairs[:, 1]
print(f"Var(X)           {X.var():9.4f}    (population value {sigma**2:.4f})")
print(f"Var(X + 10)      {(X + 10).var():9.4f}    predicted {sigma**2:.4f}")
print(f"Var(3X)           {(3*X).var():9.4f}    predicted {9*sigma**2:.4f}")
print(f"Var(X + Y)        {(X + Y).var():9.4f}    predicted {2*sigma**2:.4f}")
```

```
Var(X)           69.8402    (population value 69.9982)
Var(X + 10)       69.8402    predicted 69.9982
Var(3X)           628.5615    predicted 629.9842
Var(X + Y)        139.5885    predicted 139.9965
```

And now the failure mode. The addition rule needs a zero covariance, and it is easy to write code that quietly violates it. Here Y is not a second draw at all — it is the same student again:

```
Y_same = X                                # NOT an independent second draw
total = X + Y_same
assert np.isclose(total.var(), 2 * sigma**2, rtol=0.02), (
    f"Var(X+Y) came out {total.var():.1f}, but the independence rule "
    f"predicts {2 * sigma**2:.1f}")
)
```

AssertionError: Var(X+Y) came out 279.4, but the independence rule predicts
↪ 140.0

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[9], line 3
      1 Y_same = X                                # NOT an independent second draw
      2 total = X + Y_same
----> 3 assert np.isclose(total.var(), 2 * sigma**2, rtol=0.02), (
      4     f"Var(X+Y) came out {total.var():.1f}, but the independence rule
↪     "
      5     f"predicts {2 * sigma**2:.1f}"
      6 )
AssertionError: Var(X+Y) came out 279.4, but the independence rule predicts
↪ 140.0
```

Double, not doubled again: the variance is $4\sigma^2$ because $X + X = 2X$. In this cell the mistake is obvious. In a real analysis it looks like using the same patient twice, or treating repeated measurements as independent observations, and it inflates nothing visible — the mean comes out fine, and only the stated uncertainty is wrong.

```
print(f"Var(X + X)                                {(X + X).var():9.4f}")
print(f"Var(X) + Var(X) + 2Cov(X, X)              "
      f"{X.var() + X.var() + 2*np.cov(X, X, ddof=0)[0, 1]:9.4f}")
```

Var(X + X)	279.3607
Var(X) + Var(X) + 2Cov(X, X)	279.3607

The general formula, with the covariance term restored, gets it right. The independence version is a special case in which that term happens to be zero.

32.7 Common wrong turns

Where this usually goes wrong

Reading a density as a probability $f(x)$ can be greater than 1. It is a probability *per unit* of x , and only an area under it — a density times a width — is a probability. For a continuous variable $P(X = x) = 0$ for every single x .

Adding standard deviations Variances add across independent variables; standard deviations do not. Two independent marks with sd 8 give a total with sd $\sqrt{128} = 11.3$, not 16. Convert to variance, add, convert back.

Assuming independence because the code drew twice Independence is a property of the *design*, not of the number of lines you wrote. Two measurements on the same person, two days from the same week, two students in the same tutorial group — none of these are independent, and each carries a covariance term that the simple addition rule leaves out.

Expecting the expected value $E[X] = 1.45$ students, $E[X] = 3.5$ on a die. An expectation is a balance point and often not a possible outcome. It is what the average converges to, not what you should predict for one trial.

Believing the law of large numbers applies to short runs It says the *average* settles down eventually. It does not say a run of bad luck will be compensated by a run of good luck; the earlier draws are simply diluted by the later ones.

Using $n - 1$ when you have the whole population The correction exists because $\bar{x}$ was estimated from the same data. If the values in front of you are the entire population, divide by n . Getting this wrong is harmless at $n = 238$ and material at $n = 5$.

32.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. X has mean 10 and standard deviation 3. What is the standard deviation of $4X - 7$?
a. 5 b. 12 c. 33 d. 144
2. A continuous random variable has $f(2) = 1.8$. Is that a problem?
3. X and Y each have variance 9. Is $\text{Var}(X + Y) = 18$?
4. You average n independent measurements. By what factor must n increase to halve the standard error?
5. True or false: $E[X + Y] = E[X] + E[Y]$ requires X and Y to be independent.

 Show the answers

1. **(b), 12.** $\text{Var}(4X - 7) = 4^2 \times 9 = 144$, so the standard deviation is $\sqrt{144} = 12$. Equivalently, multiply the standard deviation by $|4|$. Option (d) is the variance, offered because reporting a variance where a standard deviation was asked for is the standard slip; the -7 affects neither.
2. **No.** A density is not a probability; it is a probability per unit of x . A value of 1.8 is fine provided the total area under f is 1 — for instance a variable spread uniformly over an interval of width 0.5 has density 2 throughout.
3. **Only if they are independent** (or, more weakly, uncorrelated). In general $\text{Var}(X + Y) = 9 + 9 + 2 \text{Cov}(X, Y)$, which can be anything from 0 (when $Y = -X$) to 36 (when $Y = X$).
4. **Four times.** The standard error is $\sigma/\sqrt{n}$, so halving it needs $\sqrt{n}$ to double, so n to quadruple. This is the single most useful piece of arithmetic in study design, and [S8](#) returns to it.
5. **False**, and this is the asymmetry worth remembering. Linearity of expectation holds for *any* X and Y , however strongly related. It is the variance rule that needs independence.

Where this shows up next

[S6](#) names the standard families of mass and density functions, and says which real mechanism produces which. [S7](#) applies $\text{sd}(\bar{X}) = \sigma/\sqrt{n}$ to the sample mean and finds that its shape becomes normal whatever X looked like. [S8](#) turns the standard error into an interval. [S11](#)

treats a fitted slope as a random variable and attaches a standard error to it by exactly the algebra in this unit.

33 Distributions chosen by mechanism

Statistics · Unit 6

S6 · Statistics Reading: 6 pages With practice: 2 hours Assumes: S5

Why this unit is here

A histogram tells you about the data you have. A **distribution** is a model, and a model lets you compute probabilities of things you have not seen: how often five failures in a row would happen by chance, how far from the mean an observation has to be before it is surprising, what a 95% interval should be wide.

The trap is choosing the distribution by looking at the histogram. Almost every family in this unit is the consequence of a **mechanism** — a story about how the numbers were generated — and the shape is what falls out of the story. Pick by the story and you can defend the choice and test it. Pick by the shape and you have drawn a curve that fits the data you happen to have.

33.1 Before you start

You should be able to:

- distinguish a probability mass function from a density — S5;
- compute an expected value and a variance — S5;
- read a histogram — S2.

Quick check. A random variable takes the value 1 with probability p and 0 otherwise. What are its mean and its variance?

💡 Show the answer

$$E[X] = p \text{ and } \text{Var}(X) = p(1 - p).$$

This is the **Bernoulli** distribution, and it is the atom the first half of this unit is built from. Note that the variance is largest at $p = 0.5$ and vanishes at $p = 0$ or 1 : a coin you cannot predict is the most variable one there is.

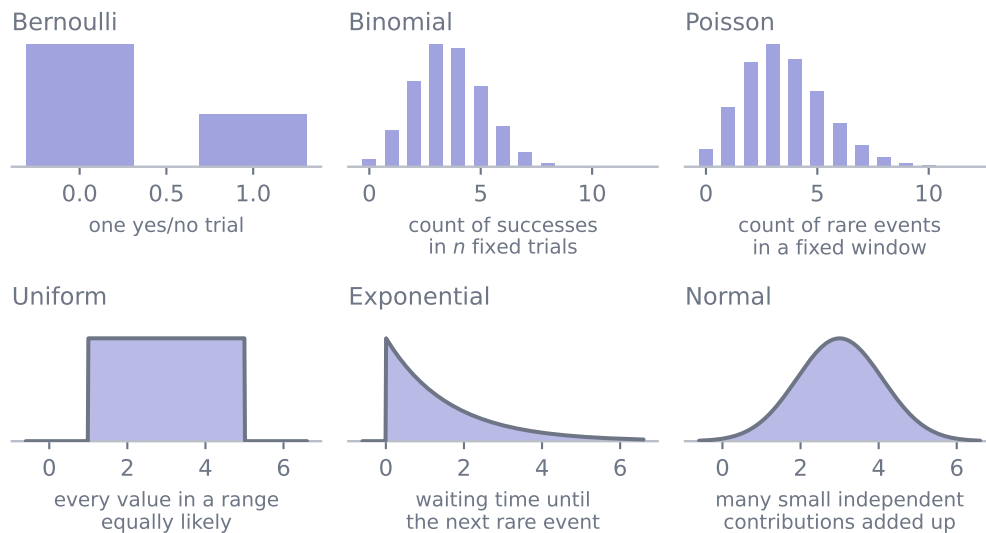


Figure 33.1: Six standard distributions, each with the mechanism that produces it. The top row is discrete — the bars are probabilities. The bottom row is continuous — the areas are probabilities. You choose between them by asking which story your data came from, not by matching the picture to a histogram.

33.2 The idea

Six families cover most of what you will meet, and each of them answers a specific question about a specific kind of process.

Read the captions rather than the shapes. “Count of successes in n fixed trials” is a claim about your data-collection process that you can check. “Looks a bit like the second one” is not.

33.3 The mechanics

33.3.1 Bernoulli and Binomial

A **Bernoulli** variable is one trial with two outcomes: $P(X = 1) = p$, $P(X = 0) = 1 - p$, mean p , variance $p(1 - p)$.

Add n **independent** Bernoulli trials that all share the same p and you count successes. That is the **Binomial**:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n$$

with $E[X] = np$ and $\text{Var}(X) = np(1 - p)$ — which follow from S5’s rules in one line each, since expectation and (under independence) variance both add.

The binomial makes two assumptions, and both fail in practice more often than people check. **The trials must be independent:** students in the same tutorial group, or repeat visits by the same customer, are not. **The probability must be the same every trial:** if p drifts — a machine wearing out, a policy changing midway — the count is no longer binomial.

It is worth being precise about which way each failure pushes the spread, because the obvious guess is wrong for one of them. Trials that stay independent but have *different* fixed probabilities $p_1, \dots, p_n$ give a count with variance $\sum p_i(1-p_i)$, which is **smaller** than $n\bar{p}(1-\bar{p})$ — spread is greatest when every trial is equally uncertain. What produces *overdispersion*, the far more common complaint, is failure of the first assumption: trials that are positively related, so that they tend to succeed or fail together.

33.3.2 Poisson

Events happen at a constant average rate λ per window, independently of each other, and you count how many fall in one window. That is the **Poisson**:

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}, \quad k = 0, 1, 2, \dots$$

with the striking property

$$E[X] = \text{Var}(X) = \lambda.$$

That equality is a **testable prediction**, and it is the first thing to check when someone proposes a Poisson model. Real count data — accidents, faults, support tickets — is usually *overdispersed*, with variance well above the mean, because the events cluster or the rate is not constant. If your counts have mean 4 and variance 15, the Poisson is already refuted, and no amount of curve-fitting repairs that.

33.3.3 Uniform

Every value in $[a, b]$ is equally likely: $f(x) = 1/(b-a)$ on that interval and zero outside it. Mean $(a+b)/2$, variance $(b-a)^2/12$. It is the distribution of “I know the range and nothing else”, and it is rarer in real data than beginners expect — genuine measurements almost always have a middle they prefer.

33.3.4 Exponential

The **waiting time** until the next event, when events arrive as a Poisson process at rate λ :

$$f(x) = \lambda e^{-\lambda x} \ (x \geq 0), \quad E[X] = \frac{1}{\lambda}, \quad \text{Var}(X) = \frac{1}{\lambda^2}.$$

Its defining property is **memorylessness**: having waited ten minutes already tells you nothing about how much longer you will wait. That is a very strong assumption. It is roughly true of radioactive decay and roughly false of almost everything involving wear, queues or human behaviour — a machine that has run for ten years is *not* as good as new.

33.3.5 Normal

The **normal** distribution arises when many small independent contributions add up, which is why it is everywhere: heights, measurement errors, exam marks built from many questions.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$$

It is symmetric about μ , and the famous fractions follow:

Table 33.1: The 68–95–99.7 rule

within	of the mean	contains about
$\mu \pm 1\sigma$	one standard deviation	68%
$\mu \pm 2\sigma$	two standard deviations	95%
$\mu \pm 3\sigma$	three standard deviations	99.7%

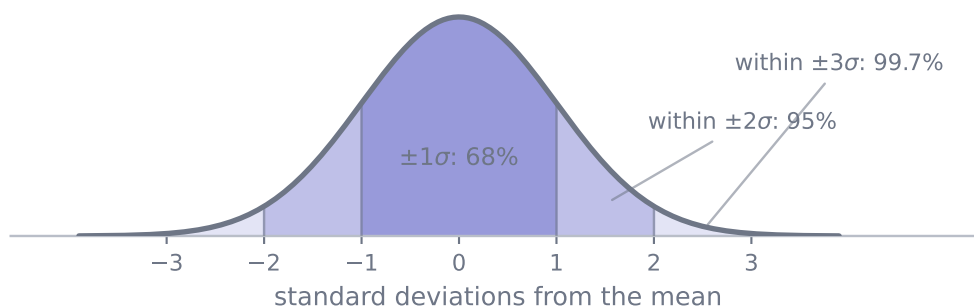


Figure 33.2: The 68–95–99.7 rule. Each band is an area under the density, so each is a probability — and the outermost band is thin enough that a three-sigma observation is genuinely rare, about 3 in 1000.

Two warnings that matter more than the formula. First, **the normal density has no elementary antiderivative** — the areas above cannot be worked out with the techniques of M7, which is why every statistics book once carried a table and every library now carries `norm.cdf`. Second, the normal has thin tails, so when the real distribution has heavier ones it badly underestimates how often extreme events happen. Financial returns are the standard cautionary tale. The error can run the other way — the cohort’s diagnostic score, below, has *lighter* tails than a normal, so a normal model would over-predict its extremes — and the general point is that a fit judged in the middle tells you very little about either tail.

33.3.6 Choosing, then checking

The procedure is always the same. Ask what generated the data, pick the family whose mechanism matches, then **check the fit against a prediction the family makes** — the mean-equals-variance property for a Poisson, the 68–95–99.7 fractions for a normal, a quantile-quantile plot for anything continuous.

A **Q-Q plot** sorts your data and plots each value against the value the model says should sit at that position. If the model is right the points fall on a straight line; the ways they leave it tell you how it is wrong.

33.4 Worked example

Two questions about the cohort, two different families.

```
cohort = pd.read_csv("../data/cohort.csv")
scored = cohort[cohort["final_score"].between(0, 100)]
scores = scored["final_score"].to_numpy()
diagnostic = scored["diagnostic_score"].to_numpy()
```

Question one: of 20 students picked at random, how many score 70 or more?

The mechanism is right there: 20 independent draws, each either a success or not, with the same probability each time. That is a binomial, and the only thing to supply is p .

```
p = (scores >= 70).mean()
n = 20
print(f"p = {(scores >= 70).sum()}/{scores.size} = {p:.4f}")
print(f"Binomial(20, {p:.3f}): mean {n*p:.2f} variance {n*p*(1-p):.2f}"
      f" sd {np.sqrt(n*p*(1-p)):.2f}")
print()
for k in [2, 4, 6, 8, 10]:
    print(f" P(exactly {k}>2) = {stats.binom.pmf(k, n, p):.4f}"
          f" P(at most {k}>2) = {stats.binom.cdf(k, n, p):.4f}")
```

$p = 69/238 = 0.2899$

Binomial(20, 0.290): mean 5.80 variance 4.12 sd 2.03

P(exactly 2) = 0.0336	P(at most 2) = 0.0434
P(exactly 4) = 0.1430	P(at most 4) = 0.2688
P(exactly 6) = 0.1907	P(at most 6) = 0.6463
P(exactly 8) = 0.1033	P(at most 8) = 0.9054
P(exactly 10) = 0.0253	P(at most 10) = 0.9869

So about six of twenty, and getting ten or more would be unusual but not extraordinary — $1 - F(9) = 3.8\%$ of the time. Notice that this is a claim about groups of twenty we never observed, produced entirely by the model.

The independence assumption deserves a moment. Twenty students drawn at random from a large population are effectively independent. Twenty students from the same tutorial group are not, and the binomial would understate the spread.

Question two: is the final score normal?

Here there is a mechanism to appeal to — a final score is built from many questions, each contributing a little — so the normal is a reasonable *prior* guess. Now check it, and check a variable where the same guess fails.

```

for name, values in [("final score", scores), ("diagnostic score",
↪ diagnostic)]:
    m, s = values.mean(), values.std(ddof=1)
    inside = [((values > m - k*s) & (values < m + k*s)).mean() for k in (1,
↪ 2, 3)]
    print(f"{name:<18} within 1sd {inside[0]:.3f}    2sd {inside[1]:.3f}    "
          f"3sd {inside[2]:.3f}")
print(f"'normal model says':<18} within 1sd 0.683    2sd 0.954    3sd 0.997")

```

final score	within 1sd 0.710	2sd 0.954	3sd 0.996
diagnostic score	within 1sd 0.601	2sd 0.979	3sd 1.000
normal model says	within 1sd 0.683	2sd 0.954	3sd 0.997

The final score is close on all three. The diagnostic score is not: only 60% of it lies within one standard deviation of the mean, against the 68% a normal predicts, and too much sits in the second band. That is the signature of a distribution that is *flatter* than a normal — which it should be, because S2 showed the cohort is five background groups with well-separated centres, and the diagnostic separates them most sharply of all.

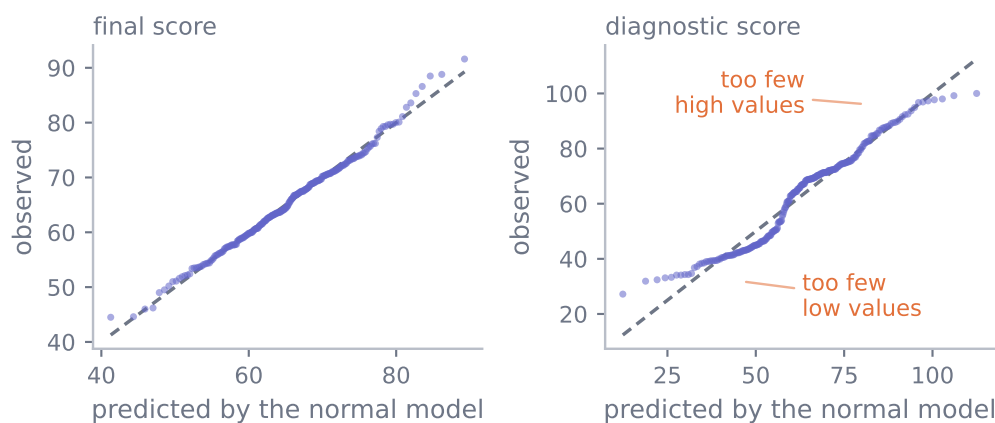


Figure 33.3: Quantile-quantile plots against a normal model. Each point puts an observed value against the value the model predicts for that position in the sorted order; a perfect fit lies on the dashed line. The final score follows it closely. The diagnostic score curves away at both ends: the low values are not as low as the model predicts and the high values not as high. That is the signature of a distribution with lighter tails than the normal, which is what a mixture of five well-separated groups produces.

The honest conclusion is the one to practise writing: *a normal model is adequate for the final score for the purposes of the intervals in S8, and not adequate for the diagnostic score, which is a mixture and should be handled by group*. Neither variable “is normal” — that is not a property data has. A model is adequate or not, for a stated purpose.

33.5 Try it yourself

Exercise 1 For each of these, name the distribution you would try first and say what mechanism justifies it.

- (a) The number of heads in 50 tosses of a fair coin.
- (b) The number of emails arriving at a server between 09:00 and 09:01.
- (c) The time until a randomly chosen radioactive atom decays.
- (d) The height of a randomly chosen adult.
- (e) The number of students in a tutorial group of 12 who complete the course, where the tutor's teaching affects everyone in the group.

 Show the solution

- (a) **Binomial(50, 0.5)** — a fixed number of independent trials with a constant success probability. Every assumption holds exactly, which is rare.
- (b) **Poisson** — a count of events in a fixed window at an approximately constant rate. Check the mean against the variance before trusting it: email arrives in bursts, and bursts produce overdispersion.
- (c) **Exponential** — the waiting time for a memoryless process. Radioactive decay is the one textbook example where memorylessness is genuinely true.
- (d) **Normal** — many small independent genetic and environmental contributions adding up. Note that adult height in a *mixed-sex* population is a mixture of two normals, and looks flatter than either, exactly like the diagnostic score in the worked example.
- (e) **Not binomial**, and this is the interesting one. The trials are not independent: a good or bad tutor moves every student in the group the same way. The count still has a mean of $12p$, but its variance exceeds $12p(1 - p)$, so binomial intervals around it would be too narrow.

Exercise 2 A support desk records the number of tickets per hour over 200 hours. The mean is 4.1 and the variance is 14.8.

- (a) Is a Poisson model tenable?
- (b) Suggest two mechanisms that could produce this pattern.
- (c) What goes wrong if you use the Poisson anyway?

 Show the solution

- (a) **No.** A Poisson predicts mean = variance. Here the variance is more than three times the mean, which is the standard diagnosis of **overdispersion**.
- (b) Either the **rate is not constant** — busier at 10am than at 4pm, busier on Mondays, a spike when a release goes out — or the **events are not independent**, because one outage generates fifty tickets at once. Both produce the same symptom, and distinguishing them needs more than the two summary numbers.
- (c) Everything derived from the model understates the spread. A Poisson would put the standard deviation at $\sqrt{4.1} = 2.0$ when it is really $\sqrt{14.8} = 3.8$, so alert thresholds fire constantly, capacity plans are built on a quiet hour that does not exist, and any interval you quote is roughly half as wide as it should be.

Exercise 3 Exam marks have mean 65 and standard deviation 8, and a normal model is adequate.

- (a) Roughly what fraction of students score above 81?
- (b) A student scores 41. How unusual is that, and what would you check before saying so out loud?

💡 Show the solution

- (a) $81 = 65 + 2 \times 8$, so this is two standard deviations above the mean. About 95% lies within two standard deviations, leaving 5% outside, split evenly between the two tails: roughly **2.5%**.
- (b) $41 = 65 - 3 \times 8$, three standard deviations below. Under a normal model about 0.3% lies outside three standard deviations, so about **0.15%** below — roughly 1 student in 700.

What to check first: whether the normal model is adequate *in the tail*. The 68–95–99.7 fractions are dominated by the middle of the distribution, and a model can match them well while getting the tails badly wrong — which is exactly where this question lives. With 240 students you have almost no information about a 1-in-700 event, so the honest answer is “unusual under this model, and this model is not well tested that far out”.

33.6 Check it in Python

The distributions in this unit make predictions, and every prediction here is one simulation away from being checked.

```
rng = np.random.default_rng(2026)
simulated = rng.binomial(n=20, p=p, size=200_000)
print(f"simulated mean {simulated.mean():.4f}    formula np      {n*p:.4f}")
print(f"simulated var  {simulated.var():.4f}    formula np(1-p)
↪ {n*p*(1-p):.4f}")
```

```
simulated mean 5.7879    formula np      5.7983
simulated var  4.1227    formula np(1-p) 4.1173
```

The Poisson’s mean-equals-variance property, and then the same statistic on data that is not Poisson:

```
poisson_counts = rng.poisson(lam=4.0, size=200_000)
print(f"true Poisson:   mean {poisson_counts.mean():.3f}    "
      f"variance {poisson_counts.var():.3f}    ratio
↪ {poisson_counts.var()/poisson_counts.mean():.3f}")

# A rate that changes hour to hour: still counts, no longer Poisson.
rates = rng.uniform(1.0, 7.0, size=200_000)
```

```

mixed_counts = rng.poisson(lam=rates)
print(f"drifting rate:  mean {mixed_counts.mean():.3f}    "
      f"variance {mixed_counts.var():.3f}    ratio "
      f"↪ {mixed_counts.var()/mixed_counts.mean():.3f}")

```

```

true Poisson:  mean 4.002    variance 4.006    ratio 1.001
drifting rate: mean 4.005    variance 7.007    ratio 1.750

```

Both sets of counts have almost the same mean. Only the variance gives the second one away, which is why the ratio is the diagnostic worth computing every time.

Now the check that fails. It is tempting to treat “roughly bell-shaped” as “normal enough”, so assert the 68% rule on the diagnostic score and watch:

```

m, s = diagnostic.mean(), diagnostic.std(ddof=1)
within_one = ((diagnostic > m - s) & (diagnostic < m + s)).mean()
assert abs(within_one - 0.683) < 0.02, (
    f"a normal model predicts 68.3% within one sd; the diagnostic score has "
    ↪ " "
    f"{100*within_one:.1f}%"
)

```

```

AssertionError: a normal model predicts 68.3% within one sd; the diagnostic
↪ score has 60.1%

```

```

-----
AssertionError                                Traceback (most recent call last)
Cell In[9], line 3
      1 m, s = diagnostic.mean(), diagnostic.std(ddof=1)
      2 within_one = ((diagnostic > m - s) & (diagnostic < m + s)).mean()
----> 3 assert abs(within_one - 0.683) < 0.02, (
      4     f"a normal model predicts 68.3% within one sd; the diagnostic
↪ score has "
      5     f"{100*within_one:.1f}%"
      6 )
AssertionError: a normal model predicts 68.3% within one sd; the diagnostic
↪ score has 60.1%

```

Eight percentage points is a large miss, and it is in the direction that says “flatter than normal”. Run the same check on the final score and it passes:

```

m, s = scores.mean(), scores.std(ddof=1)
within_one = ((scores > m - s) & (scores < m + s)).mean()
print(f"final score: {100*within_one:.1f}% within one sd - "
      f"normal predicts 68.3%")
assert abs(within_one - 0.683) < 0.05

```

```

final score: 71.0% within one sd - normal predicts 68.3%

```

Note the tolerance had to be loosened to 5 percentage points for even the good case to pass. This is a coarse check, not a test, and [S9](#) gives you the machinery to say how coarse.

33.7 Common wrong turns

Where this usually goes wrong

Choosing a distribution because the histogram looks like it With 30 observations almost any smooth family can be made to look plausible. Start from the mechanism — fixed trials, arrivals in a window, waiting times, many small additive effects — and use the histogram to *check*, not to choose.

Using a binomial when the trials are related Students in a class, measurements from one instrument, days in one week. The mean is still np , but the variance is not $np(1-p)$: positive dependence, the usual kind, inflates it, so everything built on the formula is too confident. (Negative dependence deflates it instead — drawing without replacement from a small pool is the standard example — which is safer but equally wrong.)

Using a Poisson without checking mean against variance It is one line of code and it refutes the model outright when it fails. Real counts are overdispersed far more often than not.

Assuming an exponential waiting time for something that wears out Memorylessness says a machine that has run ten years is as good as new. For radioactive decay that is true; for machinery, queues and people it is usually false and always worth stating explicitly.

Saying “the data are normally distributed” Data are not distributed; they are data. A *model* is adequate for a purpose. Say “a normal model fits well enough for this interval”, and name the check that persuaded you.

Trusting a normal model in the tails The 68–95–99.7 fractions are dominated by the middle. A model can match them and still be wrong by orders of magnitude about a 4-sigma event, and 4-sigma events are usually the ones that matter.

Expecting the central limit theorem to normalise your data It does no such thing. It says something about the distribution of the *sample mean* — S7 — and leaves the individual observations exactly as skewed as they were.

33.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A count variable has mean 6 and variance 6.1. Which family is worth trying?
 - a. Binomial
 - b. Poisson
 - c. Normal
 - d. Exponential
2. $X \sim \text{Binomial}(100, 0.2)$. What are $E[X]$ and $\text{sd}(X)$?
3. A variable is a mixture of two well-separated normal distributions **in roughly equal proportions**. Why does it fail the 68% check in the direction it does, and why does the “roughly equal” matter?
4. A colleague fits a normal model, finds it matches the 68–95–99.7 fractions well, and concludes the chance of an observation 5 standard deviations out is about 1 in 3.5 million. What is wrong with this?
5. True or false: if the individual observations are skewed, no normal-based method can be used.

💡 Show the answers

1. **(b), Poisson.** Mean and variance being nearly equal is the Poisson's signature. (a) is possible but a binomial has variance $np(1-p)$, which is below its mean np for every p strictly between 0 and 1; (c) and (d) are for continuous variables, and an exponential would have variance 36, not 6.
2. $E[X] = np = 20$ and $\text{Var}(X) = np(1-p) = 100 \times 0.2 \times 0.8 = 16$, so $\text{sd}(X) = 4$. The commonest error is stopping at the variance and reporting 16.
3. Because a balanced mixture is **flatter** than a single normal with the same standard deviation. The overall standard deviation is inflated by the distance between the two centres, so the band $\mu \pm 1\sigma$ is wide, but the data are piled at the two centres rather than in the middle — leaving *fewer* than 68% inside. The cohort's diagnostic score shows exactly this at 60%.

The proportions matter because a *lopsided* mixture behaves in the opposite way. Put 99% of the weight at 0 and 1% at 100: the standard deviation is about 10, and roughly 99% of the values sit within one standard deviation of the mean. That is not a flat distribution at all — it is a tight one with a very heavy tail, and it fails the check in the other direction.
4. The fit was checked where the data are and the conclusion is drawn where they are not. Matching the central fractions says almost nothing about the tail beyond three standard deviations, and the number quoted — 1 in 3.5 million — is a property of the normal density, not of the data. Any dataset small enough to fit on a screen contains no evidence at all about 5-sigma events.
5. **False**, and this is the most useful thing in the next unit. Skewed observations can still give a sample *mean* whose distribution is close to normal, provided n is large enough — which is what [S7](#) is about. What you must not do is apply a normal model to the individual values.

Where this shows up next

[S7](#) explains why the normal turns up so often: sums and averages of almost anything tend towards it. [S8](#) uses the tail areas of a normal — and of its close relative the t — to build intervals. [S9](#) computes the probability of data at least as extreme as yours, which is a tail area of one of these families. [S10](#) uses the binomial and the chi-square when the outcome is a count rather than a measurement.

34 Sampling distributions and the CLT

Statistics · Unit 7

S7 · Statistics Reading: 6 pages With practice: 2 hours Assumes: S6

Why this unit is here

This is the hinge of the whole track. Everything before it describes data; everything after it makes claims about a population. The bridge between them is one idea: **a statistic computed from a sample is itself a random variable**, with a distribution of its own, and knowing that distribution is what lets you say how far your number might be from the truth.

The unit's two results are the standard error, $\sigma/\sqrt{n}$, and the central limit theorem, which says the sample mean becomes normally distributed almost whatever the population looks like. Together they are why a single study can say anything at all — and the limits of the second one are why some studies cannot.

34.1 Before you start

You should be able to:

- name a distribution from the mechanism that produced it — S6;
- state what a normal distribution's 68–95–99.7 fractions mean — S6;
- use $\text{Var}(aX) = a^2 \text{Var}(X)$ and the fact that variances add — S5;
- tell a population parameter from a sample estimate — S1.

Quick check. A population has standard deviation 12. You take a sample of 9 and compute its mean. What is the standard deviation of that mean, over repeated samples?

💡 Show the answer

$$\sigma/\sqrt{n} = 12/\sqrt{9} = 12/3 = 4.$$

If you answered 12, you have given the spread of the *observations*, not of the *mean*. Distinguishing those two is what this unit is for, and everything from S8 onwards depends on it.

34.2 The idea

Take the cohort's 238 usable final scores and treat them, for the length of this unit, as an entire population whose mean we happen to know. Now imagine drawing 25 of them at random and computing the mean. Then do it again. And again.

Each of those means is a number you could have reported. Collect them and you have a **sampling distribution**: the distribution of a statistic across all the samples you might have taken.

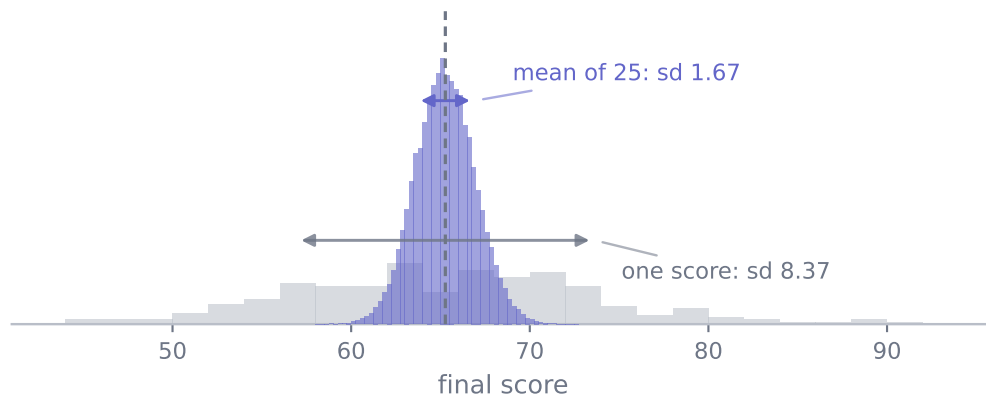


Figure 34.1: The same quantity, measured two ways. Behind, in grey, the 238 individual scores, spread over 8.4 marks. In front, the means of forty thousand samples of 25, spread over 1.67 marks — five times narrower, because five is the square root of 25. Both are centred in the same place.

Two things about that picture are the content of the unit.

The sampling distribution is much narrower. Individual students are spread over 8.4 marks; the means of 25 students are spread over 1.67. Averaging cancels some of the noise, and the cancellation is exactly the variance arithmetic of S5.

It is centred in the same place. The sample mean is *unbiased*: over many samples it neither systematically overshoots nor undershoots. That is a property of the procedure, not a guarantee about any single sample.

34.3 The mechanics

34.3.1 The standard error

For a mean of n independent draws from a population with mean μ and standard deviation σ , S5 gave

$$E[\bar{X}] = \mu, \quad \text{sd}(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

That second quantity has its own name — the **standard error of the mean** — and keeping it separate from σ is the single most important piece of vocabulary in the subject.

Table 34.1: Two numbers that are not the same

	describes	changes with more data
standard deviation σ	how spread out the <i>observations</i> are	no
standard error $\sigma/\sqrt{n}$	how much the <i>mean</i> moves between samples	shrinks

Collect more data and the observations stay exactly as spread out as they were — people do not become more alike because you measured more of them. What improves is your knowledge of where the centre is.

In practice σ is unknown, so the standard error is *estimated* by $s/\sqrt{n}$ with the sample standard deviation of S2 in place of σ . That substitution is not free, and paying for it is what the t distribution in S8 is for.

34.3.2 The central limit theorem

The standard error tells you the *width* of the sampling distribution. The central limit theorem tells you its *shape*.

If $X_1, \dots, X_n$ are independent draws from any distribution with a finite mean μ and a finite variance σ^2 , then as n grows the distribution of $\bar{X}$ approaches a normal distribution with mean μ and standard deviation $\sigma/\sqrt{n}$.

Read what it does and does not say. It is a statement about the distribution of the **mean**, not about the data. Your observations remain exactly as skewed as they always were. And it requires **finite variance** — distributions with tails heavy enough to break that condition exist, and for them the theorem simply does not apply: $\sigma/\sqrt{n}$ has nothing to describe, and no sample size makes the sample mean normal. (The mean itself may still settle down — a finite mean is enough for that — but its distribution never becomes the one the theorem promises.)

Here it is happening. The population below is exponential, which is about as lopsided as a common distribution gets.

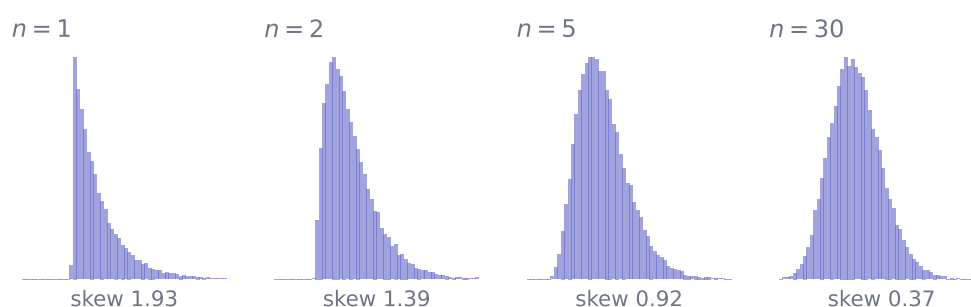


Figure 34.2: Sampling distributions of the mean of n draws from an exponential population, which is strongly right-skewed. Each panel is rescaled to the same width — the real widths shrink as $\sigma/\sqrt{n}$ — so that only the change in *shape* is visible. The lopsidedness falls steadily and by $n = 30$ the distribution is close to symmetric. The population itself, in the first panel, never changed.

34.3.3 “n is big enough” is not a number

Textbooks often say $n = 30$ is enough. That is a rule of thumb calibrated on mildly skewed populations, and it is badly wrong for strongly skewed ones. How large n has to be depends on how far the population is from normal — and the worse the population, the larger the n , with no upper bound.

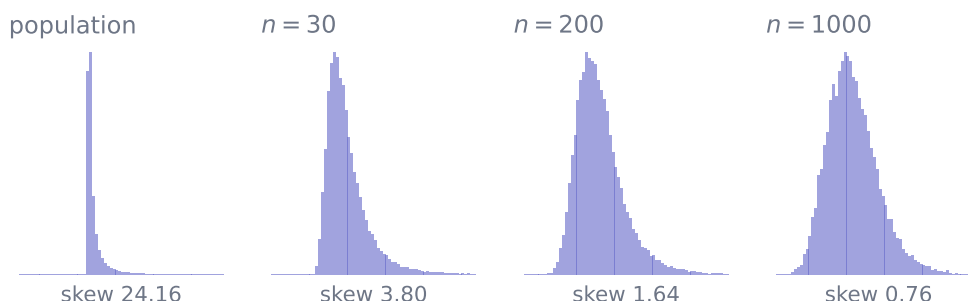


Figure 34.3: The same treatment applied to a much worse population: a lognormal whose skewness is about 24. Panels are again rescaled to a common width, and drawn over the same range as the figure above so the two can be compared directly. The convergence the theorem promises is real and it is slow — at $n = 1000$ the sample mean is still twice as lopsided as the exponential’s was at $n = 30$.

The practical rule that replaces the number: **look at your data’s skewness, and when in doubt use the bootstrap of S8**, which does not assume a shape for the sampling distribution at all.

34.3.4 Two more assumptions worth naming

Independence. Everything here assumes the n draws are independent. If they are not, the correct variance of the mean carries covariance terms:

$$\text{Var}(\bar{X}) = \frac{1}{n^2} \left(\sum_i \text{Var}(X_i) + 2 \sum_{i < j} \text{Cov}(X_i, X_j) \right).$$

The dependence you meet in practice — repeated measures, clustered data, a time series — is nearly always *positive*, so those terms add and the true standard error is larger than $\sigma/\sqrt{n}$, often much larger, and every interval built on it is too narrow. Negative dependence works the other way and makes the mean *more* precise than the formula claims; the next paragraph is an example. This is the assumption most often violated in practice and least often mentioned.

Sampling with replacement, or a small fraction. Drawing without replacement from a finite population makes the draws slightly dependent. If your sample is under about 5% of the population the effect is negligible; if you have sampled half the population, the standard error is genuinely smaller than the formula says, and there is a finite-population correction for it.

34.4 Worked example

Watching the formula come true on the cohort.

Treat the 238 usable scores as the population, and take forty thousand samples at each of three sizes.

```
print(f"population: n = {population.size}    mu = {mu:.4f}    sigma =  
↪ {sigma:.4f}")  
print()  
print(f"{'sample size':>12}{'mean of means':>16}{'sd of means':>14}"  
      f"{'sigma/sqrt(n)':>16}")  
for n in [5, 25, 100]:  
    draws = rng.choice(population, size=(40_000, n))  
    sample_means = draws.mean(axis=1)  
    print(f"{n:>12}{sample_means.mean():>16.4f}{sample_means.std():>14.4f}"  
          f"{sigma / np.sqrt(n):>16.4f}")
```

```
population: n = 238    mu = 65.2718    sigma = 8.3665
```

sample size	mean of means	sd of means	sigma/sqrt(n)
5	65.2486	3.7609	3.7416
25	65.2700	1.6761	1.6733
100	65.2712	0.8422	0.8366

Three things to take from that table.

The formula is exact, not approximate. $\sigma/\sqrt{n}$ is the true standard deviation of the sample mean for any population with finite variance, at any n . It is the *shape* that the central limit theorem makes approximate, never the width.

The means are unbiased at every sample size. Even at $n = 5$, where a single sample is nearly useless, the average of many such means lands on μ . Unbiasedness and precision are separate properties, and a procedure can have the first without the second.

Quadrupling n halves the standard error. From 25 to 100 the spread falls from 1.67 to 0.84. That is the $\sqrt{}$, and it is the arithmetic behind every complaint about how expensive precision is.

Now the shape. The cohort's scores are only mildly skewed, so the mean should become normal quickly:

```
for n in [5, 25, 100]:  
    sample_means = rng.choice(population, size=(40_000, n)).mean(axis=1)  
    centre, spread = sample_means.mean(), sample_means.std()  
    inside = ((sample_means > centre - spread) & (sample_means < centre +  
↪ spread)).mean()  
    print(f"n = {n:>3}    skewness {stats.skew(sample_means):+.3f}    "  
          f"within one sd {100*inside:.1f}%    (normal: 68.3%)")
```

$n = 5$	skewness +0.091	within one sd 68.3%	(normal: 68.3%)
$n = 25$	skewness +0.034	within one sd 68.2%	(normal: 68.3%)
$n = 100$	skewness +0.025	within one sd 68.2%	(normal: 68.3%)

Even at $n = 5$ the sampling distribution is within a percentage point of the normal fractions, because the population it came from was already close to symmetric. The lognormal in the previous figure needed more than a thousand. The theorem is the same in both cases; what differs is how long it takes to arrive.

34.5 Try it yourself

Exercise 1 A population has $\mu = 50$ and $\sigma = 20$.

- What is the standard error of the mean of a sample of 25?
- How large a sample would you need for a standard error of 1?
- A colleague reports that with $n = 400$ their sample standard deviation was 2. Is that plausible?

 Show the solution

- $20/\sqrt{25} = 20/5 = 4$.
- Set $20/\sqrt{n} = 1$, so $\sqrt{n} = 20$ and $n = 400$.
- Almost certainly they have computed the standard error and called it the sample standard deviation.** The sample standard deviation estimates $\sigma = 20$ and does not shrink with n ; the standard *error* at $n = 400$ is exactly $20/20 = 1$, and 2 is much closer to that than to 20.

Strictly, a sample standard deviation of 2 is not impossible. A population whose σ comes almost entirely from very rare enormous values can have $\sigma = 20$ while a typical sample of 400 — containing none of them — looks far tighter. Nothing in the question suggests such a population, and the mix-up is the far likelier explanation, but it is worth knowing that the sample standard deviation is itself an estimate with a distribution.

Exercise 2 Someone measures 40 patients, taking 5 blood-pressure readings from each, and computes a standard error as $s/\sqrt{200}$.

- What has gone wrong?
- Is the reported standard error too large or too small?
- What should they do instead?

 Show the solution

- The 200 readings are **not 200 independent draws**. Five readings from one patient are related to each other; the independent unit is the patient, and there are 40 of them.
- Too small.** The formula credits them with 200 independent pieces of information

when they have closer to 40. How much closer depends on how strongly readings within a patient agree: if they agreed perfectly, 200 readings would carry exactly as much information as 40.

- (c) Reduce each patient to a single number — their mean of five readings — and compute the standard error from those 40 values. That is the simplest correct analysis. (More sophisticated methods use all 200 readings and model the within-patient correlation explicitly, which recovers a little more precision; they never recover the factor of $\sqrt{5}$ the naive calculation claimed.)

Exercise 3 Household income is strongly right-skewed. A survey of 50 households reports a mean income and a symmetric interval around it.

- (a) Which assumption is under strain?
- (b) Which direction would you expect the error to go?
- (c) Name two things you could do about it.

 Show the solution

- (a) Not the standard error — $\sigma/\sqrt{n}$ holds regardless of shape. The strained assumption is the **central limit theorem's approximation at $n = 50$** : with a strongly skewed population the sampling distribution of the mean is still lopsided at that size, so a *symmetric* interval does not match it.
- (b) **Coverage falls below the stated 95%, and the misses are nearly all of one kind.** Simulating a strongly right-skewed population at $n = 50$, the usual t interval contains the true mean about 90% of the time, and in almost every failure the whole interval sits *below* it.

The mechanism is worth following. Most samples from a right-skewed population miss its rare large values, so their mean comes out too low — and because those same large values are what inflate the standard deviation, such a sample also has a small s and therefore a *narrow* interval. Too low and too narrow is exactly the combination that misses. The occasional sample that does catch a large value gets a high mean and a wide interval, and usually still covers.

- (c) Use the **bootstrap** (S8), which builds the sampling distribution empirically and produces an asymmetric interval if that is what the data imply. Or change the estimand: report the **median** income, which is what people usually meant anyway and is far less affected by the tail. Transforming to logs is a third option, but note it changes what you are estimating — the mean of the logs is not the log of the mean.

34.6 Check it in Python

The standard error formula is exact, so it should hold for a population of any shape at all. Check it on three:


```

populations = {
    "cohort scores": population,
    "exponential": rng.exponential(3.0, 200_000),
    "coin flips": rng.integers(0, 2, 200_000).astype(float),
}
n = 40
for name, pop in populations.items():
    sample_means = rng.choice(pop, size=(30_000, n)).mean(axis=1)
    print(f"{name:<15} sigma {pop.std():7.4f}    observed sd of mean "
          f"{sample_means.std():7.4f}    sigma/sqrt(n)
          ↪ {pop.std()/np.sqrt(n):7.4f}")

```

cohort scores	sigma	8.3665	observed sd of mean	1.3231	sigma/sqrt(n)	1.3229
exponential	sigma	2.9754	observed sd of mean	0.4720	sigma/sqrt(n)	0.4705
coin flips	sigma	0.5000	observed sd of mean	0.0796	sigma/sqrt(n)	0.0791

Three completely different populations — one roughly bell-shaped, one strongly skewed, one taking only the values 0 and 1 — and the width formula is right in all three.

The *shape* is a different matter, and this is where the rule of thumb fails. Assert that a sample of 30 makes the mean approximately normal, and watch it fail on a population skewed enough to matter:

```

heavy = rng.lognormal(0.0, 1.6, 300_000)
means_of_30 = rng.choice(heavy, size=(40_000, 30)).mean(axis=1)
skewness = stats.skew(means_of_30)
assert abs(skewness) < 0.5, (
    f"n = 30 was supposed to be enough, but the sample mean still has "
    f"skewness {skewness:.2f} (a normal distribution has 0)"
)

```

```

AssertionError: n = 30 was supposed to be enough, but the sample mean still
↪ has skewness 4.22 (a normal distribution has 0)

```

```

AssertionError                                Traceback (most recent call last)

```

```

Cell In[7], line 4

```

```

     2 means_of_30 = rng.choice(heavy, size=(40_000, 30)).mean(axis=1)
     3 skewness = stats.skew(means_of_30)
----> 4 assert abs(skewness) < 0.5, (
     5     f"n = 30 was supposed to be enough, but the sample mean still
↪ has "
     6     f"skewness {skewness:.2f} (a normal distribution has 0)"
     7 )

```

```

AssertionError: n = 30 was supposed to be enough, but the sample mean still
↪ has skewness 4.22 (a normal distribution has 0)

```

The same check on the cohort's scores passes comfortably at $n = 5$:

```
mild = rng.choice(population, size=(40_000, 5)).mean(axis=1)
print(f"cohort scores, n = 5:  skewness of the sample mean
↪  {stats.skew(mild):+.3f}")
assert abs(stats.skew(mild)) < 0.5
```

```
cohort scores, n = 5:  skewness of the sample mean +0.110
```

The difference is not the sample size. It is the population, and the only way to know which case you are in is to look at your data before choosing a method.

Finally, independence. Here is the same n with the draws deliberately made dependent — each sample is one student repeated:

```
n = 25
independent = rng.choice(population, size=(20_000, n)).mean(axis=1)
repeated = np.repeat(rng.choice(population, size=(20_000, 1)), n,
↪  axis=1).mean(axis=1)
print(f"25 independent students : sd of mean {independent.std():.4f}"
      f"    formula {sigma/np.sqrt(n):.4f}")
print(f"one student, 25 times    : sd of mean {repeated.std():.4f}"
      f"    formula still claims {sigma/np.sqrt(n):.4f}")
```

```
25 independent students : sd of mean 1.6766    formula 1.6733
one student, 25 times   : sd of mean 8.3366    formula still claims 1.6733
```

The second case is an extreme, but the direction is the one you should expect: positively dependent observations make the true standard error larger than the formula, and the formula gives no warning at all.

34.7 Common wrong turns

Where this usually goes wrong

Confusing the standard deviation with the standard error s describes the observations and does not shrink as you collect more. $s/\sqrt{n}$ describes the mean and does. Reporting one where the other was meant is the commonest numerical error in student work, and it is a factor of $\sqrt{n}$.

Thinking the CLT makes your data normal It makes the *sample mean* approximately normal. The observations keep whatever shape they had, and a normal model applied to them is as wrong after the theorem as before it.

Trusting $n = 30$ It is a rule of thumb for mildly skewed populations. For income, insurance claims, or anything with a long tail, thirty is nowhere near enough, and there is no sample size that works for every population.

Using $\sigma/\sqrt{n}$ on dependent observations Repeated measures, clustered samples and time series all break the assumption, and because their dependence is positive they all make the true error larger. Reduce to one number per independent unit, or use a method that models the dependence. (Negative dependence — sampling a large fraction of a finite population without replacement — makes the formula conservative instead.)

Reading unbiasedness as accuracy The sample mean is unbiased at $n = 5$, which sounds reassuring and means only that it is right *on average across many studies*. Your one study can still be a long way out. Unbiased and precise are separate virtues.

Forgetting the theorem needs a finite variance Some distributions have tails so heavy that the variance does not exist. The sample mean may still converge to the population mean, but $\sigma/\sqrt{n}$ does not exist to describe how fast, and the normal approximation never arrives. They are rare in teaching examples and not rare in finance.

34.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A population has $\sigma = 30$. You increase your sample from 100 to 900. What happens to s and to the standard error?
 - a. Both fall by a factor of 3
 - b. s stays about 30; the standard error falls from 3 to 1
 - c. s falls from 3 to 1; the standard error stays about 30
 - d. Both stay the same
2. The cohort's scores are mildly skewed and the sample mean is nearly normal by $n = 5$. Household income is strongly skewed and the sample mean is still skewed at $n = 1000$. Do these two facts contradict the central limit theorem?
3. Is the formula $\text{sd}(\bar{X}) = \sigma/\sqrt{n}$ an approximation?
4. You have measured 60% of a small population. Is your standard error $s/\sqrt{n}$?
5. A study of 2,000 people reports a mean with a very narrow standard error. The 2,000 were recruited through one social-media post. What does the standard error tell you about how close the estimate is to the population mean?

Show the answers

1. **(b).** The sample standard deviation estimates σ , which is a property of the population and does not change: it stays near 30. The standard error is $30/\sqrt{n}$, falling from $30/10 = 3$ to $30/30 = 1$. Option (c) swaps the two, which is exactly the confusion the question is testing.
2. **No.** The theorem is a statement about a limit: the sampling distribution approaches normality as $n \rightarrow \infty$. It says nothing about *how fast*, and the rate depends on the population. Both observations are the theorem behaving exactly as stated.
3. **No.** It is exact for any population with a finite variance and any n , given independent draws. The approximation in this unit is the *shape* — the claim that the sampling distribution is normal — not the width.
4. **No.** Drawing 60% of a finite population without replacement makes the draws dependent, and in this direction it *helps*: the true standard error is smaller than $s/\sqrt{n}$, by a factor of roughly $\sqrt{1 - 0.6} = 0.63$. In the limit where you sample everybody there is no sampling error at all, because there is no sampling left.

5. **Almost nothing.** The standard error measures how much the estimate would bounce around between repeated samples *drawn the same way* — and every one of those samples would be recruited through the same social-media post, with the same people systematically missing. A narrow standard error around a biased estimate is a precise answer to the wrong question, which is S1’s lesson arriving with a number attached.

Where this shows up next

S8 turns the standard error into a confidence interval, and introduces the t distribution that pays for estimating σ . S9 asks how far into the tail of this sampling distribution an observed result falls. S10 builds the sampling distribution of a *difference* between two means. S11 does the same for a fitted slope.

35 Confidence intervals and the bootstrap

Statistics · Unit 8

S8 · Statistics Reading: 5 pages With practice: 2 hours Assumes: S7

Why this unit is here

Any number you calculate from a sample is one draw from a distribution of numbers you did not see. [S7](#) established that. This unit is about saying it out loud.

The difference between reporting “*the mean score was 60.5*” and “*the mean score was 60.5, and given how few people we measured, anything from about 58.6 to 62.5 would be consistent with what we observed*” is the difference between a number and a finding. Almost every result you will produce or read for the rest of your degree comes with an interval attached, and knowing what that interval does and does not claim is most of the skill.

35.1 Before you start

You should be able to:

- explain what a sampling distribution is, and why the standard error is not the standard deviation — [S7](#);
- calculate a sample mean and sample standard deviation — [S2](#);
- say roughly what fraction of a normal distribution lies within two standard deviations of its centre — [S6](#).

Quick check. A sample of 100 measurements has a standard deviation of 20. What is the standard error of the mean?

 Show the answer

$$SE = \frac{s}{\sqrt{n}} = \frac{20}{\sqrt{100}} = 2.$$

If you answered 20, reread [S7](#) first — the distinction it draws is the one this entire unit is built on.

35.2 The idea

Take the cohort, and look only at the students who came from a life-sciences background.

```
import sys; sys.path.insert(0, "../theme")
import pandas as pd
from figures import figure, note, ACCENT, INK, CRITICAL

cohort = pd.read_csv("../data/cohort.csv")
usable = cohort.dropna(subset=["background", "final_score"])
usable = usable[usable["final_score"].between(0, 100)]

life = usable[usable["background"] == "Life sciences"]["final_score"]
print(f"n = {life.size}    mean = {life.mean():.2f}    sd =
↪ {life.std(ddof=1):.2f}")
```

```
n = 57    mean = 60.51    sd = 7.29
```

Their mean score is 60.51. But had 57 *different* life-sciences students walked through the door, the mean would have come out somewhere else — a bit higher, a bit lower. The number 60.51 is not wrong, it is simply one of many that could reasonably have happened.

A **confidence interval** is a range built from the sample, designed so that the *procedure* which builds it captures the true population value a stated fraction of the time.

Read that sentence again, because the emphasis is doing real work. The 95% in “95% confidence interval” is a property of the **method**, not of the particular interval sitting in front of you. Once you have computed (58.58, 62.45), the true mean either is in that range or it is not; nothing about it is 95% any more. What is true is that if you repeated the whole study many times, about 95% of the intervals produced this way would contain the answer.

That distinction sounds like pedantry and is not. It is the single most common misreading in applied statistics, and [S9](#) depends on you having it straight.

35.3 The mechanics

For a mean, when the population standard deviation is unknown — which is essentially always — the interval is

$$\bar{x} \pm t_{1-\alpha/2, n-1} \times \frac{s}{\sqrt{n}}$$

This is exact when the observations are independent draws from a normal population. For other populations it is an approximation, and how good an approximation depends on the sample size and on how skewed or outlier-prone the data are. It is not distribution-free.

Three parts, each doing a job:

- $\bar{x}$ is where the interval is centred: your best single guess.

- $s/\sqrt{n}$ is the standard error — how much $\bar{x}$ would bounce around between repeated samples.
- $t_{1-\alpha/2, n-1}$ is how many standard errors wide to make it. For 95% confidence it is about 2, and it is *slightly more* than 2 for small samples.

That last point is the only new idea here. If you knew the population standard deviation you would use 1.96, from the normal distribution. You do not know it; you estimated it with s , and that estimate is itself uncertain. The t distribution pays for that extra uncertainty by making the interval a little wider, and the smaller the sample the more it charges.

35.3.1 What the interval does not cover

A confidence level is a statement about sampling variation **and nothing else**. It assumes the model, the design and the arithmetic are right. It does not protect you against:

- a sample that was never representative of the population you care about;
- systematic measurement error, which shifts every observation the same way;
- observations that are not independent — repeated measures, clustered data;
- a bug in the code that produced it.

A narrow interval computed from a badly collected sample is a precise answer to the wrong question. Precision and correctness are different properties, and only one of them is visible in the output.

35.3.2 When there is no formula: the bootstrap

The formula above exists because a great deal of theory has been done for means. For a median, a correlation, a ratio of two quantities, or the difference between two percentiles, there may be no convenient formula at all.

The **bootstrap** sidesteps the theory. The reasoning is that your sample is the best picture you have of the population, so:

1. draw n rows from your sample **with replacement** — some rows appear twice, some not at all;
2. recompute the statistic on that resample;
3. repeat several thousand times;
4. take the middle 95% of the resulting values as your interval.

Sampling *with* replacement is essential. Without it you would draw the same n rows every time and see no variation at all.

The bootstrap is remarkably general, and it is not magic. It works badly for very small samples, for statistics near a boundary such as a maximum, and for strongly skewed distributions. It cannot repair bias — if your sample is unrepresentative, every resample of it is unrepresentative in the same way. And for paired or clustered data you must resample **whole units**: for a paired design, resample pairs, never individual measurements.

35.4 Worked example

An interval for the life-sciences mean, two ways.

By hand first. We have $n = 57$, $\bar{x} = 60.51$ and $s = 7.29$, so the standard error is $7.29/\sqrt{57} = 0.97$. For 95% confidence with 56 degrees of freedom the multiplier is 2.003, giving a half-width of $2.003 \times 0.97 = 1.93$ and an interval of 60.51 ± 1.93 .

```
import numpy as np
from scipy import stats

n = life.size
mean = life.mean()
se = life.std(ddof=1) / np.sqrt(n)
t_multiplier = stats.t.ppf(0.975, df=n - 1)

lower, upper = mean - t_multiplier * se, mean + t_multiplier * se
print(f"multiplier t(0.975, {n-1}) = {t_multiplier:.3f}")
print(f"standard error          = {se:.3f}")
print(f"95% interval for the mean = ({lower:.2f}, {upper:.2f})")
```

```
multiplier t(0.975, 56) = 2.003
standard error          = 0.965
95% interval for the mean = (58.58, 62.45)
```

Now the bootstrap, which needs no formula and no t distribution:

```
rng = np.random.default_rng(2026)
values = life.to_numpy()

resamples = rng.choice(values, size=(10_000, n), replace=True)
boot_means = resamples.mean(axis=1)

print(f"bootstrap interval      = ({np.quantile(boot_means, 0.025):.2f}, "
      f"{np.quantile(boot_means, 0.975):.2f})")
```

```
bootstrap interval      = (58.67, 62.46)
```

The two agree to within a few hundredths, which is what you should expect for a mean from a reasonably sized, reasonably symmetric sample. The bootstrap earns its keep on statistics where the formula runs out:

```
boot_medians = np.median(resamples, axis=1)
print(f"sample median          = {np.median(values):.2f}")
print(f"95% interval for the median = ({np.quantile(boot_medians,
↪ 0.025):.2f}, "
      f"{np.quantile(boot_medians, 0.975):.2f})")
```

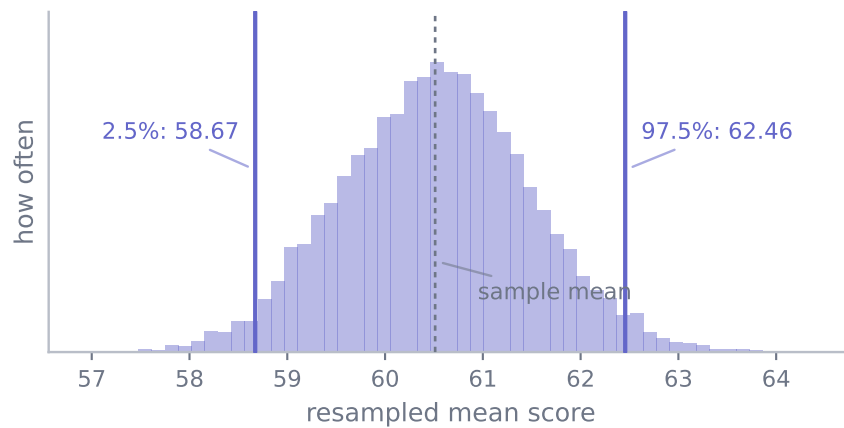



Figure 35.1: Ten thousand resampled means. The bootstrap interval is simply the middle 95% of this pile — the 2.5th and 97.5th percentiles, marked.

```
sample median          = 59.80
95% interval for the median = (57.70, 61.80)
```

Notice the median's interval is wider than the mean's. That is not a defect — the median throws away information about how far the extreme values lie, and it pays for that robustness in precision.

One last thing worth seeing. Compare two groups:

```
for group in ["Life sciences", "Mathematics or Physics"]:
    scores = usable[usable["background"] == group]["final_score"]
    k = scores.size
    err = scores.std(ddof=1) / np.sqrt(k)
    width = stats.t.ppf(0.975, df=k - 1) * err
    print(f"{group:<24} n={k:>3} mean={scores.mean():5.2f} ±{width:.2f}")
```

```
Life sciences          n= 57 mean=60.51 ±1.93
Mathematics or Physics n= 36 mean=75.28 ±2.58
```

The second group has a *wider* interval despite being a perfectly good sample. It has fewer students in it, and the width is driven by n at least as much as by the spread of the data.

35.5 Try it yourself

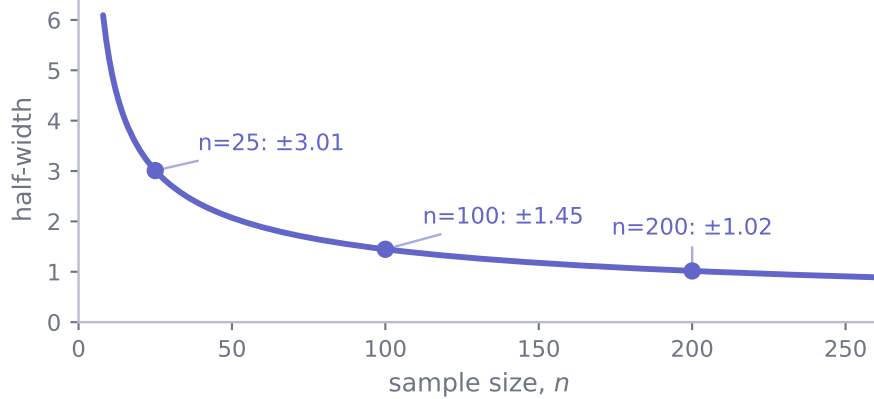
Exercise 1 A sample of 36 measurements has mean 50 and standard deviation 12. The multiplier $t_{0.975, 35}$ is 2.030. Compute the 95% interval, then say what would happen to its width if the sample were four times as large.

💡 Show the solution

SE = $12/\sqrt{36} = 2$, so the half-width is $2.030 \times 2 = 4.06$ and the interval is (45.94, 54.06). Quadrupling n to 144 divides the standard error by $\sqrt{4} = 2$, giving SE = 1. The

```
[Text(0.5, 0, 'sample size, $n$'),
Text(0, 0.5, 'half-width'),
(0.0, 262.0),
(0.0, 6.459629627947625)]
```

- (a) Half-width of the interval against sample size, holding the spread of the data fixed. Precision improves only as $1/\sqrt{n}$: the curve flattens fast, which is why the last decimal place is always the expensive one.



(b)

Figure 35.2

multiplier also shrinks slightly, to about 1.977.

Be careful what follows from this. A *different* sample of 144 would have its own mean and its own standard deviation, so you cannot predict the actual interval. What you can say is that the width would be a little under half. If the mean and standard deviation happened to come out at 50 and 12 again, the interval would be roughly (48.02, 51.98). The lesson to carry: to halve an interval you need **four times** the data. This is why precision gets expensive quickly.

Exercise 2 A colleague reports a 95% confidence interval of (12.1, 12.3) for a mean and says the estimate must be reliable because the interval is so narrow. Give two distinct reasons that conclusion does not follow.

💡 Show the solution

Any two of:

- The interval only accounts for sampling variation. If the sample was drawn from the wrong population, or people who chose not to respond differ from those who did, the interval is narrowly centred on a biased value.
- Systematic measurement error shifts every observation the same way and does not widen the interval at all — a miscalibrated instrument produces a beautifully tight interval around the wrong number.
- If the observations are not independent — several readings per person, say — the true standard error is larger than $s/\sqrt{n}$, so the interval is narrower than it should be and its real coverage is below 95%.
- Narrowness is partly just a large n . It says nothing about whether the measured quantity is the one the question was about.

Exercise 3 You have 40 patients, each measured before and after a treatment, and you want a bootstrap interval for the mean change. What exactly should you resample, and what goes wrong if you resample the 80 individual measurements instead?

💡 Show the solution

Resample the **40 patients**, each carrying its own before-and-after pair, and recompute the mean difference on each resample.

Resampling the 80 measurements independently destroys the pairing. A patient's "before" could be drawn without their "after", or paired with someone else's. That throws away exactly the structure a paired design exists to exploit — that the two readings on one person are related — and gives a miscalibrated interval.

Which way it is wrong depends on the data. When before and after are positively correlated, as they usually are, breaking the pairing re-introduces all the between-patient variation the design was built to remove, and the interval comes out too wide. If they were negatively correlated it could come out too narrow. The safe statement is that the number no longer means what it claims.

The general rule: **resample whatever unit was independently sampled.**

35.6 Check it in Python

The claim that 95% of intervals contain the truth is checkable, and checking it is the fastest way to make the definition stop feeling like a word game.

We treat the whole usable cohort as a population whose mean we happen to know. Then we take a great many small samples, build an interval from each, and count.

```
population = usable["final_score"].to_numpy()
true_mean = population.mean()

rng = np.random.default_rng(2026)
sample_size, trials = 25, 4_000

samples = rng.choice(population, size=(trials, sample_size), replace=True)
means = samples.mean(axis=1)
half_widths = (
    stats.t.ppf(0.975, df=sample_size - 1)
    * samples.std(axis=1, ddof=1) / np.sqrt(sample_size)
)
contains = (means - half_widths <= true_mean) & (true_mean <= means +
↳ half_widths)

print(f"true mean of the population      : {true_mean:.2f}")
print(f"intervals built                   : {trials}")
print(f"intervals containing the truth : {contains.sum()}  ({100 *
↳ contains.mean():.1f}%)")
```

```
true mean of the population      : 65.27
intervals built                   : 4000
```

intervals containing the truth : 3801 (95.0%)

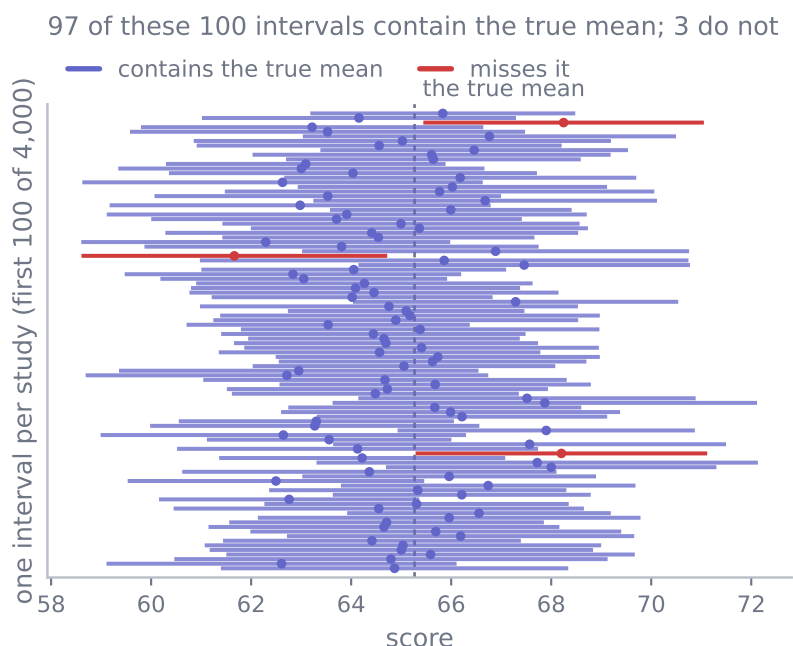


Figure 35.3: The first hundred of those intervals. Each bar is one study; the dotted line is the population mean, which in real life you would never know. The ones that miss are marked — and nothing about them looks different from the inside.

95.0% — close to 95.0% — close to the 95% the procedure promises, which is what you should expect when its assumptions roughly hold, as they do for this fairly symmetric population.

One simulation on one population does not prove the method always works. The t interval is not distribution-free, and on a badly skewed population with a small sample you would see coverage noticeably below 95%. What this shows is that the promise is a real, checkable property rather than a form of words — and that you can check it yourself for any population you are worried about.

The important part is what the failures look like. Around 200 of those intervals missed entirely, and nothing about those 200 distinguishes them from the inside. Each was computed correctly from a legitimate sample; each simply landed somewhere unlucky. **You never know which kind you have**, and that is precisely why the 95% belongs to the procedure rather than to your interval.

Two variations worth running yourself. Change 0.975 to 0.995 and watch coverage rise towards 99% as the intervals widen. Change `sample_size` to 10 and watch the intervals get much wider while the coverage stays near 95% — the method keeps its promise, it just has less to work with.

35.7 Common wrong turns

Where this usually goes wrong

“There is a 95% probability the true mean lies in this interval” The commonest error in the subject. Before you collect data, the procedure has a 95% chance of producing an

interval that covers the truth. After you have the numbers, the interval either covers it or it does not. The 95% describes the method across many hypothetical studies, not this one result.

Confusing the standard deviation with the standard error s describes how spread out the *observations* are. $s/\sqrt{n}$ describes how much the *mean* would move between samples. Adding data leaves the first roughly unchanged and shrinks the second.

Treating a narrow interval as a good analysis Width measures sampling precision only. Bias, bad design, dependence and bugs all leave the width untouched while making the centre wrong.

Expecting the bootstrap to fix a bad sample Every resample is drawn from your data. If the data are unrepresentative, so is all ten thousand of them. The bootstrap quantifies sampling variability; it cannot manufacture information that was never collected.

Resampling the wrong unit For paired, clustered or repeated-measures data, resample the independent unit — the patient, the school, the participant — never the individual rows.

Reporting the interval instead of the estimate An interval without its centre is hard to read, and a centre without its interval is overconfident. Report both, in the units of the thing measured.

35.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A 95% interval for a mean is (4.2, 6.8). Which of these is a correct reading?
 - a. 95% of the observations lie between 4.2 and 6.8
 - b. There is a 95% probability the population mean is between 4.2 and 6.8
 - c. The interval came from a procedure that captures the population mean in 95% of repeated studies
 - d. 95% of future samples will have means between 4.2 and 6.8
2. You quadruple your sample size. Roughly what happens to the width of the interval, and why is it not halved by doubling instead?
3. Give one statistic for which you would reach for the bootstrap rather than a formula, and say why.
4. In the coverage simulation, about 200 intervals missed the true mean. Was anything wrong with how those 200 were calculated?
5. A study measures the same 30 people on five occasions and treats all 150 readings as independent when computing a confidence interval. Is the interval likely to be too wide or too narrow?

 Show the answers

1. **(c).** (a) confuses the spread of the data with the precision of the mean — that would be roughly $\bar{x} \pm 2s$, not $\bar{x} \pm 2s/\sqrt{n}$. (b) is the standard misreading: after observation there is no probability left. (d) describes something else again, and is not what the procedure guarantees.

2. The width scales with $1/\sqrt{n}$, so quadrupling n roughly halves it. Doubling n divides the width by $\sqrt{2} \approx 1.41$, giving about a 29% reduction rather than 50%. Precision is bought at four times the cost you might expect.
3. Any statistic without a convenient closed-form standard error: a median, a correlation, an interquartile range, a ratio of two means, the difference between two 90th percentiles. The bootstrap needs only that you can compute the statistic on a resample.
4. **No.** Every one of them was computed correctly from a legitimate sample. They landed badly by chance, and that is the designed behaviour of a 95% procedure — a method that never missed would be far wider than necessary.
5. **Too narrow**, assuming — as is almost always the case for repeated measures — that readings on the same person are *positively* correlated. Five readings from one person then carry much less information than five readings from five people, so the effective sample size is nearer 30 than 150. Treating them as independent understates the standard error and the real coverage falls below 95%. Strictly the direction depends on the sign of that correlation; negative within-person correlation, which is rare, would push the other way.

Where this shows up next

S9 shows that a two-sided test and a confidence interval are the same calculation viewed from two sides: the test rejects exactly when the interval excludes the null value. **S10** builds intervals for the *difference* between two groups, which is almost always the quantity you actually care about. **S11** attaches an interval to a fitted slope, turning a line through a scatterplot into a claim with uncertainty attached.

36 Hypothesis tests

Statistics · Unit 9

S9 · Statistics Reading: 6 pages With practice: 2 hours Assumes: S8

Why this unit is here

A hypothesis test answers one narrow question: *if nothing were going on, how often would I see data at least as extreme as this?* That is a useful thing to know and it is much less than people take it for.

You need this unit for two reasons. Every paper you read for the rest of your degree reports p -values, and you cannot evaluate a claim you cannot decode. And the standard misreadings — that p is the probability the hypothesis is wrong, that a small p means a large effect, that a large p means no effect — are so widespread that being able to spot them is a genuine professional advantage.

The unit's other job is to show that a test and a confidence interval are the same calculation. Once you have seen that, the interval is almost always the better thing to report.

36.1 Before you start

You should be able to:

- build a confidence interval for a mean and say what the 95% refers to — S8;
- compute a standard error and say why it is not a standard deviation — S7;
- write a conditional probability and keep it the right way round — S4.

Quick check. A 95% confidence interval for a mean comes out as (58.6, 62.4). Is the value 65 a plausible value for the population mean?

 Show the answer

Not on this evidence: 65 lies outside the interval, so it is not among the values the data are consistent with at the 95% level.

You have just performed a hypothesis test. The rest of this unit is that sentence, made precise and given a number.

36.2 The idea

Take the 57 students who came from a life-sciences background. S8 found their mean final score to be 60.51 with a 95% interval of (58.58, 62.45).

Now suppose someone claims the population these students came from has a mean of 65 — the cohort’s overall average. The logic of a test is:

1. **Assume the claim is true.** Call it the **null hypothesis**, $H_0 : \mu = 65$.
2. **Work out how the data would behave if it were.** From S7, sample means from such a population would be spread around 65 with a standard error of about 0.97.
3. **See how far out the observed data actually falls.**
4. **Report how often a sample that extreme or more would occur** if the claim were true. That number is the *p*-value.

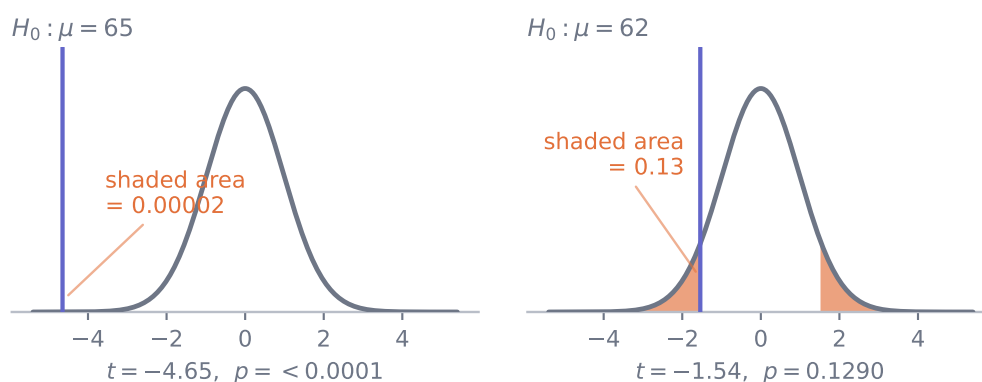


Figure 36.1: The same data against two null hypotheses. In each panel the curve is where the test statistic would fall if the null were true, the vertical line is where it actually fell, and the shaded area is the *p*-value — both tails, because the question was whether the mean differs, not whether it is smaller. Against 65 the observed value is far out in the tail. Against 62 it is unremarkable.

Against $H_0 : \mu = 65$, data this far out would occur about twice in a hundred thousand samples. Against $H_0 : \mu = 62$, it would occur about thirteen times in a hundred. The first claim looks hard to sustain; the second is perfectly consistent with what we saw.

Notice what has and has not been established. We have a statement about **how often data like this arises if the null is true**. We have said nothing about how likely the null is, and nothing at all about the second null being *correct* — only that these 57 students give no reason to doubt it.

36.3 The mechanics

36.3.1 The four pieces

Every test in the subject has the same four parts.

1. A **null hypothesis** H_0 : a specific claim precise enough to compute with. “The mean is 65”, “the two groups have the same mean”, “the slope is zero”.
2. An **alternative** H_1 , which fixes what counts as extreme. Two-sided ($\mu \neq 65$) is the default and the one to use unless you can justify otherwise.

3. A **test statistic**: a number summarising the distance between data and null, scaled by its own uncertainty. For a mean,

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}.$$

4. A **null distribution**: where that statistic would fall if H_0 were true. For the t statistic it is the t distribution with $n - 1$ degrees of freedom, the same one S8 used to set interval widths.

The **p -value** is the probability, *computed under the null*, of a test statistic at least as extreme as the observed one.

36.3.2 What a p -value is, in symbols

$$p = P(\text{data at least this extreme} \mid H_0 \text{ true})$$

Everything that goes wrong with p -values goes wrong by moving the bar. It is **not** $P(H_0 \text{ true} \mid \text{data})$ — that is the reversal of S4, and getting from one to the other would need a prior probability that the test never supplies.

Three consequences worth stating explicitly.

A small p does not mean a large effect. It means the effect is distinguishable from zero given your sample size. With the cohort’s spread of about 7.3 marks and 100,000 observations, a difference of a tenth of a mark gives $t = 4.3$ and $p \approx 10^{-5}$ — a decisive-looking result about a difference nobody could care about.

A large p does not mean no effect. It means your data cannot rule the null out. An underpowered study produces large p -values whether or not anything is going on, which is why “we found no significant difference” is nearly uninformative without the interval next to it.

$1 - p$ is not the probability the alternative is true. It is not a probability about any hypothesis at all.

36.3.3 Two ways to be wrong

Fix a threshold α — conventionally 0.05 — and reject H_0 when $p < \alpha$. Two mistakes are then possible.

Table 36.1: The two errors

	H_0 is true	H_0 is false
reject H_0	Type I error, rate α	correct
do not reject	correct	Type II error, rate β

Power is $1 - \beta$: the probability of detecting a real effect of a given size. It rises with the sample size and with the size of the effect, and falls as the data get noisier.

Note the asymmetry the picture makes obvious. You choose α ; you do **not** choose β , which follows from α , the sample size and the true effect. Shrinking α to be safer moves the line right and makes β worse.

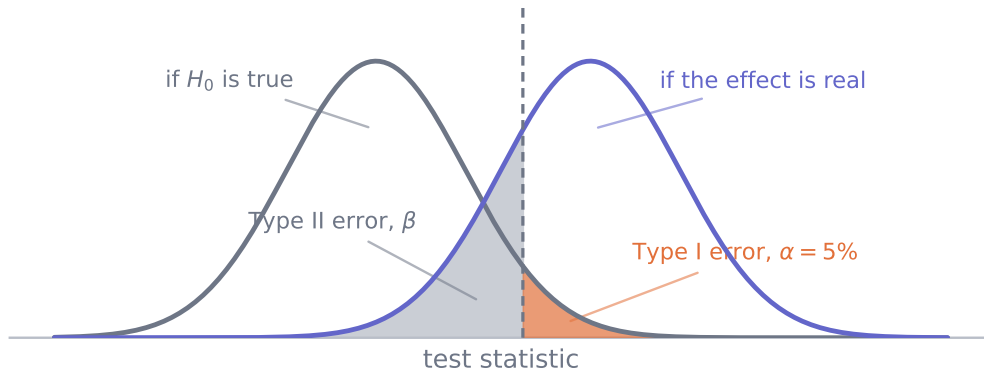


Figure 36.2: Two worlds. If the null is true the statistic is drawn from the left curve; if the real effect is the one on the right, from the right curve. The threshold is fixed by α alone, so the shaded slice of the left curve is 5% by construction. Everything of the right curve that falls short of the threshold is a missed real effect.

36.3.4 A test and an interval are the same calculation

A two-sided test at level α rejects $H_0 : \mu = \mu_0$ **exactly when** μ_0 falls outside the $(1 - \alpha)$ confidence interval. They are not two methods that usually agree; they are one calculation read two ways.

$$\left| \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \right| > t^* \iff \mu_0 \notin \left[\bar{x} - t^* \frac{s}{\sqrt{n}}, \bar{x} + t^* \frac{s}{\sqrt{n}} \right]$$

(The brackets are square because a μ_0 sitting exactly on an endpoint gives p exactly equal to α , and $p < \alpha$ does not reject. It is a boundary case of no practical consequence and of some pedagogical value: the two statements match exactly, not approximately.)

Both sides are the same inequality rearranged. This matters practically: the interval contains everything the test would tell you **and** the effect size and its precision, which the test throws away. Report the interval.

36.4 Worked example

Testing the life-sciences mean, and watching the duality hold.

```
print(f"n = {n}    mean = {mean:.4f}    sd = {life.std(ddof=1):.4f}    se =
↪ {se:.4f}")
t_star = stats.t.ppf(0.975, n - 1)
low, high = mean - t_star * se, mean + t_star * se
print(f"95% interval for the mean: ({low:.4f}, {high:.4f})")
```

```
n = 57    mean = 60.5123    sd = 7.2893    se = 0.9655
95% interval for the mean: (58.5782, 62.4464)
```

Test the claim that this population's mean is 65, by hand and then with the library:

```

null = 65
t_stat = (mean - null) / se
p_value = 2 * stats.t.sf(abs(t_stat), n - 1)
print(f"by hand : t = {t_stat:.4f}    p = {p_value:.3e}")
print(f"scipy    : {stats.ttest_1samp(life, null)}")

```

```

by hand : t = -4.6481    p = 2.081e-05
scipy    : TtestResult(statistic=-4.648132564933887, pvalue=2.080866200706556e-
05, df=56)

```

A p of 2×10^{-5} : if this population really had a mean of 65, a sample of 57 landing this far away would happen about twice in a hundred thousand studies. That is strong evidence against the claim.

The sentence to write down, though, is not the p -value. It is: *the mean final score of life-sciences students is estimated at 60.5 marks (95% CI 58.6 to 62.4), about 4.5 marks below the cohort average of 65.3*. That reports the size of the difference, its uncertainty, and — implicitly — the test.

Now the duality. Test every possible null value from 55 to 68 and see where the p -value crosses 0.05:

```

for candidate in [58.0, low, 59.0, 60.5, 62.0, high, 63.0]:
    t_c = (mean - candidate) / se
    p_c = 2 * stats.t.sf(abs(t_c), n - 1)
    verdict = "reject" if p_c < 0.05 else "do not reject"
    print(f"H0: mu = {candidate:>8.3f}    p = {p_c:.5f}    {verdict}")

```

```

H0: mu =   58.000    p = 0.01183    reject
H0: mu =   58.578    p = 0.05000    do not reject
H0: mu =   59.000    p = 0.12290    do not reject
H0: mu =   60.500    p = 0.98990    do not reject
H0: mu =   62.000    p = 0.12897    do not reject
H0: mu =   62.446    p = 0.05000    do not reject
H0: mu =   63.000    p = 0.01264    reject

```

The two interval endpoints are in that list, and both come out at p exactly 0.05 — the curve crosses the threshold precisely there and nowhere else. Every null inside the interval survives; every null outside it is rejected. One calculation, two presentations.

Finally, power, because it is the thing that decides whether a non-significant result means anything at all.

```

sd = life.std(ddof=1)
effect = 2.0                                     # a difference of 2 marks
print(f"power to detect a {effect:.0f}-mark difference (sd {sd:.2f}, alpha
↪ 0.05):")
for size in [10, 30, 57, 100, 200]:
    critical = stats.t.ppf(0.975, size - 1)

```

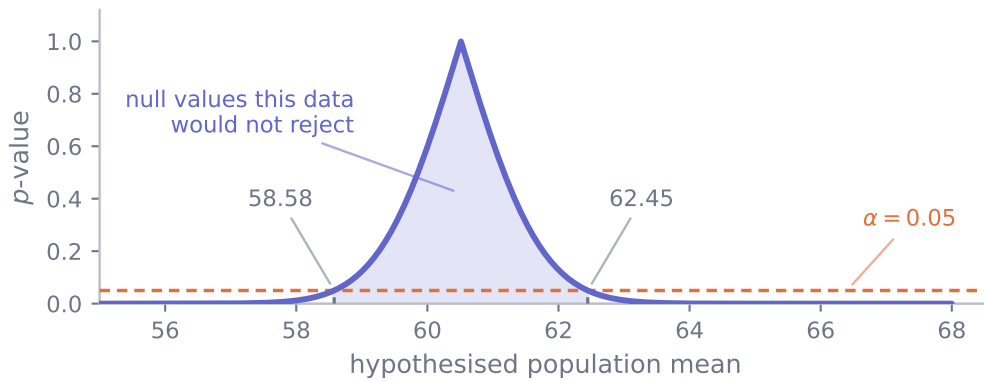


Figure 36.3: The p -value for every hypothesised mean between 55 and 68. It crosses 0.05 in exactly two places, and those two places are the ends of the 95% confidence interval — marked as vertical lines. The interval is the set of null values this data would not reject.

```
ncp = (effect / sd) * np.sqrt(size)
power = stats.nct.sf(critical, size - 1, ncp) + stats.nct.cdf(-critical,
↪ size - 1, ncp)
print(f"    n = {size:>3}    power {power:.3f}")
```

power to detect a 2-mark difference (sd 7.29, alpha 0.05):

```
n = 10    power 0.122
n = 30    power 0.306
n = 57    power 0.530
n = 100   power 0.776
n = 200   power 0.971
```

At $n = 57$ the study has a 53% chance of detecting a real 2-mark difference — barely better than a coin toss, and worse than that for anything smaller. If such a study reports $p = 0.3$, it has told you almost nothing, and saying “there is no difference” would be a serious overstatement.

36.5 Try it yourself

Exercise 1 A sample of 40 has mean 12.4 and standard deviation 3.0. Test $H_0 : \mu = 11$ against a two-sided alternative, using $t^* = 2.023$.

- Compute the standard error and the test statistic.
- Would you reject at $\alpha = 0.05$?
- Give the 95% interval, and check the two answers agree.

💡 Show the solution

- $SE = 3.0/\sqrt{40} = 0.4743$, so $t = (12.4 - 11)/0.4743 = 2.951$.
- $|t| = 2.951 > 2.023$, so **reject** at the 5% level. (The exact p -value is about 0.005.)

- (c) $12.4 \pm 2.023 \times 0.4743 = 12.4 \pm 0.960$, giving (11.44, 13.36). The null value 11 lies outside it, which is the same conclusion — as it must be, since the two calculations are the same inequality.

The interval is the better report: it says the data are consistent with any mean between 11.4 and 13.4, where the test says only “not 11”. (As always, the 95% belongs to the procedure — S8 — not to this particular pair of endpoints.)

Exercise 2 Two studies test the same drug.

- Study A: 40 patients, mean improvement 5.0 points, 95% CI (0.2, 9.8), $p = 0.04$.
- Study B: 4,000 patients, mean improvement 0.3 points, 95% CI (0.1, 0.5), $p = 0.003$.

Which shows the stronger evidence, and which shows the larger effect?

💡 Show the solution

Study B has the smaller p -value and by far the smaller effect. Its 0.3-point improvement is estimated precisely: the whole interval lies between 0.1 and 0.5, so the data have ruled out anything larger. Whether an improvement of that size is worth having is a clinical question, not a statistical one — but the interval is what puts the question in front of you, and the p -value of 0.003 does not. The tiny p reflects the sample size.

Study A shows the larger effect and knows almost nothing about it. The interval runs from 0.2 to 9.8, so the data are consistent with an improvement that is negligible and with one that is large. Its $p = 0.04$ crosses the conventional threshold and tells you very little.

The lesson: **p -values rank evidence against a null, not effects by importance.** Read the interval, in the units of the thing measured, and ask whether the values it contains would change what anyone does.

Exercise 3 A researcher runs a test, gets $p = 0.08$, and writes: “there is a 92% chance the treatment works”. Identify every error in that sentence and write a correct one.

💡 Show the solution

Three errors, stacked.

The conditional is reversed. p is the probability of data this extreme *if the null is true*, not the probability that the null is true given the data. Getting from one to the other requires a prior — see S4.

$1 - p$ is not the probability of the alternative. Even if p were a probability about the null — it is not — its complement would be about “not the null”, which includes effects of every size, most of them uninteresting.

“Works” is not what was tested. The null was almost certainly “no difference”. Rejecting it would be *evidence* of a difference — not proof, since 5% of tests reject a true null — and a difference is not the same thing as a benefit worth having.

A correct sentence, with the numbers the original omitted: “*The estimated effect was X units (95% CI a to b). Data this extreme would arise 8% of the time if the treatment had no effect at all, so these data do not rule out a null effect. What else they fail to rule out can be read off the interval — which is why it, and not the p -value, is the thing to report.*”

36.6 Check it in Python

The claim that a p -value is a probability computed under the null is testable: if the null really is true, p -values should be spread evenly between 0 and 1, and 5% of them should fall below 0.05.

```
rng = np.random.default_rng(2026)
trials, size = 20_000, 30
samples = rng.normal(loc=50, scale=10, size=(trials, size)) # null is
↳ TRUE: mu = 50
t_stats = (samples.mean(axis=1) - 50) / (samples.std(axis=1, ddof=1) /
↳ np.sqrt(size))
p_values = 2 * stats.t.sf(np.abs(t_stats), size - 1)

print(f"p below 0.05 : {(p_values < 0.05).mean():.4f} (should be 0.05)")
print(f"p below 0.10 : {(p_values < 0.10).mean():.4f} (should be 0.10)")
print(f"p below 0.50 : {(p_values < 0.50).mean():.4f} (should be 0.50)")
```

```
p below 0.05 : 0.0494 (should be 0.05)
p below 0.10 : 0.0963 (should be 0.10)
p below 0.50 : 0.4965 (should be 0.50)
```

That is what $\alpha = 0.05$ buys: when nothing is going on, one test in twenty comes out significant anyway. Those thousand “significant” results are not mistakes in anyone’s arithmetic — they are the designed behaviour of the procedure, and they are why S12 worries about how many tests were run.

Now the duality, checked exactly rather than by eye:

```
for candidate in np.linspace(55, 68, 27):
    rejects = 2 * stats.t.sf(abs((mean - candidate) / se), n - 1) < 0.05
    outside = not (low <= candidate <= high)
    assert rejects == outside, f"disagreement at mu0 = {candidate}"
print(f"test and interval agree at all 27 candidate values in [55, 68]")
print(f"interval ({low:.4f}, {high:.4f})")
```

```
test and interval agree at all 27 candidate values in [55, 68]
interval (58.5782, 62.4464)
```

Now generate data where the null is definitely **false** — a real effect of 2.5 marks — and run two thousand studies of the size we actually have:

```
truth, sd_pop = 62.5, 7.29 # the real mean is 2.5 marks above 60
studies = rng.normal(loc=truth, scale=sd_pop, size=(2000, 57))
p_values = stats.ttest_1samp(studies, 60.0, axis=1).pvalue
missed = np.flatnonzero(p_values >= 0.05)
print(f"real effect of 2.5 marks, n = 57: missed by {missed.size} of 2000 "
      f"studies ({100*missed.size/2000:.1f}%)")
```

real effect of 2.5 marks, $n = 57$: missed by 577 of 2000 studies (28.9%)

Nearly three studies in ten find nothing, and every one of them was correctly conducted on data containing a real effect. Take one of those studies and assert what a careless reader would assume — that a real effect shows up:

```
unlucky = studies[missed[0]]
result = stats.ttest_1samp(unlucky, 60.0)
assert result.pvalue < 0.05, (
    f"the null (mu = 60) is false by 2.5 marks, yet this study of 57 gives "
    f"p = {result.pvalue:.3f} and would 'find no significant difference'"
)
```

```
AssertionError: the null (mu = 60) is false by 2.5 marks, yet this study of
↳ 57 gives p = 0.071 and would 'find no significant difference'
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[11], line 3
      1 unlucky = studies[missed[0]]
      2 result = stats.ttest_1samp(unlucky, 60.0)
----> 3 assert result.pvalue < 0.05, (
      4     f"the null (mu = 60) is false by 2.5 marks, yet this study of 57
↳ gives "
      5     f"p = {result.pvalue:.3f} and would 'find no significant
↳ difference'"
      6 )
```

```
AssertionError: the null (mu = 60) is false by 2.5 marks, yet this study of
↳ 57 gives p = 0.071 and would 'find no significant difference'
```

A non-significant result is consistent with no effect *and* with an effect the study was too small to see. The p -value alone cannot tell you which; only the interval and the power can.

36.7 Common wrong turns

Where this usually goes wrong

Reading p as the probability that the null is true p is computed *assuming* the null. It is $P(\text{a statistic at least this extreme} \mid H_0)$, and turning that into $P(H_0 \mid \text{data})$ needs a base rate the test does not have. This is S4’s error in its most expensive form. (Note the “at least this extreme”: for continuous data the probability of the exact values you observed is zero, so a p -value could not be that.)

Reading a non-significant result as “no effect” It means the data could not rule out the null. An underpowered study cannot rule out anything. Write “no significant difference was found ($p = 0.3$; the 95% interval runs from -4 to $+9$, so effects of either sign remain plausible)” — and notice how much more that says.

Confusing statistical significance with importance A large enough sample makes any non-zero difference significant. The interval, in real units, is what tells you whether the difference matters.

Choosing one-sided after seeing the data A one-sided test halves the p -value, which is exactly why it must be decided before looking. Choosing the direction afterwards is not a test at 5%; it is a test at 10% reported as 5%.

Testing many things and reporting the smallest p Twenty independent tests on pure noise produce a significant result about two-thirds of the time. If you ran many tests, say so and adjust — the machinery is in [S12](#).

Treating $\alpha = 0.05$ as a law of nature It is a convention, and a fairly arbitrary one. There is nothing meaningfully different about $p = 0.049$ and $p = 0.051$, and reporting one as a discovery and the other as nothing is a failure of judgement, not a rule.

Reporting the test instead of the estimate The test discards the effect size and its precision, which are the two things a reader needs. Give the estimate with its interval; the test is a corollary.

36.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A study reports $p = 0.03$. Which statement is correct?
 - a. There is a 3% chance the null hypothesis is true
 - b. There is a 97% chance the alternative is true
 - c. If the null were true, data at least this extreme would arise 3% of the time
 - d. The effect is small but real
2. A 95% confidence interval for a difference is $(-1.2, 8.6)$. Without computing anything, what do you know about the two-sided p -value for $H_0 : \text{difference} = 0$?
3. You lower α from 0.05 to 0.01. What happens to the two error rates?
4. In the simulation above, 5% of tests were significant when the null was true. Were those tests computed wrongly?
5. Study A: $p = 0.049$. Study B: $p = 0.051$. How different is the evidence?

Show the answers

1. **(c).** That is the definition. (a) and (b) reverse the conditional; (d) confuses significance with size and cannot be read off a p -value at all — the effect could be enormous or trivial.
2. **It is greater than 0.05.** Zero lies inside the interval, and by the duality a two-sided test rejects exactly when the null value falls outside. You cannot tell *how much* greater without more information — an interval that only just contains zero corresponds to a p a little above 0.05, one that contains it centrally to a p near 1.
3. The **Type I error rate falls to 1%**, by construction — that is what α means. The **Type II error rate rises**, because the threshold has moved further out and more real effects now fall short of it. You cannot reduce both at once without more data.
4. **No.** Every one was computed correctly from data generated under the null. Five

percent false positives is what a 5% threshold *is*. A procedure that never gave a false positive would also never detect anything.

5. **Essentially not at all.** The two studies provide near-identical evidence. The threshold is a convention placed at 0.05, and nothing about the underlying quantity changes as you step across it. Treating one as a finding and the other as a null result is the single most damaging convention in applied statistics, and it is a large part of why [S12](#) exists.

Where this shows up next

[S10](#) applies all of this to the comparison people actually want: two groups, and the difference between them. [S11](#) tests whether a fitted slope differs from zero, which is the same calculation again. [S12](#) asks what happens when many tests are run, and why a significant result in a low-prior field is more likely to be false than true.

37 Comparing groups, measuring association

Statistics · Unit 10

S10 · Statistics Reading: 7 pages With practice: 2 hours Assumes: S9, S3

Why this unit is here

Almost every applied question is a comparison. Did the treated group do better than the untreated one? Do students from one background finish further ahead than students from another? Is the completion rate different for people who used the forum?

The quantity you care about is the **difference**, and the thing to report is that difference with an interval around it. This unit shows how to get one for two means and for two proportions, how to handle a table of counts, and — most importantly — why looking at two separate intervals and checking whether they overlap is not a substitute for computing the interval for the difference.

37.1 Before you start

You should be able to:

- read a p -value correctly and state the duality with a confidence interval — S9;
- build a confidence interval for a mean — S8;
- summarise the relationship between a categorical and a quantitative variable — S3.

Quick check. Two groups have means 40 and 46. Their standard errors are both 2. Roughly what is the standard error of the *difference* between the means?

💡 Show the answer

$$\sqrt{2^2 + 2^2} = \sqrt{8} = 2.83.$$

Not 2, and not 4. Standard errors, like standard deviations, combine through their squares — the variance rule from S5. Getting this wrong is the arithmetic behind the commonest mistake in the unit.

37.2 The idea

Take the two most different backgrounds in the cohort.

```

import sys; sys.path.insert(0, "../theme")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from figures import figure, note, use_style, ACCENT, WARM, INK, FAINT

cohort = pd.read_csv("../data/cohort.csv")
usable = cohort.dropna(subset=["background", "final_score"])
usable = usable[usable["final_score"].between(0, 100)]

life = usable[usable["background"] == "Life
↳ sciences"]["final_score"].to_numpy()
maths = usable[usable["background"] == "Mathematics or
↳ Physics"]["final_score"].to_numpy()
for name, group in [("Life sciences", life), ("Maths or Physics", maths)]:
    se = group.std(ddof=1) / np.sqrt(group.size)
    print(f"{name:<18} n = {group.size:>3}    mean {group.mean():6.2f}    "
          f"sd {group.std(ddof=1):5.2f}    se {se:.2f}")

```

Life sciences	n = 57	mean 60.51	sd 7.29	se 0.97
Maths or Physics	n = 36	mean 75.28	sd 7.63	se 1.27

Two means, 60.51 and 75.28. The comparison people report is often “both means with their intervals, and look, they do not overlap”. That happens to work here, and in general it is the wrong tool.

The difference is a quantity in its own right. It has an estimate, a standard error and an interval, and none of the three can be read reliably off a picture of the two groups separately.

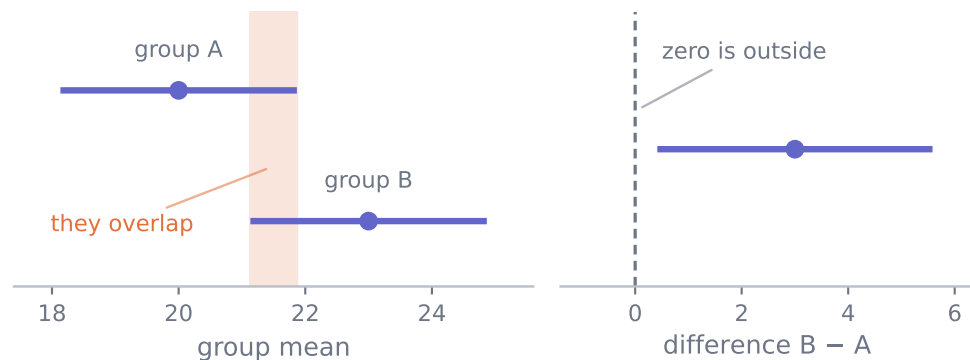


Figure 37.1: A constructed example, because the cohort’s own groups are too far apart to make the point. Two groups of 30 with means 20 and 23 and standard deviations of 5: their 95% intervals overlap, and yet the interval for the *difference* excludes zero and the test rejects at $p = 0.024$. Overlapping intervals do not mean no difference.

Two intervals that overlap can still correspond to a difference significantly away from zero, because the standard error of the difference is $\sqrt{SE_A^2 + SE_B^2}$ — always smaller than the sum $SE_A + SE_B$, which is roughly what the eyeball comparison uses.

The error runs one way, and it is worth knowing which. Non-overlapping intervals do imply a significant difference, so the eyeball test never invents a difference — it misses real ones. In the example above it only declares a difference when $|t| > 2.89$, which is a test at about the 0.5% level dressed up as one at 5%. Being conservative sounds safe until you remember that missing real differences is exactly what an underpowered analysis does.

37.3 The mechanics

37.3.1 Two means, unpaired

The estimate is $\bar{x}_A - \bar{x}_B$. Its standard error is

$$SE = \sqrt{\frac{s_A^2}{n_A} + \frac{s_B^2}{n_B}},$$

and the interval is the estimate plus or minus $t^* SE$. The test statistic is the estimate divided by its standard error, exactly as in S9.

There are two versions of this test. The **pooled** (“Student”) version assumes the two populations have the same variance and combines them into one estimate. **Welch’s** version does not, and adjusts the degrees of freedom instead. Welch is the better default: it costs almost nothing when the variances really are equal and it is substantially better when they are not, and you never actually know which case you are in. `scipy.stats.ttest_ind` uses the pooled version unless you pass `equal_var=False`, which you usually should.

37.3.2 Two means, paired

If each observation in one group is matched to one in the other — the same person before and after, the same field split into two halves — then **do not use the two-sample test**. Take the differences within pairs and analyse those single numbers with a one-sample test.

This is not a technicality; it is usually a large gain. Pairing removes the between-subject variation, which is often the largest source of noise, so the paired analysis can be dramatically more precise. Treating paired data as unpaired throws that away — and, as S8 showed for the bootstrap, the rule is always to respect the unit that was independently sampled.

37.3.3 Two proportions

For a yes/no outcome, the estimate is $\hat{p}_A - \hat{p}_B$ and

$$SE = \sqrt{\frac{\hat{p}_A(1 - \hat{p}_A)}{n_A} + \frac{\hat{p}_B(1 - \hat{p}_B)}{n_B}},$$

using the Bernoulli variance $p(1 - p)$ from S6. The normal approximation behind that interval needs a reasonable number of successes *and* failures in each group — a common rule of thumb is at least ten of each — and it fails badly for rare events, where exact or score-based intervals should be used instead.

Report the difference in percentage points, and consider also reporting the ratio; they answer different questions and a change from 1% to 2% is either “one percentage point” or “double”, depending which you choose.

37.3.4 A table of counts: the chi-square test

With two categorical variables you have the contingency table of S3. The **chi-square test of independence** asks whether the table is compatible with the two variables being unrelated.

If they were independent, S4 says the expected count in a cell would be

$$E_{ij} = \frac{(\text{row total}_i)(\text{column total}_j)}{n},$$

and the statistic accumulates the squared departures:

$$\chi^2 = \sum_{\text{cells}} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}.$$

Large values mean the observed table is far from what independence predicts. Two warnings. The approximation needs the expected counts to be reasonably large — at least 5 in most cells is the usual rule — and the test tells you only *that* the table departs from independence, never how or by how much. Always look at the cell-by-cell departures as well.

37.3.5 When the assumptions are uncomfortable

The *t*-based methods lean on the sampling distribution of the mean being roughly normal, which S7 showed can fail for small, skewed samples. Two alternatives:

- the **bootstrap** of S8, applied to the difference — it assumes no shape at all and gives you an interval directly;
- a **rank-based test** such as Mann–Whitney, which asks about the probability that a randomly chosen value from one group exceeds one from the other. Note that it answers a different question: it is not a test about means, and it is not a general test of whether the two distributions differ — two distributions with the same centre and different spreads give it nothing to find.

37.4 Worked example

How far ahead do the Maths and Physics students finish?

```
difference = maths.mean() - life.mean()
se = np.sqrt(maths.var(ddof=1) / maths.size + life.var(ddof=1) / life.size)
welch = stats.ttest_ind(maths, life, equal_var=False)
t_star = stats.t.ppf(0.975, welch.df)

print(f"difference in means : {difference:.4f} marks")
print(f"standard error       : {se:.4f}")
```

```
print(f"95% interval      : ({difference - t_star*se:.4f}, "
      f"{difference + t_star*se:.4f})")
print(f"Welch t = {welch.statistic:.4f}  df = {welch.df:.2f}  p =
      ↪ {welch.pvalue:.3g}")
```

```
difference in means : 14.7655 marks
standard error      : 1.5969
95% interval       : (11.5821, 17.9489)
Welch t = 9.2461  df = 72.01  p = 7.28e-14
```

The sentence to report: **students arriving from Mathematics or Physics finished 14.8 marks ahead of students from life sciences (95% CI 11.6 to 17.9)**. That is an estimate, in the units of the thing measured, with its uncertainty attached. The p -value adds only that a gap this large would essentially never arise from 93 students if the two populations had the same mean — which the interval, sitting a long way from zero, had already made obvious.

Both versions of the test, for comparison, and a rank-based one:

```
print(f"Welch (unequal variances) p = {welch.pvalue:.3e}")
print(f"pooled (equal variances)  p = {stats.ttest_ind(maths,
      ↪ life).pvalue:.3e}")
print(f"Mann-Whitney              p = {stats.mannwhitneyu(maths,
      ↪ life).pvalue:.3e}")
```

```
Welch (unequal variances) p = 7.277e-14
pooled (equal variances)  p = 6.028e-15
Mann-Whitney             p = 7.992e-12
```

Here the group standard deviations are 7.63 and 7.29, so pooling changes almost nothing. The habit worth forming is using Welch anyway, because the case where it matters is exactly the case where you would not have noticed.

Now all five groups at once, which is what a reader actually wants to see:

A categorical outcome. Does the chance of scoring 70 or more depend on background at all?

```
table = pd.crosstab(usable["background"], usable["final_score"] >= 70)
table.columns = ["below 70", "70 or more"]
print(table.to_string())

result = stats.chi2_contingency(table)
print(f"\nchi-square = {result.statistic:.3f}  df = {result.dof}    "
      f"p = {result.pvalue:.3g}")
print(f"smallest expected count = {result.expected_freq.min():.2f}")
```

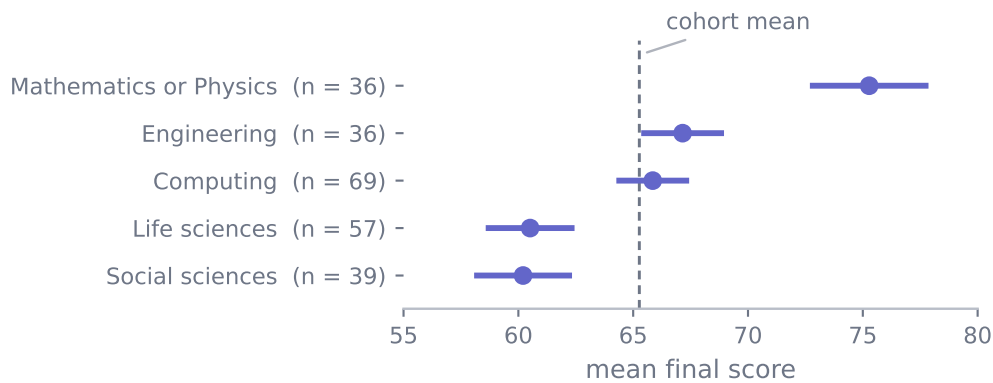


Figure 37.2: Every background's mean final score with a 95% interval, ordered. Reading pairwise comparisons off this picture is exactly what the first figure warned against — but as a description of where the five groups sit, and of how much less is known about the smaller ones, it is the right display.

background	below 70	70 or more
Computing	49	20
Engineering	22	14
Life sciences	50	7
Mathematics or Physics	12	24
Social sciences	35	4

chi-square = 40.813 df = 4 p = 2.94e-08
smallest expected count = 10.48

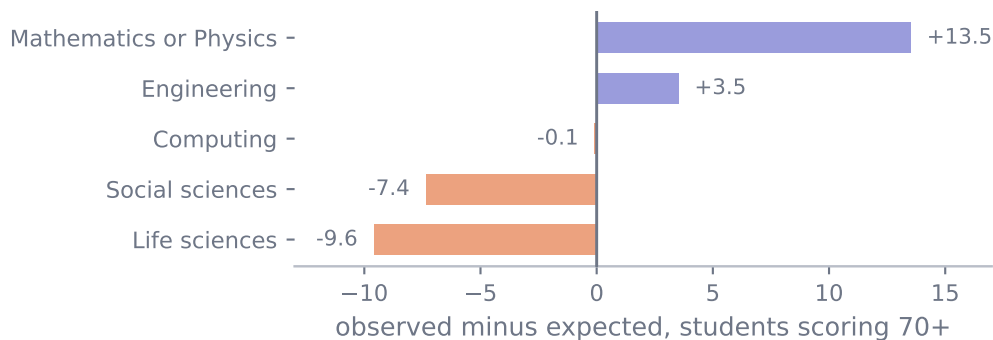


Figure 37.3: Observed counts minus the counts independence predicts, for the '70 or more' column. Two groups have far more high scorers than independence would give them and two have far fewer — which is what the chi-square statistic adds up, and what its single number then hides.

The statistic is 40.8 on 4 degrees of freedom, $p \approx 3 \times 10^{-8}$: the table is a long way from independence. But the statistic alone would not have told you *how*. The departures do: Mathematics or Physics contributes 13.5 more high scorers than independence predicts, and life sciences 9.6 fewer.

37.5 Try it yourself

Exercise 1 Group A: $n = 50$, mean 12.0, sd 4.0. Group B: $n = 50$, mean 14.0, sd 5.0.

- (a) Compute the standard error of the difference.
- (b) Give an approximate 95% interval for the difference (use $t^* \approx 2$).
- (c) Would you reject H_0 : no difference, at the 5% level?

💡 Show the solution

- (a) $SE = \sqrt{4^2/50 + 5^2/50} = \sqrt{0.32 + 0.50} = \sqrt{0.82} = 0.906$.
- (b) Take the difference as B minus A, so +2.0 — the sign is yours to choose and yours to state. $2.0 \pm 2 \times 0.906 = 2.0 \pm 1.81$, so (0.19, 3.81).
- (c) **Yes** — zero is outside the interval, so the test rejects. But notice how close it is: the interval reaches almost to zero, and the data are consistent with a difference as small as 0.2 units. If a difference of 0.2 would not matter in practice, “significant” is a very weak thing to have shown, and the interval is what makes that visible.

Exercise 2 A trial measures blood pressure in 40 patients before and after a treatment. Someone runs a two-sample t -test comparing the 40 “before” values with the 40 “after” values.

- (a) What is wrong?
- (b) What should they do?
- (c) Which analysis is likely to give the narrower interval, and why?

💡 Show the solution

- (a) The two samples are **not independent** — they are the same 40 people measured twice. The two-sample formula assumes independence between the groups, and here each “after” value is strongly related to its own “before” value.
- (b) Compute the 40 within-patient differences and analyse those with a one-sample test or interval. One number per patient, 40 independent units.
- (c) **The paired analysis, usually by a wide margin.** The two-sample standard error is driven by how much patients differ from each other, which for blood pressure is a great deal. The paired standard error is driven only by how much each patient *changed*, which is typically much smaller. Pairing removes the between-patient variation instead of measuring it.

The exception is worth knowing: if before and after were *negatively* correlated, pairing would make things worse. That is rare, and the safe rule remains to analyse the design you actually ran.

Exercise 3 Two teaching methods are compared. Method A: 30 of 100 students passed. Method B: 45 of 100 passed.

- (a) Compute the difference in proportions and its standard error.
- (b) Give an approximate 95% interval.

(c) A colleague reports the result as “method B is 50% better”. Is that fair?

💡 Show the solution

- (a) $\hat{p}_A = 0.30$, $\hat{p}_B = 0.45$. Taking the difference as B minus A — again, say which — it is +0.15.

$$SE = \sqrt{\frac{0.3 \times 0.7}{100} + \frac{0.45 \times 0.55}{100}} = \sqrt{0.0021 + 0.002475} = 0.0676.$$

- (b) $0.15 \pm 1.96 \times 0.0676 = 0.15 \pm 0.133$, giving (0.017, 0.283) — 1.7 to 28.3 percentage points. Zero is outside, but only just, and the data are consistent with almost anything from a negligible improvement to a very large one.
- (c) It is **arithmetically true and rhetorically misleading**. $0.45/0.30 = 1.5$, so the pass rate is indeed 50% higher in relative terms. But the same difference stated as “15 percentage points” sounds quite different, and the relative version conceals how uncertain it is: the interval on the *ratio* runs from about 1.05 to 2.1. Report the difference in percentage points, give the interval, and give the ratio too if it helps — never the ratio alone.

37.6 Check it in Python

First, that the standard error of a difference really does combine through squares. Simulate two independent groups and compare.

```
rng = np.random.default_rng(2026)
n_a, n_b, sd_a, sd_b = 40, 60, 8.0, 5.0
a = rng.normal(50, sd_a, size=(50_000, n_a)).mean(axis=1)
b = rng.normal(55, sd_b, size=(50_000, n_b)).mean(axis=1)

print(f"se of mean A      {a.std():.4f}    formula {sd_a/np.sqrt(n_a):.4f}")
print(f"se of mean B      {b.std():.4f}    formula {sd_b/np.sqrt(n_b):.4f}")
print(f"se of difference {(b - a).std():.4f}    "
      f"formula {np.sqrt(sd_a**2/n_a + sd_b**2/n_b):.4f}")
print(f"      NOT the sum of the two: {sd_a/np.sqrt(n_a) +
  ↳ sd_b/np.sqrt(n_b):.4f}")
```

```
se of mean A      1.2635    formula 1.2649
se of mean B      0.6464    formula 0.6455
se of difference 1.4181    formula 1.4201
      NOT the sum of the two: 1.9104
```

Now the check that fails, and it is the mistake the unit opened with. Assert that two overlapping 95% intervals mean the difference is not significant:

```

def make_group(centre, spread, size):
    """A sample of `size` values with exactly this mean and standard
    ↪ deviation."""
    z = rng.normal(0, 1, size)
    return centre + spread * (z - z.mean()) / z.std(ddof=1)

group_a, group_b = make_group(20.0, 5.0, 30), make_group(23.0, 5.0, 30)

def own_interval(values):
    half = (stats.t.ppf(0.975, values.size - 1)
            * values.std(ddof=1) / np.sqrt(values.size))
    return values.mean() - half, values.mean() + half

lo_a, hi_a = own_interval(group_a)
lo_b, hi_b = own_interval(group_b)
overlap = lo_b <= hi_a
comparison = stats.ttest_ind(group_b, group_a, equal_var=False)

assert not (overlap and comparison.pvalue < 0.05), (
    f"intervals ({lo_a:.2f}, {hi_a:.2f}) and ({lo_b:.2f}, {hi_b:.2f})
    ↪ overlap, "
    f"yet the difference has p = {comparison.pvalue:.3f}"
)

```

AssertionError: intervals (18.13, 21.87) and (21.13, 24.87) overlap, yet the
 ↪ difference has p = 0.024

 AssertionError Traceback (most recent call last)

```

Cell In[9], line 18
    15 overlap = lo_b <= hi_a
    16 comparison = stats.ttest_ind(group_b, group_a, equal_var=False)
--> 18 assert not (overlap and comparison.pvalue < 0.05), (
    19     f"intervals ({lo_a:.2f}, {hi_a:.2f}) and ({lo_b:.2f},
    ↪ {hi_b:.2f}) overlap, "
    20     f"yet the difference has p = {comparison.pvalue:.3f}"
    21 )

```

AssertionError: intervals (18.13, 21.87) and (21.13, 24.87) overlap, yet the
 ↪ difference has p = 0.024

The two intervals share the range from 21.13 to 21.87, and the difference is significant at the 5% level anyway. The eyeball test is a *different, stricter* comparison: here it would have required the two means to differ by 3.73 rather than the 2.58 the proper test asks for. It agrees with the real answer much of the time and errs by staying silent, which is the failure mode that is easiest to miss.

Finally, the chi-square statistic, computed by hand against the library, so the formula stops being a black box:

```

observed = table.to_numpy()
row_totals = observed.sum(axis=1, keepdims=True)
col_totals = observed.sum(axis=0, keepdims=True)

```

```

expected_by_hand = row_totals * col_totals / observed.sum()
chi_by_hand = ((observed - expected_by_hand) ** 2 / expected_by_hand).sum()

print(f"by hand {chi_by_hand:.6f}    scipy {result.statistic:.6f}    "
      f"agree: {np.isclose(chi_by_hand, result.statistic)}")
print(f"df = (rows - 1)(cols - 1) = "
      f"{(observed.shape[0]-1)*(observed.shape[1]-1)}    scipy says "
      f"↪ {result.dof}")

```

```

by hand 40.812864    scipy 40.812864    agree: True
df = (rows - 1)(cols - 1) = 4    scipy says 4

```

37.7 Common wrong turns

Where this usually goes wrong

Comparing two intervals by eye instead of computing the difference Overlapping intervals can hide a significant difference and non-overlapping ones can exaggerate. The difference has its own standard error, $\sqrt{SE_A^2 + SE_B^2}$, and its own interval. Compute it.

Adding standard errors instead of adding their squares $SE_A + SE_B$ is always too big. Square, add, take the root — the variance rule of S5.

Using a two-sample test on paired data Before-and-after on the same people, matched pairs, two measurements per site. Take the within-pair differences and analyse those. Ignoring the pairing throws away the design’s whole advantage.

Reporting only the p -value for a comparison “The groups differed significantly ($p < 0.001$)” tells the reader nothing about *how much*. Give the difference in real units with its interval; that is the finding.

Treating a significant chi-square as an explanation It says the table departs from independence and nothing more. Look at the cell-by-cell departures, and be careful with small expected counts, where the approximation is unreliable.

Confusing a rank test with a test about means Mann–Whitney compares distributions, not means. Two distributions with the same mean can differ by it, and two with different means need not. Report what you actually tested.

Comparing many groups and reporting the biggest gap Five groups make ten pairwise comparisons, and with ten tests at 5% a “significant” result is likely from noise alone. Say how many comparisons you made — S12 says what to do about it.

37.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

- Two group means have standard errors 3 and 4. What is the standard error of their difference?
 - 1
 - 5
 - 7
 - 25
- Two 95% intervals for group means overlap slightly. What can you conclude about the difference?

3. When would you prefer Welch's test to the pooled two-sample test?
4. A chi-square test on a 4×3 table gives $p = 0.001$. What have you learned?
5. A study reports that a treatment "halves the risk". The risk went from 0.02% to 0.01%. Is the reporting fair?

 Show the answers

1. **(b), 5.** $\sqrt{3^2 + 4^2} = \sqrt{25} = 5$. Option (c) adds the standard errors, which is what you would get only if the two estimates were perfectly *negatively* correlated — the covariance term $-2\rho SE_A SE_B$ is largest when $\rho = -1$; (d) is the variance, not the standard error.
2. **Almost nothing on its own.** Overlap is compatible with a difference that is significant and with one that is not; the answer depends on the two sample sizes and standard deviations. Compute the interval for the difference — it takes one line and it is the quantity of interest anyway.
3. **Essentially always.** Welch does not assume equal variances, costs almost nothing when they happen to be equal, and is much safer when they are not — particularly when the group sizes also differ, which is when the pooled test goes wrong most badly. The one thing you should not do is test for equal variances first and choose on the result, which distorts the error rate of everything downstream.
4. That the table is **inconsistent with the two variables being independent**. You have not learned which cells drive it, in which direction, or how large the departure is in any practical sense — and with a large enough table and sample, a trivial departure produces a small p . Look at observed minus expected next.
5. **Not on its own.** The relative statement is true and the absolute change is one in ten thousand. Both should be given: "the risk fell from 0.02% to 0.01%, a halving, or one case avoided per 10,000 people treated." A relative risk with no baseline is the standard way to make a negligible effect sound dramatic.

Where this shows up next

S11 replaces the two-group comparison with a line, which lets the explanatory variable be a number rather than a label — and lets you adjust for a second variable at the same time. **S12** returns to the ten pairwise comparisons this unit resisted making, and to what happens when a difference between groups is really a difference in who is in them.

38 Simple linear regression

Statistics · Unit 11

S11 · Statistics Reading: 6 pages With practice: 2 hours Assumes: S10, M11

Why this unit is here

S10 compared groups. Regression does the same job when the explanatory variable is a *number* rather than a label — and it gives you the answer in the units people actually want: not “these groups differ”, but “**each extra hour is worth this many marks, give or take this much**”.

M11 already showed how the line is computed. This unit supplies the part that makes it a statistical statement rather than an arithmetic one: the slope is an estimate, it has a standard error, and there is a specific list of assumptions under which that standard error means what it says. It also supplies the two numbers people most often misuse — R^2 and the prediction interval.

38.1 Before you start

You should be able to:

- fit a straight line by least squares and say what it minimises — **M11**;
- build a confidence interval and read a p -value — **S9**, **S8**;
- interpret a correlation and say what it cannot see — **S3**.

Quick check. A fitted line is $\hat{y} = 70 - 0.24x$, where y is a final score in marks and x is hours of practice. What are the units of -0.24 , and what does the 70 mean?

 Show the answer

The slope is in **marks per hour**: the fitted line falls by 0.24 marks for each extra hour. The intercept is the fitted value at $x = 0$ — 70 marks for a student who practised not at all.

Whether that intercept means anything depends on whether $x = 0$ is inside the range of the data. Here the smallest observed practice time is one hour, so 70 is an extrapolation, and one to be careful with.

38.2 The idea

The model behind a fitted line is

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i,$$

where β_0 and β_1 are fixed unknown numbers — parameters, in the sense of S1 — and ε_i is everything else: measurement noise, omitted causes, individual variation. The ε_i are assumed to have mean zero, the same variance for every i , and to be independent of each other.

Least squares gives you $\hat{\beta}_0$ and $\hat{\beta}_1$, and the crucial point of this unit is that those hats are doing the same work they did in S1: **they are estimates, computed from one sample, and a different sample would have produced different ones.**

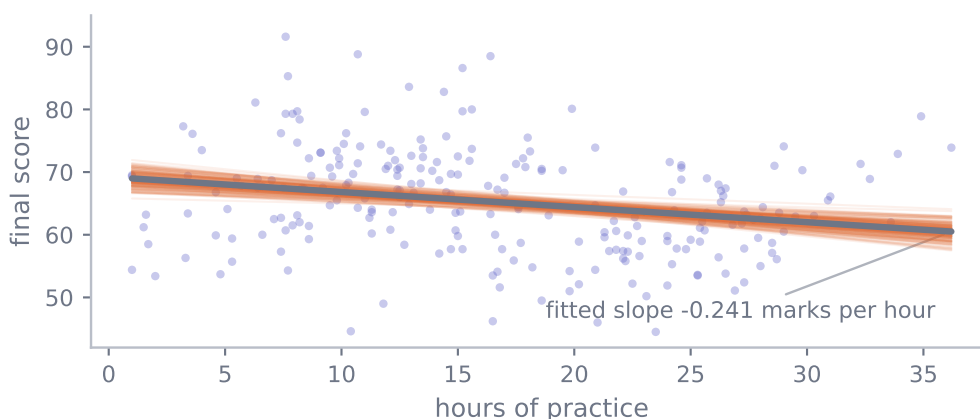


Figure 38.1: Two hundred lines, each fitted to a resample of the 238 students, drawn faintly behind the fit to the real data. The fan they trace out is what the standard error of the slope measures. A single line drawn on a scatterplot hides this completely, which is why the slope must be reported with an interval.

Every one of those faint lines is a slope somebody could have reported, from data generated the same way. Some are noticeably steeper than the fit and some are noticeably flatter. Quantifying that fan is what the rest of the unit does.

38.3 The mechanics

38.3.1 The estimates

From M11, least squares gives

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = r \frac{s_y}{s_x}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

The second form for the slope is worth carrying: it is the correlation of S3, converted from unitless into marks-per-hour by the ratio of the two standard deviations. Correlation and slope carry the same sign and answer different questions.

38.3.2 The uncertainty

Let s_e be the **residual standard deviation**, the typical size of a vertical miss:

$$s_e = \sqrt{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

The divisor is $n - 2$ because two parameters have been estimated from the same data — the same argument that gave S2 its $n - 1$. Then

$$\text{SE}(\hat{\beta}_1) = \frac{s_e}{\sqrt{\sum (x_i - \bar{x})^2}}.$$

Read what makes a slope precise. **Small residuals** help, obviously. So does a **large** n . And so does a **wide spread of** x — the denominator is the total spread of the explanatory variable, so measuring people whose practice hours vary a lot pins the slope down far better than measuring the same number of people who all practised about the same amount. That is a design lesson, and it is invisible unless you look at the formula.

The interval and test then follow the pattern of S9 exactly:

$$\hat{\beta}_1 \pm t_{n-2}^* \text{SE}(\hat{\beta}_1), \quad t = \frac{\hat{\beta}_1 - 0}{\text{SE}(\hat{\beta}_1)}.$$

38.3.3 R^2 , and what it is not

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

— the proportion of the variance in y that the fitted line accounts for. In simple regression it is exactly the square of the correlation.

Three things it does **not** tell you. It does not say the model is correct: a curved relationship can produce a respectable R^2 while the line is wrong everywhere. It does not say the slope is useful: a slope can be precisely estimated, obviously real, and account for very little of the variation, which is the normal situation with human data. And it does not say anything about cause.

A low R^2 with a tight interval on the slope means “this variable matters a little, and we know that quite precisely” — a perfectly respectable finding, and the one this unit’s worked example produces.

38.3.4 Two kinds of interval

Asking “what is the fitted value at x_0 ?” has two different answers, and confusing them is the commonest error in applied regression.

- A **confidence interval for the mean response** says where the *average* y for everyone at x_0 lies. It shrinks as n grows.
- A **prediction interval for a new observation** says where *one new individual* at x_0 is likely to fall. It carries the residual scatter as well, so it stays wide however much data you collect.

$$SE_{\text{mean}} = s_e \sqrt{\frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum (x_i - \bar{x})^2}}, \quad SE_{\text{pred}} = s_e \sqrt{1 + \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum (x_i - \bar{x})^2}}$$

The extra 1 under the second root is the individual’s own noise, and it is usually the dominant term.

38.3.5 The assumptions, and how to check them

Table 38.1: What to check, and where to look

assumption	how it fails	how to look
the relationship is linear	curvature	residuals against fitted values
constant variance	a fan shape	residuals against fitted values
errors independent	clustering, time trends	know the design; plot in time order
errors roughly normal	skew, heavy tails	Q-Q plot of residuals

Only the last is about normality, and it is the least important of the four: the intervals rely on the sampling distribution of $\hat{\beta}_1$, which S7’s central limit theorem makes approximately normal for moderate n even when the errors are not. Independence is the one that will hurt you most and shows up in no plot.

38.4 Worked example

How much is an hour of practice worth?

```
print(f"n = {n}")
print(f"slope      {fit.slope:+.4f} marks per hour    (se {fit.stderr:.4f})")
print(f"intercept {fit.intercept:+.4f} marks          (se
↪ {fit.intercept_stderr:.4f})")
t_star = stats.t.ppf(0.975, n - 2)
print(f"95% CI for the slope: ({fit.slope - t_star*fit.stderr:+.4f}, "
      f"{fit.slope + t_star*fit.stderr:+.4f})")
print(f"t = {fit.slope/fit.stderr:.4f}    p = {fit.pvalue:.3g}    R^2 =
↪ {fit.rvalue**2:.4f}")
```



```

n = 238
slope      -0.2408 marks per hour    (se 0.0652)
intercept +69.2261 marks              (se 1.1948)
95% CI for the slope: (-0.3693, -0.1123)
t = -3.6921   p = 0.000276   R^2 = 0.0546

```

The reportable sentence: **across the cohort, each additional hour of practice goes with 0.24 marks *fewer* on the final score (95% CI 0.11 to 0.37 marks lower), and practice hours account for 5.5% of the variation in scores.**

Three separate things are true of that sentence and worth separating.

The slope is negative and the interval excludes zero. This is not noise.

The effect is small and the fit is poor. $R^2 = 0.055$ means practice hours explain about a twentieth of the variation. Read together with the interval, the finding is “a real but modest association, estimated reasonably precisely” — which is what most findings about people look like.

It is not a statement about what practice does. S1 established that the sign reverses inside every background group. The fit above is a correct summary of the pooled cloud and a bad guide to any student’s decision.

```

fitted = fit.intercept + fit.slope * hours
residuals = score - fitted
s_e = np.sqrt((residuals**2).sum() / (n - 2))
sxx = ((hours - hours.mean())**2).sum()

print(f"residual sd s_e    {s_e:.4f}    (sd of the scores themselves
↳ {score.std(ddof=1):.4f})")
print(f"se by hand        {s_e/np.sqrt(sxx):.6f}")
print(f"se from scipy     {fit.stderr:.6f}")
print(f"R^2 by hand       {1 - (residuals**2).sum()/((score -
↳ score.mean())**2).sum():.6f}")

```

```

residual sd s_e    8.1693    (sd of the scores themselves 8.3841)
se by hand         0.065219
se from scipy      0.065219
R^2 by hand        0.054606

```

The residual standard deviation is 8.17 against the scores’ own 8.38. Knowing a student’s practice hours has bought you almost nothing about where they will land — which is the same fact as $R^2 = 0.055$, in marks rather than as a proportion.

The two intervals, drawn.

```

x0 = 20.0
leverage0 = 1/n + (x0 - hours.mean())**2 / sxx
predicted = fit.intercept + fit.slope * x0
print(f"at {x0:.0f} hours, the fitted value is {predicted:.2f} marks")
print(f" 95% CI for the mean score there : ")

```

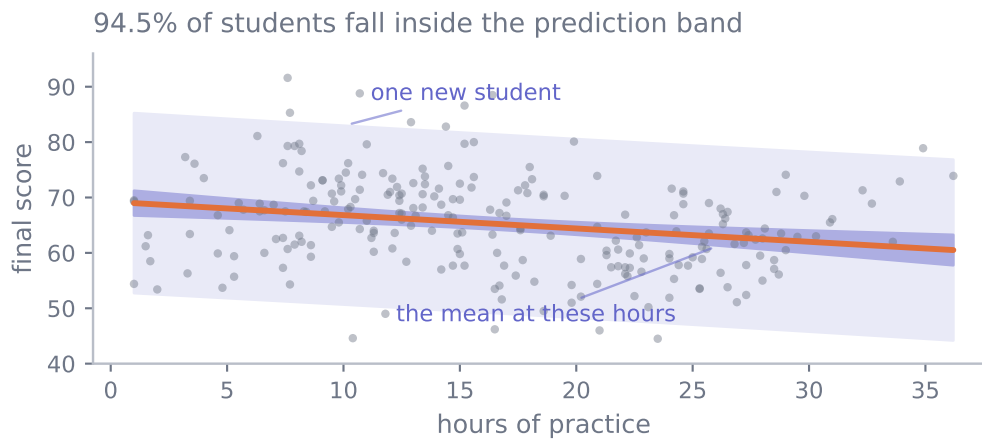


Figure 38.2: The fitted line with two bands. The narrow inner band is a 95% confidence interval for the *mean* score of students at each level of practice — it is not about where individual students fall, and most of them are outside it. The wide outer one is a 95% prediction interval for a single student, and 94.5% of the observed students sit inside it. Those students were used to fit the line, so that figure is a sanity check rather than an independent test.

```
f"({predicted - t_star*s_e*np.sqrt(leverage0):.2f}, "  
f"{predicted + t_star*s_e*np.sqrt(leverage0):.2f})"  
print(f" 95% prediction for one student : "  
f"({predicted - t_star*s_e*np.sqrt(1 + leverage0):.2f}, "  
f"{predicted + t_star*s_e*np.sqrt(1 + leverage0):.2f})")
```

```
at 20 hours, the fitted value is 64.41 marks  
95% CI for the mean score there : (63.27, 65.55)  
95% prediction for one student : (48.28, 80.54)
```

The mean is pinned down to about a mark either way. An individual student is pinned down to about sixteen marks either way. Quoting the first when you meant the second is the standard way a regression is oversold.

Check the assumptions.

No curvature, no fan, and residuals that track a normal model closely. Linearity, constant variance and approximate normality are all in reasonable shape.

Neither plot can check the other two things that matter. **Independence** is a property of the design, not of the residuals: you establish it by knowing how the data were collected. And no residual plot at all can tell you that a variable is **missing** from the model — the fit can look immaculate and still be measuring something other than what you meant. That is precisely this dataset's situation, and the next section shows what it costs.

38.5 Try it yourself

Exercise 1 A regression of weekly sales (in £000) on advertising spend (in £000) gives $\hat{\beta}_1 = 3.2$ with a standard error of 1.5, from $n = 40$ shops.

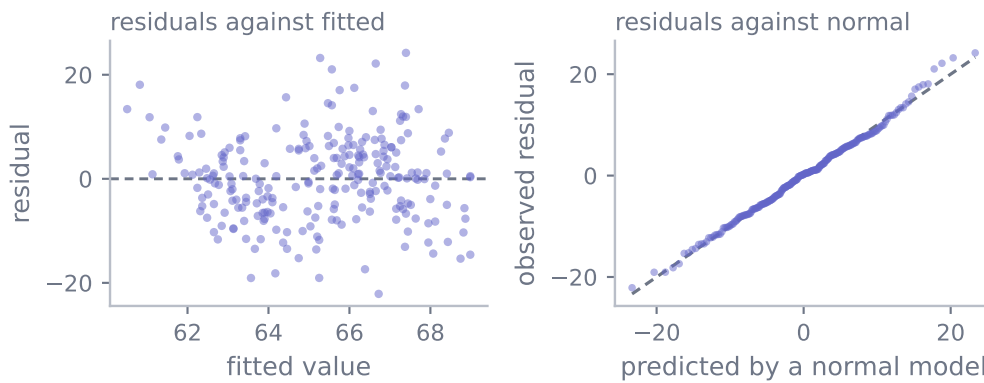


Figure 38.3: Diagnostics. Left: residuals against fitted values, which should show no pattern — no curve, no fan. Right: a Q-Q plot of the residuals against a normal model. Both look acceptable here, which clears the two assumptions these plots can see and says nothing about the two they cannot.

- Give an approximate 95% interval for the slope.
- Interpret the slope in words, with units.
- The R^2 is 0.11. What does that add?

💡 Show the solution

- With $t_{38}^* \approx 2.02$: $3.2 \pm 2.02 \times 1.5 = 3.2 \pm 3.03$, giving $(0.17, 6.23)$. Zero is outside — just.
- Each extra £1,000 of advertising goes with about **£3,200 more in weekly sales**, on average across shops. The interval says the data are consistent with anything from £170 to £6,230, which spans “not worth doing” and “extremely worth doing”.
- $R^2 = 0.11$ says advertising spend accounts for 11% of the variation in sales between shops. The other 89% is location, size, season, competition and everything else. It does not weaken the slope estimate, and it does warn you that predicting one shop’s sales from its advertising alone will be poor.

(The three numbers are not independent, incidentally: in simple regression $t^2 = (n-2)R^2/(1-R^2)$, so a slope, its standard error, the sample size and R^2 determine each other. Here $t = 3.2/1.5 = 2.13$ and $R^2 = 2.13^2/(2.13^2 + 38) = 0.11$, which is where the figure came from.)

Exercise 2 Which of these would you expect to make the standard error of a fitted slope smaller?

- Collecting more observations.
- Choosing x values spread over a wider range.
- Reducing measurement noise in y .
- Adding a constant to every x .

💡 Show the solution

(a), (b) and (c). Look at $SE(\hat{\beta}_1) = s_e / \sqrt{\sum (x_i - \bar{x})^2}$:

- more observations add terms to the sum in the denominator — though only if they add *spread*: an observation placed exactly at $\bar{x}$ contributes nothing to it, and one with a wild y can inflate s_e enough to make the standard error worse. In the ordinary case of more observations across the same range of x , it shrinks;
- a wider spread of x makes each term larger — this is often the cheapest of the three and the one people forget;
- less noise in y shrinks s_e .

(d) **does nothing**. $\sum (x_i - \bar{x})^2$ is unchanged by a shift, because $\bar{x}$ shifts with it. The slope estimate is unchanged too; only the intercept moves, since it is the fitted value at the new $x = 0$.

A caution on (b): widening the range only helps if the relationship really is straight over the wider range. Extending the x values into territory where the line stops being a good model buys precision in the estimate of the wrong thing.

Exercise 3 Using this unit's fitted line, a student who has practised 20 hours is predicted to score 64.4 marks.

- A colleague says “so a student who practises 20 hours will get about 64”. What is wrong?
- The same line predicts 69.2 marks at zero hours. What is wrong with that?
- Would it be reasonable to advise a student to practise less?

💡 Show the solution

- It confuses the **mean response with an individual prediction**. The average score among students at 20 hours is pinned down to about (63.3, 65.6). A single student's score has a 95% prediction interval of about (48.3, 80.5) — a range of 32 marks, covering a fail and a first. The fitted value is a centre, not a forecast.
- It is an **extrapolation**. No student in the data practised fewer than one hour, so nothing in the sample speaks to zero hours. The intercept is a mathematical feature of the line, not an estimate of anything observed.
- No**, and this is the important one. The fit describes an association across a cohort in which practice hours are entangled with prior background: students who arrive stronger practise less. Within every background group the slope is *positive*, at about one mark per hour. Advising an individual requires the within-group relationship, and even that is observational — students chose their own hours. S12 takes this apart properly.

38.6 Check it in Python

First, that the slope really is the correlation rescaled, and that the “by hand” standard error matches the library:

```

r = np.corrcoef(hours, score)[0, 1]
print(f"slope from r * sy/sx  {r * score.std(ddof=1) /
↪  hours.std(ddof=1):+.6f}")
print(f"slope from scipy      {fit.slope:+.6f}")
print(f"R^2 from r squared    {r**2:.6f}    from scipy {fit.rvalue**2:.6f}")

```

```

slope from r * sy/sx  -0.240793
slope from scipy      -0.240793
R^2 from r squared    0.054606    from scipy 0.054606

```

Second, the claim that the standard error measures the fan in the opening figure. Resample the students, refit, and compare the spread of the resulting slopes with the formula:

```

rng = np.random.default_rng(11)
slopes = np.empty(4000)
for i in range(slopes.size):
    pick = rng.integers(0, n, n)
    slopes[i] = stats.linregress(hours[pick], score[pick]).slope

print(f"sd of 4000 resampled slopes  {slopes.std():.6f}")
print(f"formula se                    {fit.stderr:.6f}")
print(f"bootstrap 95% interval        ({np.quantile(slopes, 0.025):+.4f}, "
    f"{np.quantile(slopes, 0.975):+.4f})")
print(f"formula 95% interval          ({fit.slope - t_star*fit.stderr:+.4f},
↪  "
    f"{fit.slope + t_star*fit.stderr:+.4f})")

```

```

sd of 4000 resampled slopes  0.065154
formula se                    0.065219
bootstrap 95% interval        (-0.3724, -0.1186)
formula 95% interval          (-0.3693, -0.1123)

```

Two completely different routes — one algebraic, one brute force — agreeing to three decimal places.

Now the check that fails, and it is the one this dataset exists to teach. A pooled slope is not the slope within any group:

```

grouped = scored.dropna(subset=["background"])
by_group = {name: stats.linregress(part["hours_practice"],
↪  part["final_score"]).slope
    for name, part in grouped.groupby("background")}
for name, slope in by_group.items():
    print(f" {name:<24} {slope:+.4f} marks per hour")

assert all(slope < 0 for slope in by_group.values()), (
    f"the pooled slope is {fit.slope:+.4f}, but every one of the "
    f"{len(by_group)} within-group slopes is positive"
)

```

Computing	+0.6624 marks per hour
Engineering	+0.8454 marks per hour
Life sciences	+1.1526 marks per hour
Mathematics or Physics	+1.0112 marks per hour
Social sciences	+1.3445 marks per hour

AssertionError: the pooled slope is -0.2408, but every one of the 5
↳ within-group slopes is positive

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[9], line 7
      4 for name, slope in by_group.items():
      5     print(f" {name:<24} {slope:+.4f} marks per hour")
----> 7 assert all(slope < 0 for slope in by_group.values()), (
      8     f"the pooled slope is {fit.slope:+.4f}, but every one of the "
      9     f"{len(by_group)} within-group slopes is positive"
     10 )
```

AssertionError: the pooled slope is -0.2408, but every one of the 5
↳ within-group slopes is positive

Pooled, the line falls by 0.24 marks an hour. Within each background it rises by between 0.66 and 1.34 marks an hour. Both are correct least-squares fits to the data in front of them.

Adding the background information to the model — as [M11](#) does with the arrival diagnostic — flips the sign of the practice coefficient:

```
design = np.column_stack([np.ones(n), hours,
↳ scored["diagnostic_score"].to_numpy()])
theta, *_ = np.linalg.lstsq(design, score, rcond=None)
print(f"score ~ 1 + hours           slope {fit.slope:+.4f}")
print(f"score ~ 1 + hours + diagnostic slope {theta[1]:+.4f}")
```

score ~ 1 + hours	slope -0.2408
score ~ 1 + hours + diagnostic	slope +0.9156

Same data, same method, opposite sign. Which coefficient answers your question depends entirely on what you were asking, and nothing in the output tells you which one you wanted.

38.7 Common wrong turns

Where this usually goes wrong

Reporting a slope without an interval A slope is an estimate. “0.24 marks per hour” and “0.24 marks per hour, 95% CI 0.11 to 0.37” are different claims, and only the second can be argued with.

Confusing a confidence band with a prediction band The first is about the mean at a given x and is narrow; the second is about one new individual and is much wider, because it carries the residual scatter. In this unit’s fit they differ by a factor of about fourteen.

Reading a low R^2 as “no relationship” $R^2 = 0.055$ here, with a slope whose interval excludes zero. Small and real are compatible, and with human data they are the normal case.

Reading a high R^2 as “the model is right” A curved relationship can produce a high R^2 with a line that is wrong everywhere. Look at the residuals against the fitted values, always.

Interpreting the intercept outside the data The intercept is the fitted value at $x = 0$. If no observation is near zero, it is an extrapolation and usually meaningless — sometimes physically impossible.

Predicting outside the range of x The line is a summary of the region you observed. Nothing in the data says the relationship continues, and the further out you go the more confidently wrong the prediction becomes.

Reading the slope as an effect A regression coefficient describes an association in the data you fitted, given the other variables in the model. Making it causal needs a design that supports the claim, not a smaller p -value.

38.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A fitted slope is 2.0 with standard error 0.5, from $n = 100$. Which is true?
 - a. The relationship explains most of the variation
 - b. A 95% interval for the slope is roughly (1.0, 3.0)
 - c. The correlation is 2.0
 - d. Each unit of x causes 2 units of y
2. You want a more precise estimate of a slope but cannot collect more data. What else could you change about the design?
3. Two regressions have identical slopes and identical standard errors, but R^2 of 0.9 and 0.2. What differs?
4. Why does a prediction interval not shrink towards zero width as n grows?
5. The cohort’s pooled slope is -0.24 and every within-background slope is positive. Which is the “real” relationship between practice and score?

Show the answers

1. **(b).** With $n = 100$ the multiplier is close to 2, so the interval is about 2.0 ± 1.0 . (a) confuses the slope with R^2 — nothing here says how much variation is explained; (c) confuses the slope with the correlation, which cannot exceed 1; (d) upgrades an association to a cause, which no regression output can license.
2. **Spread the x values further apart.** The standard error is $s_e / \sqrt{\sum (x_i - \bar{x})^2}$, so choosing observations across a wider range of x shrinks it without a single extra data point. Reducing measurement noise in y works too. The caveat from Exercise 2 stands: the relationship must stay straight over the wider range.
3. **The residual scatter relative to the spread of y .** The regression with $R^2 = 0.2$ has much more variation in y that the line does not account for. Since

the standard errors match, both slopes are estimated equally precisely — which is a clean demonstration that R^2 and the precision of a coefficient are separate things.

4. Because it contains the individual's own noise. As n grows the *line* is pinned down and the mean-response band shrinks towards nothing, but a new student still scatters about that line with standard deviation s_e . The prediction interval converges to roughly $\hat{y} \pm 2s_e$, not to a point.
5. **Neither is “the” relationship; they answer different questions.** The pooled slope describes the cohort as a whole, in which practice hours partly stand in for prior background. The within-group slopes describe students compared with others who arrived in a similar position. For advising a student the within-group figure is much closer to the question — and, since nobody assigned practice hours, still not a causal answer. [S12](#) is next.

Where this shows up next

[S12](#) takes the reversal in this unit's failing check, names it, and sets it alongside the other ways an analysis produces a confident wrong answer. [M12](#) is where regression with many correlated predictors goes when the columns start to overlap.

39 How analyses go wrong

Statistics · Unit 12

S12 · Statistics Reading: 7 pages With practice: 1.5 hours Assumes: S11

Why this unit is here

Every technique in this track can be executed perfectly and still produce a confident, reproducible, wrong answer. That is not a flaw in the techniques; it is what happens when a correct calculation is pointed at the wrong question, or run on a sample selected in a way nobody wrote down, or repeated until something interesting appears.

The failures in this unit are not exotic. They are the ordinary ones, they account for most of the retracted findings you will read about during your degree, and every one of them is visible in advance if you know the shape to look for. This is the unit that makes the previous eleven usable.

39.1 Before you start

You should be able to:

- fit and interpret a straight line with an interval on its slope — [S11](#);
- read a p -value and state what it is conditional on — [S9](#);
- name the unit, population, estimand and design of a question — [S1](#).

Quick check. A test is significant at $p < 0.05$. Twenty independent tests are run on data where nothing is going on. How many significant results would you expect, and what is the chance of at least one?

 Show the answer

You expect $20 \times 0.05 = 1$. The chance of at least one is $1 - 0.95^{20} = 0.64$.

So in a study with twenty comparisons and no real effects anywhere, the odds are roughly two to one that you will find something to write up. That single calculation is the engine behind a large fraction of this unit.

39.2 The idea

The cohort used throughout these notes contains a planted trap, and by now you have walked into it four times.

```
import sys; sys.path.insert(0, "../theme")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from figures import figure, note, no_axes, use_style, ACCENT, WARM, COOL,
↪ INK, FAINT

cohort = pd.read_csv("../data/cohort.csv")
scored = cohort[cohort["final_score"].between(0, 100)]
grouped = scored.dropna(subset=["background"])

pooled = stats.linregress(scored["hours_practice"], scored["final_score"])
print(f"pooled slope, all {len(scored)} students: {pooled.slope:+.4f} marks
↪ per hour")
for name, part in grouped.groupby("background"):
    within = stats.linregress(part["hours_practice"], part["final_score"])
    print(f"    {name:<24} n = {len(part):>3}    {within.slope:+.4f}")
```

pooled slope, all 238 students: -0.2408 marks per hour

Computing	n = 69	+0.6624
Engineering	n = 36	+0.8454
Life sciences	n = 57	+1.1526
Mathematics or Physics	n = 36	+1.0112
Social sciences	n = 39	+1.3445

Pooled, practice looks harmful. Within every background group, it looks helpful, by between roughly three and five times as much as the pooled slope says it hurts. This is **Simpson's paradox**: an association that reverses when a third variable is taken into account.

The mechanism is not mysterious. Students from quantitative backgrounds start further ahead and practise less; students from other backgrounds start further behind and practise more. Pooling the two puts a strong negative *between-group* pattern on top of a positive *within-group* one, and the between-group pattern wins.

Nothing in the pooled analysis is wrong. The slope is correctly computed, its interval is honest, its *p*-value is small. It answers a question — *among these 238 students, how does score vary with practice hours?* — and that question is almost never the one anybody wanted answered.

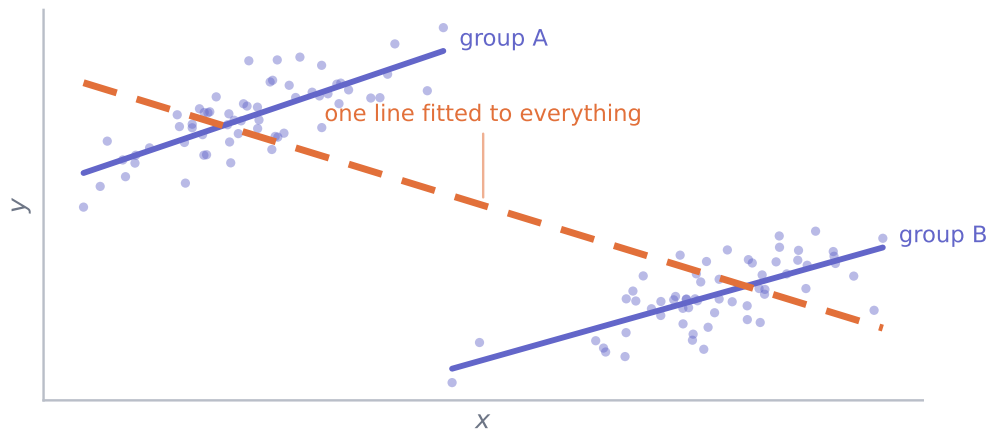


Figure 39.1: The mechanism, drawn cleanly. Two groups, each with a rising relationship inside it. The right-hand group starts higher on the vertical axis and lower on the horizontal one, so the two clouds are offset along a downward diagonal — and a line fitted to everything follows the offset between the groups rather than the trend inside them.

39.3 The mechanics

39.3.1 Confounding

Simpson’s paradox is the dramatic case of a general problem. A **confounder** is a common cause: something that influences both the thing you varied and the thing you measured. Its influence is credited to whatever you happened to put in your model.

“Related to both” is not enough to make a variable a confounder, and the difference decides what to do about it. A variable on the causal path *from* your predictor *to* your outcome is a mediator, and adjusting for it changes which question you are answering rather than removing a bias. A variable that is a *consequence* of both is a collider, and adjusting for it manufactures a bias that was not there. Which kind you are looking at is a claim about how the world works, and no amount of data settles it.

There are three responses, in decreasing order of strength.

1. **Randomise.** If treatment is assigned by a coin, no variable — measured or not — can be systematically related to it. This is why experiments are worth their cost.
2. **Adjust.** Include the confounder in the model, or compare within its levels, as the group-by-group fits above do. This works only for confounders you thought of and measured.
3. **Say so.** If neither is available, state which confounders you could not rule out and in which direction each would bias the answer.

The third is not a failure. It is what an honest observational analysis looks like, and it is far more useful than an unqualified claim.

39.3.2 Selection, and the sample you did not see

S1 warned that a design decides which population an answer is about. The failures come in a family:

- **Self-selection** — people who respond differ from those who do not.
- **Survivorship** — you only see the units that made it: the funds still trading, the customers who did not leave, the aircraft that came back.
- **Collider bias** — conditioning on a *consequence* of two variables creates an association between them that does not exist in the population. Restrict attention to students who were admitted, and admission criteria will manufacture a negative relationship between the things that earn admission.

Sample size does not help with any of them; it makes the wrong answer more precise.

39.3.3 Multiple comparisons

Each test at $\alpha = 0.05$ has a one-in-twenty false-positive rate. Run many and false positives become the expected outcome rather than the exception.

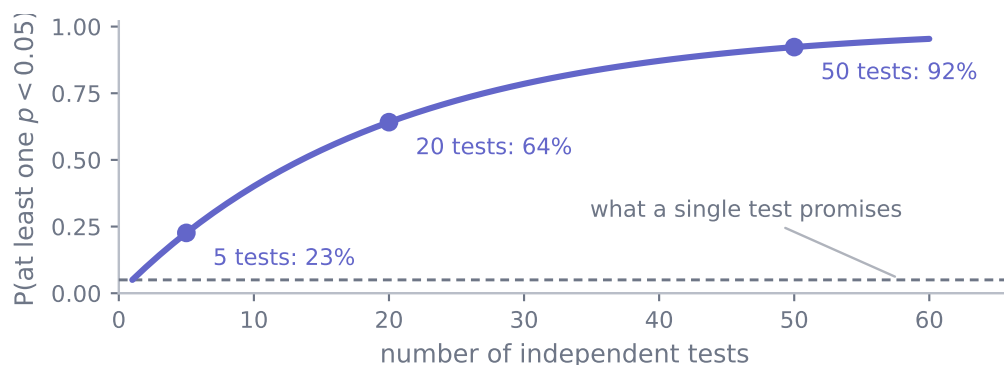


Figure 39.2: The probability of at least one significant result when nothing at all is going on, against the number of independent tests run. At five tests it is already better than one in five; by twenty it is nearly two in three. Nothing about any individual test has changed.

The count that matters is **every comparison you could have made**, not the ones you wrote down. Trying three outcome measures, two subgroups and two ways of excluding outliers is twelve analyses even if you report one. This is the “garden of forking paths”, and it does its damage without anyone intending to cheat: each decision felt reasonable at the time, and each was made after seeing the data.

Two repairs. **Bonferroni**: test at α/k for k comparisons, which controls the chance of any false positive and costs power. **False discovery rate** (Benjamini–Hochberg): control the *expected* proportion of your discoveries that are false — an average across many studies, not a guarantee about yours — which is usually the more sensible target when screening many hypotheses. Both are far less important than the habit of saying how many comparisons were made.

39.3.4 Base rates, again

S4 showed that a good test for a rare condition mostly produces false positives. Replace “condition” with “true hypothesis” and the same arithmetic applies to research.

If a fraction π of the hypotheses in a field are true, a test with power $1 - \beta$ and level α produces true positives at rate $(1 - \beta)\pi$ and false ones at rate $\alpha(1 - \pi)$, so

$$P(\text{true} \mid \text{significant}) = \frac{(1 - \beta) \pi}{(1 - \beta) \pi + \alpha (1 - \pi)}.$$

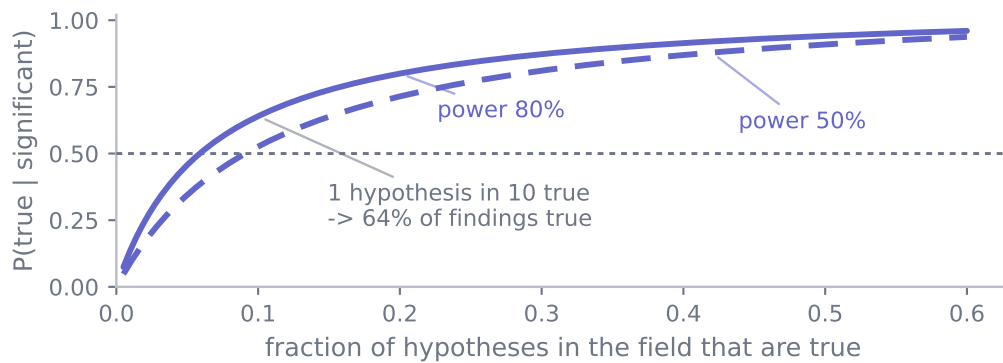


Figure 39.3: The chance that a significant finding is true, against the fraction of hypotheses in the field that are true, at the conventional 5% level. Even with 80% power, a field in which one hypothesis in ten holds up produces significant findings that are right only about two thirds of the time — and at 50% power, barely half.

Two things follow. **The p -value alone cannot tell you how likely a finding is to be true**, because two of the three numbers in that formula are properties of the field rather than of your study. Read them off the figure: with 80% power and one hypothesis in ten holding up, 64% of significant findings are real; drop the prior to one in twenty and it is 46%; drop the power to 50% as well and it is 35%. And **power is not just about your own study**: low power raises the share of published findings that are false, because it shrinks the numerator while leaving the false-positive rate alone.

39.3.5 Regression to the mean

If a measurement is part signal and part noise, the units that scored highest were partly lucky, and on a second measurement they will average closer to the middle. This happens with no cause at all, and it is routinely mistaken for one.

It is why the “worst-performing hospitals” improve after an intervention, why the sequel to a breakout success disappoints, and why anyone who praises after a good outcome and criticises after a bad one will conclude that criticism works.

The defence is a control group. Without one you cannot separate the effect of what you did from the movement that would have happened anyway.

39.3.6 Overfitting

A model with enough freedom can fit any dataset exactly, including its noise. It will then perform badly on data it has not seen. The symptom is a gap between performance on the data you fitted and on data you held back; the defence is to hold data back before you start, and never to touch it until the model is final.

39.4 Worked example

Three failures, on the cohort.

One: the reversal, quantified. Fit the practice slope with and without the information about where each student started.

```
hours = scored["hours_practice"].to_numpy()
score = scored["final_score"].to_numpy()
diagnostic = scored["diagnostic_score"].to_numpy()
ones = np.ones(hours.size)

for name, design in [("score ~ 1 + hours", np.column_stack([ones, hours])),
                    ("score ~ 1 + hours + diagnostic",
                     np.column_stack([ones, hours, diagnostic]))]:
    theta, *_ = np.linalg.lstsq(design, score, rcond=None)
    print(f"{name:<32} hours coefficient {theta[1]:+.4f}")
```

```
score ~ 1 + hours           hours coefficient -0.2408
score ~ 1 + hours + diagnostic hours coefficient +0.9156
```

Adding one column moves the coefficient on practice from -0.24 to $+0.92$ marks per hour. Neither fit is a mistake. The first answers “how does score vary with practice across the cohort?”, the second “how does score vary with practice among students who arrived equally prepared?” — and only the second is close to the question a student asking whether to practise actually has.

Two: torturing the data. Split the cohort at random — a variable that cannot possibly matter — and test whether the two halves differ. Do it twenty times.

```
rng = np.random.default_rng(2026)
p_values = np.array([
    stats.ttest_ind(score[flag], score[~flag], equal_var=False).pvalue
    for flag in (rng.integers(0, 2, score.size).astype(bool) for _ in
        ↪ range(20))
])
print(f"20 random splits of the cohort")
print(f"  smallest p           {p_values.min():.4f}")
print(f"  number below 0.05     {(p_values < 0.05).sum()}")
print(f"  Bonferroni threshold for 20 tests: {0.05/20:.4f}")
```

```
20 random splits of the cohort
smallest p           0.0136
number below 0.05     1
Bonferroni threshold for 20 tests: 0.0025
```

One split in twenty comes out “significant” at $p = 0.014$, and a report of that split alone — “*students in group X scored significantly higher*” — would be entirely true about the

arithmetic and entirely false about the world. Against the Bonferroni threshold of 0.0025 it survives nothing.

Three: regression to the mean. Take the students who scored highest on the arrival diagnostic and follow them to the final. Both variables are standardised, so the comparison is like for like.

```
z_diagnostic = (diagnostic - diagnostic.mean()) / diagnostic.std(ddof=1)
z_final = (score - score.mean()) / score.std(ddof=1)
print(f"correlation between the two: {np.corrcoef(z_diagnostic, z_final)[0,
↪ 1]:.4f}\n")
for label, mask in [("top 10% on arrival", z_diagnostic >=
↪ np.quantile(z_diagnostic, 0.9)),
                    ("bottom 10% on arrival", z_diagnostic <=
↪ np.quantile(z_diagnostic, 0.1))]:
    print(f"{label:<24} n = {mask.sum():>3}      "
          f"diagnostic {z_diagnostic[mask].mean():+.3f} sd      "
          f"final {z_final[mask].mean():+.3f} sd")
```

correlation between the two: 0.6579

top 10% on arrival	n = 24	diagnostic +1.700 sd	final +1.428 sd
bottom 10% on arrival	n = 24	diagnostic -1.491 sd	final -0.709 sd

The strongest starters were 1.70 standard deviations above average on arrival and 1.43 above it at the end. The weakest were 1.49 below on arrival and 0.71 below at the end. Both groups moved towards the middle, and nobody did anything to them.

Now imagine the department had run a support programme for the weakest tenth. Their improvement — most of the way back to average — would have looked like a triumph, and essentially all of it would have happened anyway.

39.5 Try it yourself

Exercise 1 A hospital reports that its overall surgical survival rate is lower than a neighbouring hospital's, but that its survival rate is higher for every individual severity category.

- (a) Is one of the two figures wrong?
- (b) What explains it?
- (c) Which figure should a patient use?

Show the solution

- (a) **No.** Both can be correct simultaneously; this is Simpson's paradox with the roles of practice and background played by severity.
- (b) The hospital with the lower overall rate almost certainly takes **more severe cases**. Severity drives both which hospital you go to and whether you survive, so pooling across severity credits the case mix to the hospital.

- (c) **The category-specific figures**, matched to their own severity — that is the comparison that holds the confounder fixed. The overall figure answers a question about the two hospitals' *patients*, not about their surgeons.

Worth adding: this is exactly why publishing unadjusted mortality league tables creates an incentive to refuse difficult cases, and why the adjusted figures are the ones that get published.

Exercise 2 A team tests 4 outcome measures in 3 subgroups and reports the one result with $p = 0.03$ as their finding.

- (a) How many comparisons were made?
(b) Roughly what is the chance of at least one $p < 0.05$ if nothing is going on?
(c) What should they have written?

 Show the solution

- (a) $4 \times 3 = 12$ at minimum — and more if any decisions about exclusions or transformations were made after seeing the data.
(b) $1 - 0.95^{12} = 0.46$, if the twelve tests are independent. Close to a coin flip, from a study in which nothing whatever is going on.

They will not be exactly independent — the same people appear in several comparisons, and the four outcomes are probably related — which pulls the number down. It does not pull it down to anything reassuring, and the honest version of the calculation is still “we made twelve comparisons”.

- (c) Something like: “*We examined 4 outcomes in 3 subgroups. One comparison reached $p = 0.03$ (uncorrected); against a Bonferroni threshold of 0.0042 it is not significant, and we report it as a hypothesis for a future study rather than as a finding.*”

Note what is **not** required: they do not have to hide the result. They have to report the denominator. A result that arrives with its number of comparisons attached is useful; the same result without it is misleading.

Exercise 3 During the Second World War, returning aircraft were examined to see where they had been hit, so that armour could be added where the damage was densest.

- (a) What is wrong with the plan?
(b) Where should the armour go?
(c) Name a modern analysis with the same structure.

 Show the solution

- (a) **Survivorship bias.** The sample consists only of aircraft that came back. Hits are presumably scattered fairly evenly, so the places where returning aircraft show *few* holes are the places where a hit meant not returning.
(b) **Where the returning aircraft show no damage** — the engines, in the original study. The observed pattern is a map of survivable hits.

- (c) Many: studying only customers who did not churn, only funds that still exist, only patients who completed a course of treatment, only startups that are still trading. In every case the units that would have been most informative are the ones missing from the file, and no amount of data on the survivors recovers them.

39.6 Check it in Python

The multiple-comparisons arithmetic is worth watching rather than believing. Here nothing is going on anywhere — two groups drawn from the same population — and we count how often at least one test in a batch comes out significant.

```
rng = np.random.default_rng(7)
for tests in [1, 5, 20]:
    hits = 0
    for _ in range(4000):
        batch = [stats.ttest_ind(rng.normal(0, 1, 30), rng.normal(0, 1,
↪ 30)).pvalue
                        for _ in range(tests)]
        hits += min(batch) < 0.05
    print(f"{tests:>2} tests per study: at least one significant in "
          f"{100*hits/4000:5.1f}% of studies    (formula {100*(1 -
↪ 0.95**tests):5.1f}%)")
```

1 tests per study:	at least one significant in	5.5% of studies	(formula	5.0%)
5 tests per study:	at least one significant in	23.5% of studies	(formula	22.6%)
20 tests per study:	at least one significant in	63.7% of studies	(formula	64.2%)

And now the check that fails — the one this whole unit is about. It is tempting to believe that if an association is real and strong in the data, adjusting for another variable can only change it a little:

```
simple, *_ = np.linalg.lstsq(np.column_stack([ones, hours]), score,
↪ rcond=None)
adjusted, *_ = np.linalg.lstsq(np.column_stack([ones, hours, diagnostic]),
                               score, rcond=None)
assert np.sign(simple[1]) == np.sign(adjusted[1]), (
    f"the practice coefficient is {simple[1]:+.4f} on its own and "
    f"{adjusted[1]:+.4f} once the arrival diagnostic is included - "
    f"opposite signs, same data"
)
```

```
AssertionError: the practice coefficient is -0.2408 on its own and +0.9156
↪ once the arrival diagnostic is included - opposite signs, same data
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[9], line 4
```

```

1 simple, *_ = np.linalg.lstsq(np.column_stack([ones, hours]), score,
↪ rcond=None)
2 adjusted, *_ = np.linalg.lstsq(np.column_stack([ones, hours,
↪ diagnostic]),
3                                     score, rcond=None)
----> 4 assert np.sign(simple[1]) == np.sign(adjusted[1]), (
5     f"the practice coefficient is {simple[1]:+.4f} on its own and "
6     f"{adjusted[1]:+.4f} once the arrival diagnostic is included - "
7     f"opposite signs, same data"
8 )

```

AssertionError: the practice coefficient is -0.2408 on its own and +0.9156
↪ once the arrival diagnostic is included - opposite signs, same data

Not a little: the opposite direction. There is no rule that says adding a variable makes a small change, and the size of the change is exactly the size of the confounding.

Finally, regression to the mean, on data with **no relationship between the two occasions except noise around a shared underlying ability**:

```

rng = np.random.default_rng(3)
ability = rng.normal(0, 1, 5000)
first = ability + rng.normal(0, 1, 5000)      # nothing happens between the
↪ two
second = ability + rng.normal(0, 1, 5000)

# Standardise both occasions, so the two are on a common scale.
z_first = (first - first.mean()) / first.std(ddof=1)
z_second = (second - second.mean()) / second.std(ddof=1)

worst = z_first <= np.quantile(z_first, 0.1)
print(f"bottom 10% at time 1: {z_first[worst].mean():+.3f} sd"
      f" -> time 2 {z_second[worst].mean():+.3f} sd")
print(f"apparent 'improvement': "
      f"{z_second[worst].mean() - z_first[worst].mean():+.3f} sd, "
      f"from an intervention that did not exist")

```

bottom 10% at time 1: -1.747 sd -> time 2 -0.932 sd
apparent 'improvement': +0.815 sd, from an intervention that did not exist

Nothing was done to anyone. The improvement is entirely the noise that put them at the bottom in the first place, failing to repeat itself.

39.7 Common wrong turns

Where this usually goes wrong

Reporting a pooled result when the groups disagree If the direction reverses inside subgroups, the pooled number is not a summary of anything anyone asked about. Report both, and say which question each answers.

Adjusting for everything you have Adjustment is not automatically safer. Conditioning on a variable that is a *consequence* of both your predictor and your outcome creates bias rather than removing it. Which variables to adjust for is a question about how the world works, not about which columns are in the file.

Counting only the tests you reported The relevant number is every comparison you could have made, including decisions about exclusions and transformations taken after seeing the data. Report the denominator.

Explaining regression to the mean If you selected units for being extreme, expect them to move towards the middle next time. Attributing that movement to whatever you did in between requires a control group that was equally extreme and left alone.

Treating a large sample as protection Size fixes sampling noise and nothing else. Confounding, selection and survivorship all survive intact into a dataset of any size, and become harder to argue with because the intervals are so tight.

Evaluating a model on the data you fitted Any sufficiently flexible model fits its training data. Hold data back before you start, and do not look at it until the model is final — including not looking at it “just to check”.

Believing that a significant result is probably true How likely it is to be true depends on three numbers — the fraction of hypotheses in the field that hold up, the power, and α — and the p -value contains only the last. In an exploratory field with modest power the share of significant findings that are false can easily exceed half. That is base rates, and it is arithmetic rather than cynicism.

39.8 Check yourself

Answers are at the foot of the page. Give a reason as well as an answer.

1. A study finds a positive association between coffee and heart disease. Which would most change your interpretation?
 - a. A larger sample
 - b. A smaller p -value
 - c. Information about smoking
 - d. A higher R^2
2. Ten subgroups are tested and one gives $p = 0.04$. How much evidence is that?
3. A school selects its 20 weakest readers for a programme. After a term they have improved relative to the school average. Did the programme work?
4. Adding a variable to a regression changes a coefficient's sign. Which fit is right?
5. Why does low statistical power make the *published* literature less reliable, rather than just making individual studies less informative?

 Show the answers

1. (c). Smoking is associated with both coffee drinking and heart disease, so it is a candidate confounder, and the association could be entirely its doing. (a), (b) and (d) all make the estimate more precise or the fit tighter without touching the reason it might be wrong — which is the standard shape of this unit's lesson.

2. **Very little on its own.** With ten tests, the chance of at least one $p < 0.05$ under no effect at all is $1 - 0.95^{10} = 0.40$. Reported with its denominator, and treated as a hypothesis to test on new data, it is worth something. Reported alone as a finding, it is close to worthless.
3. **You cannot tell.** They were selected for being extreme, so regression to the mean predicts improvement with no programme at all. Without a comparison group of equally weak readers who did not take part, the programme's effect and the statistical artefact are inseparable.
4. **Both are correct fits, and they answer different questions.** The unadjusted coefficient describes how the outcome varies with the predictor across the whole sample; the adjusted one describes how it varies among units alike on the added variable. Which one you want depends on what role the added variable plays: a **common cause** of predictor and outcome (adjust, to remove confounding); something **on the path** from predictor to outcome (adjusting is legitimate but answers a narrower question — the part of the effect that does not run through it); or a **consequence of both** (adjusting creates a bias that was not there). Nothing in the regression output distinguishes the three.
5. Because power sits in the numerator of $P(\text{true} \mid \text{significant})$. Halving power halves the rate of true positives while leaving the false-positive rate at α , so the proportion of significant findings that are false goes *up*. A literature built from underpowered studies contains a higher share of false claims than the same literature built from well-powered ones, even with everyone behaving honestly.

Where this shows up next

This is the last unit of the Statistics track, and the habits in it are the ones worth carrying into everything else you do.

Write the question down before you touch the data — [S1](#). Report an estimate with an interval, in the units of the thing measured — [S8](#), [S10](#). Say how many comparisons you made. Say which confounders you could not rule out. And when a result surprises you, look for the third variable before you look for a publisher.

Part V

Lookup

40 Formula card

A lookup sheet, not teaching material. Nothing here is introduced for the first time — every row links to the unit that explains it, and the explanation is where the meaning lives.

40.1 Reading notation

Written	Read as	Where
$\sum_{i=1}^n x_i$	add up x_1 through x_n	M1
$f: \mathbb{R} \rightarrow \mathbb{R}$	f takes a real number and returns one	M2
$\bar{x}, \hat{p}$	computed from a sample	S1
μ, σ, p	properties of a population	S1
$x \in A$	x is a member of the set A	M1
$A \cap B, A \cup B$	A and B ; A or B	S4
$P(A \mid B)$	probability of A given B	S4

40.2 Exponentials and logarithms

$$\ln(ab) = \ln a + \ln b, \quad \ln \frac{a}{b} = \ln a - \ln b, \quad \ln(a^b) = b \ln a$$

$$\ln(e^x) = x, \quad e^{\ln x} = x \quad (x > 0), \quad a^x = b \implies x = \frac{\ln b}{\ln a}$$

— [M2](#).

40.3 Differentiation

Table 40.1: The four you need

Function	Derivative
c (a constant)	0
x^n	nx^{n-1}
e^x	e^x
$\ln x$	$1/x \quad (x > 0)$

$$(u + v)' = u' + v', \quad (uv)' = u'v + uv', \quad \left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$$

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x) \quad (\text{chain rule})$$

— M3, M4.

Finding a best value. Solve $f'(x) = 0$ for candidates, then check whether each is a maximum, a minimum or neither — and check the endpoints of the allowed range separately. M5.

Local linear approximation. $f(x) \approx f(a) + f'(a)(x - a)$. M3.

Several variables. The gradient ∇f collects the partial derivatives; it points in the direction of steepest increase, and gradient descent steps against it. M6.

40.4 Integration

$$\int_a^b f(x) dx = F(b) - F(a) \quad \text{where } F' = f \quad (\text{fundamental theorem})$$

$$\int x^n dx = \frac{x^{n+1}}{n+1} + C \quad (n \neq -1), \quad \int \frac{1}{x} dx = \ln|x| + C, \quad \int e^x dx = e^x + C$$

— M7.

40.5 Vectors and matrices

$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta, \quad \|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$$

Shapes: $(m \times k)(k \times n) \rightarrow (m \times n)$; the inner dimensions must match. $(AB)^\top = B^\top A^\top$. M8, M9.

Least squares. The normal equations are $X^\top X \hat{\theta} = X^\top y$; compute with `np.linalg.lstsq(X, y, rcond=None)`, which never forms $X^\top X$. M11.

Eigenvectors and the SVD. $A\mathbf{v} = \lambda\mathbf{v}$; a symmetric matrix has orthogonal eigenvectors, and principal components are the eigenvectors of the covariance (or correlation) matrix. M12.

40.6 Describing data

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s = \sqrt{s^2}$$

Five-number summary: minimum, Q_1 , median, Q_3 , maximum; $\text{IQR} = Q_3 - Q_1$. Tukey whiskers reach the furthest point within 1.5 IQR of the box. S2.

$$\text{cov}(x, y) = \frac{1}{n-1} \sum (x_i - \bar{x})(y_i - \bar{y}), \quad r = \frac{\text{cov}(x, y)}{s_x s_y} \in [-1, 1]$$

— S3.

40.7 Probability

$$P(\text{not } A) = 1 - P(A), \quad P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(A \cap B) = P(A | B)P(B)$$

Independent means $P(A \cap B) = P(A)P(B)$, equivalently $P(A | B) = P(A)$. Mutually exclusive means $P(A \cap B) = 0$, which is a different and nearly opposite thing.

$$P(A) = \sum_i P(A | B_i)P(B_i) \quad P(B | A) = \frac{P(A | B)P(B)}{P(A)}$$

— S4.

40.8 Random variables

$$E[X] = \sum_x x p(x) \text{ or } \int x f(x) dx, \quad \text{Var}(X) = E[X^2] - (E[X])^2$$

Rule	Condition
$E[aX + b] = aE[X] + b$	none
$E[X + Y] = E[X] + E[Y]$	none
$\text{Var}(aX + b) = a^2 \text{Var}(X)$	none
$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$	$\text{Cov}(X, Y) = 0$

— S5.

40.9 Distributions

Family	Mechanism	Mean	Variance
Bernoulli(p)	one yes/no trial	p	$p(1 - p)$
Binomial(n, p)	n independent trials, same p	np	$np(1 - p)$
Poisson(λ)	rare events in a fixed window	λ	λ
Uniform(a, b)	every value equally likely	$(a + b)/2$	$(b - a)^2/12$
Exponential(λ)	waiting time, memoryless	$1/\lambda$	$1/\lambda^2$
Normal(μ, σ^2)	many small effects added	μ	σ^2

Normal: 68% within $\mu \pm \sigma$, 95% within $\mu \pm 2\sigma$, 99.7% within $\mu \pm 3\sigma$. S6.

40.10 Inference

$$\text{SE}(\bar{x}) = \frac{s}{\sqrt{n}}, \quad \bar{x} \pm t_{n-1}^* \frac{s}{\sqrt{n}}$$

— S7, S8.

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \quad (\text{df } n-1) \quad p = P(\text{a statistic at least this extreme} \mid H_0)$$

— S9.

Two means (Welch): estimate $\bar{x}_A - \bar{x}_B$, with

$$\text{SE} = \sqrt{\frac{s_A^2}{n_A} + \frac{s_B^2}{n_B}}.$$

Two proportions: estimate $\hat{p}_A - \hat{p}_B$, with

$$\text{SE} = \sqrt{\frac{\hat{p}_A(1 - \hat{p}_A)}{n_A} + \frac{\hat{p}_B(1 - \hat{p}_B)}{n_B}}.$$

A table of counts: $E_{ij} = (\text{row}_i)(\text{col}_j)/n$ and $\chi^2 = \sum (O_{ij} - E_{ij})^2 / E_{ij}$, with $(\text{rows} - 1)(\text{cols} - 1)$ degrees of freedom. S10.

40.11 Regression

$$\hat{\beta}_1 = r \frac{s_y}{s_x}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}, \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$$s_e = \sqrt{\frac{1}{n-2} \sum (y_i - \hat{y}_i)^2}, \quad \text{SE}(\hat{\beta}_1) = \frac{s_e}{\sqrt{\sum (x_i - \bar{x})^2}}$$

Interval for the slope: $\hat{\beta}_1 \pm t_{n-2}^* \text{SE}(\hat{\beta}_1)$. For a prediction at x_0 , add the individual's own scatter: $\text{SE}_{\text{pred}}^2 = \text{SE}_{\text{mean}}^2 + s_e^2$. S11.

41 Python card

The library calls these notes actually use, in one place. Nothing here is introduced for the first time; each row links to the unit that explains what the call is for and what it does when the data are not what you assumed.

Everything below assumes the standard imports:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
```

41.1 Core Python

Task	Call	Where
divide, floor-divide, remainder	<code>a / b, a // b, a % b</code>	P2
format a number into text	<code>f"{value:.2f}"</code>	P2
slice a list	<code>xs[1:4], xs[-1], xs[::-2]</code>	P3
build a list from a loop	<code>[f(x) for x in xs if keep(x)]</code>	P4
key-value store	<code>{"a": 1}, d.get(k, default)</code>	P3
a set of unique values	<code>set(xs)</code>	P3
define a function	<code>def name(arg, other=default):</code>	P5
assert an assumption	<code>assert condition, "message"</code>	P4

41.2 NumPy

Task	Call	Where
make an array	<code>np.array(...), np.zeros(n), np.arange(n)</code>	P6
evenly spaced values	<code>np.linspace(lo, hi, count)</code>	P6
shape, reshape	<code>a.shape, a.reshape(rows, cols)</code>	P6
select by condition	<code>a[a > 70], np.where(cond, x, y)</code>	P6
summarise	<code>a.mean(), a.std(ddof=1), a.sum()</code>	P7
summarise along an axis	<code>a.mean(axis=0) (down columns)</code>	P7
quantiles	<code>np.quantile(a, [0.25, 0.5, 0.75])</code>	S2
correlation	<code>np.corrcoef(x, y)[0, 1]</code>	S3
covariance	<code>np.cov(x, y, ddof=1)[0, 1]</code>	S3
compare floats	<code>np.isclose(a, b) — never ==</code>	P2

Task	Call	Where
solve least squares	<code>np.linalg.lstsq(X, y, rcond=None)</code>	M11
solve a square system	<code>np.linalg.solve(A, b)</code>	M10
eigen-decomposition	<code>np.linalg.eigh(C)</code> (symmetric)	M12
singular values	<code>np.linalg.svd(X, full_matrices=False)</code>	M12
condition number	<code>np.linalg.cond(A)</code>	M10
random numbers	<code>rng = np.random.default_rng(seed)</code>	P11
draw from data	<code>rng.choice(values, size=n, replace=True)</code>	S8

! `numpy.std` and `numpy.var` divide by n by default; `pandas.std` and `pandas.var` divide by $n - 1$. Pass `ddof=` and say which you meant. [S2](#).

41.3 pandas

Task	Call	Where
load a file	<code>pd.read_csv(path)</code>	P8
first look	<code>df.head()</code> , <code>df.info()</code> , <code>df.describe()</code>	P8
one column	<code>df["score"]</code>	P8
rows by condition	<code>df[df["score"] > 70]</code>	P8
rows by label and column	<code>df.loc[mask, "score"]</code>	P8
add a column	<code>df["z"] = ...</code> or <code>df.assign(z=...)</code>	P8
find missing values	<code>df.isna().sum()</code>	P9
drop missing values	<code>df.dropna(subset=["score"])</code>	P9
range check	<code>df["score"].between(0, 100)</code>	P9
force a column numeric	<code>pd.to_numeric(col, errors="coerce")</code>	P9
counts of a category	<code>df["background"].value_counts()</code>	P9
group and summarise	<code>df.groupby("g")["y"].agg(["count", "mean"])</code>	P9
two-way table	<code>pd.crosstab(df["a"], df["b"])</code>	S3
join two frames	<code>pd.merge(left, right, on="id", validate="1:1")</code>	P9

! `NaN > 70` is `False`, so a filter drops missing values without telling you. Count them before and after. [P9](#).

41.4 Matplotlib

Task	Call	Where
a figure and axes	<code>fig, ax = plt.subplots(figsize=(w, h))</code>	P10
scatter, line, bar	<code>ax.scatter(x, y)</code> , <code>ax.plot(x, y)</code> , <code>ax.bar(x, h)</code>	P10
histogram	<code>ax.hist(values, bins=24)</code>	S2
boxplot	<code>ax.boxplot(values, vert=False)</code>	S2
labels and limits	<code>ax.set(xlabel=..., ylabel=..., ylim=(0, 100))</code>	P10
a reference line	<code>ax.axvline(x)</code> , <code>ax.axhline(y)</code>	P10
several panels	<code>plt.subplots(rows, cols, sharey=True)</code>	P10
tidy and save	<code>fig.tight_layout()</code> , <code>fig.savefig(path, dpi=150)</code>	P10

41.5 scipy.stats

Task	Call	Where
normal density, cdf, quantile	<code>stats.norm.pdf / .cdf / .ppf</code>	S6
t multiplier for a 95% interval	<code>stats.t.ppf(0.975, df)</code>	S8
binomial and Poisson probabilities	<code>stats.binom.pmf</code> , <code>stats.poisson.pmf</code>	S6
one-sample t -test	<code>stats.ttest_1samp(values, mu0)</code>	S9
two-sample t -test (Welch)	<code>stats.ttest_ind(a, b, equal_var=False)</code>	S10
rank correlation	<code>stats.spearmanr(x, y)</code>	S3
rank-based comparison	<code>stats.mannwhitneyu(a, b)</code>	S10
table of counts	<code>stats.chi2_contingency(table)</code>	S10
fit a straight line	<code>stats.linregress(x, y)</code>	S11

`stats.linregress` returns `.slope`, `.intercept`, `.rvalue`, `.pvalue`, `.stderr` and `.intercept_stderr` — the last two are the ones S11 is about.

41.6 Work someone else can rerun

Habit	How	Where
pin the versions	a <code>requirements.txt</code> with <code>==</code>	P11
seed every random draw	<code>np.random.default_rng(2026)</code>	P11
paths relative to the script	<code>Path(__file__).parent / "data"</code>	P11
assert your assumptions	<code>assert len(df) == 240</code>	P11
never edit the raw data	read it, write derived files elsewhere	P11

42 Glossary

One line each, in alphabetical order, with a link to the unit that does the explaining. Where two terms are routinely confused they are defined next to each other on purpose.

Alternative hypothesis. What counts as evidence against the null; it fixes which results are “extreme”. [S9](#)

Base rate. How common something is before you look at any evidence. Without it a conditional probability cannot be reversed. [S4](#)

Bootstrap. Estimating uncertainty by resampling your own data with replacement, thousands of times. [S8](#)

Broadcasting. NumPy’s rule for combining arrays of different shapes by stretching a dimension of size 1. [P7](#)

Central limit theorem. The sample *mean* becomes approximately normal as n grows, whatever the population looks like, provided the variance is finite. It says nothing about the individual observations. [S7](#)

Chain rule. The derivative of $f(g(x))$ is $f'(g(x)) \cdot g'(x)$; the second factor is the one people forget. [M4](#)

Condition number. How much a system of equations amplifies small errors in its inputs. [M10](#)

Confidence interval. A range built by a procedure that captures the true value a stated fraction of the time across repeated studies. The percentage belongs to the procedure, not to the interval in front of you. [S8](#)

Confounder. A variable related to both the thing you varied and the thing you measured, whose influence gets credited to something else. [S12](#)

Correlation (r). A unitless measure of *linear* association between -1 and $+1$. $r = 0$ means no linear relationship, not no relationship. [S3](#)

Covariance. The unscaled version of correlation; its sign is informative and its size depends on the units you chose. [S3](#)

Density. For a continuous variable, the function whose *area* gives probability. Its height is not a probability and may exceed 1. [S5](#)

Derivative. The rate at which a function responds to a small change in its input; the gradient of the tangent. [M3](#)

Design. How units got from the population into your table. It decides which conclusions are defensible. [S1](#)

Eigenvector. A direction a matrix stretches without turning; its eigenvalue is the stretch factor. [M12](#)

Estimand. The number you want, defined on the population. Distinct from the **estimate**, which is the number you compute from a sample. [S1](#)

Expected value. The long-run average of a random variable; a balance point, often not a possible outcome. [S5](#)

Gradient. The vector of partial derivatives; points in the direction of steepest increase. [M6](#)

Independence. $P(A \cap B) = P(A)P(B)$: knowing one tells you nothing about the other. Not the same as **mutually exclusive**, which means they cannot both happen and makes them strongly dependent. [S4](#)

Integral. Accumulated area under a curve; the reverse of differentiation. [M7](#)

Interquartile range (IQR). $Q_3 - Q_1$, the width of the middle half. [S2](#)

Least squares. Choosing coefficients to minimise the sum of squared residuals. [M11](#)

Mean. The balance point of a set of values; sensitive to extremes. Compare the **median**, the half-way point by count, which is not. [S2](#)

Multiple comparisons. Running many tests, so that false positives become the expected outcome rather than the exception. [S12](#)

Nominal / ordinal. Categories without an order, and categories with one but without known spacing. A median makes sense for the second and not the first. [S1](#)

Normal distribution. The bell-shaped family that arises when many small independent effects add up; 68–95–99.7 within one, two and three standard deviations. [S6](#)

Null hypothesis. The specific claim a test assumes while computing how surprising the data would be. [S9](#)

Overdispersion. Count data more variable than a Poisson allows — variance above the mean. [S6](#)

Parameter. A fixed, usually unknown, property of a population, written with a Greek letter. [S1](#)

Population. The entire group a question is about, as opposed to the **sample**, which is what you measured. [S1](#)

Power. The probability of detecting a real effect of a given size. Low power makes non-significant results uninformative and published findings less reliable. [S9](#)

Prediction interval. A range for one *new* observation; wider than a confidence interval for the mean, and it does not shrink to nothing as n grows. [S11](#)

p -value. The probability, computed assuming the null is true, of a test statistic at least as extreme as the observed one. Not the probability that the null is true. [S9](#)

Quantile. A value at or below which approximately a stated proportion of the data lies; the median is the 0.5-quantile. [S2](#)

R^2 . The proportion of variance in the outcome accounted for by the fitted model. Not a measure of whether the model is correct. [S11](#)

Random variable. A rule assigning a number to each outcome of a random process. [S5](#)

Regression to the mean. Units selected for being extreme are closer to average next time, with no cause required. [S12](#)

Residual. Observed value minus fitted value; what the model did not account for. [S11](#)

Sampling distribution. The distribution of a statistic across all the samples you might have taken. [S7](#)

Simpson's paradox. An association that reverses when a third variable is taken into account. [S12](#)

Standard deviation (s , σ). How spread out the *observations* are. Does not shrink as you collect more data. [S2](#)

Standard error. How much a *statistic* moves between samples; for a mean, $s/\sqrt{n}$. Shrinks as you collect more data. [S7](#)

Survivorship bias. Drawing conclusions from only the units that made it into the data. [S12](#)

Transpose. Flipping a matrix's rows and columns; $(AB)^T = B^T A^T$. [M9](#)

t distribution. The distribution used in place of the normal when the population standard deviation has been estimated from the data; slightly wider, and more so for small samples. [S8](#)

Type I / Type II error. Rejecting a true null; failing to reject a false one. Their rates are α and β , and you choose only the first. [S9](#)

Unit. What one row of your table represents, and the thing that must be independently sampled for the standard formulas to apply. [S1](#)

Variance. The average squared distance from the mean; the square of the standard deviation, in squared units. [S2](#), [S5](#)